

IBM Storage Protect  
8.2.1

*Using the Application Programming  
Interface*



**Note:**

Before you use this information and the product it supports, read the information in [“Notices” on page 241.](#)

**Edition notice**

This edition applies to version 8, release 2, modification 1 of IBM Storage Protect (product numbers 5725-W98, 5725-W99, and 5725-X15) and to all subsequent releases and modifications until otherwise indicated in new editions.

© **Copyright International Business Machines Corporation 1993, 2026.**

US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

---

# Contents

|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>About this publication.....</b>                            | <b>vii</b> |
| Who should read this publication.....                         | vii        |
| Publications .....                                            | vii        |
| Conventions used in this publication.....                     | vii        |
| <b>What's new in version 8.2.0.....</b>                       | <b>ix</b>  |
| <b>Chapter 1. API overview.....</b>                           | <b>1</b>   |
| Understanding configuration and options files.....            | 1          |
| Setting up the API environment.....                           | 3          |
| <b>Chapter 2. Installing and upgrading.....</b>               | <b>5</b>   |
| Installing the API.....                                       | 5          |
| Building and running the sample API application.....          | 5          |
| UNIX or Linux sample application source files.....            | 5          |
| Windows 64-bit sample application.....                        | 7          |
| <b>Chapter 3. Configuring and administrating APIs.....</b>    | <b>9</b>   |
| Considerations for designing an application.....              | 9          |
| Determining size limits.....                                  | 12         |
| Maintaining API version control.....                          | 12         |
| Using multithreading.....                                     | 14         |
| Signals and signal handlers.....                              | 15         |
| Starting or ending a session.....                             | 15         |
| Object names and IDs.....                                     | 20         |
| Accessing objects as session owner.....                       | 21         |
| Accessing objects across nodes and owners.....                | 22         |
| Managing file spaces.....                                     | 22         |
| Associating objects with management classes.....              | 24         |
| Expiration/deletion hold and release.....                     | 26         |
| Querying the IBM Storage Protect system.....                  | 28         |
| Server efficiency.....                                        | 31         |
| Sending data to a server.....                                 | 32         |
| Set up the API to send performance data.....                  | 34         |
| Sending objects to the server.....                            | 36         |
| Data deduplication.....                                       | 44         |
| Application failover.....                                     | 48         |
| Example flow diagrams for backup and archive.....             | 51         |
| File grouping.....                                            | 54         |
| Receiving data from a server.....                             | 57         |
| Updating and deleting objects on the server.....              | 64         |
| Logging events.....                                           | 65         |
| State diagram summary for the IBM Storage Protect API.....    | 65         |
| Understanding interoperability.....                           | 67         |
| Backup-archive client interoperability.....                   | 67         |
| Operating system interoperability.....                        | 69         |
| Backing up multiple nodes with client node proxy support..... | 70         |
| Using the API with Unicode.....                               | 71         |
| When to use Unicode.....                                      | 71         |
| Setting up Unicode.....                                       | 71         |

## **Chapter 4. API commands, options, and utilities.....73**

|                                              |     |
|----------------------------------------------|-----|
| API function calls.....                      | 73  |
| dsmBeginGetData.....                         | 75  |
| dsmBeginQuery.....                           | 76  |
| dsmBeginTxn.....                             | 81  |
| dsmBindMC.....                               | 82  |
| dsmChangePW.....                             | 83  |
| dsmCleanUp.....                              | 84  |
| dsmDeleteAccess.....                         | 84  |
| dsmDeleteFS.....                             | 85  |
| dsmDeleteObj.....                            | 86  |
| dsmEndGetData.....                           | 87  |
| dsmEndGetDataEx.....                         | 87  |
| dsmEndGetObj.....                            | 88  |
| dsmEndQuery.....                             | 88  |
| dsmEndSendObj.....                           | 89  |
| dsmEndSendObjEx.....                         | 89  |
| dsmEndTxn.....                               | 90  |
| dsmEndTxnEx.....                             | 91  |
| dsmGetData.....                              | 93  |
| dsmGetBufferData.....                        | 93  |
| dsmGetNextQObj.....                          | 94  |
| dsmGetObj.....                               | 97  |
| dsmGroupHandler.....                         | 98  |
| dsmInit.....                                 | 99  |
| <b>dsmInitEx</b> .....                       | 102 |
| dsmLogEvent.....                             | 106 |
| dsmLogEventEx.....                           | 107 |
| dsmQueryAccess.....                          | 108 |
| dsmQueryApiVersion.....                      | 109 |
| dsmQueryApiVersionEx.....                    | 109 |
| dsmQueryCliOptions.....                      | 110 |
| dsmQuerySessInfo.....                        | 110 |
| dsmQuerySessOptions.....                     | 111 |
| dsmRCMsg.....                                | 112 |
| dsmRegisterFS.....                           | 113 |
| dsmReleaseBuffer.....                        | 114 |
| dsmRenameObj.....                            | 115 |
| dsmRequestBuffer.....                        | 116 |
| dsmRetentionEvent.....                       | 117 |
| dsmSendBufferData.....                       | 118 |
| dsmSendData.....                             | 119 |
| dsmSendObj.....                              | 120 |
| dsmSetAccess.....                            | 123 |
| dsmSetUp.....                                | 124 |
| dsmTerminate.....                            | 126 |
| dsmUpdateFS.....                             | 126 |
| dsmUpdateObj.....                            | 127 |
| dsmUpdateObjEx.....                          | 129 |
| API return codes source file: dsmrc . h..... | 131 |
| API type definitions source files.....       | 140 |
| API function definitions source file.....    | 175 |

## **Appendix A. API return codes source file: dsmrc . h..... 185**

## **Appendix B. API type definitions source files.....195**

|                                                              |            |
|--------------------------------------------------------------|------------|
| <b>Appendix C. API function definitions source file.....</b> | <b>231</b> |
| <b>Appendix D. Accessibility.....</b>                        | <b>239</b> |
| <b>Notices.....</b>                                          | <b>241</b> |
| <b>Glossary.....</b>                                         | <b>245</b> |
| <b>Index.....</b>                                            | <b>247</b> |



## About this publication

---

This publication provides information to help you to perform the following tasks:

- Add IBM® Storage Protect application program interface calls to an existing application
- Write programs with general-use program interfaces that obtain the services of IBM Storage Protect.

In addition to the application programming interface (API), the following programs are included on several operating systems:

- A backup-archive client program that backs up and archives files from your workstation or file server to storage, and restores and retrieves backup versions and archived copies of files to your local file systems.
- A Web backup-archive client that an authorized administrator, support person, or end user can use to perform backup, restore, archive, and retrieve services using a Web browser on a remote machine.
- An administrative client program that you can access from a Web browser or from the command line. An administrator controls and monitors server activities, defines storage management policies for backup, archive, and space management services, and sets up schedules to perform these services at regular intervals.

## Who should read this publication

---

This publication provides instructions for you to add API calls to an existing application. You should be familiar with C programming language and IBM Storage Protect functions.

## Publications

---

The IBM Storage Protect product family includes, IBM Storage Protect for Virtual Environments, IBM Storage Protect for Databases, and several other storage management products from IBM.

To view IBM product documentation, see [IBM Documentation](#).

## Conventions used in this publication

---

This publication uses the following typographical conventions:

| Example                    | Description                                                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| autoexec.ncf<br>hsmgui.exe | A series of lowercase letters with an extension indicates program file names.                                                                                        |
| DSMI_DIR                   | A series of uppercase letters indicates return codes and other values.                                                                                               |
| <b>dsmQuerySessInfo</b>    | Boldface type indicates a command that you type on a command line, the name of a function call, the name of a structure, a field within a structure, or a parameter. |
| <b><i>timeformat</i></b>   | Boldface italic type indicates a backup-archive client option. The bold type is used to introduce the option, or used in an example.                                 |
| <i>dateformat</i>          | Italic type indicates an option, the value of an option, a new term, a placeholder for information you provide, or for special emphasis in the text.                 |
| maxcmdretries              | Monospace type indicates fragments of a program or information as it might appear on a display screen, such a command example.                                       |

| Example       | Description                                                                       |
|---------------|-----------------------------------------------------------------------------------|
| plus sign (+) | A plus sign between two keys indicates that you press both keys at the same time. |



## What's new in version 8.2.0

---

IBM Storage Protect 8.2.0 introduces new features and updates.

For a list of new features and updates in this release, see [API updates](#).

Any new and changed information in this product documentation is indicated by a vertical bar (|) to the left of the change.



---

# Chapter 1. API overview

The IBM Storage Protect application program interface (API) enables an application client to use storage management functions.

The API includes function calls that you can use in an application to perform the following operations:

- Start or end a session
- Assign management classes to objects before they are stored on a server
- Back up or archive objects to a server
- Restore or retrieve objects from a server
- Query the server for information about stored objects
- Manage file spaces
- Send retention events

When you, as an application developer, install the API, you receive the files that an end user of an application needs:

- The API shared library.
- The messages file.
- The sample client options files.
- The source code for the API header files that your application needs.
- The source code for a sample application, and the makefile to build it.

For 64-bit applications, all compiles should be performed using compiler options that enable 64-bit support. For example, '-q64' should be used when building API applications on AIX®, and '-m64' should be used on Linux®. See the sample make files for more information.

**Important:** When you install the API, ensure that all files are at the same level.

For information about installing the API, see [Installing the IBM Storage Protect backup-archive clients](#).

References to UNIX and Linux include AIX, HP-UX, Linux, Mac OS X, and Oracle Solaris operating systems.

---

## Understanding configuration and options files

Configuration and options files set the conditions and boundaries under which your session runs.

You, an administrator, or an end user can set option values to:

- Set up the connection to a server
- Control which objects are sent to the server and the management class to which they are associated

You define options in one or two files when you install the API on your workstation.

On UNIX and Linux operating systems, the options reside in two options files:

- dsm.opt - the client options file
- dsm.sys - the client system options file

On other operating systems, the client options file (dsm.opt) contains all of the options.

**Restriction:** The API does not support the following backup-archive client options:

- autofsrrename
- changingretries
- domain
- eventlogging

- groups
- subdir
- users
- virtualmountpoint

You also can specify options on the **dsmInitEx** function call. Use the option string parameter or the API configuration file parameter.

The same option can derive from more than one configuration source. When this happens, the source with the highest priority takes precedence. [Table 1 on page 2](#) lists the priority sequence.

*Table 1. Configuration sources in order of decreasing priority*

| Priority | UNIX and Linux                             | Windows                                 | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------|--------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1        | dsm.sys file<br>(client system options)    | not applicable                          | <p>This file contains options that a system administrator sets only for UNIX and Linux.</p> <p><b>Tip:</b> If your dsm.sys file contains server stanzas, ensure that the passwordaccess option specifies the same value (either prompt or generate) in each of the stanzas.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 2        | Option string<br>(client options)          | Option string<br>(all options)          | <p>One of these options takes effect when it is passed as a parameter to a <b>dsmInitEx</b> call. The list can contain client options such as compressalways, servername (UNIX and Linux only), or tcpserveraddr (non-UNIX).</p> <p>With the API option string, an application client can make changes to the option values in the API configuration file and the client options file. For example, your application might query the end user if compression is required. Depending on the user responses, you can construct an API option string with this option and pass it into the call to <b>dsmInitEx</b>.</p> <p>For information about the API option string format, see “<a href="#">dsmInitEx</a>” on <a href="#">page 102</a>. You also can set this parameter to NULL. This indicates that there is no API option string for this session.</p> |
| 3        | API configuration file<br>(client options) | API configuration file<br>(all options) | <p>The values that you set in the API configuration file override the values that you set in the client options file. Set up the options in the API configuration file with values that are appropriate in the IBM Storage Protect session for the user. The values take effect when the API configuration file name is passed as a parameter in the <b>dsmInitEx</b> call.</p> <p>You also can set this parameter to NULL. This indicates that there is no API configuration file for this session.</p>                                                                                                                                                                                                                                                                                                                                                   |

Table 1. Configuration sources in order of decreasing priority (continued)

| Priority | UNIX and Linux                   | Windows                       | Description                                                                                                                                                                                                                                          |
|----------|----------------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4        | dsm.opt file<br>(client options) | dsm.opt file<br>(all options) | On UNIX and Linux operating systems the dsm.opt file contains the user options only. On other operating systems, the dsm.opt file contains all options. To override the options in these files, follow the methods that are described in this table. |

### Related information

[Processing options](#)

## Setting up the API environment

The API uses unique environment variables to locate files. You can use different files for API applications from those that the backup-archive client uses. Applications can use the **dsmSetup** function call to override the values that the environment variables set.

**Tip:** On Windows, the default installation directory is: %SystemDrive%\Program Files\Common Files\Tivoli\TSM\api

Table 2 on page 3 lists the API environment variables by operating system.

Table 2. API environment variables

| Variables   | UNIX and Linux                                                                                                                                                                 | Windows                                                                                                                                                                                   |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DSMI_CONFIG | The fully-qualified name for the client options file (dsm.opt).                                                                                                                | The fully-qualified name for the client options file (dsm.opt).                                                                                                                           |
| DSMI_DIR    | Points to the path that contains the dsm.sys, en_US subdirectory, and any other national language support (NLS) language. The en_US subdirectory must contain dsmclientV3.cat. | Points to the path that contains dscenu.txt and any NLS message file.                                                                                                                     |
| DSMI_LOG    | Points to the path for the dserror.log file.                                                                                                                                   | Points to the path for the dserror.log file.<br><br>If the client <b>errorlogname</b> option is set, the location specified by that option overrides the directory specified by DSMI_LOG. |



---

# Chapter 2. Installing and upgrading

## Installing the API

---

Information about installing the IBM Storage Protect application programming interface (API) is provided in the installation procedures for the backup-archive client.

## Building and running the sample API application

---

The API package includes sample applications that demonstrate the API function calls in context. Install a sample application and review the source code to understand how you can use the function calls.

Select one of the following sample API application packages:

- The interactive, single-threaded application package (dapi\*)
- The multithreaded application package (callmt\*)
- The logical object grouping test application (dsmgrp\*)
- The event-based retention policy sample application (callevnt)
- The deletion hold sample application (callhold)
- The data retention protection sample application (callret)
- The IBM Storage Protect data buffer sample program (callbuff)

To help you get started, review the procedure to build the sample dapismp sample application by your platform:

- For UNIX or Linux applications, see [“UNIX or Linux sample application source files”](#) on page 5.
- For Windows applications, see [“Windows 64-bit sample application”](#) on page 7.

The dapismp sample application creates its own data streams when backing up or archiving objects. It does not read or write objects to the local disk file system. The object name does not correspond to any file on your workstation. The "seed string" that you issue generates a pattern that can be verified when the object is restored or retrieved. Once you compile the sample application and run **dapismp** to start it, follow the instructions that display on your screen.

## UNIX or Linux sample application source files

To build and run the sample UNIX or Linux sample application, you need to ensure you have certain source files. Once you build the sample application you can compile and run it.

The files that are listed in [Table 3 on page 5](#) include the source files and other files that you need to build the sample application that is included with the API package.

*Table 3. Files that you need to build the UNIX or Linux API sample application*

| File names     | Description                                            |
|----------------|--------------------------------------------------------|
| README_api_enu | README file                                            |
| dsmrc.h        | Return codes header file                               |
| dsmapi.h       | Common type definitions header file                    |
| dsmapios.h     | Operating system-specific type definitions header file |
| dsmapi.h       | Function prototype header file                         |
| release.h      | Release values header file                             |

Table 3. Files that you need to build the UNIX or Linux API sample application (continued)

| File names                                                                                                   | Description                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dapibkup.c<br>dapidata.h<br>dapiinit.c<br>dapint64.h<br>dapint64.c<br>dapipref.c<br>dapiproc.c<br>dapiproc.h | dapipw.c<br>dapiqry.c<br>dapirc.c<br>dapismp.c<br>dapitype.h<br>dapiutil.h<br>dapiutil.c<br>Modules for the command line-driven sample application           |
| makesmp[64].xxx                                                                                              | Makefile to build dapismp for your operating system. The xxx indicates the operating system.                                                                 |
| callmt1.c<br>callmt2.c                                                                                       | Multi-threaded sample files                                                                                                                                  |
| callmtu1.c<br>callmtu2.c                                                                                     | Multi-threaded Unicode sample files                                                                                                                          |
| libApiDS.xx<br>libApiDS64.xx, or<br>libApiTSM64.xx                                                           | Shared library (the suffix is platform-dependent)                                                                                                            |
| dsmgrp.c<br>callevnt.c<br>callhold.c<br>callret.c<br>callbuff.c<br>dpstthread.c                              | Grouping sample files<br>Event-based retention policy sample source code<br>Deletion hold sample source code<br>Data retention protection sample source code |

## Building the UNIX or Linux sample application

You build the **dapismp** sample API application by using a compiler for your operating system.

### About this task

You must install the following compilers to build the UNIX or Linux API sample application:

- IBM AIX - IBM Visual Age compiler version 6 or later
- HP-IA64 - aCC compiler A.05.50 or later
- Linux - GCC compiler version 3.3.3 or later
- Mac OS X - GCC compiler version 4.0 or later
- Oracle Solaris - Oracle Studio C++ compiler version 11 or later

### Procedure

1. To build the API samples, run the following command:

```
gmake -f makesmp[64].xxx
```

Where xxx indicates the operating system.

2. After you build the samples, set up your environment variables, including the DSMI\_DIR, and your options files. For more information, see [“Understanding configuration and options files”](#) on page 1.



3. The first time you log on, log on as the root user to register your password.

**Tip:** Setting the `compressalways` option to no might not resend an object uncompressed. This behavior depends on the application functionality.

To specify the Shared Memory communications method on AIX, the IBM Storage Protect API client user must comply with one of the following conditions:

- Must be logged in as the root user.
- Must have the same UID as the process that is running the IBM Storage Protect server.

For more information, see the application program documentation.

4. Run the **dapism** command to start the application.
5. Choose from the list of options that is displayed. Ensure that you run the sign-on action before you run any other actions.

**Requirement:** Always prefix the file space, high-level, and low-level names with the correct path delimiter (/) when you enter the name, for example: /myfilespace. You must use this prefix even when you specify the asterisk (\*) wildcard character.

### Related information

[Environment variables \(UNIX and Linux systems\)](#)

## Windows 64-bit sample application

To build and run the sample application for Microsoft Windows 64-bit systems, you must install the IBM Storage Protect API and ensure that you have certain source files.

### Restrictions:

- For best results, use dynamic loading. For an example, see the file `dynaload.c` and the implementation in the sample code.
- Files for the sample application are in the following directories:
  - api64\obj**  
Contains the API sample program object files.
  - api64\samprun**  
Contains the sample program **dapism**. The sample program contains the execution directory.
- The DLL `tsmapi64.dll` is a 64-bit DLL.
- Use the Microsoft C/C++ Compiler Version 15 and the makefile `makesmp64.mak` to compile the API sample application **dapism**. You might have to adjust the makefiles to fit your environment, specifically the library or the include directories.
- After you compile the application, run the sample application by issuing the command **dapism** from the `api64\samprun` directory.
- Choose from the list of options displayed that are displayed. Ensure that you run the sign-on action before you run any other actions.
- Always prefix the file space, high-level, and low-level names with the correct path delimiter (\) when you enter the name, for example: \myfilespace. You must use this prefix even when you specify the asterisk (\*) wildcard character.

For Windows operating systems, the source files that you must have to build the sample application are listed in Table 4 on page 7. The sample application is included in the API package. For your convenience, a precompiled executable (`dapism.exe`) is also included.

Table 4. Files for building the Windows 64-bit API sample application

| File names                | Description |
|---------------------------|-------------|
| <code>api.txt</code>      | README file |
| <code>tsmapi64.dll</code> | API DLLs    |

*Table 4. Files for building the Windows 64-bit API sample application (continued)*

| <b>File names</b>                                                                                                                                                    | <b>Description</b>                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dsmrc.h                                                                                                                                                              | Return codes header file                                                                                                                                                     |
| dsmapi64.h                                                                                                                                                           | Common type definitions header file                                                                                                                                          |
| dsmapi64.h                                                                                                                                                           | Operating system-specific type definitions header file                                                                                                                       |
| dsmapi64.h                                                                                                                                                           | Function prototype header file                                                                                                                                               |
| dsmapi64.h                                                                                                                                                           | Dynamically loaded function prototype header file                                                                                                                            |
| release.h                                                                                                                                                            | Release values header file                                                                                                                                                   |
| dapi64.h<br>dapi64.h<br>dapi64.h<br>dapi64.h                                                                                                                         | Source code header files                                                                                                                                                     |
| tsmapi64.lib                                                                                                                                                         | Implicit library                                                                                                                                                             |
| dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c<br>dapi64.c | Source code files for dapi64.exe                                                                                                                                             |
| makesmp64.mak<br>(Windows x64)<br>makesmp64.mak<br>(Windows IA64)                                                                                                    | Makefiles to build sample applications                                                                                                                                       |
| callmt1.c<br>callmt2.c<br>callmtu164.c<br>callmtu264.c                                                                                                               | Multithreaded sample files                                                                                                                                                   |
| dpsthread.c                                                                                                                                                          | Sample file source code                                                                                                                                                      |
| callevnt.c<br>callhold.c<br>callret.c<br>callbuff.c                                                                                                                  | Event-Based retention policy source code<br>Deletion hold sample source code<br>Data retention protection sample source code<br>Shared buffer ( no copy) sample source code. |

# Chapter 3. Configuring and administrating APIs

## Considerations for designing an application

When you design an application, you must have a broad understanding of many aspects of the API.

To gain an understanding of the API, review the following topics:

- [“Determining size limits” on page 12](#)
- [“Maintaining API version control” on page 12](#)
- [“Using multithreading” on page 14](#)
- [“Signals and signal handlers” on page 15](#)
- [“Starting or ending a session” on page 15](#)
- [“Object names and IDs” on page 20](#)
- [“Controlling access to password files” on page 18](#)
- [“Accessing objects as session owner” on page 21](#)
- [“Accessing objects across nodes and owners” on page 22](#)
- [“Managing file spaces” on page 22](#)
- [“Associating objects with management classes” on page 24](#)
- [“Expiration/deletion hold and release” on page 26](#)
- [“Querying the IBM Storage Protect system” on page 28](#)
- [“Sending data to a server” on page 32](#)
- [“Example flow diagrams for backup and archive” on page 51](#)
- [“File grouping” on page 54](#)
- [“State diagram summary for the IBM Storage Protect API” on page 65](#)

When you design your application, review the considerations in [Table 5 on page 9](#). Start structures with **memset** fields might change in subsequent releases. The **stVersion** value increments with each product enhancement.

Table 5. API Considerations for designing an application

| Design item    | Considerations                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Setting locale | <p>The application must set the locale before the API is called. To set the locale to the default value, add the following code to the application:</p> <pre>setlocale(LC_ALL, "");</pre> <p>To set the locale to another value, use the same call with the proper locale in the second parameter. Check for specifics in the documentation for each operating system that you are using.</p> |

Table 5. API Considerations for designing an application (continued)

| Design item     | Considerations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Session control | <p>Apply the following guidelines to session control:</p> <ul style="list-style-type: none"> <li>Assign a unique node name for each IBM Storage Protect backup-archive client and IBM Storage Protect API client product. The following products are examples of these clients: <ul style="list-style-type: none"> <li>IBM Storage Protect for Mail</li> <li>or IBM Storage Protect HSM for Windows</li> </ul> </li> <li>Use a consistent owner name across a backup and restore procedure.</li> <li>Use the passwordaccess option to manage access to the protected password file.</li> <li>Ensure that sessions for data movement end when the task is completed so that devices on the server are freed for use by other sessions.</li> <li>To permit LAN-free data transfer, use the <b>dsmSetup</b> function call with the multithread flag set to on.</li> <li>On AIX, when you are using multithreaded applications or LAN-free, especially running on machines with multiple processors, set the environment variable AIXTHREAD_SCOPE to S in the environment before you start the application, for better performance and more solid scheduling. For example: <pre>EXPORT AIXTHREAD_SCOPE=S</pre> <p>By setting AIXTHREAD_SCOPE to S, user threads that are created with default attributes are placed into system-wide contention scope. The user thread is bound to a kernel thread and is scheduled by the kernel. The underlying kernel thread is not shared with any other user thread. For more information, see <a href="#">“Using multithreading” on page 14</a>.</p> </li> <li>Ensure that only one thread in a session calls any API function at any time. Applications that use multiple threads with the same session handle must synchronize the API calls. For example, use a <b>mutex</b> to synchronize API calls: <pre>getTSMMutex() issue TSM API call releaseTSMMutex()</pre> <p>Use this approach only when the threads share a handle. You can use parallel calls to API functions if the calls have different session handles.</p> </li> </ul> |

Table 5. API Considerations for designing an application (continued)

| Design item                    | Considerations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Session control<br>(continued) | <ul style="list-style-type: none"> <li>Implement a threaded consumer/producer model for data movement. API calls are synchronous and the calls for <b>dsmGetData function</b> and <b>dsmSendData function</b> block until they are finished. By using a consumer/producer model, the application can read the next buffer during waiting periods for the network. Also, decoupling the data read/write and the network increases performance when there is a network bottleneck or delays. In general, the following holds: <div data-bbox="446 457 1237 510" data-label="Text"> <pre>Data thread &lt;--&gt; shared queue of buffers &lt;--&gt; communication thread (issue calls to the IBM Storage Protect API)</pre> </div> </li> <li>Use the same session for multiple operations to avoid incurring an overhead. For applications that deal with many small objects, implement session-pooling so that the same session can be used across multiple small operations. An overhead is associated with opening and closing a session to the IBM Storage Protect server. The <b>dsmInit/</b><b>dsmInitEX</b> call is serialized so even in a multithreaded application only one thread can sign on at any time. Also, during sign-on the API sends a number of one-time queries to the server so that the server can do all operations. These queries include policy, option, file spaces, and local configuration.</li> </ul> |
| Operation sequence             | <p>The IBM Storage Protect server locks file space database entries during some operations. The following rules apply when you are designing IBM Storage Protect API applications:</p> <ul style="list-style-type: none"> <li>Queries lock the file space during the entire transaction.</li> <li>The query lock can be shared with other query operations, so multiple query operations on the same file space can execute concurrently.</li> <li>The following operations are used to modify the IBM Storage Protect server database (<b>DB Chg</b>): send, get, rename, update, and delete.</li> <li>Completion of a <b>DB Chg</b> operation requires a file space lock during the database change at the end of the transaction.</li> <li>Multiple <b>DB Chg</b> operations on the same file space can execute concurrently. There might be a delay while the sequence waits for the lock at the end transaction.</li> <li>The query lock cannot be shared with <b>DB Chg</b> operations. A <b>DB Chg</b> operation delays the beginning of a query on the same file space, so design your applications to separate and serialize queries from <b>DB Chg</b> operations on the same file space.</li> </ul>                                                                                                                                                                                                                   |
| Object naming                  | <p>When you name objects, consider the following factors:</p> <ul style="list-style-type: none"> <li>The specific object names are the high-level and low-level object names. If a unique identifier, such as a date stamp, is included in the name, then backup objects are always active. The objects expire only when they are marked inactive by the <b>dsmDeleteObj</b> function call.</li> <li>The restore method for objects determines how to format the name for easy queries. If you plan to use a partial object restore (POR), you cannot use compression. To suppress compression, use the <b>dsmSendObj objAttr objCompressed=bTrue</b> function.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Object grouping                | <p>Group objects logically by using file spaces. A file space is a container on the server that provides a grouping category for the objects. The API queries all file spaces during the initial sign-on and also during queries, so the number of file spaces must be restricted. A reasonable assumption is that an application sets up 20 - 100 file spaces per node. The API can cater for more file spaces, but each file space incurs an overhead for the session. To create a more granular separation, use the directory object in the application.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Table 5. API Considerations for designing an application (continued)

| Design item             | Considerations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Object handling</b>  | <p>Do not store <code>objectID</code> values to use for future restores. These values are not guaranteed to be persistent during the life of the object.</p> <p>During a restore, pay special attention to the restore order. After the query, sort on this value before the restore. If you are using multiple types of serial media, then access the different types of media in separate sessions. For more information, see the following topic:</p> <p><a href="#">“Selecting and sorting objects by restore order” on page 59</a></p> |
| <b>Management class</b> | <p>Consider how much control the application must have over the management class that is associated with the application objects. You can define include statements, or you can specify a name on the <b><code>dsmSendObj</code></b> function call.</p>                                                                                                                                                                                                                                                                                     |
| <b>Object size</b>      | <p>IBM Storage Protect needs to know a size estimate for each object. Consider how your application estimates the size of an object. An overestimation of the object size is better than an underestimation.</p>                                                                                                                                                                                                                                                                                                                            |

## Determining size limits

Certain data structures or fields in the API have size limits. These structures are often names or other text fields that cannot exceed a predetermined length.

The following fields are examples of data structures that have size limits:

- Application type
- Archive description
- Copy group destination
- Copy group name
- File space information
- Management class name
- Object owner name
- Password

These limits are defined as constants within the header file `dsmapi.h`. Any storage allocation is based on these constants rather than on numbers that you enter. For more information, see [“API type definitions source files” on page 140](#).

## Maintaining API version control

All APIs have some form of version control. The API version that you use in your application must be compatible with the version of the API library that is installed on the user workstation.

The **`dsmQueryApiVersionEx`** should be the first API call that you enter when you use the API. This call performs the following tasks:

- Confirms that the API library is installed and available on the end user's system
- Returns the version level of the API library that the application accesses

The API is designed to be upwardly compatible. Applications that are written to older versions or releases of the API library operate correctly when you run a later version.

Determining the release of the API library is very important because some releases might have different memory requirements and data structure definitions. Downward compatibility is unlikely. See [Table 6 on page 13](#) for information about your platform.

Table 6. Platform compatibility information

| Platform      | Description                                                       |
|---------------|-------------------------------------------------------------------|
| Windows       | The message files must be at the same level as the library (DLL). |
| UNIX or Linux | The API library and the message files must be at the same level.  |

The **dsmQueryApiVersionEx** call returns the version of the API library that is installed on the end user workstation. You can then compare the returned value with the version of the API that the application client is using.

The API version number of the application client is entered in the compiled object code as a set of four constants defined in `dsmapi.h`:

```
DSM_API_VERSION  
DSM_API_RELEASE  
DSM_API_LEVEL  
DSM_API_SUB_LEVEL
```

See [“API type definitions source files”](#) on page 140.

The API version of the application client should be less than, or equal to, the API library that is installed on the user's system. Be careful about any other condition. You can enter the **dsmQueryApiVersionEx** call at any time, whether the API session has been started or not.

Data structures that the API uses also have version control information in them. Structures have version information as the first field. As enhancements are made to structures, the version number is increased. When initializing the version field, use the defined structure Version value in `dsmapi.h`.

Figure 1 on page 14 demonstrates the type definition of the structure, **dsmApiVersionEx** from the header file, `dsmapi.h`. The example then defines a global variable that is named **apiLibVer**. It also demonstrates how you can use it in a call to **dsmQueryApiVersionEx** to return the version of the end user's API library. Finally, the returned value is compared to the API version number of the application client.

```

typedef struct
{
    dsUint16_t stVersion;      /* Structure version          */
    dsUint16_t version;       /* API version                */
    dsUint16_t release;       /* API release                */
    dsUint16_t level;         /* API level                  */
    dsUint16_t subLevel;      /* API sub level              */
} dsmApiVersionEx;

dsmApiVersionEx apiLibVer;

memset(&apiLibVer, 0x00, sizeof(dsmApiVersionEx));
dsmQueryApiVersionEx(&apiLibVer);

/* check for compatibility problems */
dsInt16_t appVersion= 0, libVersion = 0;
appVersion=(DSM_API_VERSION * 10000)+(DSM_API_RELEASE * 1000) +
            (DSM_API_LEVEL * 100) + (DSM_API_SUBLEVEL);
libVersion = (apiLibVer.version * 10000) + (apiLibVer.release * 1000) +
            (apiLibVer.level * 100) + (apiLibVer.subLevel);
if (libVersion < appVersion)
{
    printf("\n*****\n");
    printf("The IBM Storage Protect API library is lower than the application version\n");
    printf("Install the current library version.\n");
    printf("*****\n");
    return 0;
}

printf("* API Library Version = %d.%d.%d.%d  *\n",
    apiLibVer.version,
    apiLibVer.release,
    apiLibVer.level,
    apiLibVer.subLevel);

```

Figure 1. An example of obtaining the version level of the API

## Using multithreading

The multithreaded API permits applications to create multiple sessions with the IBM Storage Protect server within the same process. The API can be entered again. Any calls can run in parallel from within different threads.

**Tip:** When you run applications that assume a multithreaded API, use the **dsmQueryAPIVersionEx** call.

To run the API in multithreaded mode, set the *mtFlag* value to **DSM\_MULTITHREAD** on the **dsmSetUp** call. The **dsmSetUp** call must be the first call after the **dsmQueryAPIVersionEx** call. This call must return before any thread calls the **dsmInitEx** call. When all threads complete processing, enter a call to **dsmCleanUp**. The primary process should not end before all the threads complete processing. See *callmt1.c* in the sample application.

**Restriction:** The default for the API is single-thread mode. If an application does not call **dsmSetUp** with the *mtFlag* value set to **DSM\_MULTITHREAD**, the API permits only one session for each process.

Once **dsmSetUp** successfully completes, the application can begin multiple threads and enter multiple **dsmInitEx** calls. Each **dsmInitEx** call returns a handle for that session. Any subsequent calls on that thread for that session must use that handle value. Certain values are process-wide, environmental variables (values that are set on **dsmSetUp**). Each **dsmInitEx** call parses options again. Each thread can run with different options by specifying an overwrite file or an options string on the **dsmInitEx** call. This enables different threads to go to different servers, or use different node names.

**Recommendation:** On HP, set the thread stack to 64K or greater. The default value of the thread stack (32K) might not be sufficient

To permit application users to have a LAN-free session, use **dsmSetUp** *mtFlag* **DSM\_MULTITHREAD** in your application. This is necessary even if the application is single threaded. This flag activates the threading necessary for the IBM Storage Protect LAN-free interface.



## Signals and signal handlers

The application handles signals from the user or the operating system. If the user enters a **CTRL+C** keystroke sequence, the application must catch the signal and send **dsmTerminate** calls for each of the active threads. Then, call **dsmCleanUp** to exit. If sessions are not closed properly, unexpected results might occur on the server.

The application requires signal handlers, such as SIGPIPE and SIGUSR1, for signals that cause the application to end. The application then receives the return code from the API. For example, to ignore SIGPIPE add the following instruction in your application: `signal(SIGPIPE, SIG_IGN)`. After this information is added, instead of the application exiting on a broken pipe, the proper return code is returned.

## Starting or ending a session

IBM Storage Protect is a session-based product, and all activities must be performed within an IBM Storage Protect session. To start a session, the application starts the **dsmInitEx** call. This call must be performed before any other API call other than **dsmQueryApiVersionEx**, **dsmQueryCliOptions**, or **dsmSetUp**.

The **dsmQueryCliOptions** function can be called only before the **dsmInitEx** call. The function returns the values of important options, such as option files, compression settings, and communication parameters. The **dsmInitEx** call sets up a session with the server as indicated in the parameters that are passed in the call or defined in the options files.

The client node name, the owner name, and the password parameters are passed to the **dsmInitEx** call. The owner name is case-sensitive, but the node name and password are not. The application client nodes must be registered with the server before a session starts.

Each time an API application client starts a session with the server, the client application type is registered with the server. Always specify an operating system abbreviation for the application type value because this value is entered in the platform field on the server. The maximum string length is `DSM_MAX_PLATFORM_LENGTH`.

The **dsmInitEx** function call establishes the IBM Storage Protect session with the API configuration file and option list of the application client. The application client can use the API configuration file and option list to set a number of IBM Storage Protect options. These values override the values that are set in the user configuration files during installation. Users cannot change the options that the administrator defines. If the application client does not have a specific configuration file and option list, you can set both of these parameters to NULL. For more information about configuration files, see the following topic:

[“Understanding configuration and options files” on page 1](#)

The **dsmInitEx** function call establishes the IBM Storage Protect session, by using parameters that permit extended verification.

Check the **dsmInitEx** function call and the **dsmInitExOut** information return code. The administrator canceled the last session if the return code is okay (RC=ok) and the information return code (infoRC) is `DSM_RC_REJECT_LASTSESS_CANCELED`. To end the current session immediately, call **dsmTerminate**.

The **dsmQuerySessOptions** call returns the same fields as the **dsmQueryCliOptions** call. The call can be sent only within a session. The values reflect the client options that are valid during that session, from option files, and from any overrides from the **dsmInitEx** call.

After a session starts, the application can send a call to **dsmQuerySessInfo** to determine the server parameters that are set for this session. Items such as the policy domain and transaction limits are returned to the application with this call.

End sessions with a **dsmTerminate** call. Any connection with the server is closed and all resources that are associated with this session are freed.

For an example of starting and ending a session, see the following topic:

[An example of starting and ending a session](#)

The example defines a number of global and local variables that are used in calls to **dsmInitEx** and **dsmTerminate**.

The **dsmInitEx** call takes a pointer to `dsmHandle` as a parameter. The **dsmTerminate** call takes the `dsmHandle` as a parameter.

The example `Details of rcApiOut` displays the details of **rcApiOut**. The function **rcApiOut** calls the API function **dsmRCMsg**, which translates a return code into a message.

The **rcApiOut** call then prints the message for the user. A version of **rcApiOut** is included in the API sample application. The **dsmApiVersion** function is a type definition that is found in the header file `dsmapiTd.h`.

## Session security

The IBM Storage Protect session-based system has security components that permit applications to start sessions in a secure manner. These security measures prohibit unauthorized access to the server and help to insure system integrity.

Every session that is started with the server must complete a sign-on process, requires a password. When the password is coupled with the node name of the client, it insures proper authorization when connecting to the server. The application client provides this password to the API to start the session.

Two methods of password processing are available: *passwordaccess=prompt* or *passwordaccess=generate*. If you use the *passwordaccess=prompt* option, you must include the password value on each **dsmInitEx** call. Or, you can supply the node name and owner name on the **dsmInitEx** call.

Passwords have expiration times associated with them. If a **dsmInitEx** call fails with a password-expired return code (DSM\_RC\_REJECT\_VERIFIER\_EXPIRED), the application client must enter the **dsmChangePW** call using the handle that is returned by **dsmInitEx**. This updates the password before the session can be established successfully. The example in [Figure 4 on page 18](#) demonstrates the procedure to change a password by using **dsmChangePW**. The login owner must use a root user ID or an authorized user ID to change the password.

The second method, *passwordaccess=generate*, encrypts and stores the password value in a file. The node name and owner name cannot be supplied on the **dsmInitEx** call, and the system default values are used. This protects the security of the password file. When the password expires, the *generate* parameter creates a new one and updates the password file automatically.

### Tips:

1. If two different physical machines have the same IBM Storage Protect node name or multiple paths are defined on one node using several server stanzas, *passwordaccess=generate* might only work for the stanza which is used first after password expiration. During the first client-server contact, the user is prompted for the same password for each server stanza separately, and for each stanza, a copy of the password is stored separately. When the password expires, a new password is generated for the stanza which connects the first client-server contact. All subsequent attempts to connect via other server stanzas fail, because there is no logical link between their respective copies of the old password, and the updated copy generated by the stanza used first after password expiration. In this case, you must update the passwords prior to expiration or after expiration as a recovery from the situation, as follows:
  - a. Run **dsmadmc** and update the password on the server.
  - b. Run **dsmc -servername=stanza1** and use the new password to generate a proper entry.
  - c. Run **dsmc -servername=stanza2** and use the new password to generate a proper entry.
2. For UNIX or Linux: Only the root user or an authorized user can change the password when using *passwordaccess=prompt*. Only the root user or an authorized user can start the password file when using *passwordaccess=generate*.

**Restriction:** The options `users` and `groups` are not recognized.

An application can restrict user access by other means, such as setting access filters.

Applications that use multiple IP connections to a single IBM Storage Protect server should use the same node name and IBM Storage Protect client password for each session. Follow these steps to enable this support:

1. Define one IBM Storage Protect server stanza in the dsm.sys file.
2. For the connections not using the default IP address, specify the option values for *TCPserver* address and *TCPport* on the **dsmInitEx** call.

These values override the IP connection information, but the session still uses the same dsm.sys stanza node and password information.

**Note:** Nodes in a cluster share a single password.

```
dsmApiVersionEx * apiApplVer;
char             *node;
char             *owner;
char             *pw;
char             *confFile = NULL;
char             *options = NULL;
dsInt16_t        rc = 0;
dsUInt32_t        dsmHandle;
dsmInitExIn_t     initIn;
dsmInitExOut_t    initOut;
char             *userName;
char             *userNamePswd;

memset(&initIn, 0x00, sizeof(dsmInitExIn_t));
memset(&initOut, 0x00, sizeof(dsmInitExOut_t));
memset(&apiApplVer, 0x00, sizeof(dsmApiVersionEx));
apiApplVer.version = DSM_API_VERSION; /* Set the applications compile */
apiApplVer.release = DSM_API_RELEASE; /* time version.          */
apiApplVer.level   = DSM_API_LEVEL;
apiApplVer.subLevel = DSM_API_SUBLEVEL;

printf("Doing signon for node %s, owner %s, with password %s\n", node, owner, pw);

initIn.stVersion = dsmInitExInVersion;
initIn.dsmApiVersionP = &apiApplVer
initIn.clientNodeNameP = node;
initIn.clientOwnerNameP = owner ;
initIn.clientPasswordP = pw;
initIn.applicationTypeP = "Sample-API AIX";
initIn.configfile = confFile;
initIn.options = options;
initIn.userNameP = userName;
initIn.userPasswordP = userNamePswd;
rc = dsmInitEx(&dsmHandle, &initIn, &initOut);

if (rc == DSM_RC_REJECT_VERIFIER_EXPIRED)
{
    printf("*** Password expired. Select Change Password.\n");
    return(rc);
}
else if (rc)
{
    printf("*** Init failed: ");
    rcApiOut(dsmHandle, rc); /* Call function to print error message */
    dsmTerminate(dsmHandle); /* clean up memory blocks */
    return(rc);
}
```

Figure 2. An example of starting and ending a session

```

void rcApiOut (dsUint32_t handle, dsInt16_t rc)
{
    char *msgBuf ;

    if ((msgBuf = (char *)malloc(DSM_MAX_RC_MSG_LENGTH+1)) == NULL)
    {
        printf("Abort:  Not enough memory.\n") ;
        exit(1) ;
    }

    dsmRCMsg(handle, rc, msgBuf);
    printf("
    free(msgBuf) ;
    return;
}

```

Figure 3. Details of rcApiOut

```

printf("Enter your current password:");
gets(current_pw);
printf("Enter your new password:");
gets(new_pw1);
printf("Enter your new password again:");
gets(new_pw2);
/* If new password entries don't match, try again or exit. */
/* If they do match, call dsmChangePW. */

rc = dsmChangePW(dsmHandle,current_pw,new_pw1);
if (rc)
{
    printf("*** Password change failed. Rc =
}
else
{
    printf("*** Your new password has been accepted and updated.\n");
}
return 0;

```

Figure 4. An example of changing a password

## Controlling access to password files

To control access to the protected password files on UNIX and Linux systems, you can log on as an authorized user and set the passwordaccess option to generate.

### Procedure

Complete the following steps when you set the passwordaccess to generate:

1. Write the application with a call to **dsmSetUp** which passes *argv[0]*. The *argv[0]* contains the name of the application that calls the API. The application is permitted to run an authorized user; however, the administrator must decide on the login name for the authorized user.
2. Set the effective user ID bit (S bit) for the application executable to On. The owner of the application executable file can then become an authorized user and can create a password file, update passwords, and run applications. The owner of the application executable file must be the same as the user ID that runs the program. In the following example, *User* is *user1*, the name of the application executable file is *applA*, and *user1* has read/write permissions on the */home/user1* directory. The *applA* executable file has the following permissions:

```
-rwsr-xr-x user1  group1  applA
```

3. Instruct the users of the application to use the authorized user name to log in. IBM Storage Protect verifies that the login ID matches the application executable owner before it permits access to the protected password file.
4. Set the passworddir option in the *dsm.sys* file to point to a directory where this user has read/write access.  
For example, enter the following line in the server stanza of the *dsm.sys* file:

```
passworddir /home/user1
```

5. Create the password file and ensure that the authorized user owns the file.
6. Log on as user1 and run app1A.
7. Call **dsmSetUp** and pass in *argv*.

## Creating an administrative user with client owner authority

An administrative user with client owner authority can set parameters on the **dsmInitEx** function call to start sessions. This user can function as an "administrative user" with backup and restore authority for the defined nodes.

### Procedure

To receive client owner authority, complete the following steps on the server:

1. Define the administrative user:

```
REGister Admin admin_name password
```

Where:

- *admin\_name* is the administrative user name.
- *password* is the admin password.

2. Define the authority level. Users with system or policy authority also have client owner authority.

```
Grant Authority admin_name classes authority node
```

Where:

- *admin\_name* is the administrative user.
- *classes* is the node.
- *authority* has one of the following levels of authority:
  - owner: full backup and restore authority for the node
  - node: single node
  - domain: group of nodes

3. Define access to a single node.

```
Register Node node_name password userid=user_id
```

Where:

- *node\_name* is the client user node
- *password* is the client user node password
- *user\_id* is the administrative user name

### Results

When the application uses the administrative user, the **dsmInitEx** function is called with the *userName* and *userNamePswd* parameters.

```
dsmInitEx
    clientNodeName = NULL
    clientOwnerName = NULL
    clientPassword = NULL
    userName = 'administrative user' name
    userNamePswd = 'administrative user' password
```

You can set the `passwordaccess` option to `generate` or `prompt`. With either parameter, the `userNamePswd` value starts the session. When the session starts, any backup or restore process can occur for that node.

## Object names and IDs

The IBM Storage Protect server is an object storage server whose primary function is to efficiently store and retrieve named objects. The object ID is unique for each object and remains with the object for the life of the object *except* when you use export or import.

To meet this requirement IBM Storage Protect has two main storage areas, database and data storage.

- The database contains all metadata, such as the name or attributes associated with objects.
- The data storage contains the object data. The data storage is actually a storage hierarchy that the system administrator defines. Data are efficiently stored and managed on either online or offline media, depending on cost and access needs.

Each object that is stored on the server has a name associated with it. The client controls the following key components of that name:

- File space name
- High-level name
- Low-level name
- Object type

When making decisions about naming objects for an application, you might need to use an external name for the full object names to the end user. Specifically, the end user might need to specify the object in an Include or Exclude statement when the application is run. The exact syntax of the object name in these statements is platform-dependent. On the Windows operating system, the drive letter associated with the file space rather than the file space name itself is used in the Include or Exclude statement.

The object ID value that was assigned when you created the object might not be the same as when you perform a restore process. Applications should save the object name and then query to obtain the current object ID before doing a restore.

### File space name

The file space name is one of the most important storage components. It can be the name of a file system, disk drive, or any other high-level qualifier that groups related data together.

IBM Storage Protect uses the file space to identify the file system or disk drive on which the data are located. In this way, actions can be performed on all entities within a file space, such as querying all objects within a specified file space. Because the file space is such an important component of the IBM Storage Protect naming convention, you use special calls to register, update, query, and delete file spaces.

The server also has administrative commands to query the file spaces on any node in IBM Storage Protect storage, and delete them if necessary. All data stored by the application client must have a file space name associated with it. Select the name carefully to group similar data together in the system.

To avoid possible interference, an application client should select different file space names from those that a backup-archive client would use. The application client should publish its file space names so that end users can identify the objects for include-exclude statements, if necessary.

**Note:** On Windows platforms, a drive letter is associated with a file space. When you register or update a file space, you must supply the drive letter. Because the include-exclude list refers to the drive letter, you must keep track of each letter and its associated file space. In the sample program `dapismpr`, the drive letter is set to "G" by default.

See [“Building and running the sample API application” on page 5](#) for more information on the sample programs.

## High-level and low-level names

Two other components of the object name are the high-level name qualifier and the low-level name qualifier. The high-level name qualifier is the directory path in which the object belongs, and the low-level name qualifier is the actual name of the object in that directory path.

When the file space name, high-level name, and low-level name are concatenated, they must form a syntactically correct name on the operating system on which the client runs. It is not necessary for the name to exist as an object on the system or resemble the actual data on the local file system. However, the name must meet the standard naming rules to be properly processed by the **dsmBindMC** calls. See [“Understanding backup and archive objects” on page 37](#) for naming considerations that are related to policy management.

## Object type

The object type identifies the object as either a file or a directory. A file is an object that contains both attributes and binary data, and a directory is an object that contains only attributes.

Table 7 on page 21 shows what the application client would code is for object names by platform.

| Table 7. Application object name examples by platform |                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform                                              | Client code for object name                                                                                                                                                                                                                                                                       |
| UNIX or Linux                                         | /myfs/highlev/lowlev                                                                                                                                                                                                                                                                              |
| Windows                                               | "myvol\\highlev\\lowlev"<br><br><b>Note:</b> On a Windows platform, a double backslash translates into a single backslash, because a backslash is the escape character. File space names start with a slash on the UNIX or Linux platform, but do not start with a slash on the Windows platform. |

## Accessing objects as session owner

Each object has an owner name associated with it. The rules determining what objects are accessed depend on what owner name is used when a session is started. Use this session owner value to control access to the object.

The session owner is set during the call to **dsmInitEx** in the *clientOwnerNameP* parameter. If you start a session with **dsmInitEx** owner name of *NULL* and you use *passwordaccess=prompt*, that session owner is handled with session (root or authorized user) authority. This is also true if you log in with a root user ID or an authorized user ID and you use *passwordaccess=generate*. During a session started in this manner, you can perform any action on any object that is owned by this node regardless of the actual owner of that object.

If a session is started with a specific owner name, the session can only perform actions on objects that have that object owner name associated with them. Backups or archives into the system all must have this owner name associated with them. Any queries performed return only the values that have this owner name associated with them. The object owner value is set during the **dsmSendObj** call in the **Owner** field of the **ObjAttr** structure. An owner name is case-sensitive. Table 8 on page 21 summarizes the conditions under which a user has access to an object.

Table 8. Summary of user access to objects

| Session owner             | Object owner       | User access |
|---------------------------|--------------------|-------------|
| NULL (root, system owner) | " " (empty string) | Yes         |
| NULL                      | Specific name      | Yes         |
| Specific name             | " " (empty string) | No          |
| Specific name             | Same name          | Yes         |

Table 8. Summary of user access to objects (continued)

| Session owner | Object owner   | User access |
|---------------|----------------|-------------|
| Specific name | Different name | No          |

## Accessing objects across nodes and owners

Three function calls support cross-node, cross-owner access on the same platform: **dsmSetAccess**, **dsmDeleteAccess**, and **dsmQueryAccess**. These functions, along with the *-fromnode* and *-fromowner* string options that are passed on **dsmInitEx**, permit a complete cross-node query, restore and retrieve process through the API.

For example, User A on node A uses the **dsmSetAccess** function call to give access to its backups under the /db file space to User B from Node B. The access rule is displayed as:

| ID | Type   | Node   | User   | Path    |
|----|--------|--------|--------|---------|
| 1  | Backup | Node B | User B | /db/*/* |

When User B logs on at Node B, the option string to **dsmInitEx** is:

```
-fromnode=nodeA -fromowner=userA
```

These options are set for this session. Any queries access the file spaces, and files of Node A. Backups and archives are not permitted. Only query, restore, and retrieve processes are permitted from the file spaces for which User B has access. If the application tries to execute any operation using a **dsmBeginTxn** (for examples, backup or update) while signed in with a *-fromnode* or *-fromowner* option set, then the **dsmBeginTxn** fails with the return code DSM\_RC\_ABORT\_NODE\_NOT\_AUTHORIZED. See the individual function calls and [“dsmInitEx” on page 102](#) for more information.

**Tip:** On UNIX and Linux you can specify *-fromowner=root* in the option string that is passed on the **dsmInitEx** function call. This permits non-root users access to files that the root owns if a set access was performed.

Use the *asnodename* option on the **dsmInitEx** option string with the appropriate function to back up, archive, restore, retrieve, query or delete data under the target node name on the IBM Storage Protect server. See [“Backing up multiple nodes with client node proxy support” on page 70](#) for information on enabling this option.

## Managing file spaces

Because file spaces are important to the operation of the system, a separate set of calls is used to register, update, and delete file space identifiers. Before you can store any objects that are associated with a file space on the system, you must first register the file space with IBM Storage Protect.

Use the **dsmRegisterFS** call to accomplish this task. For more information about object names and IDs, see [“Object names and IDs” on page 20](#).

The file space identifier is the top-level qualifier in a three-part name hierarchy. Grouping related data together within a file space makes management of that data much easier. For example, either the application client or the IBM Storage Protect server administrator can delete a file space and all the objects within that file space.

File spaces also permit the application client to provide information about the file space to the server that the administrator can then query. This information is returned on the query in the **qryRespFSData** structure and includes the following file system information:

| Type          | Definition                                                                              |
|---------------|-----------------------------------------------------------------------------------------|
| <b>fstype</b> | The file space type. This field is a character string that the application client sets. |



| Type                           | Definition                                                                                  |
|--------------------------------|---------------------------------------------------------------------------------------------|
| <b>fsAttr[platform].fsInfo</b> | A client information field that is used for client-specific data.                           |
| <b>capacity</b>                | The total amount of space in the file space.                                                |
| <b>occupancy</b>               | The amount of space that is currently occupied in the file space.                           |
| <b>backStartDate</b>           | The time stamp when the latest backup started (set by sending a <b>dsmUpdateFS</b> call).   |
| <b>backCompleteDate</b>        | The time stamp when the latest backup completed (set by sending a <b>dsmUpdateFS</b> call). |

Using capacity and occupancy depends on the application client. Some applications might not need information about the size of the file space, in which case these fields can default to 0. For more information about querying file spaces, see [“Querying the IBM Storage Protect system”](#) on page 28.

After a file space is registered with the system, you can back up or archive objects at any time. To update the occupancy and the capacity fields of the file space after a backup or archive operation, call **dsmUpdateFS**. This call ensures that the values for the occupancy and capacity of the file system are current. You can also update the **fsinfo**, **backupstart**, and **backupcomplete** fields.

If you want to monitor your last backup dates, enter a **dsmUpdateFS** call before you start the backup. Set the update action to DSM\_FSUPD\_BACKSTARTDATE. This forces the server to set the **backStartDate** field of the file space to the current time. After the backup is complete for that file space, enter a **dsmUpdateFS** call with the update action that is set to DSM\_FSUPD\_BACKCOMPLETEDATE. This call creates a time stamp on the end of the backup.

If a file space is no longer needed, you can delete it with the **dsmDeleteFS** command. On a UNIX or Linux operating system, only the root user or authorized users can delete file spaces.

The examples in [Figure 5 on page 23](#) demonstrate how to use the three file space calls for UNIX or Linux. For an example of how to use the three file space calls for Windows, see the sample program code that is installed on your system.

```
/* Register the file space if it has not already been done. */

dsInt16      rc;
regFSData    fsData;
char         fsName[DSM_MAX_FSNAME_LENGTH];
char         smpAPI[] = "Sample-API";

strcpy(fsName, "/home/tallan/text");
memset(&fsData, 0x00, sizeof(fsData));
fsData.stVersion = regFSDataVersion;
fsData.fsName = fsName;
fsData.fsType = smpAPI;
strcpy(fsData.fsAttr.unixFSAttr.fsInfo, "Sample API FS Info");
fsData.fsAttr.unixFSAttr.fsInfoLength =
    strlen(fsData.fsAttr.unixFSAttr.fsInfo) + 1;
fsData.occupancy.hi=0;
fsData.occupancy.lo=100;
fsData.capacity.hi=0;
fsData.capacity.lo=300;

rc = dsmRegisterFS(dsmHandle, fsData);
if (rc == DSM_RC_FS_ALREADY_REGED) rc = DSM_RC_OK; /* already done */
if (rc)
{
    printf("Filespace registration failed: ");
    rcApiOut(dsmHandle, rc);
    free(bkup_buff);
    return (RC_SESSION_FAILED);
}
```

*Figure 5. An example of working with file spaces, Part 1*

```

/* Update the file space. */
dsmFSUpd      updFilespace;          /* for update FS */

updFilespace.stVersion = dsmFSUpdVersion;
updFilespace.fsType = 0;              /* no change */
updFilespace.occupancy.hi = 0;
updFilespace.occupancy.lo = 50;
updFilespace.capacity.hi = 0;
updFilespace.capacity.lo = 200;
strcpy(updFilespace.fsAttr.unixFSAttr.fsInfo,
       "My update for filesystem");
updFilespace.fsAttr.unixFSAttr.fsInfoLength =
    strlen(updFilespace.fsAttr.unixFSAttr.fsInfo);

updAction = DSM_FSUPD_FSINFO |
            DSM_FSUPD_OCCUPANCY |
            DSM_FSUPD_CAPACITY;

rc = dsmUpdateFS (handle, fsName, &updFilespace, updAction);
printf("dsmUpdateFS rc=%d\n", rc);

```

Figure 6. An example of working with file spaces, Part 2

```

/* Delete the file space. */

printf("\nDeleting file space
rc = dsmDeleteFS (dsmHandle, fsName, DSM_REPOS_ALL);
if (rc)
{
    printf(" FAILED!!! ");
    rcApiOut(dsmHandle, rc);
}
else printf(" OK!\n");

```

Figure 7. An example of working with file spaces, Part 3

## Associating objects with management classes

A primary feature of IBM Storage Protect is the use of policies (management classes) to define how objects are stored and managed in IBM Storage Protect storage. An object is associated with a management class when the object is backed up or archived.

This management class determines:

- How many versions of the object are kept if backed up
- How long to keep archive copies
- Where to insert the object in the storage hierarchy on the server

Management classes consist of both backup copy groups and archive copy groups. A copy group is a set of attributes that define the management policies for an object that is being backed up or archived. If a backup operation is being performed, the attributes in the backup copy group apply. If an archive operation is being performed, the attributes in the archive copy group apply.

The backup or archive copy group in a particular management class can be empty or NULL. If an object is bound to the NULL backup copy group, that object cannot be backed up. If an object is bound to the NULL archive copy group, the object cannot be archived.

Because the use of a policy is a very important component of IBM Storage Protect, the API requires that all objects sent to the server are first assigned a management class by using the **dsmBindMC** call. With IBM Storage Protect software, you can use an include-exclude list to affect management class binding. The **dsmBindMC** call uses the current Include-Exclude list to perform management class binding.

Include statements can associate a specific management class with a backup or archive object. Exclude statements can prevent objects from being backed up but not from being archived.

The API requires that **dsmBindMC** is called before you back up or archive an object. The **dsmBindMC** call returns a mcBindKey structure that contains information on management class and copy groups that

are associated with the object. Check the copy group destination before proceeding with a send. When you send multiple objects in a single transaction, they must have the same copy group destination. The **dsmBindMC** function call returns the following information:

*Table 9. Information returned on the dsmBindMC call*

| Information              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Management Class         | The name of the management class that was bound to the object. The application client can send the <b>dsmBeginQuery</b> call to determine all attributes of this management class.                                                                                                                                                                                                                                                                                                                 |
| Backup Copy Group        | Informs you if a backup copy group exists for this management class. If a backup operation is being performed and a backup copy group does not exist, this object cannot be sent to storage. You receive an error code if you attempted to send it using the <b>dsmSendObj</b> call.                                                                                                                                                                                                               |
| Backup Copy Destination  | This field identifies the storage pool to which the data is sent. If you are performing a multiple object backup transaction, all copy destinations within that transaction must be the same. If an object has a different copy destination than previous objects in the transaction, end the current transaction and begin a new transaction before you can send the object. You receive an error code if you attempt to send objects to different copy destinations within the same transaction. |
| Archive Copy Group       | Informs you if an archive copy group exists for this management class. If an archive operation is being performed and an archive copy group does not exist, this object cannot be sent to storage. You receive an error code if you attempted to send it using the <b>dsmSendObj</b> call.                                                                                                                                                                                                         |
| Archive Copy Destination | This field identifies the storage pool to which the data are sent. If you are performing a multiple object archive transaction, all copy destinations within that transaction must be the same. If an object has a different copy destination than previous objects in the transaction, end the current transaction and begin a new transaction before you send the object. You receive an error code if you attempt to send objects to different copy destinations within the same transaction.   |

Backup copies of an object can be rebound to a different management class if a subsequent back up with the same object name is done that uses a management class different than the original. For example, if you back up ObjectA and bind it to Mgmtclass1, and later you back up ObjectA and bind it to Mgmtclass2, the most current backup rebinds any inactive copies to Mgmtclass2. The parameters defined in Mgmtclass2 would now control all copies. However the data does not move if the destination is different.

You can also rebind backup copies to a different management class using the **dsmUpdateObj** or **dsmUpdateObjEx** call with the DSM\_BACKUPD\_MC action.

#### **Related information**

[Deduplication option](#)

## **Query management classes**

Applications can query management classes to determine what management classes are possible for a given node and to determine what the attributes are within the management class.

You can only bind objects to management classes by using the **dsmBindMC** call. You might want your applications to query the management class attributes and display them to end users. See [“Querying the IBM Storage Protect system” on page 28](#) for more information.

In the example in [Figure 8 on page 26](#), a switch statement is used to distinguish between backup and archive operations when calling **dsmBindMC**. The information returned from this call is stored in the **MCBindKey** structure.

```

dsUInt16_t    send_type;
dsUInt32_t    dsmHandle;
dsmObjName    objName;      /* structure containing the object name */
mcBindKey     MCBindKey;    /* management class information */
char          *dest;        /* save destination value */

switch (send_type)
{
    case (Backup_Send) :
        rc = dsmBindMC(dsmHandle,&objName,stBackup,&MCBindKey);
        dest = MCBindKey.backup_copy_dest;
        break;
    case (Archive_Send) :
        rc = dsmBindMC(dsmHandle,&objName,stArchive,&MCBindKey);
        dest = MCBindKey.archive_copy_dest;
        break;
    default : ;
}

if (rc)
{
    printf("*** dsmBindMC failed: ");
    rcApiOut(dsmHandle, rc);
    rc = (RC_SESSION_FAILED);
    return;
}

```

Figure 8. An example of associating a management class with an object

## Expiration/deletion hold and release

You can hold deletion and expiration of specific archive objects in response to a pending or ongoing action that requires that particular data be held. In the event an action is initiated that might require access to data, that data must be available until the action is concluded and access to the data is no longer required as part of that process. After determining that the suspension is no longer required (released), normal deletion and expiration timing resumes per the original retention period.

### Before you begin

Verify that the server is licensed by issuing a test **dsmRetentionEvent** call:

1. Query for one object you want to hold and get the ID.
2. Issue the **dsmBeginTxn**, **dsmRetentionEvent** with Hold, and **dsmEndTxn**.
3. If the server is not licensed, you receive a vote of abort with reason code DSM\_RC\_ABORT\_LICENSE\_VIOLATION.

### Restrictions:

1. You cannot issue more than one **dsmRetentionEvent** call in a single transaction.
2. You cannot issue a hold on an object that is already under hold.

### Procedure

1. To hold objects, complete the following steps:
  - a) Query the server for all the objects that you want to place under hold. Get the object ID for each object.
  - b) Issue a **dsmBeginTxn** call, then issue a **dsmRetentionEvent** call with the list of objects, followed by a **dsmEventType**: eventHoldObj call. If the number of objects exceeds the value of maxObjPerTxn, use multiple transactions.
  - c) Use the qryRespArchiveData response on the **dsmGetNextQObj** function call to confirm that the objects are put under hold. Check the value of objHeld in qryRespArchiveData.
2. To release objects from hold, complete the following steps:
  - a) Query the server for all the objects that you want to release from hold. Get the object ID for each object.

- b) Issue a **dsmBeginTxn** call, then issue a **dsmRetentionEvent** call with the list of objects, followed by a **dsmEventType: eventReleaseObj** call. If the number of objects exceeds the value of **maxObjPerTxn**, use multiple transactions.
- c) Use the **qryRespArchiveData** response on the **dsmGetNextQObj** function call to confirm if the objects were released from hold. Check the value of **objHeld** in **qryRespArchiveData**.

## Archive data retention protection

Data that is under the control of IBM Storage Protect cannot be modified by unauthorized agents, such as an individual or a program. This protection extends to preventing the deletion of data, such as archive objects, by any agent before the expiration of the retention period.

### About this task

Protecting archive retention helps to ensure that no individual or program can maliciously or accidentally delete data that is under the control of IBM Storage Protect. An archive object that is sent to an archive retention protection server is protected from accidental deletes and has an enforced retention period. Archive retention protection has the following restrictions:

- Only archive operations are allowed on a retention protection server.
- Any object that is not bound explicitly to a management class through a value in the **dsmBindMc** function call or through include-exclude statements is bound to the explicit name of the default management class. For example, if the default management class in the node policy is MC1, the object is bound explicitly to MC1 rather than to DEFAULT. On a query response, the object displays as bound to MC1.
- After you enable archive data retention protection, any attempt to delete an object before the retention period expires returns the code **DSM\_RC\_ABORT\_DELETE\_NOT\_ALLOWED** on the end transaction.

See the IBM Storage Protect server documentation for instructions for setting retention protection for an archive object.

### Procedure

To set up archive data retention protection, complete the following steps:

1. On a new server installation with no previous data, run the **SET ARCHIVERETENTIONPROTECTION ON** command.
2. In the API option string on the **dsmInit** or **dsmInitEx** function calls, enter the following instruction:

```
-ENABLEARCHIVERETENTIONPROTECTION=yes
```

You can also set the **enablearchiveretentionprotection** option in your **dsm.opt** file on systems other than UNIX, or in your **dsm.sys** file on UNIX systems:

```
SERVNAME srvr1.ret
TCPPOPT      1500
TCPSEVERADDRESS node.domain.company.com
COMMMETHOD TCPIP
ENABLEARCHIVERETENTIONPROTECTION YES
```

For more information about this option, see [“The enablearchiveretentionprotection option”](#) on page 27.

3. Issue a query to the server to confirm that the IBM Storage Protect server is enabled for archive retention protection. Check the value of the **archiveRetentionProtection** field in the **dsmQuerySessInfo** structure.

### *The enablearchiveretentionprotection option*

The **enablearchiveretentionprotection** option specifies whether to enable data retention protection for archive objects on the IBM Storage Protect server that is dedicated for this purpose. Your server administrator must activate data retention protection on a new server that does not already have

stored objects (backup, archive, or space-managed). If the API application attempts to store a backup version or space-managed object on the server, an error message is issued.

The note in [“Considerations for designing an application”](#) on page 9 states: “Do not store objectID values to use for future restores. They are not guaranteed to be persistent during the life of the object.” can be relaxed for Archive manager applications since the archive-manager server does not support export or import. Archive-manager applications can save and use the objectID to improve the performance during object restore.

If the server issues the **SET ARCHIVERETENTIONPROTECTION ON** command, you cannot delete an archived object from the server by using the **delete filesystem** command, until the policy parameters of the archive copy group are satisfied. See the appropriate server documentation for information about how to set up a management class.

### ***Event-based retention policy***

In an event-based retention policy, the retention time of an archive object is initiated by a business event, such as closing a bank account. Event-based retention closely aligns the IBM Storage Protect data retention policy with business requirements for data. When the event occurs, the application sends an **eventRetentionActivate** event for that object to the server to initiate the retention.

### **Procedure**

To use an event-based retention policy, complete the following steps:

1. On the server, create a management class with an archive **copygroup** of type EVENT. For more information, see the IBM Storage Protect server documentation.
2. Query the management class to confirm that the class is event-based. If the management class is event-based, the **retainInit** field in the **archDetailCG** structure is ARCH\_RETINIT\_EVENT.
3. Bind the objects to the event-based management class by using include, **archmc**, or explicitly through the **mcNameP** attribute in the **ObjAttr** structure on the **dsmSendObj** function call.
4. At the point that you want to start the retention for the object, query the server for all of the objects that are affected. Check to see whether they are in a PENDING state, and get the object ID. In a pending state, the **retentionInitiated** field in the **qryRespArchiveData** structure indicates DSM\_ARCH\_RETINIT\_PENDING.
5. Issue a **dsmBeginTxn** call, then issue a **dsmRetentionEvent** call with the list of objects, followed by a **dsmEventType: eventRetentionActivate** call. If the number of objects exceeds the value of **maxObjPerTxn**, use multiple transactions.

**Restriction:** You can issue only one **dsmRetentionEvent** call per transaction.

6. Query the objects to confirm that the retention is activated. If retention is initiated, the **retentionInitiated** field in the **qryRespArchiveData** structure has a value of I.

## **Querying the IBM Storage Protect system**

The API has several queries, such as management class query, that applications can use.

### **Procedure**

All queries that use the **dsmBeginQuery** call follow these steps:

1. Send the **dsmBeginQuery** call with the appropriate query type:
  - Backup
  - Archive
  - Active backed-up objects
  - File space
  - Management class

The **dsmBeginQuery** call informs the API of the data format that is returned from the server. The appropriate fields can be placed in the data structures that are passed by the **dsmGetNextQObj** calls. The begin query call also permits the application client to set the scope of the query by properly specifying the parameters on the begin query call.

**Restriction:** On UNIX or Linux systems, only the root user can query active backed-up objects. This query type is known as "fast path".

2. Enter the **dsmGetNextQObj** call to obtain each record from the query. This call passes a buffer that is large enough to hold the data that is returned from the query. Each query type has a corresponding data structure for the data returned. For example, a backup query type has an associated **qryRespBackupData** structure that is populated when the **dsmGetNextQObj** call is sent.
3. The **dsmGetNextQObj** call usually returns one of the following codes:
  - DSM\_RC\_MORE\_DATA: Send the **dsmGetNextQObj** call again.
  - DSM\_RC\_FINISHED: There is no more data. Send the **dsmEndQuery** call.
4. Send the **dsmEndQuery** call. When all query data are retrieved or more query data are not needed, enter the **dsmEndQuery** call to end the query process. The API flushes any remaining data from the query stream and releases any resources that were used for the query.

## Results

Figure 9 on page 29 displays the state diagram for query operations.

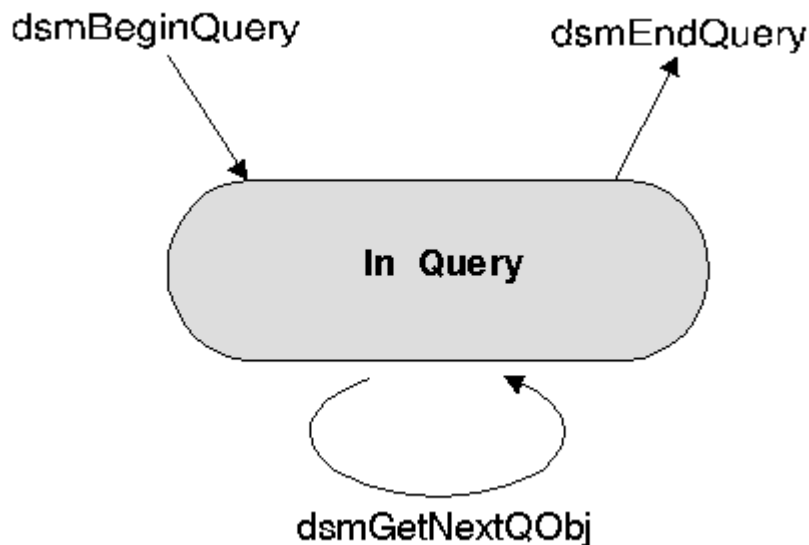


Figure 9. State diagram for general queries

Figure 10 on page 30 displays the flowchart for query operations.

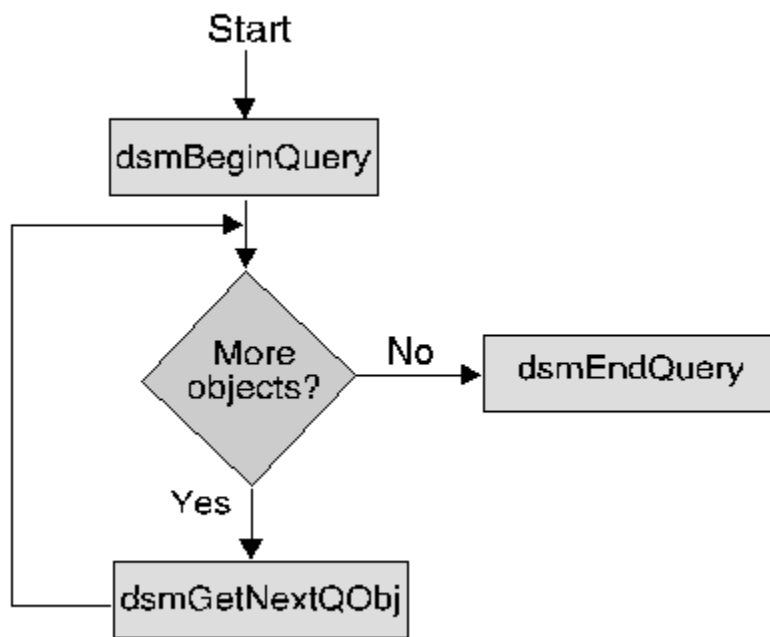


Figure 10. Flowchart for general queries



## Example of querying the system

In this example a management class query prints out the values of all the fields in the backup and archive copy groups for a particular management class.

```
dsInt16          rc;
qryMCData        qMCData;
DataBlk          qData;
qryRespMCDetailData qRespMCData, *mcResp;
char             *mc, *s;
dsBool_t         done = bFalse;
dsUInt32_t       qry_item;

/* Fill in the qMCData structure with the query criteria we want */
qMCData.stVersion = qryMCDataVersion; /* structure version */
qMCData.mcName    = mc;                /* management class name */
qMCData.mcDetail  = bTrue;             /* want full details? */

/* Set parameters of the data block used to get or send data */
qData.stVersion = DataBlkVersion;
qData.bufferLen = sizeof(qryRespMCDetailData);
qData.bufferPtr = (char *)&qRespMCData;

qRespMCData.stVersion = qryRespMCDetailDataVersion;

if ((rc = dsmBeginQuery(dsmHandle, qtMC, (dsmQueryBuff *)&qMCData)))
{
    printf("*** dsmBeginQuery failed: ");
    rcApiOut(dsmHandle, rc);
    rc = (RC_SESSION_FAILED);
}
else
{
    done = bFalse;
    qry_item = 0;
    while (!done)
    {
        rc = dsmGetNextQObj(dsmHandle, &qData);
        if ((rc == DSM_RC_MORE_DATA) || (rc == DSM_RC_FINISHED))
            && qData.numBytes)
        {
            qry_item++;
            mcResp = (qryRespMCDetailData *)qData.bufferPtr;
            printf("Mgmt. Class\n");
            printf("    Name:\n");
            printf("    Backup CG Name:\n");
            . /* other fields of backup and archive copy groups */
            printf("    Copy Destination:\n");
        }
        else
        {
            done = bTrue;
            if (rc != DSM_RC_FINISHED)
            {
                printf("*** dsmGetNextQObj failed: ");
                rcApiOut(dsmHandle, rc);
            }
        }
        if (rc == DSM_RC_FINISHED) done = bTrue;
    }
    rc = dsmendQuery (dsmHandle);
}
```

Figure 11. An example of performing a system query

## Server efficiency

Use these guidelines when you retrieve from, or send objects to, the IBM Storage Protect server.

- When you retrieve objects from the IBM Storage Protect server, follow these guidelines:

- Retrieve data in the restore order that is provided by the IBM Storage Protect server. The restore order is especially important for tape devices, because retrieving data that is not ordered can result in tape rewinds and mounts.
- Even when data is stored on a disk device, you can save time when the retrieves are ordered.
- Perform as much work as possible in a single IBM Storage Protect server session.
- Do not start and stop multiple sessions.
- When you send objects to the IBM Storage Protect server, follow these guidelines:
  - Send multiple objects in a single transaction.
  - Avoid sending one object per transaction, especially when the data is sent directly to a tape device. Part of the tape device transaction is to ensure that the data in the RAM buffers of the tape is written to media.

### Related concepts

[“Selecting and sorting objects by restore order” on page 59](#)

After the backup or archive query is performed, the application client must determine which objects, if any, are to be restored or retrieved.

### Related information

[“Starting or ending a session” on page 15](#)

IBM Storage Protect is a session-based product, and all activities must be performed within an IBM Storage Protect session. To start a session, the application starts the **dsmInitEx** call. This call must be performed before any other API call other than **dsmQueryApiVersionEx**, **dsmQueryCliOptions**, or **dsmSetUp**.

## Sending data to a server

The API permits application clients to send data or named objects and their associated data to IBM Storage Protect server storage.

**Tip:** You can either back up or archive data. Perform all send operations within a transaction.

### The transaction model

All data sent to IBM Storage Protect storage during a backup or archive operation is done within a transaction. A transaction model provides a high level of data integrity, but it does impose some restrictions that an application client must take into consideration.

Start a transaction by a call to **dsmBeginTxn** or end a transaction by a call to **dsmEndTxn**. A single transaction is an atomic action. Data sent within the boundaries of a transaction is either committed to the system at the end of the transaction or rolled back if the transaction ends prematurely.

Transactions can consist of either single object sends or multiple object sends. To improve system performance by decreasing system overhead, send smaller objects in a multiple object transaction. The application client determines whether single or multiple transactions are appropriate.

Send all objects within a multiple object transaction to the same copy destination. If you need to send an object to a different destination than the previous object, end the current transaction and start a new one. Within the new transaction, you can send the object to the new copy destination.

**Note:** Objects that do not contain any bit data ( *sizeEstimate=0* ) are not checked for copy destination consistency.

IBM Storage Protect limits the number of objects that can be sent in a multiple object transaction. To find this limit, call **dsmQuerySessInfo** and examine the **maxObjPerTxn** field. This field displays the value of the *TXNGroupmax* option that is set on your server.

The application client must keep track of the objects sent within a transaction to perform retry processing or error processing if the transaction ends prematurely. Either the server or the client can stop a transaction at any time. The application client must be prepared to handle sudden transaction ends that it did not start.

## File aggregation

IBM Storage Protect servers use a function that is called file aggregation. With file aggregation, all objects sent in a single transaction are stored together, which saves space and improves performance. You can still query and restore the objects separately.

To use this function, all of the objects in a transaction should have the same file space name. If the file space name changes within a transaction, the server closes the existing aggregated object and begins a new one.

## LAN-free data transfer

The API can take advantage of LAN-free data transfer if the **dsmSetUp** option for multithreading is ON. The API returns the existence of a LAN-free destination in the **Query Mgmt Class** response structure **archDetailCG** or **backupDetailCG** field **bLanFreeDest**.

You can use LAN-free operations on platforms that are supported by the storage agent. Macintosh platform is excluded.

LAN-free information is provided in the following output structures. The out structure (**dsmEndGetDataExOut\_t**) for **dsmEndGetData** includes the field, **totalLFBytesRecv**. This is the total number of LAN-free bytes that are received. The out structure (**dsmEndSendObjExOut\_t**) for **dsmEndSendObjEx** includes the field, **totalLFBytesSent**. This is the total number of LAN-free bytes that were sent.

## Simultaneous-write operations

You can configure IBM Storage Protect server storage pools to write simultaneously to a primary storage pool and copy storage pool or pools during a backup or archive. Use this configuration to create multiple copies of the object.

If a simultaneous-write operation fails, the return code on the **dsmEndTxn** function might be **DSM\_RC\_ABORT\_STGPOOL\_COPY\_CONT\_NO**, which indicates that the write to one of the copy storage pools failed, and the IBM Storage Protect storage pool option **COPYCONTINUE** was set to **NO**. The application terminates and the problem must be resolved by the IBM Storage Protect server administrator.

For more information about setting up simultaneous-write operations, see the IBM Storage Protect server documentation.

## Enhancing API performance

You can use the **tcpbuffsize** and **tcpnodelay** client options and the **DataBlk** API parameter to enhance API performance.

Table 10 on page 33 describes the actions that you can take to enhance the API performance.

| Table 10. Backup-archive options and the API parameter that enhance performance |                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backup-archive client options                                                   | Description                                                                                                                                                                                             |
| tcpbuffsize                                                                     | Specifies the size of the TCP buffer. The default value is 31 KB. To enhance performance, set the value to 32 KB.                                                                                       |
| tcpnodelay                                                                      | Specifies whether to send small buffers to the server rather than holding them. To enhance performance, set this option to <i>yes</i> for all platforms. This option is valid for Windows and AIX only. |
| API parameter                                                                   | Description                                                                                                                                                                                             |

Table 10. Backup-archive options and the API parameter that enhance performance (continued)

| Backup-archive client options | Description                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DataBlk</b>                | This parameter is used with the <b>dsmSendData</b> function call to determine the application buffer size. For best results, set the parameter as a multiple of the <i>tcpbuffsize</i> value that is specified with the <i>tcpbuffsize</i> minus 4 bytes. For example, set a value of 28 for this parameter when the value of <i>tcpbuffsize</i> is set to 32 KB. |

Each **dsmSendData** call is synchronous and does not return until the data transferred to the API in the **dataBlkPtr** is flushed to the network. The API adds a 4-byte overhead to each transaction buffer that is placed on the network.

For example, when the transaction buffer size is 32 KB and the application **DataBlk** buffer size is 31 KB, then each application **DataBlk** buffer fits in a communications buffer and can be flushed immediately. However, if the application **DataBlk** buffer is exactly 32 KB, and because the API is adding 4 bytes per transaction buffer, two flushes are required; one of 32 KB and one of 4 bytes. Also, if you set the *tcpnodelay* option to no, flushing the 4 bytes might take up to 200 milliseconds.

## Set up the API to send performance data to the client performance monitor

The client performance monitor is a component of the Tivoli Storage Manager Administration Center that is used to display performance data that is collected by the API. The client performance monitor records and displays performance data for client backup, archive, and restore operations.

With performance monitoring enabled, you can display performance data that is collected by the API by using the performance monitor; the performance monitor is available in the Tivoli Storage Manager Administration Center. Starting with version 7.1, the Administration Center component is no longer included in Tivoli Storage Manager or IBM Storage Protect distributions. If you have an Administration Center that was installed with a previous server release, you can continue to use it to display performance data. If you do not already have an Administration Center installed, you can download the previously-released version from <ftp://public.dhe.ibm.com/storage/tivoli-storage-management/maintenance/admincenter/v6r3/>. For information about using the performance monitor, see the Tivoli Storage Manager version 6.3 server documentation.

## Configuring client performance monitor options

You enable IBM Storage Protect clients to use the performance monitor by specifying parameters in the client options file. You specify these options for each client that you want to monitor.

### Before you begin

When you monitor performance on UNIX and Linux computers, set the open file descriptor limit to at least 1024, by using the following command:

```
ulimit -n 1024
```

### Procedure

To configure the client performance monitor options, complete the following steps:

1. Open the client options file for each client that you are monitoring. Depending on your configuration, the client options are in one of the following files:

- *dsm.opt*
- *dsm.sys*

2. Add the following options to the client options file:

```
PERFMONTCPSERVERADDRESS
PERFMONTCPPORT
```

**PERFMONTCPSERVERADDRESS**

The PERFMONTCPSERVERADDRESS option specifies the host name or IP address of the system where the client performance monitor is installed.

**Supported clients**

This option is platform independent and is supported for all clients.

**Options file**

Set this option in the client options file (dsm.opt or dsm.sys).

**Syntax**

➤ PERFMONTCPServeraddress *server* ➤

**Parameters*****server***

The server host name or IP address of the system that has the client performance monitor installed (this is the same server that runs the Administration Center).

**Examples****Options file:**

PERFMONTCPSERVERADDRESS 131.222.10.5

**Command line:**

This option cannot be set using the command line.

**PERFMONTCPPORT**

The port number that the client performance monitor listens on for performance data from the clients.

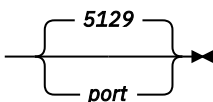
**Supported clients**

This option is platform independent and is supported for all clients.

**Options file**

Set this option in the client options file (dsm.opt or dsm.sys).

**Syntax**

➤ PERFMONTCPPort  ➤

**Parameters*****port***

The port that is monitored for client performance data. Port 5129 is the default port.

**Examples****Options file:**

PERFMONTCPPORT 5000

**Command line:**

This option cannot be set using the command line.

## PERFMONCOMMTIMEOUT

Specifies the maximum time, in seconds, that the `dsmTerminate` call waits for performance data to arrive after a session is ended.

### Supported clients

This option is platform independent and is supported for all clients.

### Options file

Set this option in the client options file (`dsm.opt` or `dsm.sys`).

### Syntax



### Parameters

#### *seconds*

The time to wait for remaining performance data to arrive, before ending the session.

### Examples

#### Options file:

```
PERFMONCOMMTIMEOUT 60
```

#### Command line:

This option cannot be set using the command line.

## Sending objects to the server

Application clients can send data or named objects and their associated data to IBM Storage Protect storage by using the API backup and archive functions. The backup and archive components of the system permit use of different management procedures for data that is sent to storage.

The size estimate attribute is an estimate of the total size of the data object to send to the server. If the application does not know the exact object size, set the *sizeEstimate* to a higher estimate. If the estimate is smaller than the actual size, the IBM Storage Protect server uses extra resources to manage extra space allocations.

#### Tips:

- Be as accurate as is possible when you make this size estimate. The server uses this attribute for efficient space allocation and object placement within its storage resources.
- If the estimate is smaller than the actual size, a server with caching does not allocate extra space and stops the send.

You might encounter problems if the *sizeEstimate* is much too large. The server might not have enough space for the estimated size but does have space for the actual size; or the server might use slower devices.

You can back up or archive objects that are larger than two gigabytes in size. The objects can be either compressed or uncompressed.

To start a send operation, call **`dsmSendObj`**. If you have more data than you can send at one time, you can make repeated calls to **`dsmSendData`** to transfer the remainder of the information. Call **`dsmEndSendObj`** to complete the send operation.

## Understanding backup and archive objects

The backup component of the IBM Storage Protect system supports several versions of named objects that are stored on the server.

Any object backed up to the server that has the same name as an object that is already stored on the server from that client is subject to version control. Objects are considered to be in active or inactive states on the server. The latest copy of an object on the server that has not been deactivated is in the active state. Any other object with the same name, whether it is an older version or a deactivated copy, is considered inactive. Management class constructs define different management criteria. They are assigned to active and inactive objects on the server.

Table 11 on page 37 lists the copy group fields that apply to active and inactive states:

| Table 11. Backup copy group fields |                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------|
| Field                              | Description                                                                            |
| VEREXISTS                          | The number of inactive versions if active versions exist.                              |
| VERDELETED                         | The number of inactive versions if active versions do not exist.                       |
| RETEXTRA                           | The number of days to keep inactive versions.                                          |
| REONLY                             | The number of days to keep the last inactive versions if active versions do not exist. |

If backup versions each have a unique name, such as using a time stamp in the name, then versioning does not happen automatically: every object is active. Active objects never expire, so an application would be responsible for deactivating these with the **dsmDeleteObj** call. In this situation, the application would need the deactivated objects to expire as soon as possible. The user would define a backup copy group with VERDELETED=0 and RETONLY=0.

The archive component of the IBM Storage Protect system permits objects to be stored on the server with retention or expiration period controls instead of version control. Each object stored is unique, even though its name might be the same as an object already archived. Archive objects have a description field associated with the metadata that can be used during query to identify a specific object.

Every object on the IBM Storage Protect server is assigned a unique object ID. The persistence of the original value is not guaranteed during the life of an object (specifically, after an export or import). Therefore, an application should not query and save the original object ID for use on later restores. Rather, an application should save the object name and insert date. You can use this information during a restore to query objects and verify the insert date. Then, the current object ID can be used to restore the object.

## Compression

Configuration options on a given node and the **dsmSendObj** objCompressed option, determine whether IBM Storage Protect compresses the object during a send. Also, objects with a sizeEstimate less than DSM\_MIN\_COMPRESS\_SIZE are never compressed.

If the object is compressed already (objCompressed=bTrue), it is not compressed again. If it is not compressed, IBM Storage Protect decides whether to compress the object, based on the values of the compression option that is set by the administrator and that is set in the API configuration sources.

The administrator can change compression thresholds on the server by using the register node command (compression=yes, no, or client-determined). If this is client-determined, then the compression behavior is determined by the compression option value in the configuration sources.

Some types of data, such as data that is already compressed, might actually get bigger when processed with the compression algorithm. When this happens, the return code DSM\_RC\_COMPRESS\_GREW is generated. If you realize that this might happen, but you want the send operation to continue anyway, tell the end users to specify the following option in their options file:

If, during a **dsmSendData** function, with compression enabled, you get DSM\_RC\_COMPRESS\_GREW return code, you might want to start over and send the object again without compression. To enforce this, set the **dsmSendObj** `ObjAttr.objCompressed` to `bTrue`.

Information about the actual compression behavior during a **dsmSendObj** is returned by the **dsmEndSendObjEx** call. `objCompressed` specifies if compression was done. `totalBytesSent` is the number of bytes sent by the application. `totalCompressedSize` is the number of bytes after compression. The **dsmEndSendObjEx** call also has a `totalLFBytesSent` field that contains the total bytes sent over LAN-free.



**Attention:** If your application plans to use partial object restore or retrieve, you cannot compress the data while sending it. To enforce this, set the **dsmSendObj** `ObjAttr.objCompressed` to `bTrue`.

### Compression type

The type of compression that the client uses is determined by the combination of compression and client-side data deduplication that is used during backup or archive processing.

The compression algorithm that is used by the client is reported by the API in a new field in the **qryRespArchiveData** and **qryRespBackupData** structures:

```
dsChar_t      compressAlg[20]; /* compression algorithm name */
```

The following types of compression are reported:

#### LZ4

A faster and more efficient compression method that the client uses in any of the following situations:

- The client-side deduplicated object is sent to a container storage pool on the IBM Storage Protect server. The server must be at version 7.1.5 or later.
- The object does not undergo client-side data deduplication.
- The object undergoes only traditional server-side data deduplication.

#### LZW

A traditional type of compression that the client uses is when client-deduplicated objects are sent to traditional (non-container) storage pools on the server.

#### Blank field

The object is not compressed by the client. The object is not compressed because the compression option is set to *no*, or the option is not specified during backup or archive processing. Although the object is not compressed by the client, it might be compressed by the server.

The compression type is not configurable. It is determined by the backup-archive client at the time of backup or archive processing.

### Example

The following example shows the `Compresssion Type` field in the output of the backup and archive queries from the 64-bit sample application **dapismp**:

```
Enter selection ==>1
                        Filespace:\fs1
                        Highlevel:\hl
                        Lowlevel:\ll
                        Object Type(D/F/A):f
Active(A),Inactive(I),Both(B):a
If root, query all owners? (Y/N):
                        Object Owner Name:
                        point in time date (MMDDYYYY):
                        point in time time (hhmm):
                        Show detailed output? (Y/N):y
On Restore, Wait for mount?(Y/N):
Are the above responses correct (y/n/q)?
```



```

Item 1: \fs1\hl\ll
  Object type: File
  Object state: Active
  Insert date: 2016/2/3 10:57:41
  Expiration date: 0/0/0 0:0:0
  Owner:
  Restore order: 0-0-0-0-0
  Object id: 0-40967
  Copy group: 1
  Media class: Fixed
  Mgmt class: DEFAULT
  Object info is          :IBM Storage Protect API Verify Data
  Object info length is :73
  Estimated size : 0 4000
  Compression : YES
  Compression Type: LZ4
  Encryption : NO
  Encryption Strength : NONE
  Client Deduplicated : YES

```

## Buffer copy elimination

The buffer copy elimination function removes the copy of data buffers between an application and the IBM Storage Protect server, which results in better processor utilization. For maximum effect, use this approach in a LAN-free environment.

The buffers for data movement are allocated by IBM Storage Protect and a pointer is passed back to the application. The application places the data in the provided buffer, and that buffer is passed through the communication layers to the storage agent by using shared memory. The data is then moved to the tape device, which eliminates copies of data. This function can be used with either backup or archive operations.



**Attention:** When you use this method, pay extra attention to proper buffer handling and sizes of buffers. The buffers are shared between the components and any memory overwrite that is a result of a programming error results in severe errors.

The overall sequence of calls for backup/archive is as follows:

```

dsmInitEx (UseTsmBuffers = True, numTsmBuffers = [how many IBM Storage Protect
          -allocated buffers the application needs to allocate])
dsmBeginTxn
for each object in the txn
  dsmBindMC
  dsmSendObject
  dsmRequestBuffer
  dsmSendBufferData (sends and release the buffer used)
  dsmEndSendObjEx
dsmEndTxn
for each buffer still held
  dsmReleaseBuffer
dsmTerminate

```

The **dsmRequestBuffer** function can be called multiple times, up to the value that is specified by the **numTsmBuffers** option. An application can have two threads: a producer thread that fills buffers with data; and a consumer thread that sends those buffers to IBM Storage Protect with the **dsmSendBufferData** call. When a **dsmRequestBuffer** call is issued and the **numTsmBuffers** is reached, the **dsmRequestBuffer** call blocks until a buffer is released. The buffer release can happen by either calling **dsmSendBufferData**, which sends and releases a buffer or by calling **dsmReleaseBuffer**. For more information, see `callbuff.c` in the API sample directory.

If at any point there is a failure in the send, the application must release all the buffers that are held and terminate the session. For example:

```

If failure
  for each data buffer held by application
    call dsmReleaseBuffer
dsmTerminate

```

If an application calls **dsmTerminate** and a buffer is still held, the API does not exit. The following code is returned: **DSM\_RC\_CANNOT\_EXIT\_MUST\_RELEASE\_BUFFER**. If the application cannot release the buffer, the application must exit the process to force a cleanup.

### ***Buffer copy elimination and restore and retrieve***

The IBM Storage Protect server controls the amount of data to be placed in the buffer, based on tape access optimization with restore and retrieve. This method is not as beneficial to the application as the normal method of getting data. During prototyping, check the performance of the buffer copy elimination method and use this method only if you see a worthwhile improvement.

The maximum amount of data in a single buffer that is returned by the IBM Storage Protect server is (256K bytes – header overhead). As a consequence, only applications that deal with small buffer writes benefit from this data retrieval mechanism. The application must give special attention to the number of bytes in the buffer, depending on the object size, the network, and other boundary conditions. In some situations, the use of buffer copy elimination can actually perform worse than the normal restore. The API normally caches the data and returns a fixed length to the application. The application can then control the number of data writes back to the disk.

If you use buffer copy elimination, create a data-caching mechanism for buffers that are less than the preferred write buffer size. For example, if an application writes 64K data blocks to disk, the application must take these actions:

1. Call **dsmGetBufferData**.
2. Write out blocks of 64K.
3. On the final block, copy the remainder to a **tempBuff**, issue another **dsmGetBufferData** call, and fill the **tempBuff** with the rest of the data.
4. Continue writing blocks of 64K:

```
dsmGetBufferData #1 get 226K      dsmGetBufferData #2 get 240K
Block1 64K - write to disk        Block1 30K - copy to tempbuff-write to disk
Block2 64K - write to disk        Block2 64K - write to disk
Block3 64K - write to disk        Block3 64K - write to disk
Block4 34K - copy to tempbuff     Block4 64K - write to disk
Block5 18K - write to tempbuff    etc
```

In this example, six disk writes are direct and 1 is cached.

The overall sequence of calls for restore and retrieve is as follows:

**dsmInitEx** (UseTsmBuffers = True numTsmBuffers = how many buffers the application wants to allocate).

```
dsmBeginGetData
While obj id
  dsmGetObj (no data restored on this call- buffer set to NULL)
  While data to read
    dsmGetBufferData (returns the data in the data buffer)
    ...process data...
  dsmReleaseBuffer
dsmEndGetObj
dsmEndGetData
```

For every **dsmGetBufferData** call, implement a **dsmReleaseBuffer** call. The **dsmGetBufferData** and corresponding **dsmReleaseBuffer** do not need to be consecutive. An application might issue multiple **dsmGetBufferData** calls first to get several buffers, and then issue the corresponding **dsmReleaseBuffer** calls later. For sample code that uses this function, see `callbuff.c` in the API sample directory.

**Restriction:** Because the API provides the buffer and the goal is to minimize processor utilization, more processing of the data in the buffer is not permitted. The application cannot use encryption and compression with buffer copy elimination because both of these operations require data processing and copies.

Implement both the regular data movement path and the buffer copy elimination to enable the user to switch between both paths, based on their needs. If the user must compress or encrypt data, then use the existing mechanism. If there is a processor constraint, then use the new mechanism. Both of these mechanisms are complementary and do not completely replace each other.

## API encryption

Two methods are available to encrypt data: application-managed encryption and IBM Storage Protect client encryption.

Select and use only one of these methods to encrypt data. The methods are mutually exclusive and if you encrypt data by using both methods, you will be unable to restore or retrieve some data. For example, assume that an application uses application-managed encryption to encrypt object A, and then uses IBM Storage Protect client encryption to encrypt object B. During a restore operation, if the application sets the option to use IBM Storage Protect client encryption and it tries to restore both objects, only object B can be restored; object A cannot be restored because it was encrypted by the application, not by the client.

Regardless of the encryption method that is used, the IBM Storage Protect must enable password authentication. By default, the server uses SET AUTHENTICATION ON.

The API uses either AES 128-bit or AES 256-bit encryption. AES 256-bit data encryption provides a higher level of data encryption than AES 128-bit data encryption. Files that are backed up by using AES 256-bit encryption cannot be restored with an earlier client. Encryption can be enabled with or without compression. If you use encryption, you cannot use the partial object restore and retrieve and buffer copy elimination functions.

### Application-managed encryption

With application-managed encryption, the application provides the key password to the API (using key DSM\_ENCRYPT\_USER) and it is the application's responsibility to manage the key password.



**Attention:** If the encryption key is not saved, and you forgot the key, your data is unrecoverable.

The application provides the key password in the **dsmInitEx** call and must provide the proper key password at restore time.



**Attention:** If the key password is lost, there is no way to restore the data.

The same key password must be used for backup and restore (or archive and retrieve) operations for the same object. This method does not have a dependency on the IBM Storage Protect server level. To set up this method, the application needs to follow these steps:

1. Set the `bEncryptKeyEnabled` variable to `bTrue` in the call to **dsmInitEx**, and set the `encryptionPasswordP` variable to point to a string with the encrypt key password.
2. Set the `include.encrypt` for the objects to encrypt. For example, to encrypt all data, set:

```
include.encrypt /.../* (UNIX)
```

and

```
include.encrypt *\...\* (Windows)
```

To encrypt the object /FS1/DB2/FULL, set:

```
include.encrypt /FS1/DB2/FULL
```

3. Set `ENCRYPTKEY=PROMPT|SAVE` in the option string that is passed to the API in the **dsmInitEx** call on Windows. This option can also be set in `dsm.opt` (Windows) or `dsm.sys` (UNIX or Linux).

By default, the `encryptkey` option is set to `prompt`. This setting ensures that the key does not get stored automatically. If `encryptkey save` is specified, the key is stored by IBM Storage Protect on the local

machine but then only one key can be valid for all IBM Storage Protect operations with the same node name.

After a send of an object, the **dsmEndSendObjEx** specifies whether an object was encrypted and which method was used. Possible values in the *encryptionType* field:

- DSM\_ENCRYPT\_NO
- DSM\_ENCRYPT\_USER
- DSM\_ENCRYPT\_CLIENTENCRKEY

The following table lists the API encryption types, prerequisites, and the functions that are available.

| Table 12. API encryption types, prerequisites, and functions available |              |                                                                                                                                                  |
|------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Type                                                                   | Prerequisite | Function available                                                                                                                               |
| ENCRYPTIONTYPE                                                         | None         | Set the ENCRYPTIONTYPE in the option string that is passed to the API in the <b>dsmInitEx</b> call on Windows. ENCRYPTIONTYPE=AES128 by default. |
| EncryptKey=save                                                        | None         | API and backup-archive                                                                                                                           |
| EncryptKey=prompt                                                      | None         | API and backup-archive                                                                                                                           |
| EncryptKey=generate                                                    | None         | API and backup-archive                                                                                                                           |
| EnableClientEncryptKey                                                 | None         | API only                                                                                                                                         |

**Note:** It is advised that the server has authentication turned ON. If authentication is turned OFF, the key is not encrypted, but the data is still encrypted. However, this is not recommended.

Table 13 on page 42 shows how both Authorized Users and non-Authorized Users can encrypt or decrypt data during a backup or restore operation, depending on the value that is specified for the passwordaccess option. To perform the following authorized-user and non-authorized-user operations, the TSM.KDB, TSM.sth, and TSM.IDX files must exist. The authorized user sets the encryptkey option to **save** and the passwordaccess option to **generate**.

| Table 13. Encrypting or decrypting data with application managed key on UNIX or Linux |                       |                   |                                                                        |
|---------------------------------------------------------------------------------------|-----------------------|-------------------|------------------------------------------------------------------------|
| Operation                                                                             | passwordaccess option | encryptkey option | Result                                                                 |
| Authorized user backup                                                                | generate              | save              | Data encrypted.                                                        |
|                                                                                       | generate              | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |
|                                                                                       | prompt                | save              | Data encrypted if encryptionPasswordP contains an encryption password. |
|                                                                                       | prompt                | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |
| Authorized user restore                                                               | generate              | save              | Data encrypted.                                                        |
|                                                                                       | generate              | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |
|                                                                                       | prompt                | save              | Data encrypted if encryptionPasswordP contains an encryption password. |
|                                                                                       | prompt                | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |
| Non-authorized user backup                                                            | generate              | save              | Data encrypted.                                                        |
|                                                                                       | generate              | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |
|                                                                                       | prompt                | save              | Data encrypted if encryptionPasswordP contains an encryption password. |
|                                                                                       | prompt                | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |

Table 13. Encrypting or decrypting data with application managed key on UNIX or Linux (continued)

| Operation                   | passwordaccess option | encryptkey option | Result                                                                 |
|-----------------------------|-----------------------|-------------------|------------------------------------------------------------------------|
| Non-authorized user restore | generate              | save              | Data encrypted.                                                        |
|                             | generate              | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |
|                             | prompt                | save              | data encrypted if encryptionPasswordP contains an encryption password. |
|                             | prompt                | prompt            | Data encrypted if encryptionPasswordP contains an encryption password. |

### IBM Storage Protect client encryption

IBM Storage Protect client encryption uses the key that is managed by the DSM\_ENCRYPT\_CLIENTENCRKEY value to protect your data. Client encryption is transparent to the application that is using the API, with the exception that partial object restore operations and retrieve operations are not possible for objects that were encrypted or compressed.

For both IBM Storage Protect client encryption and application-managed encryption, the encryption password refers to a string value that is used to generate the actual encryption key. The value for the encryption password option is 1-63 characters in length, but the key that is generated from it is always 8 bytes for 56 DES, 16 bytes for 128 AES, and 32 bytes for 256 AES.



**Attention:** If the encryption key is not available, data cannot be restored or retrieved. When you use ENABLECLIENTENCRYPTKEY for encryption, the encryption key is stored on the server database. For objects that use this method, the server database must exist and have the proper values for the objects for a proper restore. Ensure that you back up the server database frequently to prevent data loss.

This is the simpler method to implement, where one random encryption key is generated per session and it is stored on the IBM Storage Protect server with the object in the server database. During restore, the stored key is used for decryption. Using this method, the management of the key is the responsibility of IBM Storage Protect, and the application does not have to deal with the key at all. Because the key is stored in the server database, you must have a valid IBM Storage Protect database for a restore operation of an encrypted object. When the key is transmitted between the API and the server, it is also encrypted. The transmission of the key is secure, and when the key is stored in the IBM Storage Protect server database it is encrypted. The only time that the key is placed in the clear with the export data stream is when a node's data are exported between servers.

To enable IBM Storage Protect client encryption, complete the following steps:

1. Specify -ENABLECLIENTENCRYPTKEY=YES in the option string that is passed to the API on the dsmInitEx call or set the option in the system option file dsm.opt (Windows) or dsm.sys (UNIX or Linux).
2. Set the include.encrypt for the objects to encrypt. For example, to encrypt all data, set:

```
include.encrypt /.../* (UNIX)
```

and

```
include.encrypt *\...\* (Windows)
```

To encrypt the object /FS1/DB2/FULL, set:

```
include.encrypt /FS1/DB2/FULL
```

# Data deduplication

Data deduplication is a method of reducing storage needs by eliminating redundant data.

## Overview

Two types of data deduplication are available on IBM Storage Protect: *client-side data deduplication* and *server-side data deduplication*.

*Client-side data deduplication* is a data deduplication technique that is used on the backup-archive client to remove redundant data during backup and archive processing before the data is transferred to the IBM Storage Protect server. Using client-side data deduplication can reduce the amount of data that is sent over a local area network.

*Server-side data deduplication* is a data deduplication technique that is done by the server. The IBM Storage Protect administrator can specify the data deduplication location (client or server) to use with the **DEDUP** parameter on the **REGISTER NODE** or **UPDATE NODE** server command.

## Enhancements

With client-side data deduplication, you can:

- Exclude specific files on a client from data deduplication.
- Enable a data deduplication cache that reduces network traffic between the client and the server. The cache contains extents that were sent to the server in previous incremental backup operations. Instead of querying the server for the existence of an extent, the client queries its cache.

Specify a size and location for a client cache. If an inconsistency between the server and the local cache is detected, the local cache is removed and repopulated.

**Note:** For applications that use the IBM Storage Protect API, the data deduplication cache must not be used because of the potential for backup failures caused by the cache being out of sync with the IBM Storage Protect server. If multiple, concurrent backup-archive client sessions are configured, there must be a separate cache configured for each session.

- Enable both client-side data deduplication and compression to reduce the amount of data that is stored by the server. Each extent is compressed before it is sent to the server. The tradeoff is between storage savings and the processing power that is required to compress client data. In general, if you compress and deduplicate data on the client system, you are using approximately twice as much processing power as data deduplication alone.

The server can work with deduplicated, compressed data. In addition, backup-archive clients earlier than V6.2 can restore deduplicated, compressed data.

Client-side data deduplication uses the following process:

- The client creates extents. *Extents* are parts of files that are compared with other file extents to identify duplicates.
- The client and server work together to identify duplicate extents. The client sends non-duplicate extents to the server.
- Subsequent client data-deduplication operations create new extents. Some or all of those extents might match the extents that were created in previous data-deduplication operations and sent to the server. Matching extents are not sent to the server again.

## Benefits

Client-side data deduplication provides several advantages:

- It can reduce the amount of data that is sent over the local area network (LAN).
- The processing power that is required to identify duplicate data is offloaded from the server to client nodes. Server-side data deduplication is always enabled for deduplication-enabled storage pools.

However, files that are in the deduplication-enabled storage pools and that were deduplicated by the client, do not require additional processing.

- The processing power that is required to remove duplicate data on the server is eliminated, allowing space savings on the server to occur immediately.

Client-side data deduplication has a possible disadvantage. The server does not have whole copies of client files *until* you back up the primary storage pools that contain client extents to a non-deduplicated copy storage pool. (*Extents* are parts of a file that are created during the data-deduplication process.) During storage pool backup to a non-deduplicated storage pool, client extents are reassembled into contiguous files.

By default, primary sequential-access storage pools that are set up for data deduplication must be backed up to non-deduplicated copy storage pools before they can be reclaimed and before duplicate data can be removed. The default ensures that the server always has copies of whole files, in either a primary storage pool or a copy storage pool.

**Important:** For further data reduction, you can enable client-side data deduplication and compression together. Each extent is compressed before it is sent to the server. Compression saves space, but it increases the processing time on the client workstation.

The following options pertain to data deduplication:

- Deduplication
- Dedupcachepath
- Dedupcachesize
- Enablededupcache
- Exclude.dedup
- Include.dedup

## API client-side data deduplication

*Client-side data deduplication* is used by the API on the backup-archive client, to remove redundant data during backup and archive processing before the data is transferred to the IBM Storage Protect server.

Client-side data deduplication is used by the API, to remove redundant data during backup and archive processing before the data is transferred to the IBM Storage Protect server. Using client-side data deduplication can reduce the amount of data that is sent over a local area network. Using client-side data deduplication can also reduce the IBM Storage Protect server storage space.

When the client is enabled for client-side data deduplication, and you perform a backup or archive operation, the data is sent to the server as extents. The next time a backup or archive operation is performed, the client and server identify which data extents have already been backed up or archived, and send only the unique extents of data to the server.

For client-side data deduplication, the server and API must be at version 6.2 or later.

Before you use client-side data deduplication to back up or archive your files, the system must meet the following requirements:

- The client must have the deduplication option enabled.
- The server must enable the client for client-side data deduplication with the **DEDUP=CLIENTORSERVER** parameter on either the **REGISTER NODE** or **UPDATE NODE** command.
- The storage pool destination for the data must be a data deduplication-enabled storage pool. The data deduplication-enabled storage pool is file device type only.
- Ensure that the files are bound to the correct management class.
- A file can be excluded from client-side data deduplication processing. By default, all files are included.
- Files must be larger than 2 KB.

- The server can limit the maximum transaction size for data deduplication by setting the CLIENTDEDUPTXNLIMIT option on the server. See the server documentation information about this option.

If any of these requirements are not met, data is processed normally, with no client-side data deduplication.

Here are some data deduplication restrictions:

- LAN-free data movement and client-side data deduplication are mutually exclusive. If you enable both LAN-free data movement and client-side data deduplication, LAN-free data movement operations complete and client-side data deduplication is ignored.
- Encryption and client-side data deduplication are mutually exclusive. If you enable both encryption and client-side data deduplication, encryption operations complete and client-side data deduplication is ignored. Encrypted files, and files that are eligible for client-side data deduplication, can be processed in the same operation, but are done in separate transactions.

#### **Requirements:**

1. In any transaction, all files must be either included for data deduplication or excluded. If the transaction has mixed files, the transaction fails, and a return code of DSM\_RC\_NEEDTO\_ENDTXN is returned by the API.
  2. Use storage device encryption together with client-side data deduplication. Because SSL is used in combination with client-side deduplication, there is no need for client encryption.
- The following functions are not available for client-side data deduplication:
    - IBM Storage Protect for Space Management (HSM) client
    - API shared buffer
    - NAS
    - Subfile backup
  - Buffer copy elimination cannot be used with data transformations like compression, encryption, and data deduplication.
  - If you use client-side deduplication, the API detects and fails (with RC=254) backups of file extents that are marked as expired on the server during sending data to the server. If you want to retry the operation, you need to include that programming in the calling application.
  - Simultaneous-write operations on the server takes precedence over client-side data deduplication. If simultaneous-write operations are enabled, client-side data deduplication does not occur.

**Restriction:** When client side data deduplication is enabled, the API cannot recover from a state where the server has run out of storage on the destination pool, even if there is a next pool defined. A stop reason code of DSM\_RS\_ABORT\_DESTINATION\_POOL\_CHANGED is returned and the operation fails. There are two ways to recover from this situation:

1. Ask the administrator to add more scratch volumes to the original filepool.
2. Retry the operation with data deduplication disabled.

For even greater bandwidth savings, you can enable a local cache for data deduplication. The local cache saves queries from going to the IBM Storage Protect server. The default value for ENABLEDEDUPCACHE is NO, so that the cache is not out of sync with the server. If the cache is out of sync with the server, the application resends all data. If your application can retry on a failed transaction, and you want to use the local cache, set the ENABLEDEDUPCACHE option to YES in the dsm.opt (Windows) or dsm.sys (UNIX) file.

At the end of a restore, if *all* of the data was restored through the API, and the object was deduplicated by the client, an end-to-end digest is calculated and compared to the value calculated at backup time. If those values do not match, error DSM\_RC\_DIGEST\_VALIDATION\_ERROR is returned. If an application receives this error, the data is corrupt. This error can also be a result of a transient error on the network, so try the restore or retrieve again.



Here is an example of the query session command showing data deduplication information:

```
dsmQuerySessInfo Values:
Server Information:
Server name: SERVER1
Server Host: AVI
Server port: 1500
Server date: 2009/10/6 20:48:51
Server type: Windows
Server version: 6.2.0.0
Server Archive Retention Protection : NO
Client Information:
Client node type: API Test1
Client filespace delimiter: :
Client hl & ll delimiter: \
Client compression: Client determined (3u)
Client archive delete: Client can delete archived objects
Client backup delete: Client CANNOT delete backup objects
Maximum objects in multiple object transactions: 4096
Lan free Enabled: NO
Deduplication : Client Or Server
General session info:
Node: AVI
Owner:
API Config file:
```

Here is an example of the query management class command showing data deduplication information:

```
Policy Information:
Domain name: DEDUP
Policyset name: DEDUP
Policy activation date: 0/0/0 0:0:0
Default management class: DEDUP
Backup retention grace period: 30 days
Archive retention grace period: 365 days
Mgmt. Class 1:
Name: DEDUP
Description: dedup - values like standard
Backup CG Name: STANDARD
Frequency: 0
Ver. Data Exists: 2
Ver. Data Deleted: 1
Retain Extra Ver: 30
Retain Only Ver: 60
Copy Destination: AVIFILEPOOL
Lan free Destination: NO
Deduplicate Data: YES

Archive CG Name: STANDARD
Frequency: 10000
Retain versions: 365
Copy Destination: AVIFILEPOOL
Lan free Destination: NO
Retain Init : CREATE
Retain Minimum : 65534
Deduplicate Data: YES
```

## Related information

[Deduplication option](#)

### ***Exclude and include files for data deduplication***

You can choose to exclude or include backup or archive files for data deduplication.

### **Exclude files from data deduplication**

To exclude files from data deduplication processing, follow these steps:

1. Set the `exclude.dedup` option for the objects to exclude.

For example, to exclude all dedup data for UNIX systems, set:

```
exclude.dedup /.../*
```

2. To exclude all dedup data for Windows systems, set:

```
exclude.dedup *\...\*
```

**Important:** If an object is sent to a data deduplication pool, data deduplication occurs on the server, even if the object is excluded from client-side data deduplication.

## Include files for data deduplication

You can choose to include backup or archive files for data deduplication.

To refine the list of files to be included, the `include.dedup` option can be used in combination with the `exclude.dedup` option.

By default, all eligible objects are included for data deduplication.

Here are some UNIX and Linux examples:

```
exclude.dedup /FS1/...\*  
include.dedup /FS1/archive/*
```

Here are some Windows examples:

```
exclude.dedup E:\myfiles\...\*  
include.dedup E:\myfiles\archive\*
```

## Server-side data deduplication

Server-side data deduplication is data deduplication that is performed by the server.

The IBM Storage Protect administrator can specify the data deduplication location (client or server) to use with the **DEDUP** parameter on the **REGISTER NODE** or **UPDATE NODE** server command.

In a data deduplication-enabled storage pool (file pool), only one instance of a data extent is retained. Other instances of the same data extent are replaced with a pointer to the retained instance.

For more information about server-side data deduplication, see the IBM Storage Protect server documentation.

## Application failover

When the IBM Storage Protect server becomes unavailable because of an outage, applications that use the API can automatically fail over to a failover server for data recovery.

The IBM Storage Protect server that the client and API connects to during normal production processes is called the *primary server*. When the primary server is set up for node replication, that server is also known as the *source replication server*. The client node data on the source replication server can be replicated to the *target replication server*. This server is also known as the *failover server*, and is the server that the client automatically fails over to when the primary server fails.

The client and API must be configured for automated client failover, and must connect to a version 7.1 (or later) server that replicates client node data. The configuration for the API is the same as the configuration for the backup-archive client.

During normal operations, connection information for the failover server is automatically sent to the client from the primary server during the logon process. The failover server information is automatically saved to the client options file.

Each time the client application logs on to the IBM Storage Protect server, it attempts to contact the primary server. If the primary server is unavailable, the application automatically fails over to the failover server by using the failover server information in the client options file. In failover mode, the application can query the failover server and restore or retrieve replicated data.

You must back up the application at least one time to the primary server. The API can fail over to the failover server to recover data only if the data from the client node was replicated from the primary server to the failover server.

## Related information

[Automated client failover configuration and use](#)

## Failover status information

The API provides status information that applications can use to determine the failover status and the status of replicated client data on the secondary server also known as the failover server.

The replication status indicates whether the most recent backup was replicated to the failover server. If the time stamp of the most recent backup operation on the API matches the time stamp of the backup on the failover server, the replication status is current. If the two time stamps do not match, the replication status is not current and the replicated data might be out-of-date.

The following replication status information is returned on the **query filespace** response on the **dsmGetNextQObj** function call in the **qryRespFSData** structure:

| Table 14. Replication status information reported by the API |                                 |                                                                       |
|--------------------------------------------------------------|---------------------------------|-----------------------------------------------------------------------|
| Status information                                           | Type                            | Definition                                                            |
| Start of last replication                                    | <b>lastReplStartDate</b>        | The last time replication was started.                                |
| End of last replication                                      | <b>lastReplCmpltDate</b>        | The last time replication was completed, even if there was a failure. |
| Last backup store date (Server)                              | <b>lastBackOpDateFromServer</b> | The last store time stamp that was saved on the server.               |
| Last backup store date (Local)                               | <b>lastBackOpDateFromLocal</b>  | The last store time stamp that was saved on the client.               |

The failover status is reported by the **bIsFailOverMode** field in the **dsmInitExOut\_t** structure.

See “API type definitions source files” on page 140 for the structure and type definitions of the API.

The DSM\_RC\_SIGNON\_FAILOVER\_MODE return code indicates that the client and API failed over to the failover server, and is running in failover mode.

## Example of signon during a failover

The following sample output is an example of signing on the server during a failover:

```
signon
Doing signon for node khoyt, owner , with password khoytpass
ANS2106I Connection to primary IBM Storage Protect server 123.45.6.78 failed
ANS2107I Attempting to connect to failover server TARGET at 123.45.6.79 : 1501
ANS2108I Connected to failover server TARGET.
Handle on return = 1

*****
After dsmInitEx:
Server TARGET ver/rel/lev 7/1/0/0
userNameAuthorities : Owner
Replication Server name : TARGET
Home Server name : MINE
Connected to replication server
*****
```

## Example of query session command

The following sample output is an example of the **query session** command that shows the failover (secondary or replication) server information:

```
query session
dsmQuerySessInfo Values:
  Server Information:
    Server name      : TARGET
    Server Host      : 123.45.6.79
    Server port      : 1500
    Server date      : 2013/5/21  14:13:32
    Server type      : Windows
    Server version: 7.1.0.0
    Server Archive Retention Protection : NO
  Replication Server Information
    Home Server name : MINE
    Replication Server name : TARGET
      Host          : 123.45.6.79
      Port          : 1501
    Fail over status : Connected to replication server
  Client Information:
    Client node type : Unix
    Client filespace delimiter: /
    Client hl & ll delimiter : /
    Client compression : Client determined (3u)
    Client archive delete : Client can delete archived objects
    Client backup delete : Client CANNOT delete backup objects
    Maximum objects in multiple object transactions: 4096
    Lan free Enabled : NO
    Deduplication : Server Only
  General session info:
    Node : KHOYT
    Access Node :
    Owner :
    API Config file:
  Policy Information:
    Domain name : STANDARD
    Policyset name : STANDARD
    Policy activation date : 0/0/0 0:0:0
    Default management class : STANDARD
    Backup retention grace period : 30 days
    Archive retention grace period: 365 days
```

## Example of query filespace command

The following sample output is an example of the **query filespace** command that shows the replication status of a file space on the failover server:

```
filespace query
Filespace pattern to query:*
Are the above responses correct (y/n/q)?

Filespace Name  Type          Occupancy  Capacity  Start          End
-----
/fs             API:Sample          100        300      0/0/0 0:0:0    0/0/0 0:0:0

  Start of last Replication : 2013/5/21 21:3:2
  End of last Replication   : 2013/5/21 21:3:3
                             Server
  Last backup store date    : 2013/5/21 21:18:25      Local 2013/5/21 21:18:25
  Last archive store date   : 0/0/0 0:0:0            0/0/0 0:0:0
  Last HSM store date       : 0/0/0 0:0:0            0/0/0 0:0:0
  FSINFO : Sample API FS Info
```

## Related reference

[“dsmGetNextQObj” on page 94](#)

The **dsmGetNextQObj** function call gets the next query response from a previous **dsmBeginQuery** call and places the response in the caller buffer.

## Example flow diagrams for backup and archive

The API is designed for straightforward logic flows and clear transitions between the various states of the application client. This clean state transition catches logic flaws and program errors early in the development cycle, greatly enhancing the quality and reliability of the system.

For example, you cannot make a **dsmSendObj** call unless a transaction was started and a **dsmBindMC** call was previously made for the object that you are backing up.

Figure 12 on page 51 displays the state diagram for performing backup or archive operations within a transaction. The arrow pointing from "In Send Object" to **dsmEndTxn** indicates that a **dsmEndTxn** call can be started after a call to **dsmSendObj** or **dsmSendData**. You might want to do this if an error condition occurred during the send of an object and you want to stop the entire operation. In this case, you must use a vote of DSM\_VOTE\_ABORT. In normal circumstances, however, call **dsmEndSendObj** before you end the transaction.

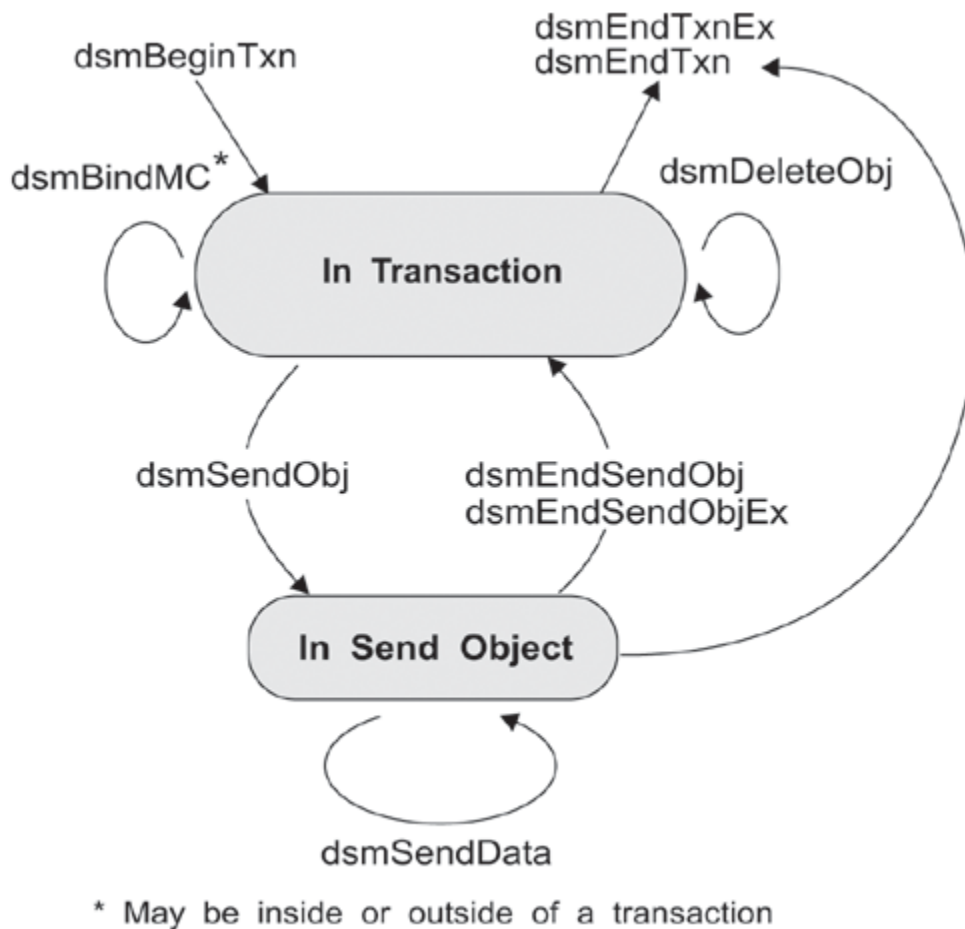


Figure 12. State diagram for backup and archive operations

Figure 13 on page 52 displays the flowchart for performing backup or archive operations within a transaction.

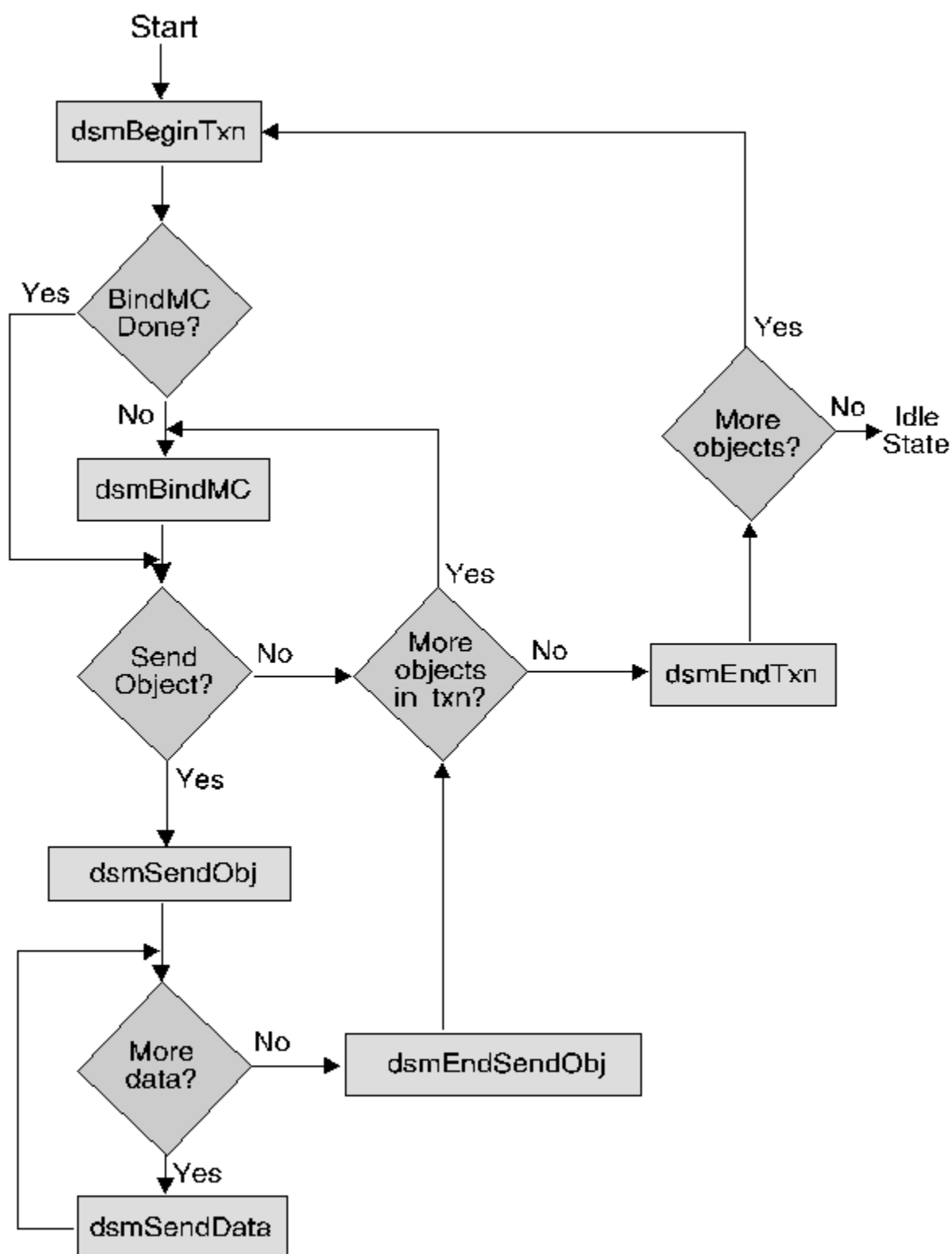


Figure 13. Flowchart for backup and archive operations

The primary feature in these two diagrams is the loop between the following API calls from within a transaction:

- **dsmBindMC**
- **dsmSendObj**

- **dsmSendData**
- **dsmEndSendObj**

The **dsmBindMC** call is unique in that you can start it from inside or outside of a transaction boundary. You can also start it from a different transaction, if required. The only requirement for the **dsmBindMC** call is that it is made prior to backing up or archiving an object. If the object that you are backing up or archiving is not associated with a management class, an error code is returned from **dsmSendObj**. In this situation, the transaction is ended by calling **dsmEndTxn** (this error condition is not shown in the flowchart).

The flowchart illustrates how an application would use multiple object transactions. It shows where decision points can be placed to determine if the object that is sent fits within the transaction or whether to start a new transaction.

## Code example of API functions that send data to IBM Storage Protect storage

This example demonstrates the use of the API functions that send data to IBM Storage Protect storage. The **dsmSendObj** call appears inside a switch statement, so that different parameters can be called depending on whether a backup or archive operation is being performed.

The **dsmSendData** call is called from inside a loop that repeatedly sends data until a flag is set that permits the program execution to exit the loop. The entire send operation is performed from within the transaction.

The third parameter on the **dsmSendObj** call is a buffer that contains the archive description. Because backup objects do not have a description, this parameter is NULL when backing up an object.

[Figure 8 on page 26](#) displays an example that shows the use of the **dsmBindMC** function call.

```

if ((rc = dsmBeginTxn(dsmHandle)) )      /* API session handle */
{
    printf("*** dsmBeginTxn failed: ");
    rcApiOut(dsmHandle, rc);
    return;
}

/* Call dsmBindMC if not done previously */
objAttr.sizeEstimate.hi = 0;      /* estimate of */
objAttr.sizeEstimate.lo = 32000; /* object size */
switch (send_type)
{
    case (Backup_Send) :
        rc = dsmSendObj(dsmHandle,stBackup,
            NULL,&objName,&objAttr,NULL);
        break;
    case (Archive_Send) :
        archData.stVersion = sndArchiveDataVersion;
        archData.descr = desc;
        rc = dsmSendObj(dsmHandle,stArchive,
            &archData,&objName,&objAttr,NULL);
        break;
    default : ;
}
if (rc)
{
    printf("*** dsmSendObj failed: ");
    rcApiOut(dsmHandle, rc);
    return;
}
done = bFalse;
while (!done)
{
    dataBlk.stVersion = DataBlkVersion;
    dataBlk.bufferLen = send_amt;
    dataBlk.numBytes = 0;
    dataBlk.bufferPtr = bkup_buff;
    rc = dsmSendData(dsmHandle,&dataBlk);
    if (rc)
    {
        printf("*** dsmSendData failed: ");
        rcApiOut(dsmHandle, rc);
        done = bTrue;
    }
    /* Adjust the dataBlk buffer for the next piece to send */
}
rc = dsmEndSendObj(dsmHandle);
if (rc)
{
    printf("*** dsmEndSendObj failed: ");
    rcApiOut(dsmHandle, rc);
}
txn_reason = 0;
rc = dsmEndTxn(dsmHandle,      /* API session handle */
               DSM_VOTE_COMMIT, /* Commit transaction */
               &txn_reason);   /* Reason if txn aborted */
if (rc || txn_reason)
{
    printf("*** dsmEndTxn failed: rc = ");
    rcApiOut(dsmHandle, rc);
    printf("    reason =
}

```

Figure 14. An example of sending data to a server

## File grouping

The IBM Storage Protect API has a logical file grouping protocol that relates several individual objects together. You can reference and manage these groups as a logical group on the server. A logical group requires that all group members and the group leader belong to the same node and file space on the server.

Each logical group has a group leader. If the group leader is deleted, the group is deleted. You cannot delete a member that is part of a group. Expiration of all members in a group is dependent on the group leader. For example, if a member is marked for expiration, the member does not expire unless the group



leader expires. However, if a member is not marked for expiration, and the group leader is expired, then all members are expired.

File groups contain backup data only, and cannot contain archive data. Archive objects can use the **Archive Description** field to facilitate a type of grouping if required by an application.

The **dsmGroupHandler** call groups the operations. The **dsmGroupHandler** function must be called from within a transaction. Most group error conditions are caught on either the **dsmEndTxnl** or **dsmEndTxnEx** calls.

The out structure in **dsmEndTxnEx** includes a new field, **groupLeaderObjId**. This field contains the object ID of the group leader if a group was opened in that transaction. You can create a group across more than one transaction. A group is not committed, or saved, on the server until a close is performed. The **dsmGroupHandler** is an interface that can accept five different operations. They include:

- DSM\_GROUP\_ACTION\_OPEN
- DSM\_GROUP\_ACTION\_CLOSE
- DSM\_GROUP\_ACTION\_ADD
- DSM\_GROUP\_ACTION\_ASSIGNTO
- DSM\_GROUP\_ACTION\_REMOVE

Table 15 on page 55 lists the **dsmGroupHandler** function call actions:

| Table 15. dsmGroupHandler functions |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Action                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| OPEN                                | The OPEN action creates a group. The next object that is sent becomes the group leader. The group leader cannot have content. All objects after the first object become members that are added to the group. To create a group, open a group and pass in a unique string to identify the group. This unique identifier allows several groups with the same name to be opened. After the group is opened, the next object that is sent is the group leader. All other objects that are sent are group members.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| CLOSE                               | <p>The CLOSE action commits and saves an open group. To close the group, pass in the object name and the unique string that is used in the open operation. The application must check for open groups and, if necessary, close or delete the groups. A group is not committed or saved until the group is closed. A CLOSE action fails in the following conditions:</p> <ul style="list-style-type: none"><li>• The group that you are trying to close has the same name as an existing open group.</li><li>• A management class incompatibility exists between the current closed group and the new group to be closed of the same name. In this case, complete the following steps:<ol style="list-style-type: none"><li>1. Query the previous closed group.</li><li>2. If the management class of the existing closed group is different from the management class associated with the current open group, issue a <b>dsmUpdateObject</b> with type DSM_BACKUPD_MC. This command updates the existing group to the new management class.</li><li>3. Issue the CLOSE action.</li></ol></li></ul> |
| ADD                                 | The ADD action appends an object to a group. All objects that are sent after the ADD action are assigned to the group.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

| Table 15. <i>dsmGroupHandler</i> functions (continued) |                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Action                                                 | Description                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| ASSIGNTO                                               | The ASSIGNTO action permits the client to assign objects that exist on the server to the declared peer group. This transaction sets up the PEER group relationship. The ASSIGNTO action is similar to the ADD action, with the following exceptions: <ul style="list-style-type: none"> <li>• The ADD action applies to objects within an in-flight transaction.</li> <li>• The ASSIGNTO action applies to an object that is on the server.</li> </ul> |
| REMOVE                                                 | The REMOVE action removes a member, or a list of members, from a group. A group leader cannot be removed from a group. A group member must be removed before the member can be deleted.                                                                                                                                                                                                                                                                |

Use the following query types for group support:

- **qtBackupGroups**
- **qtOpenGroups**

The **qtBackupGroups** queries groups that are closed while **qtOpenGroups** queries groups that are open. The query buffer for the new types has fields for **groupLeaderObjId** and **objType**. The query performs differently depending on the values for these two fields. The following table includes some query possibilities:

Table 16. *Examples of queries*

| <b>groupLeaderObjId.<br/>hi</b> | <b>groupLeaderObjId.l<br/>o</b> | <b>objType</b> | <b>Result</b>                                                                                                                                                                                       |
|---------------------------------|---------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0                               | 0                               | NULL           | Returns a list of all group leaders                                                                                                                                                                 |
| grpLdrObjId.hi                  | grpLdrObjId.lo                  | 0              | Returns a list for all group members that are assigned to the specified group leader ( <b>grpLdrObjId</b> ).                                                                                        |
| grpLdrObjId.hi                  | grpLdrObjId.lo                  | objType        | Returns a list by using <b>BackQryRespEnhanced3</b> , for each group member that is assigned to the specified group leader ( <b>grpLdrObjId</b> ), and matching the object type ( <b>objType</b> ). |

The response structure (**qryRespBackupData**) from **dsmGetNextQObj** includes two fields for group support:

- **isGroupLeader**
- **isOpenGroup**

These fields are Boolean flags. The following example displays the creation of the group, adding members to the group, and closing the group to commit the group on the IBM Storage Protect server.

```

dsmBeginTxn
    dsmGroupHandler (PEER, OPEN, leader, uniqueId)
    dsmBeginSendObj
        dsmEndSendObj
    dsmEndTxnEx (With objId of leader)
Loop for multiple txns
{
    dsmBeginTxn
        dsmGroupHandler (PEER, ADD, member, groupLeaderObjID)
        Loop for multiple objects
        {
            dsmBeginSendObj
                Loop for data
                {
                    dsmSendData
                }
            dsmEndSendObj
        }
    dsmEndTxn
}
dmBeginTxn
    dsmGroupHandler(CLOSE)
dsmEndTxn

```

Figure 15. Example of pseudo-code that is used to create a group

For a code example, see the sample group program `dsmgrp.c` that is included in the `API samprc` directory.

## Receiving data from a server

Application clients can receive data or named objects and their associated data from IBM Storage Protect storage by using the restore and retrieve functions. The restore function accesses objects that previously were backed up, and the retrieve function accesses objects that previously were archived.

**Restriction:** The API can only restore or retrieve objects that were backed up or archived using API calls.

Both restore and retrieve functions start with a query operation. The query returns different information depending on whether the data was originally backed up or archived. For instance, a query on backup objects returns information on whether an object is active or inactive, while a query on archive objects returns information such as object descriptions. Both queries return object IDs that are used to uniquely identify the object on the server.

## Partial object restore or retrieve

The application client can receive only a portion of the object. This is called a partial object restore or a partial object retrieve.



**Attention:** Partial restore or retrieve of compressed or encrypted objects produces unpredictable results.

**Note:** If you code your application to use a partial object restore or retrieve, you cannot compress the data while sending it. To enforce this, set `ObjAttr.objCompressed` to `bTrue`.

To perform a partial object restore or retrieve, associate the following two data fields with each object **GetList** entry:

### offset

The byte offset into the object from which to begin returning data.

### length

The number of object bytes to return.

Use `DSM_MAX_PARTIAL_GET_OBJ` to determine the maximum number of objects that can perform a partial object restore or retrieve for a specific **dsmBeginGetData** list.

The following data fields, used on the **dsmBeginGetData** call, determine what portion of the object is restored or retrieved:

- If both the offset and length are zero, the entire object is restored or retrieved from IBM Storage Protect storage.
- If the offset is greater than zero, but the length is zero, the object is restored or retrieved from the offset to the end.
- If the length is greater than zero, only the portion of the object from the offset for the specified length is restored or retrieved.

## Restoring or retrieving data

After a query is made and a session is established with the IBM Storage Protect server, you can run a procedure to restore or retrieve data.

### Procedure

To restore or retrieve data, complete the following steps:

1. Query the IBM Storage Protect server for either backup or archive data.
2. Determine the objects to restore or retrieve from the server.
3. Sort the objects on the **Restore Order** field.
4. Send the **dsmBeginGetData** call with the list of objects that you want to access.
5. Send the **dsmGetObject** call to obtain each object from the system. Multiple **dsmGetData** calls might be needed for each object to obtain all associated object data. Send the **dsmEndGetObject** call after all data for an object is obtained.
6. Send the **dsmEndGetData** call after all data for all objects is received, or to end the receive operation.

### Querying the server

Before you begin any restore or retrieve operation, first query the IBM Storage Protect server to determine what objects you can receive from storage.

To send the query, the application must enter the parameter lists and structures for the **dsmBeginQuery** call. The structure must include the file space that the query examines and pattern-match entries for the high-level and low-level name fields. If the session was initialized with a NULL owner name, you do not need to specify the owner field. However, if the session was initialized with an explicit owner name, only objects that are associated with that owner name are returned.

The point-in-time **BackupQuery** query provides a snapshot of the system at a specific time. By specifying a valid date, you can query all files that are backed up to that time. Even if an object has an active backup from a later date, point-in-time overrides an object state so that the previous inactive copy is returned. For more information, see the following example: [pitDate](#).

A query returns all information that is stored with the object, in addition to the information in the following table.

*Table 17. Query to the server return information*

| Field           | Description                                                                                                                                                                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| copyId          | The copyIdHi and copyIdLo values provide an 8-byte number that uniquely identifies this object for this node in IBM Storage Protect storage. Use this ID to request a specific object from storage for restore or retrieve processing.                                 |
| restoreOrderExt | The restoreOrderExt value provides a mechanism for receiving objects from IBM Storage Protect storage in the most efficient manner possible. Sort the objects to restore on this value to ensure that tapes are mounted only one time and are read from front to back. |

You must keep some or all of the query information for later processing. Keep the `copyId` and `restoreOrderExt` fields because they are needed for the actual restore operation. You must also keep any other information needed to open a data file or identify a destination.

Call **`dsmEndQuery`** to finish the query operation.

### ***Selecting and sorting objects by restore order***

After the backup or archive query is performed, the application client must determine which objects, if any, are to be restored or retrieved.

Then you sort the objects in ascending order (low to high). This sorting is very important to the performance of the restore operation. Sorting the objects on the **`restoreOrderExt`** fields ensures that the data is read from the server in the most efficient order.

All data on disk is restored first, followed by data on media classes that require volume mounts (such as tape). The **`restoreOrderExt`** field also ensures that data on tape is read in order with processing starting at the front of a tape and progressing towards the end.

Properly sorting on the **`restoreOrderExt`** field means that duplicate tape mounts and unnecessary tape rewinds do not occur.

A non-zero value in the **`restoreOrderExt.top`** field correlates to a unique serial access device on the IBM Storage Protect server. Since a serial access device can only be used by one session / mount point at a time, the application should ensure that if it uses multiple sessions there are not concurrent restores with the same **`restoreOrderExt.top`** value. Otherwise the first session are able to access the objects, but other sessions wait until the first session terminates and the device becomes available.

The following example shows how to sort objects by using **Restore Order** fields.

*Figure 16. Sorting objects with the restore order fields*

```
typedef struct {
    dsStruct64_t      objId;
    dsUint160_t       restoreOrderExt;

} SortOrder;          /* struct used for sorting */

=====
/* the code for sorting starts from here */
dsmQueryType         queryType;
qryBackupData         queryBuffer;
DataBlk              qDataBlkArea;
qryRespBackupData     qbDataArea;
dsInt16_t             rc;
dsBool_t             done = bFalse;
int i = 0;
int qry_item;
SortOrder sortorder[100]; /* sorting can be done up to 100 items
                           only right now. Set appropriate
                           array size to fit your needs */

/*-----+
| NOTE: Make sure that proper initializations have been done to
|   queryType,
|   queryBuffer, qDataBlkArea, and qbDataArea.
|-----*/

qDataBlkArea.bufferPtf = (char*) &qbDataArea;

rc = dsmBeginQuery(dsmHandle, queryType, (void *) &queryBuffer);

/*-----+
| Make sure to check rc from dsmBeginQuery
+-----*/
while (!done)
{
    rc = dsmGetNextQObj(dsmHandle, &qDataBlkArea);
    if ((rc == DSM_RC_MORE_DATA) ||
        (rc == DSM_RC_FINISHED))
        &&( qDataBlkArea.numBytes))
    {
        /*-----+
        /* transferring restoreOrderExt and objId */

```

```

        /*****
        sortorder[i].restoreOrderExt = qbDataArea.restoreOrderExt;
        sortorder[i].objId = qbDataArea.objId;

    } /* if ((rc == DSM_RC_MORE_DATA) || (rc == DSM_RC_FINISHED)) */
    else
    {
        done = bTrue;
        /*****
        /* take appropriate action. */
        /*****/
    }

    i++;
    qry_item++;

} /* while (!done) */
rc = dsmEndQuery(dsmHandle);
/*check rc */
/*****/
/* sorting the array using qsort. After the call, */
/* sortorder will be sorted by restoreOrderExt field */
/*****/

qsort(sortorder, qry_item, sizeof(SortOrder), SortRestoreOrder);

/*-----+
| NOTE: Make sure to extract sorted object ids and store them in
| any data structure you want.
|-----*/

/*-----+
| int SortRestoreOrder(SortOrder *a, SortOrder *b)
| This function compares restoreOrder fields from two structures.
| if (a > b)
|     return(GREATERTHAN);
| if (a < b)
|     return(LESSTHAN);
| if (a == b)
|     return(EQUAL);
|-----*/
int SortRestoreOrder(SortOrder *a, SortOrder *b)
{
    if (a->restoreOrderExt.top > b->restoreOrderExt.top)
        return(GREATERTHAN);
    else if (a->restoreOrderExt.top < b->restoreOrderExt.top)
        return(LESSTHAN);
    else if (a->restoreOrderExt.hi_hi > b->restoreOrderExt.hi_hi)
        return(GREATERTHAN);
    else if (a->restoreOrderExt.hi_hi < b->restoreOrderExt.hi_hi)
        return(LESSTHAN);
    else if (a->restoreOrderExt.hi_lo > b->restoreOrderExt.hi_lo)
        return(GREATERTHAN);
    else if (a->restoreOrderExt.hi_lo < b->restoreOrderExt.hi_lo)
        return(LESSTHAN);
    else if (a->restoreOrderExt.lo_hi > b->restoreOrderExt.lo_hi)
        return(GREATERTHAN);
    else if (a->restoreOrderExt.lo_hi < b->restoreOrderExt.lo_hi)
        return(LESSTHAN);
    else if (a->restoreOrderExt.lo_lo > b->restoreOrderExt.lo_lo)
        return(GREATERTHAN);
    else if (a->restoreOrderExt.lo_lo < b->restoreOrderExt.lo_lo)
        return(LESSTHAN);
    else
        return(EQUAL);
}

```

### ***Starting the dsmBeginGetData call***

After you select and sort the objects to receive, submit them to IBM Storage Protect for either a restore or retrieve operation. The **dsmBeginGetData** call begins a restore or retrieve operation. The objects are returned to the application client in the order you requested.

Complete the information for these two parameters in these calls:

### **mountWait**

This parameter tells the server whether the application client waits for offline media to be mounted in order to obtain data for an object, or whether that object should be skipped during processing of the restore or retrieve operation.

### **dsmGetObjListP**

This parameter is a data structure that contains the **objId** field which is a list of all object IDs that are restored or retrieved. Each **objId** is associated with a **partialObjData** structure that describes whether the entire **objId** or only a particular section of the object will be retrieved.

Each **objId** is eight bytes in length, so a single restore or retrieve request can contain thousands of objects. The number of objects to request in a single call is limited to DSM\_MAX\_GET\_OBJ or DSM\_MAX\_PARTIAL\_GET\_OBJ.

### ***Receiving each object to restore or retrieve***

After the **dsmBeginGetData** call is sent, you can perform a procedure to receive each object that is sent from the server.

### **About this task**

The DSM\_RC\_MORE\_DATA return code means that a buffer was returned and that you should call **dsmGetData** again. Check the **DataBlk.numBytes** for the actual number of returned bytes.

When you obtain all data for an object, you must send a **dsmEndGetObj** call. If more objects will be received, send **dsmGetObj** again.

If you want to stop the process, for example, to discard any remaining data in the restore stream for all objects that are not yet received, send the **dsmEndGetData** call. This call flushes the data from the server to the client. However, using this method might take time to complete. If you want to end a restore operation, use **dsmTerminate** to close the session.

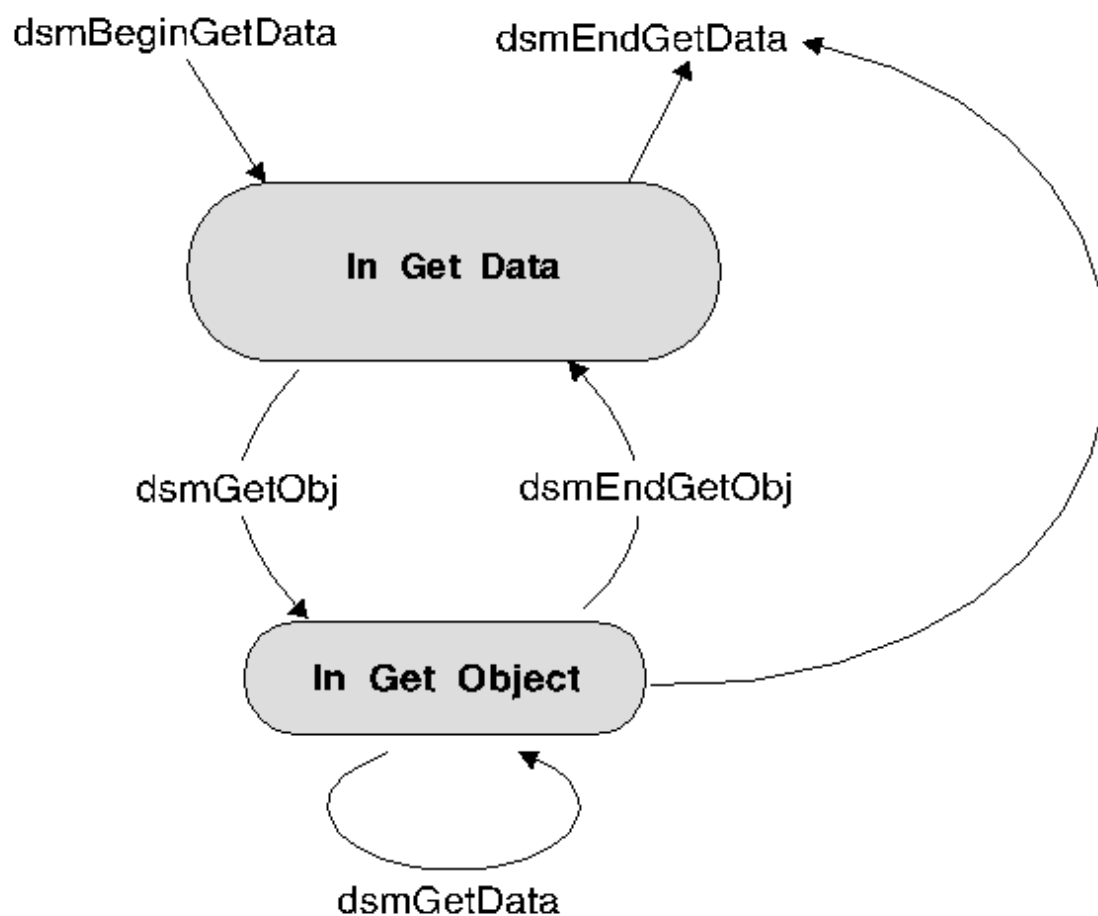
### **Procedure**

1. Send the **dsmGetObj** call to identify the object that you requested from the data stream and to obtain the first block of data that is associated with the object.
2. Send more **dsmGetData** calls, as necessary to obtain the remaining object data.

### **Example flow diagrams for restore and retrieve**

A state diagram and a flowchart can be used to illustrate how to perform restore or retrieve operations.

The arrow pointing from "In Get Object" to **dsmEndGetData** indicates that you can send a **dsmEndGetData** call after a call to **dsmGetObj** or **dsmGetData**. You might need to do this if an error condition occurred while getting an object from IBM Storage Protect storage and you want to stop the operation. In normal circumstances, however, call **dsmEndGetObj** first.



*Figure 17. State diagram for restore and retrieve operations*

[Figure 18 on page 63](#) displays the flowchart for performing restore or retrieve operations.



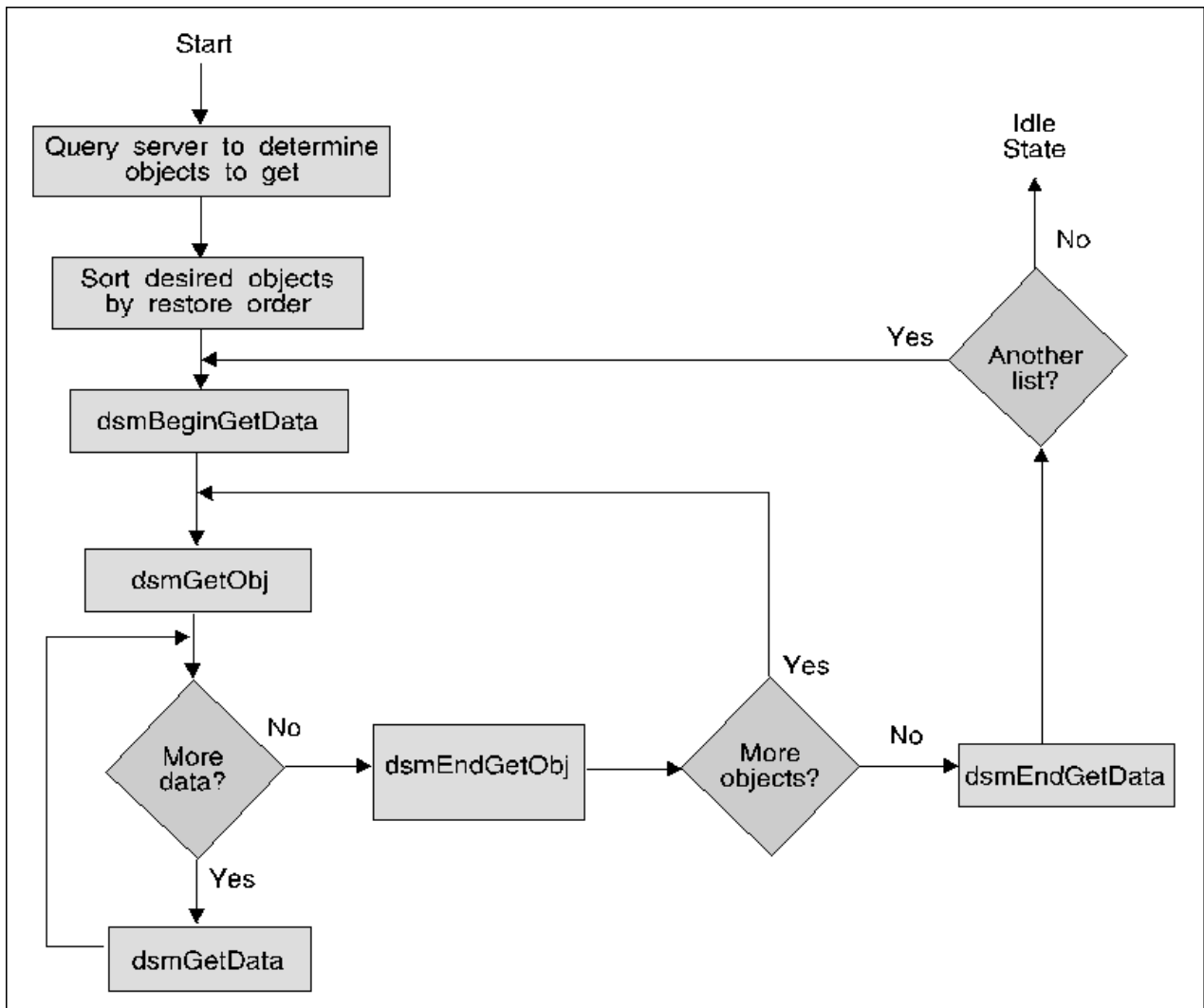


Figure 18. Flowchart for restore and retrieve operations

## Code example of receiving data from a server

This example demonstrates using the API functions to retrieve data from IBM Storage Protect storage.

The **dsmBeginGetData** function call appears inside a switch statement, so that different parameters can be called depending on whether a restore or retrieve operation is being performed. The **dsmGetData** function call is called from inside a loop that repeatedly gets data from the server until a flag is set that permits the program execution to exit the loop.

Figure 19. An example of receiving data from a server

```

/* Call dsmBeginQuery and create a linked list of objects to restore. */
/* Process this list to create the proper list for the GetData calls. */
/* Set up the getList structure to point to this list. */
/* This example is set up to perform a partial object retrieve. To */
/* retrieve only complete objects, set up: */
/*     getList.stVersion = dsmGetListVersion; */
/*     getList.partialObjData = NULL; */
dsmGetList getList;
getList.stVersion = dsmGetListPORVersion; /* structure version */
getList.numObjId = items; /* number of items in list */
getList.objId = (ObjID *)rest_ibuff;
/* list of object IDs to restore */
getList.partialObjData = (PartialObjData *) part_ibuff;
/* list of partial object data */
switch(get_type)
{
    case (Restore_Get) :

```

```

        rc = dsmBeginGetData(dsmHandle,bFalse,gtBackup,&getList);
        break;
    case (Retrieve_Get) :
        rc = dsmBeginGetData(dsmHandle,bFalse,gtArchive,&getList);
        break;
    default : ;
}
if (rc)
{
    printf("*** dsmBeginGetData failed: ");
    rcApiOut(dsmHandle, rc);
    return rc;
}
/* Get each object from the list and verify whether it is on the */
/* server. If so, initialize structures with object attributes for */
/* data validation checks. When done, call dsmGetObj. */
rc = dsmGetObj(dsmHandle,objId,&dataBlk);
done = bFalse;
while(!done)
{
    if ( (rc == DSM_RC_MORE_DATA)
        || (rc == DSM_RC_FINISHED))
    {
        if (rc == DSM_RC_MORE_DATA)
        {
            dataBlk.numBytes = 0;
            rc = dsmGetData(dsmHandle,&dataBlk);
        }
        else
            done = bTrue;
    }
    else
    {
        printf("*** dsmGetObj or dsmGetData failed: ");
        rcApiOut(dsmHandle, rc);
        done = bTrue;
    }
}
/* while */
rc = dsmEndGetObj(dsmHandle);
/* check rc from dsmEndGetObj */
/* check rc from dsmEndGetData */
rc = dsmEndGetData(dsmHandle);
return 0;

```

## Updating and deleting objects on the server

Your API applications can use the **dsmUpdateObj** or **dsmUpdateObjEx** function call to update objects that were archived or backed up. Use either call in the session state only, updating one object at a time. Use **dsmUpdateObjEx** to update any of several archive objects containing the same name.

To select an archive object, set the **dsmSendType** function call to **stArchive**.

- With **dsmUpdateObj**, only the latest archive object with the assigned name is updated.
- With **dsmUpdateObjEx**, any archived object can be updated by specifying the proper object ID.

For an archived object, the application can update the following fields:

- Description
- Object information
- Owner

To select a backup object, set **dsmSendType** to **stBackup**. For backed-up objects, only the active copy is updated.

For a backed-up object, the application can update the following fields:

- Management class
- Object information
- Owner

## Deleting objects from the server

API applications can make calls to either delete objects that were archived or turn off objects that were backed up. Deleting archived objects is dependent on the node authorization that was given when the administrator registered the node. Administrators can specify that nodes can delete archived objects.

Use the **dsmDeleteObj** function call to delete archived objects and turn off backup objects. Using this **delType** removes the backup object from the server. This is based on **objID**, deletes an object from the server database. Only an owner of an object can delete it. You can delete any version (active or inactive) of an object. The server reconciles the versions. If you delete an active version of an object, the first inactive version becomes active. If you delete an inactive version of an object, all older versions advance. The node must be registered with **backDel** permission.

An archived object is marked for deletion in storage when the system performs its next object expiration cycle. Once you delete an archived object from the server, you cannot retrieve it.

When you inactivate a backup object at the server, the object moves from an active state to an inactive state. These states have different retention policies associated with them that are based on the management class that is assigned.

Similar to the **dsmSendObj** call, a call to **dsmDeleteObj** is sent within the boundary of a transaction. The state diagram in [Figure 12 on page 51](#) displays how a call to **dsmDeleteObj** is preceded by a call to **dsmBeginTxn** and followed by a call to **dsmEndTxn**.

## Logging events

An API application can log event messages to central locations. The application can direct logging to the IBM Storage Protect server, the local machine, or both. The **dsmLogEventEx** function call is performed in a session. To view messages logged on the server, use the query **actlog** command through the administrative client.

Use the IBM Storage Protect client option, **errorlogretention**, to prune the client error log file if the application writes numerous client messages to the client log **dsmLogType**, either **logLocal** or **logBoth**.

For more information about IBM Storage Protect logs, see the IBM Storage Protect server documentation.

## State diagram summary for the IBM Storage Protect API

Once you review all the considerations for creating your own application with the IBM Storage Protect API, review this state diagram summary of an entire application.

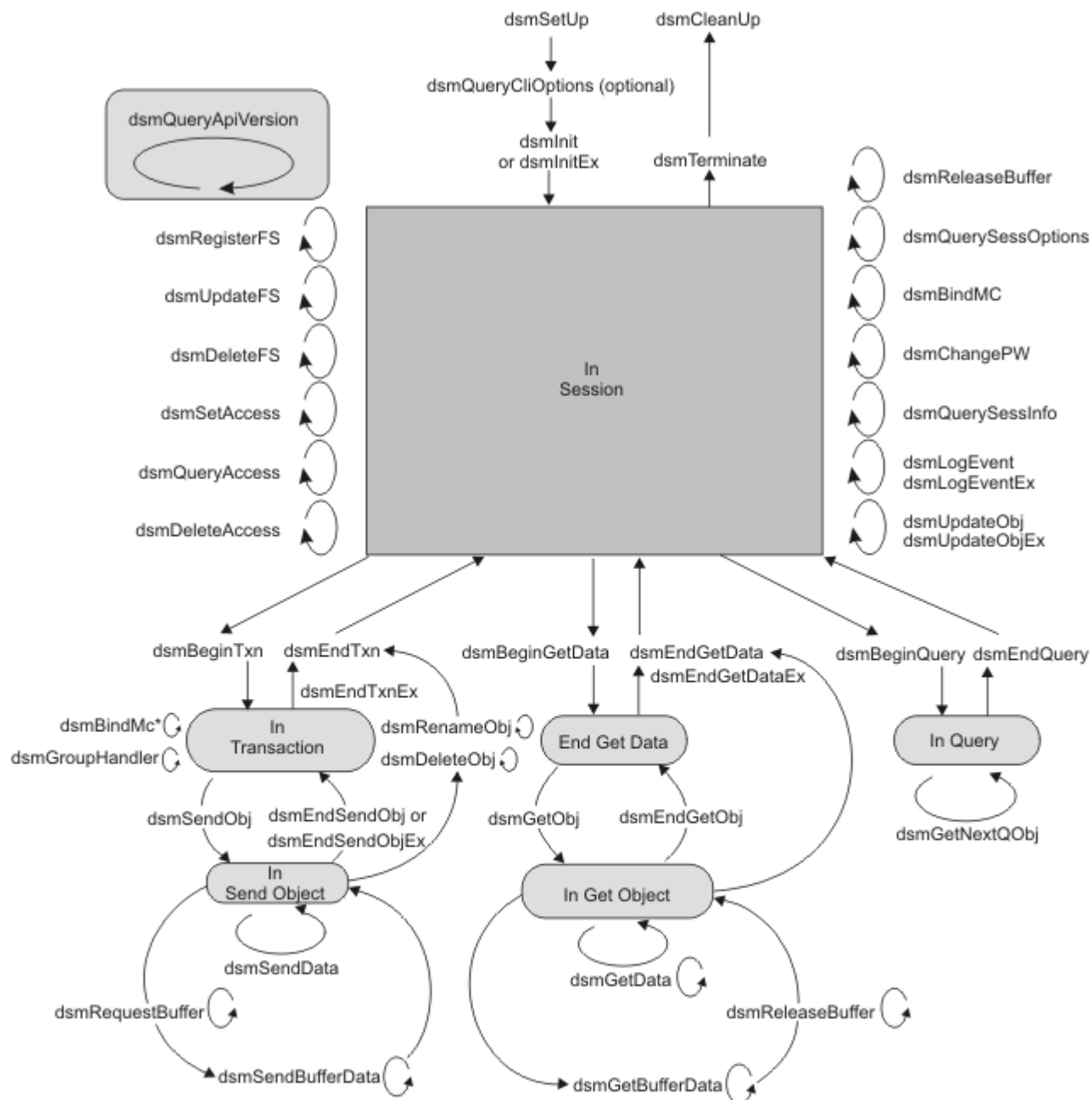
[Figure 20 on page 66](#) contains the state diagram for the API. It contains all previously displayed state diagrams in addition to several other calls previously not displayed.

The points in this diagram include:

- Call **dsmQueryApiVersionEx** at any time. It has no state associated with it. See [Figure 1 on page 14](#) for an example.
- Call **dsmQueryCliOptions** before a **dsmInitEx** call only.
- Use **dsmRegisterFS**, **dsmUpdateFS**, and **dsmDeleteFS** to manage file spaces. These calls are made from within an idle session state. Use the **dsmBeginQuery** call to query file spaces. For more information about file space calls, see [“Managing file spaces” on page 22](#).
- Send the **dsmBindMC** call from within an idle session state or from within a send object transaction state. See the example in [Figure 8 on page 26](#).
- Send the **dsmChangePW** call from within an idle session state.

**Note:** If the **dsmInitEx** call returns with a password-expired return code, the **dsmChangePW** call must be made before you start a valid session. See [Figure 4 on page 18](#) for an example that uses **dsmChangePW**.

- If a call returns with an error, the state remains as it was. For example, if **dsmGetObj** returns with an error, the state remains In Get Data, and a call to **dsmEndGetObj** is a call sequence error.



\* Can be inside or outside of a transaction

Figure 20. Summary state diagram for the API

## Understanding interoperability

---

The API has two types of interoperability: between the backup-archive client and API applications and between different operating systems.

### Backup-archive client interoperability

The backup-archive command line can access API objects to provide limited interoperability. API objects can only be viewed and accessed from the backup-archive command line client and cannot be viewed or accessed from any of the graphical interfaces. The backup-archive command-line client can only restore content of the file and nothing else, so you should only use it for a salvage type of operation.

The following command-line actions are provided:

- Delete archive
- Delete filespace
- Query
- Restore
- Retrieve
- Set access

The path information is actual directories for backup-archive client objects. In contrast, the API object path information might not have any relationship to existing directories: the path might be completely contrived. Interoperability does not change this aspect of these object types. To use this feature successfully, follow the restrictions and conventions.

#### Notes:

1. There is no interoperability between the backup-archive client and API objects stored on a retention protection server.
2. You cannot use the backup-archive client GUIs to access files that were stored using the API client. You can only use the command line to access these files.

### Naming your API objects

Establish a consistent naming convention for API object names. The naming convention must cater for the file space name, the high-level qualifier, and the low-level qualifier. The file space name and high-level qualifiers can refer to actual directory names. Each object name can consist of more than one directory name that applies to the low-level qualifier.

For convenience, use the name of the object that is not prefixed with directory information as the low-level qualifier. For more information, see [“Object names and IDs” on page 20](#).

File space names must be fully qualified when they are referred to from either the API or the backup-archive command line. For example, on a UNIX or Linux operating system, you register the following file spaces:

- /a
- /a/b

When you refer to /a, objects that are related only to file space /a are displayed. To view objects that are related to /a/b, you must specify /a/b as the file space name.

After you register both file spaces, if you back up object b into file space /a, then a query for /a/b continues to display objects that are related only to file space /a/b.

The exception to this restriction occurs in file space references when you attempt to query or delete file spaces with the API. In both cases, the file space names do not have to be fully qualified if you use a wildcard character. For example, /a\* refers to both /a and /a/b.

**Tip:** If interoperability is important for you, then avoid file space names that overlap.

On Windows systems, enclose file space names in braces { } for API objects when you access the objects from the backup-archive command line interface. Windows operating systems automatically place file space names in uppercase letters when you register or refer the names. However, this automatic function does not occur for the remainder of the object name specification. If you want full interoperability, place the high-level qualifier and the low-level qualifier in uppercase letters in the application when you back up API objects. If your application does not uppercase high-level qualifiers (directory names) and low-level qualifiers (file names) before it sends objects to the server, you will be unable to access the objects directly by name through the backup-archive client.

For example, if an object is stored on the server as {"FileSpaceName"}\TEST\MYDIRNAME\file.txt, you cannot directly restore or query the file.txt object because your application did not uppercase the file name before the file was copied to the server. The only way to manipulate these objects is to use wildcard characters. For example, to query \TEST\MYDIRNAME\file.txt, a backup-archive client user must use wildcard characters for all parts of the object name that were not uppercased before they were sent to the server. The following command must be used to query this file.txt file:

```
dsmc query backup {"FileSpaceName"}\TEST\MYDIRNAME\*
```

If any other of the other qualifiers are also saved in lowercase text, those qualifiers must also be queried by using wildcards. For example, to query an object that is stored as {"FileSpaceName"}\TEST\mydirname\file.txt, use the following command:

```
dsmc query backup {"FileSpaceName"}\TEST\*\*
```

The examples that follow demonstrate these concepts. In both Windows and UNIX or Linux environments, you do not have to specify either the complete high-level or low-level qualifier. However, if you do not specify the complete qualifier, then you must use the wildcard character.

| Platform      | Example                                                                                                                                                                                                              |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows       | To query all backed-up files in file space MYFS, enter the following string: <div>dsmc q ba "{MYFS}\*\*</div> <div>You must use at least one asterisk (*) for each of the high-level and low-level qualifiers.</div> |
| UNIX or Linux | To query all backed-up files in file space /A, enter the following string: <div>dsmc q ba "/A/*/*"</div> <div>You must use at least one asterisk (*) for each of the high-level and low-level qualifiers.</div>      |

## Backup-archive client commands you can use with the API

You can use a subset of backup-archive client commands within an application. For example, you can view and manage objects that other users own either on the same node or on a different node.

To view and manage objects that other users own either on the same node or on a different node, perform these steps:

1. Give access with the **set access** command.
2. Specify the owner and the node. Use the *fromowner* and *fromnode* options from the backup-archive command line to specify the owner and the node. For example:

```
dsmc q ba "/A/*/*" -fromowner=other_owner -fromnode=other_node
```

[Table 18 on page 69](#) describes the commands that you can use with API objects.

Table 18. Backup-archive client commands you can use with API objects

| Command                 | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Delete Archive</b>   | Archived files that the current user owns can be deleted. The set access command settings have no effect on this command.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Delete Filespace</b> | The <b>delete filespace</b> command affects API objects.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Query</b>            | <p>From the backup-archive command line, you can query backed up and archived API objects and objects that other users own, or that exist on other nodes. See <a href="#">“Naming your API objects” on page 67</a> for information about querying API objects.</p> <p>Use the existing <i>-fromowner</i> option to query objects that a different user owns for which the set access permission has been given. Use the existing <i>-fromnode</i> option to query objects that exist on another node for which the set access permission has been given. For more information, see <a href="#">“dsmInitEx” on page 102</a>.</p> |
| <b>Restore Retrieve</b> | <p><b>Note:</b> Use these commands only for exception situations. API objects that are encrypted using the application managed key can be restored or retrieved if the encryption key is known or saved in the password file. API objects encrypted by using transparent encryption cannot be restored or retrieved by using the backup-archive client.</p> <p>These commands return data as bit files that are created by using default file attributes. You can restore or retrieve API objects that other users own, or that are from a different node. The set access command determines which objects qualify.</p>         |
| <b>Set Access</b>       | The <b>set access</b> command permits users to manage API objects that another user owns, or that are from another node.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

## Examples

```
dsmc query backup f:\*\*
```

|     | Size |  | Backup Date         | Mgmt Class | A/I File               |
|-----|------|--|---------------------|------------|------------------------|
|     | ---- |  | -----               | -----      | -----                  |
| API | 1 B  |  | 11/14/2018 08:22:24 | DEFAULT    | A \\viola\fs\dir1\test |
|     | 1 B  |  | 11/14/2018 08:21:41 | DEFAULT    | A \\viola\fs\dir1\test |

```
dsmc query backup "/home/*/*"
```

|     | Size |  | Backup Date         | Mgmt Class | A/I File           |
|-----|------|--|---------------------|------------|--------------------|
|     | ---- |  | -----               | -----      | -----              |
| API | 1 B  |  | 11/15/2018 08:22:24 | DEFAULT    | A /home/user1/test |
|     | 1 B  |  | 11/15/2018 08:21:41 | DEFAULT    | A /home/user1/test |

## Operating system interoperability

The IBM Storage Protect API supports cross-platform interoperability. Applications on a UNIX or Linux system can operate on file spaces and objects that are backed up from a Windows system. Similarly, a Windows system can operate on file spaces and objects that are backed up from a UNIX or Linux system.

### About this task

By default, the names of objects from one UNIX system are compatible with the names of objects from other UNIX systems. By default, names of objects from Windows systems are not compatible with names of objects from UNIX systems. Several parameters control the naming of objects in IBM Storage Protect file spaces. If you set up an application appropriately, the names of objects can be used by applications that run on both Windows systems and UNIX systems. Use the same parameters to back up and restore objects.

**Restriction:** A Windows application that uses Unicode creates a file space that is not compatible with applications that run on UNIX systems.

## Procedure

To achieve interoperability, complete the following setup tasks:

1. Establish a consistent naming convention. Select a character for the `dir` delimiter, such as forward slash (/) or backslash (\). Place the directory delimiter character in front of the file space name, the high-level qualifier, and the low-level qualifier.
2. When you call **dsmInitEx**, set the value of the **dirDelimiter** field to the directory delimiter character that you selected and set **bCrossPlatform** to **bTrue**.
3. Set the **useUnicode** flag to **bFalse** when you use the IBM Storage Protect interface. Unicode file names are not compatible with non-Unicode file names.

## Backing up multiple nodes with client node proxy support

Backups of multiple nodes which share storage can be consolidated to a common target node name on the IBM Storage Protect server. This method is useful when the system that runs the backup can change over time, such as with a cluster. You can also use the `asnodename` option to restore data from a different system other than the one which ran the backup.

### About this task

Use the `asnodename` option on the **dsmInitEx** option string to back up, archive, restore, and retrieve, query, or delete data under the target node name on the IBM Storage Protect server. You can also specify the `asnodename` option in the `dsm.opt` or `dsm.sys` file.

**Restriction:** Do not use target nodes as traditional nodes, especially if you encrypt your files before you back up to the server.

## Procedure

To enable this option, complete the following steps:

1. Install the API client on all nodes in a shared data environment.
2. If not already registered, register each node with the IBM Storage Protect server. Register the common "target" node name to be shared by each of the agent nodes that are used in your shared data environment.
3. Register each of the agent nodes in the shared data environment with the server. The agent node name is used for authentication. Data is not stored by using the agent node name when the `asnodename` option is used.
4. Ask your administrator to grant proxy authority to all nodes in the shared environment to access the target node name on the IBM Storage Protect server, by using the **grant proxynode** command.
5. Use the **query proxynode** administrative client command to display the client nodes that have the authority to perform client operations on behalf of another node. This authority is granted by the **grant proxynode** command. Or use the **dsmQuery** command with the query type **qtProxyNodeAuth** to see the nodes to which this node can proxy.
6. If the application is using user encryption of data, not TSMENCRKEY, ensure that all nodes use the same encryption key. You must use the same encryption key for all files that are backed up in the shared node environment.

### Related information

[Backing up data with client-node proxy support \(UNIX and Linux systems\)](#)

[Backing up data with client-node proxy support \(Windows systems\)](#)



## Using the API with Unicode

---

The IBM Storage Protect API supports Unicode UCS2, a fixed length, double-byte code page that has code points for all known code pages, such as Japanese, Chinese, or German. It supports as many as 65,535 unique code points.

**Restriction:** This feature is only available on Windows.

With Unicode, your application can back up and restore file names in any character set from the same machine. For example, on an English machine, you can back up and restore file names in any other language code page.

### When to use Unicode

You can simplify your application that supports multiple languages by writing a Unicode application and by taking advantage of the IBM Storage Protect Unicode interface.

Use the IBM Storage Protect Unicode interface if any of the following conditions are true:

- If your application is already compiled for Unicode and it was converting to a multibyte character set (mbcs) before calling the IBM Storage Protect API.
- If you are writing a new application and want to enable your application to support Unicode.
- If your application uses a string passed to it from an operating system or other application that uses Unicode.

If you do not need Unicode, it is not necessary to compile your application again.

The API continues to support the dsm interface. The API SDK contains `callmtu1.c` and `callmtu2.c` sample programs that demonstrate how to use the Unicode API. Use **makentu** to compile these programs.

### Setting up Unicode

To set up and use Unicode you must perform a particular procedure so the API registers a Unicode file space on the server and all file names in that file space become Unicode strings.

**Restriction:** You cannot store Unicode and non-Unicode file names in the same file space.

1. Compile the code with the `-DUNICODE` flag.
2. All strings in your application must be `wchar` strings.
3. Follow the structures in the `tsmapitd.h` file, and the function definitions in the `tsmapifp.h` file for calls to the API.
4. Set the `useUnicode` flag to `bTrue` on the **tsmInitEx** function call. Any new file space is registered as a Unicode file space.

When you send data to previously registered, non-Unicode file spaces, the API continues to send file names as non-Unicode. Rename the old file spaces on the server to `fname_old` and start a new Unicode file space for new data. The API restores non-Unicode data from the old file spaces. Use the **bIsUnicode** field in the **tsmQryRespFSDData** structure that is returned on a query file space to determine whether or not a file space is Unicode.

Each **dsmXXX** function call has a matching **tsmXXX** function call. The difference between the two are the structures that are used. All **tsmXXX** function call structures have `dsChar_t` types for string values when they are compiled with the `UNICODE` flag. The `dsChar_r` maps to `wchar`. There is no other difference between these interfaces.

**Restriction:** Use either one interface or the other. Do not mix the **dsmXXX** function call and **tsmXXX** function call interfaces. Ensure that you use the IBM Storage Protect structures and IBM Storage Protect version definitions.

Some constants continue to be defined in the `dsmapitd.h` file, so you need both the `dsmapitd.h` and the `tsmapitd.h` files when you compile.

You can use the IBM Storage Protect interface on other operating systems, such as UNIX or Linux, but on these operating systems, the `dsChar_t` type maps to `char` because Unicode is supported on only Windows operating systems. You can write only one variation of the application and compile on more than one operating system using the IBM Storage Protect interface. If you are writing a new application, use the IBM Storage Protect interface.

If you are upgrading an existing application:

1. Convert the **dsXXXX** function call structures and calls to the IBM Storage Protect interface.
2. Migrate existing file spaces.
3. Back up new file spaces with the *useUnicode* flag set to *true*.

**Note:** After you use a Unicode-enabled client to access a node, you cannot connect to the same node name with an older version of the API or with an API from another operating system. If your application uses cross-platform capability, do not use the `Unicode` flag. There is no cross-platform support between Unicode and non-Unicode operating systems.

When you enable the *useUnicode* flag, all string structures are treated as Unicode strings. On the server, only the following fields are true Unicode:

- File space name
- High level
- Low level
- Archive description

All remaining fields convert to mbcs in the local code page before they are sent to the server. Fields, such as `nodename`, are `wchar` strings. They must be valid in the current locale. For example, on a Japanese machine, you can back up files with Chinese names, but the node name must be a valid string in Japanese. The option file remains in the current code page. If you need to create a Unicode include-exclude list, use the *incl excl* option with a file name and create a Unicode file with Unicode patterns in it.

### **Related information**

[incl excl option](#)

## Chapter 4. API commands, options, and utilities

### API function calls

Table 19 on page 73 provides an alphabetical list of the API function calls, a brief description and the location of more detailed information about the function call, which includes:

| Element             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>      | Describes the function call.                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Syntax</b>       | <p>Contains the actual C code for the function call. This code is copied from the UNIX or Linux version of the <code>dsmapifp.h</code> header file. See <a href="#">“API function definitions source file”</a> on page 175.</p> <p>This file differs slightly on other operating systems. Application programmers for other operating systems should check their version of the header file, <code>dsmapifp.h</code>, for the exact syntax of the API definitions.</p> |
| <b>Parameters</b>   | Describes each parameter in the function call, identifying it as either input (I) or output (O), depending on how it is used. Some parameters are designated as both input and output (I/O). The data types that are referenced in this section are defined in the <code>dsmapitd.h</code> header file. See <a href="#">“API type definitions source files”</a> on page 140.                                                                                           |
| <b>Return codes</b> | Contains a list of the return codes that are specific to the function call. General system errors, such as communication errors, server problems, or user errors that might appear on any call are not listed. The return codes are defined in the <code>dsmsrc.h</code> header file. See <a href="#">“API return codes source file: dsmsrc.h”</a> on page 131.                                                                                                        |

Table 19. API function calls

| Function call and location                   | Description                                                                                                        |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <a href="#">“dsmBeginGetData”</a> on page 75 | Starts a restore or retrieve operation on a list of objects in storage.                                            |
| <a href="#">“dsmBeginQuery”</a> on page 76   | Starts a query request to IBM Storage Protect for information.                                                     |
| <a href="#">“dsmBeginTxn”</a> on page 81     | Starts one or more transactions that begins a complete action. Either all of the actions succeed, or none succeed. |
| <a href="#">“dsmBindMC”</a> on page 82       | Associates, or binds, a management class to the object that is passed.                                             |
| <a href="#">“dsmChangePW”</a> on page 83     | Changes an IBM Storage Protect password.                                                                           |
| <a href="#">“dsmCleanUp”</a> on page 84      | This call is used if <b>dsmSetUp</b> was called.                                                                   |
| <a href="#">“dsmDeleteAccess”</a> on page 84 | Deletes current authorization rules for backup versions or archived copies of your objects.                        |
| <a href="#">“dsmDeleteFS”</a> on page 85     | Deletes a file space from storage.                                                                                 |
| <a href="#">“dsmDeleteObj”</a> on page 86    | Turns off backup objects, or deletes archive objects in storage.                                                   |
| <a href="#">“dsmEndGetData”</a> on page 87   | Ends a <b>dsmBeginGetData</b> session that gets objects from storage.                                              |
| <a href="#">“dsmEndGetDataEx”</a> on page 87 | Provides the total of LAN-free bytes that were sent.                                                               |
| <a href="#">“dsmEndGetObj”</a> on page 88    | Ends a <b>dsmGetObj</b> session that obtains data for a specified object.                                          |

Table 19. API function calls (continued)

| Function call and location                         | Description                                                                                                                                   |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">“dsmEndQuery” on page 88</a>           | Signifies the end of a <b>dsmBeginQuery</b> action.                                                                                           |
| <a href="#">“dsmEndSendObj” on page 89</a>         | Indicates the end of data that is sent to storage.                                                                                            |
| <a href="#">“dsmEndSendObjEx” on page 89</a>       | Provides compression information and the number of bytes that were sent.                                                                      |
| <a href="#">“dsmEndTxn” on page 90</a>             | Ends an IBM Storage Protect transaction.                                                                                                      |
| <a href="#">“dsmEndTxnEx” on page 91</a>           | Provides group leader object ID information to use with the <b>dsmGroupHandler</b> function call.                                             |
| <a href="#">“dsmGetData” on page 93</a>            | Obtains a byte stream of data from IBM Storage Protect and place it in the caller's buffer.                                                   |
| <a href="#">“dsmGetBufferData” on page 93</a>      | Gets an IBM Storage Protect-allocated buffer of data from the IBM Storage Protect server.                                                     |
| <a href="#">“dsmGetNextQObj” on page 94</a>        | Gets the next query response from a previous <b>dsmBeginQuery</b> call and places it in the caller's buffer.                                  |
| <a href="#">“dsmGetObj” on page 97</a>             | Obtains the requested object data from the data stream and places it in the caller's buffer.                                                  |
| <a href="#">“dsmGroupHandler” on page 98</a>       | Performs an action on a logical file group depending on the input that is given.                                                              |
| <a href="#">“dsmInit” on page 99</a>               | Starts an API session and connects the client to storage.                                                                                     |
| <a href="#">“dsmInitEx” on page 102</a>            | Starts an API session using the additional parameters that permit extended verification.                                                      |
| <a href="#">“dsmLogEvent” on page 106</a>          | Logs a user message to the server log file, to the local error log, or to both.                                                               |
| <a href="#">“dsmLogEventEx” on page 107</a>        | Logs a user message to the server log file, to the local error log, or to both.                                                               |
| <a href="#">“dsmQueryAccess” on page 108</a>       | Queries the server for all access authorization rules for either backup versions or archived copies of your objects.                          |
| <a href="#">“dsmQueryApiVersion” on page 109</a>   | Performs a query request for the API library version that the application client accesses.                                                    |
| <a href="#">“dsmQueryApiVersionEx” on page 109</a> | Performs a query request for the API library version that the application client accesses.                                                    |
| <a href="#">“dsmQueryCliOptions” on page 110</a>   | Queries important option values in the user's option files.                                                                                   |
| <a href="#">“dsmQuerySessInfo” on page 110</a>     | Starts a query request to IBM Storage Protect for information that is related to the operation of the specified session in <b>dsmHandle</b> . |
| <a href="#">“dsmQuerySessOptions” on page 111</a>  | Queries important option values that are valid in the specified session in <b>dsmHandle</b> .                                                 |
| <a href="#">“dsmRCMsg” on page 112</a>             | Obtains the message text that is associated with an API return code.                                                                          |
| <a href="#">“dsmRegisterFS” on page 113</a>        | Registers a new file space with the server.                                                                                                   |
| <a href="#">“dsmReleaseBuffer” on page 114</a>     | Returns an IBM Storage Protect-allocated buffer.                                                                                              |
| <a href="#">“dsmRenameObj” on page 115</a>         | Renames the high-level or low-level object name.                                                                                              |

Table 19. API function calls (continued)

| Function call and location                      | Description                                                                                                                                                                 |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">“dsmRequestBuffer” on page 116</a>  | Obtains an IBM Storage Protect-allocated buffer for buffer copy elimination.                                                                                                |
| <a href="#">“dsmRetentionEvent” on page 117</a> | Sends a list of object IDs to the server with a retention event operation to be performed on these objects.                                                                 |
| <a href="#">“dsmSendBufferData” on page 118</a> | Sends data from an IBM Storage Protect-allocated buffer.                                                                                                                    |
| <a href="#">“dsmSendData” on page 119</a>       | Sends a byte stream of data to IBM Storage Protect via a buffer.                                                                                                            |
| <a href="#">“dsmSendObj” on page 120</a>        | Starts a request to send a single object to storage.                                                                                                                        |
| <a href="#">“dsmSetAccess” on page 123</a>      | Gives other users, or nodes, access to backup versions or archived copies of your objects, access to all your objects, or access to a selective set.                        |
| <a href="#">“dsmSetUp” on page 124</a>          | Overwrites environment variable values.                                                                                                                                     |
| <a href="#">“dsmTerminate” on page 126</a>      | Ends a session with the server and cleans up the IBM Storage Protect environment.                                                                                           |
| <a href="#">“dsmUpdateFS” on page 126</a>       | Updates a file space in storage.                                                                                                                                            |
| <a href="#">“dsmUpdateObj” on page 127</a>      | Updates the objInfo information that is associated with an active backup object already on the server, or it updates archived objects.                                      |
| <a href="#">“dsmUpdateObjEx” on page 129</a>    | Updates the objInfo information that is associated with a specific archive object even when there are multiple objects with same name, or it updates active backup objects. |

## Related information

[API return codes](#)

## dsmBeginGetData

The **dsmBeginGetData** function call starts a restore or retrieve operation on a list of objects in storage. This list of objects is contained in the **dsmGetList** structure. The application creates this list with values from the query that preceded a call to **dsmBeginGetData**.

The caller first must use the restore order fields that are obtained from the object query to sort the list that is contained in this call. This ensures that the objects are restored from storage in the most efficient way possible without rewinding or remounting data tapes.

When getting whole objects, the maximum *dsmGetList.numObjID* is DSM\_MAX\_GET\_OBJ. When getting partial objects, the maximum is DSM\_MAX\_PARTIAL\_GET\_OBJ.

Follow the call to **dsmBeginGetData** with one or more calls to **dsmGetObj** to obtain each object within the list. After each object is obtained, or additional data for the object is not needed, the **dsmEndGetObj** call is sent.

When all objects are obtained, or the **dsmEndGetObj** is canceled, the **dsmEndGetData** call is sent. You then can start the cycle again.

## Syntax

```
dsInt16_t dsmBeginGetData (dsUInt32_t      dsmHandle,
                          dsBool_t        mountWait,
                          dsmGetType      getType,
                          dsmGetList      *dsmGetObjListP);
```

## Parameters

### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **dsBool\_t mountWait (I)**

A Boolean true or false value indicates whether or not the application client waits for offline media to be mounted if the data that is needed is currently offline. If **mountWait** is true, the application waits for the server to mount the required media. The application waits until the media is mounted or the request is canceled.

### **dsmGetType getType (I)**

An enumerated type consisting of **gtBackup** and **gtArchive** that indicates what type of object to get.

### **dsmGetList \*dsmGetObjListP (I)**

The structure that contains information about the objects or partial objects to restore or retrieve. The structure points to a list of object IDs and, in the case of a partial object restore or retrieve, a list of associated offsets and lengths. If your application uses the partial object restore or retrieve function, set the **dsmGetList.stVersion** field to **dsmGetListPORVersion**. In a partial object restore or retrieve, you cannot compress data while sending it. To enforce this, set **ObjAttr.objCompressed** to **bTrue**.

See Figure 19 on page 63 and “API type definitions source files” on page 140 for more information on this structure.

See “Partial object restore or retrieve” on page 57 for more information on partial object restore or retrieve.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 20. Return codes for *dsmBeginGetData*

| Return code                      | Explanation                                                                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_ABORT_INVALID_OFFSET (33) | The offset that was specified during a partial object retrieve is greater than the length of the object.                                                                             |
| DSM_RC_ABORT_INVALID_LENGTH (34) | The length that was specified during a partial object retrieve is greater than the length of the object, or the offset in addition to the length extends past the end of the object. |
| DSM_RC_NO_MEMORY (102)           | There is no RAM remaining to complete the request.                                                                                                                                   |
| DSM_RC_NUMOBJ_EXCEED (2029)      | The <b>dsmGetList.numObjId</b> is greater than <b>DSM_MAX_GET_OBJ</b> .                                                                                                              |
| DSM_RC_OBJID_NOTFOUND (2063)     | The object ID was not found. The object was not restored.                                                                                                                            |
| DSM_RC_WRONG_VERSION_PARM (2065) | The API version of the application client is different from the IBM Storage Protect library version.                                                                                 |

## dsmBeginQuery

The **dsmBeginQuery** function call starts a query request to the server for information about data, file spaces, and management classes.

Specifically, **dsmBeginQuery** can query:

- Archived data
- Backed-up data
- Active backed-up data

- File spaces
- Management classes

The query data that is returned from the call is obtained by one or more calls to **dsmGetNextQObj**. When the query is complete, the **dsmEndQuery** call is sent.

## Syntax

```
dsInt16_t dsmBeginQuery (dsUInt32_t      dsmHandle,
                        dsmQueryType queryType,
                        dsmQueryBuff *queryBuffer);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmQueryType queryType (I)

Identifies the type of query to run. Assign one of the following options:

#### qtArchive

Queries archived objects.

#### qtBackup

Queries backed-up objects.

#### qtBackupActive

Queries active, backed-up objects only for the entire file space name that you pass. This query is called a "fast path" and is an efficient way to query active objects from storage.

**Prerequisite:** You must be logged on as a root user on a UNIX or Linux operating system.

#### qtFilespace

Queries registered file spaces.

#### qtMC

Queries defined management classes.

#### qtBackupGroups

Queries groups that are closed.

#### qtOpenGroups

Queries groups that are open.

#### qtProxyNodeAuth

Queries nodes to which this node can proxy.

#### qtProxyNodePeer

Queries peer nodes with the same target.

### dsmQueryBuff \*queryBuffer (I)

Identifies a pointer to a buffer that is mapped to a particular data structure. This structure is associated with the query type that you pass. These structures contain the selection criteria for each query type. Complete the fields in each structure to specify the scope of the query that you want to run. The `stVersion` field in each structure contains the structure version number.

The data structures and their related fields include the following items:

## **qryArchiveData**

### **objName**

The complete object name. You can use a wildcard character, such as an asterisk (\*) or a question mark (?), in the high-level or low-level portion of the name. An asterisk matches zero or more characters, and a question mark matches one character. The objType field of objName can have one of the following values:

- DSM\_OBJ\_FILE
- DSM\_OBJ\_DIRECTORY
- DSM\_OBJ\_ANY\_TYPE

For more information about high-level and low-level names, see the following topic: [“High-level and low-level names” on page 21.](#)

### **owner**

The owner name of the object.

### **insDateLowerBound**

The lower boundary for the insert date that the object was archived. Set the year component to DATE\_MINUS\_INFINITE for an unbounded lower boundary.

### **insDateUpperBound**

The upper boundary for the insert date that the object was archived. Set the year component to DATE\_PLUS\_INFINITE for an unbounded upper boundary.

### **expDateLowerBound**

The lower boundary for the expiration date. Set the year component to DATE\_MINUS\_INFINITE for an unbounded lower boundary.

### **expDateUpperBound**

The upper boundary for the expiration date. To match a management class **RETVer** setting of NOLIMIT, set the year component to DATE\_PLUS\_INFINITE.

### **descr**

The archive description. Enter an asterisk (\*) to search all descriptions.

## **qryBackupData**

### **objName**

The complete object name. You can use a wildcard character, such as an asterisk (\*) or a question mark (?), in the high-level or low-level portion of the name. An asterisk matches zero or more characters, and a question mark matches one character. The objType field of objName can have one of the following values:

- DSM\_OBJ\_FILE
- DSM\_OBJ\_DIRECTORY
- DSM\_OBJ\_ANY\_TYPE

For more information about high-level and low-level names, see the following topic: [“High-level and low-level names” on page 21.](#)

### **owner**

The owner name of the object.

### **objState**

You can query for one of the following object states:

- DSM\_ACTIVE



- DSM\_INACTIVE
- DSM\_ANY\_MATCH

### pitDate

The point-in-time value. A query with this field returns the most recent object that is backed up before this date and time. The objState can be active or inactive. Objects that are deleted before the pitDate are not returned. For example:

```
Mon - backup ABC(1), DEF, GHI
Tue - backup ABC(2), delete DEF
Thu - backup ABC(3)
```

On Friday, call the query with a point-in-time value of Wednesday at 12:00:00 a.m. The call returns the following information:

```
ABC(2) - an Inactive copy
GHI    - an Active copy
```

The call does not return DEF because that object was deleted prior to the point-in-time value.

### qryABackupData

#### objName

The complete object name. You can use a wildcard character, such as an asterisk (\*) or a question mark (?), in the high-level or low-level portion of the name. An asterisk matches zero or more characters, and a question mark matches one character. The objType field of objName can have one of the following values:

- DSM\_OBJ\_FILE
- DSM\_OBJ\_DIRECTORY
- DSM\_OBJ\_ANY\_TYPE

For more information about high-level and low-level names, see the following topic: [“High-level and low-level names” on page 21.](#)

### qryFSData

#### fsName

Enter the name of a specific file space in this field, or enter an asterisk (\*) to retrieve information about all registered file spaces.

### qryMCData

#### mcName

Enter the name of a specific management class, or enter an empty string (" ") to retrieve information about all management classes.

**Note:** You cannot use an asterisk (\*).

#### mcDetail

Determines whether information on the backup and archive copy groups of the management class is returned. The following values are valid:

- bTrue
- bFalse

### qryBackupGroup:

#### groupType

The group type is DSM\_GROUPTYPE\_PEER.

#### fsName

The file space name.

**owner**

The owner ID.

**groupLeaderObjId**

The group leader object ID.

**objType**

The object type.

**qryProxyNodeAuth:****targetNodeName**

The target node name.

**peerNodeName**

The peer node name.

**hlAddress**

The peer address of the high-level name.

**llAddress**

The peer address of the low-level name.

**qryProxyNodePeer:****targetNodeName**

The target node name.

**peerNodeName**

The peer node name.

**hlAddress**

The peer address of the high-level name.

**llAddress**

The peer address of the low-level name.

**Return codes**

The following table describes the return codes for the **dsmBeginQuery** function call.

*Table 21. Return codes for dsmBeginQuery*

| Return code                 | Return code number | Explanation                                         |
|-----------------------------|--------------------|-----------------------------------------------------|
| DSM_RC_NO_MEMORY            | 102                | There is not enough memory to complete the request. |
| DSM_RC_FILE_SPACE_NOT_FOUND | 124                | The specified file space was not found.             |
| DSM_RC_NO_POLICY_BLK        | 2007               | Server policy information was not available.        |
| DSM_RC_INVALID_OBJTYPE      | 2010               | Invalid object type.                                |
| DSM_RC_INVALID_OBJOWNER     | 2019               | Invalid object owner name.                          |
| DSM_RC_INVALID_OBJSTATE     | 2024               | Invalid object condition.                           |

Table 21. Return codes for *dsmBeginQuery* (continued)

| Return code               | Return code number | Explanation                                                                                          |
|---------------------------|--------------------|------------------------------------------------------------------------------------------------------|
| DSM_RC_WRONG_VERSION_PARM | 2065               | The API version of the application client is different from the IBM Storage Protect library version. |

## dsmBeginTxn

The **dsmBeginTxn** function call begins one or more IBM Storage Protect transactions that begin a complete action; either all the actions succeed or none succeed. An action can be either a single call or a series of calls. For example, a **dsmSendObj** call that is followed by a number of **dsmSendData** calls can be considered a single action. Similarly, a **dsmSendObj** call with a **dataBlkPtr** that indicates a data area containing the object to back up is also considered a single action.

Try to group more than one object together in a single transaction for data transfer operations. Grouping objects results in significant performance improvements in the IBM Storage Protect system. From both a client and a server perspective, a certain amount of overhead is incurred by starting and ending each transaction.

There are limits to what you can perform within a single transaction. These restrictions include:

- A maximum number of objects that you can send or delete in a single transaction. This limit is located in the data that **dsmQuerySessInfo** returns in the *ApiSessInfo.maxObjPerTxn* field. This corresponds to the *TxnGroupMax* server option.
- All objects that are sent to the server (either backup or archive) within a single transaction must have the same copy destination that is defined in the management class binding for the object. This value is located in the data that **dsmBindMC** returns in the **mcBindKey.backup\_copy\_dest** or **mcBindKey.archive\_copy\_dest** fields.

With the API, either the application client can monitor and control these restrictions, or the API can monitor these restrictions. If the API is monitoring restrictions, appropriate return codes from the API calls inform the application client when one or more restrictions are reached.

Always match a **dsmBeginTxn** call with a **dsmEndTxn** call to optimize the set of actions within a pair of **dsmBeginTxn** and **dsmEndTxn** calls.

## Syntax

```
dsInt16_t dsmBeginTxn (dsUInt32_t dsmHandle);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 22. Return codes for *dsmBeginTxn*

| Return code                           | Explanation                                              |
|---------------------------------------|----------------------------------------------------------|
| DSM_RC_ABORT_NODE_NOT_AUTHORIZED (36) | FROMNODE or FROMOWNER is not allowed for TXN operations. |

# dsmBindMC

The **dsmBindMC** function call associates, or binds, a management class to the passed object. The object is passed through the include-exclude list that is pointed to in the options file. If a match is not found in the Include list for a specific management class, the default management class is assigned. The Exclude list can prevent objects from a backup but not from an archive.

The application client can use the parameters that are returned in the **mcBindKey** structure to determine if this object should be backed up or archived, or whether a new transaction must be started because of different copy destinations. See **dsmBeginTxn** for more information.

Call **dsmBindMC** before you call **dsmSendObj** because every object must have a management class associated with it. This call can be performed within a transaction or outside of a transaction. For example, within a multiple object transaction, if **dsmBindMC** indicates that the object has a different copy destination than the previous object, the transaction must be ended and a new transaction started. In this case, another **dsmBindMC** is not required because one has already been performed for this object.

## Syntax

```
dsInt16_t dsmBindMC (dsUInt32_t      dsmHandle,
                    dsmObjName *objNameP,
                    dsmSendType  sendType,
                    mcBindKey   *mcBindKeyP);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmObjName \*objNameP (I)

A pointer to the structure that contains the file space name, high-level object name, low-level object name, and object type.

### dsmSendType sendType (I)

Identifies whether this management class bind is performed for archive or backup sends. The possible values for this call include:

| Name                      | Description       |
|---------------------------|-------------------|
| <b>stBackup</b>           | A backup object   |
| <b>stArchive</b>          | An archive object |
| <b>stBackupMountWait</b>  | A backup object   |
| <b>stArchiveMountWait</b> | An archive object |

For the **dsmBindMC** call, **stBackup** and **stBackupMountWait** are equivalent, and **stArchive** and **stArchiveMountWait** are equivalent.

### mcBindKey \*mcBindKeyP (O)

This is the address of an **mcBindKey** structure where the management class information is returned. The application client can use the information that is returned here to determine if this object fits within a multiple object transaction, or to perform a management class query on the management class that is bound to the object.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 23. Return codes for *dsmBindMC*

| Return code                      | Explanation                                                                               |
|----------------------------------|-------------------------------------------------------------------------------------------|
| DSM_RC_NO_MEMORY (102)           | There is no RAM remaining to complete the request.                                        |
| DSM_RC_INVALID_PARM (109)        | One of the parameters that was passed has an invalid value.                               |
| DSM_RC_TL_NOBCG (184)            | The management class for this file has no valid backup copy group.                        |
| DSM_RC_TL_EXCLUDED (185)         | The backup object is excluded and cannot be sent.                                         |
| DSM_RC_TL_NOACG (186)            | The management class for this file has no valid archive copy group.                       |
| DSM_RC_INVALID_OBJTYPE (2010)    | Invalid object type.                                                                      |
| DSM_RC_INVALID_SENDDTYPE (2022)  | Invalid send type.                                                                        |
| DSM_RC_WRONG_VERSION_PARM (2065) | Application client API version is different from the IBM Storage Protect library version. |

**Note:** Ensure that a valid copy group exists for a management class, otherwise the **dsmBindMC** function can return DSM\_RC\_TL\_NOBCG (184) or DSM\_RC\_TL\_NOACG (186) error messages.

## dsmChangePW

The **dsmChangePW** function call changes an IBM Storage Protect password. On a multiple-user operating system such as UNIX or Linux, only the root user or the authorized user can use this call.

On Windows operating systems, you can specify the password in the dsm.opt file. In this situation, **dsmChangePW** does not update the dsm.opt file. After the call to **dsmChangePW** is made, you must update the dsm.opt file separately.

This call must process successfully if **dsmInitEx** returns DSM\_RC\_VERIFIER\_EXPIRED. The session ends if the **dsmChangePW** call fails in this situation.

If **dsmChangePW** is called for some other reason, the session remains open regardless of the return code.

## Syntax

```
dsInt16_t dsmChangePW (dsUInt32_t dsmHandle,
    char *oldPW,
    char *newPW);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### char \*oldPW (I)

The old password of the caller. The maximum length is DSM\_MAX\_VERIFIER\_LENGTH.

### char \*newPW (I)

The new password of the caller. The maximum length is DSM\_MAX\_VERIFIER\_LENGTH.

## Return codes

The return code numbers are provided in parentheses ().

Table 24. Return codes for *dsmChangePW*

| Return code                   | Explanation                                               |
|-------------------------------|-----------------------------------------------------------|
| DSM_RC_ABORT_BAD_VERIFIER (6) | An incorrect password was entered.                        |
| DSM_RC_AUTH_FAILURE (137)     | Authentication failure. Old password is incorrect.        |
| DSM_RC_NEWPW_REQD (2030)      | A value must be entered for the new password.             |
| DSM_RC_OLDPW_REQD (2031)      | A value must be entered for the old password.             |
| DSM_RC_PASSWD_TOOLONG (2103)  | The specified password is too long.                       |
| DSM_RC_NEED_ROOT (2300)       | The API caller must be a root user or an authorized user. |

## dsmCleanUp

The **dsmCleanUp** function call is used if **dsmSetUp** was called. The **dsmCleanUp** function call should be called after **dsmTerminate**. You cannot make any other calls after you call **dsmCleanUp**.

There are no return codes that are specific to this call.

### Syntax

```
dsInt16_t DSMLINKAGE dsmCleanUp
(dsBool_t mtFlag);
```

### Parameters

#### dsBool\_t mtFlag (I)

This parameter specifies that the API was used either in a single thread or a multithread mode. Possible values include:

- DSM\_SINGLETHREAD
- DSM\_MULTITHREAD

## dsmDeleteAccess

The **dsmDeleteAccess** function call deletes current authorization rules for backup versions or archived copies of your objects. When you delete an authorization rule, you revoke the access a user has to any files that are specified by the rule.

When you use **dsmDeleteAccess**, you can only delete one rule at a time. Obtain the rule ID through the **dsmQueryAccess** command.

There are no return codes that are specific to this call.

### Syntax

```
dsInt16_t DSMLINKAGE dsmDeleteAccess
(dsUInt32_t dsmHandle,
 dsUInt32_t ruleNum) ;
```

### Parameters

#### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### **dsUInt32\_t ruleNum (I)**

The rule ID for the access rule that is deleted. This value is obtained from a **dsmQueryAccess** function call.

## **dsmDeleteFS**

The **dsmDeleteFS** function call deletes a file space from storage. To delete a file space, you must have the appropriate permissions that your IBM Storage Protect administrator gave you. To determine whether you have the necessary permissions, call **dsmQuerySessInfo**. This function call returns a data structure of type *ApiSessInfo*, that includes two fields, *archDel* and *backDel*.

### **Note:**

- On a UNIX or Linux operating system, only a root user or an authorized user can delete a file space.
- If the file space that you need to delete contains backup versions, you must have backup delete authority (**backDel** = BACKDEL\_YES). If the file space contains archive copies, you must have archive delete authority (*archDel* = ARCHDEL\_YES). If the file space contains both backup versions and archive copies, you must have both types of delete authority.
- When using an archive manager server, a file space cannot actually be removed. This function call returns *rc=0* even though the file space was not actually deleted. The only way to verify that the file space has been deleted is to issue a filespace query to the server.
- The IBM Storage Protect server delete file-space function is a background process. If errors other than those detected before passing a return code happen, they are recorded in the IBM Storage Protect server log.

## **Syntax**

```
dsInt16_t dsmDeleteFS (dsUInt32_t      dsmHandle,  
                      char             *fsName,  
                      unsigned char    repository);
```

## **Parameters**

### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **char \*fsName (I)**

A pointer to the file space name to delete. The wildcard character is not permitted.

### **unsigned char repository (I)**

Indicates whether the file space to delete is a backup repository, archive repository, or both. The possible values for this field include:

```
DSM_ARCHIVE_REP    /* archive repository */  
DSM_BACKUP_REP     /* backup repository */  
DSM_REPOS_ALL      /* all repository types */
```

## **Return codes**

The return code numbers are provided in parentheses ( ).

Table 25. Return codes for *dsmDeleteFS*

| Return code                      | Explanation                                                       |
|----------------------------------|-------------------------------------------------------------------|
| DSM_RC_ABORT_NOT_AUTHORIZED (27) | You do not have the necessary authority to delete the file space. |
| DSM_RC_INVALID_REPOS (2015)      | Invalid value for repository.                                     |

Table 25. Return codes for `dsmDeleteFS` (continued)

| Return code                   | Explanation                     |
|-------------------------------|---------------------------------|
| DSM_RC_FSNAME_NOTFOUND (2060) | File space name not found.      |
| DSM_RC_NEED_ROOT (2300)       | API caller must be a root user. |

## dsmDeleteObj

The **dsmDeleteObj** function call inactivates backup objects, deletes backup objects, or it deletes archive objects in storage. The **dtBackup** type inactivates the currently active backup copy only. The **dtBackupID** type removes from the server whichever object ID is specified. Call this function from within a transaction.

See **dsmBeginTxn** for more information.

**Restriction:** You cannot delete backup objects that are contained in a retention set. To satisfy long-term data retention requirements, these files remain in server storage and expire according to the retention set's own expiration date after which they are eligible for deletion.

Before you send **dsmDeleteObj**, send the query sequence that is described in “Querying the IBM Storage Protect system” on page 28 to obtain the information for **delInfo**. The call to **dsmGetNextQObj** returns a data structure named **qryRespBackupData** for backup queries or **qryRespArchiveData** for archive queries. These data structures contain the information that you need for **delInfo**.

The value of **maxObjPerTxn** determines the maximum number of objects that you can delete in a single transaction. To obtain this value, call **dsmQuerySessInfo**.

**Tip:** Your node must have the appropriate permission that your administrator set. To delete archive objects, you must have archive delete authority. You do not need backup delete authority to inactivate a backup object.

## Syntax

```
dsInt16_t dsmDeleteObj (dsUInt32_t      dsmHandle,
                        dsmDelType delType,
                        dsmDelInfo delInfo)
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmDelType delType (I)

Indicates what type of object (backup or archive) to delete. Possible values include:

| Name | Description |
|------|-------------|
|------|-------------|

|                  |                                               |
|------------------|-----------------------------------------------|
| <b>dtArchive</b> | The object to delete was previously archived. |
|------------------|-----------------------------------------------|

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <b>dtBackup</b> | The object to inactivate was previously backed up. |
|-----------------|----------------------------------------------------|

|                   |                                                |
|-------------------|------------------------------------------------|
| <b>dtBackupID</b> | The object to delete was previously backed up. |
|-------------------|------------------------------------------------|

**Restriction:** Using this **delType** with *objID* removes the backup object from the server. Only an owner of an object can delete it.

You can delete any version (active or inactive) of an object. The server reconciles the versions. If you delete an active version of an object, the first inactive version becomes active. If you delete an inactive version of an object, all older versions will advance. The node must be registered with **backDel** permission.



### **dsmDelInfo delInfo (I)**

A structure whose fields identify the object. The fields are different, depending on whether the object is a backup object or an archive object. The structure to inactivate a backup object, **delBack**, contains the object name and the object copy group. The structure for an archive object, **delArch**, contains the object ID.

The structure to remove a backup object, **delBackID**, contains the object ID.

### **Return codes**

The return code numbers are provided in parentheses ( ).

*Table 26. Return codes for dsmDeleteObj*

| <b>Return code</b>               | <b>Explanation</b>                                                                        |
|----------------------------------|-------------------------------------------------------------------------------------------|
| DSM_RC_FS_NOT_REGISTERED (2061)  | File space name is not registered.                                                        |
| DSM_RC_WRONG_VERSION_PARM (2065) | Application client API version is different from the IBM Storage Protect library version. |

### **dsmEndGetData**

The **dsmEndGetData** function call ends a **dsmBeginGetData** session that obtains objects from storage.

The **dsmEndGetData** function call starts after all objects that you want to restore are processed, or ends the get process prematurely. Call **dsmEndGetData** to end a **dsmBeginGetData** session before you can continue other processing.

Depending on when **dsmEndGetData** is called, the API might need to finish processing a partial data stream before the process can be stopped. The caller, therefore, should not expect an immediate return from this call. Use **dsmTerminate** if the application needs to close the session and end the restore immediately.

There are no return codes that are specific to this call.

### **Syntax**

```
dsInt16_t dsmEndGetData (dsUInt32_t dsmHandle);
```

### **Parameters**

#### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **dsmEndGetDataEx**

The **dsmEndGetDataEx** function call provides the total of LAN-free bytes that were sent. It is an extension of the **dsmEndGetData** function call.

### **Syntax**

There are no return codes that are specific to this call.

```
dsInt16_t dsmEndGetDataEx (dsmEndGetDataExIn_t * dsmEndGetDataExInP,  
                           dsmEndGetDataExOut_t * dsmEndGetDataExOutP);
```

## Parameters

### **dsmEndGetDataExIn\_t \*dsmEndGetDataExInP (I)**

Passes the end get object dsmHandle that identifies the session and associates it with subsequent calls.

### **dsmEndGetDataExOut\_t \*dsmEndGetDataExOutP (O)**

This structure contains this input parameter:

#### **totalLFBytesRecv**

The total LAN-free bytes that are received.

## dsmEndGetObj

The **dsmEndGetObj** function call ends a **dsmGetObj** session that obtains data for a specified object.

Start the **dsmEndGetObj** call after an end of data is received for the object. This indicates that all data was received, or that no more data will be received for this object. Before you can start another **dsmGetObj** call, you must call **dsmEndGetObj**.

Depending on when **dsmEndGetObj** is called, the API might need to finish processing a partial data stream before the process can stop. Do not expect an immediate return from this call.

## Syntax

```
dsInt16_t dsmEndGetObj (dsUInt32_t dsmHandle);
```

## Parameters

### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

## Return codes

The return code numbers are provided in parentheses ().

Table 27. Return codes for dsmEndGetObj

| Return code            | Explanation                                        |
|------------------------|----------------------------------------------------|
| DSM_RC_NO_MEMORY (102) | There is no RAM remaining to complete the request. |

## dsmEndQuery

The **dsmEndQuery** function call signifies the end of a **dsmBeginQuery** action. The application client sends **dsmEndQuery** to complete a query. This call either is sent after all query responses are obtained through **dsmGetNextQObj**, or it is sent to end a query before all data are returned.

**Tip:** IBM Storage Protect continues to send the query data from the server to the client in this case, but the API discards any remaining data.

Once a **dsmBeginQuery** is sent, a **dsmEndQuery** must be sent before any other activity can start.

There are no return codes that are specific to this call.

## Syntax

```
dsInt16_t dsmEndQuery (dsUInt32_t dsmHandle);
```

## Parameters

### **dsUint32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

## **dsmEndSendObj**

The **dsmEndSendObj** function call indicates the end of data that is sent to storage.

Enter the **dsmEndSendObj** function call to indicate the end of data from the **dsmSendObj** and **dsmSendData** calls. A protocol violation occurs if this is not performed.

## Syntax

```
dsInt16_t dsmEndSendObj (dsUint32_t dsmHandle);
```

## Parameters

### **dsUint32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 28. Return codes for *dsmEndSendObj*

| Return code            | Explanation                                         |
|------------------------|-----------------------------------------------------|
| DSM_RC_NO_MEMORY (102) | There is no RAM remaining to complete this request. |

## **dsmEndSendObjEx**

The **dsmEndSendObjEx** function call provides additional information regarding the number of bytes processed. The information includes: total bytes sent, compression information, lan-free bytes, and deduplication information.

The **dsmEndSendObjEx** function call is an extension of the **dsmEndSendObj** function call.

## Syntax

```
dsInt16_t dsmEndSendObjEx (dsmEndSendObjExIn_t *dsmEndSendObjExInP,  
                           dsmEndSendObjExOut_t *dsmEndSendObjExOutP);
```

## Parameters

### **dsmEndSendObjExIn\_t \*dsmEndSendObjExInP (I)**

This parameter passes the end send object dsmHandle that identifies the session and associates it with subsequent calls.

### **dsmEndSendObjExOut\_t \*dsmEndSendObjExOutP (O)**

This parameter passes the end send object information:

| Name                  | Description                                                   |
|-----------------------|---------------------------------------------------------------|
| <b>totalBytesSent</b> | The total number of bytes that are read from the application. |

| Name                       | Description                                                     |
|----------------------------|-----------------------------------------------------------------|
| <b>objCompressed</b>       | A flag that displays if the object was compressed.              |
| <b>totalCompressedSize</b> | The total byte size after compression.                          |
| <b>totalLFBytesSent</b>    | The total LAN-free bytes that were sent.                        |
| <b>objDeduplicated</b>     | A flag that displays if the object was deduplicated by the API. |
| <b>totalDedupSize</b>      | Total bytes sent after deduplication.                           |

## Return codes

The return code numbers are provided in parentheses ( ).

Table 29. Return codes for *dsmEndSendObjEx*

| Return code            | Explanation                                         |
|------------------------|-----------------------------------------------------|
| DSM_RC_NO_MEMORY (102) | There is no RAM remaining to complete this request. |

## dsmEndTxn

The **dsmEndTxn** function call ends an IBM Storage Protect transaction.

Pair the **dsmEndTxn** function call with **dsmBeginTxn** to identify the call or set of calls that are considered a transaction.

The application client can specify on the **dsmEndTxn** call whether the transaction must be committed or ended.

Perform all of the following calls within the bounds of a transaction:

- **dsmSendObj**
- **dsmSendData**
- **dsmEndSendObj**
- **dsmDeleteObj**

## Syntax

```
dsInt16_t dsmEndTxn (dsUInt32_t dsmHandle,
                    dsUInt8_t vote,
                    dsUInt16_t *reason);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsUInt8\_t vote (I)

Indicates whether the application client commits all the actions that are done between the previous **dsmBeginTxn** call and this call. The following values are possible:

```
DSM_VOTE_COMMIT    /* commit current transaction */
DSM_VOTE_ABORT     /* roll back current transaction */
```

Use DSM\_VOTE\_ABORT only if your application finds a reason to stop the transaction.

### **dsUInt16\_t \*reason (0)**

If the call to **dsmEndTxn** ends with an error, or the value of vote is not agreed to, this parameter has a reason code that indicates why the vote failed. The return code for the call might be zero, and the reason code might be non-zero. Therefore, the application client must always check for errors on both the return code and the reason (`if (rc || reason)`) before you can assume a successful completion.

If the application specifies a vote of `DSM_VOTE_ABORT`, the reason code is `DSM_RS_ABORT_BY_CLIENT (3)`. See “API return codes source file: `dsmrc.h`” on page 131 for a list of the possible reason codes. Numbers 1 through 50 in the return codes list are reserved for the reason codes. If the server ends the transaction, the return code is `DSM_RC_CHECK_REASON_CODE`. In this case, the reason value contains more information on the cause of the abort.

## **Return codes**

The return code numbers are provided in parentheses ( ).

*Table 30. Return codes for `dsmEndTxn`*

| <b>Return code</b>                                    | <b>Explanation</b>                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DSM_RC_ABORT_CRC_FAILED (236)</code>            | The CRC that was received from the server does not match the CRC that was calculated by the client.                                                                                                                                                                                                                                                                                                                                                            |
| <code>DSM_RC_INVALID_VOTE (2011)</code>               | The value that was specified for vote is not valid.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>DSM_RC_CHECK_REASON_CODE (2302)</code>          | The transaction was aborted. Check the reason field.                                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>DSM_RC_ABORT_STGPPOOL_COPY_CONT_NO (241)</code> | The write to one of the copy storage pools failed, and the IBM Storage Protect storage pool option <code>COPYCONTINUE</code> is set to <code>NO</code> . The transaction terminates.                                                                                                                                                                                                                                                                           |
| <code>DSM_RC_ABORT_RETRY_SINGLE_TXN (242)</code>      | <p>This abort code indicates that the current transaction was aborted because of a problem during a store operation. The problem can be resolved by sending each file in an individual transaction. This error is typical in the following circumstances:</p> <ul style="list-style-type: none"><li>• The next storage pool has a different copy storage pool list.</li><li>• The operation is switched to this pool in the middle of a transaction.</li></ul> |

## **dsmEndTxnEx**

The **dsmEndTxnEx** function call provides group leader object ID information for you to use with the **dsmGroupHandler** function call. It is an extension of the **dsmEndTxn** function call.

### **Syntax**

```
dsInt16_t dsmEndTxnEx (dsmEndTxnExIn_t *dsmEndTxnExInP  
                      dsmEndTxnExOut_t *dsmEndTxnExOutP);
```

### **Parameters**

#### **dsmEndTxnExIn\_t \*dsmEndTxnExInP (1)**

This structure contains the following parameters:

## **dsmHandle**

The handle that identifies the session and associates it with subsequent IBM Storage Protect calls.

## **dsUInt8\_t vote (I)**

Indicates whether or not the application client commits all the actions that are done between the previous **dsmBeginTxn** call and this call. The possible values are:

```
DSM_VOTE_COMMIT    /* commit current transaction */
DSM_VOTE_ABORT     /* roll back current transaction */
```

Use DSM\_VOTE\_ABORT only if your application has found a reason to stop the transaction.

## **dsmEndTxnExOut\_t \*dsmEndTxnExOutP (O)**

This structure contains the following parameters:

### **dsUInt16\_t \*reason (O)**

If the call to **dsmEndTxnEx** ends with an error or the value of *vote* is not agreed to, this parameter has a reason code indicating why the vote failed.

**Tip:** The return code for the call might be zero, and the reason code might be non-zero. Therefore, the application client must always check for errors on both the return code and the reason (if `(rc || reason)`) before you can assume a successful completion.

If the application specifies a vote of DSM\_VOTE\_ABORT, the reason code is DSM\_RS\_ABORT\_BY\_CLIENT (3). See [“API return codes source file: dsmsrc.h” on page 131](#) for a list of the possible reason codes. Numbers 1 through 50 in the return codes list are reserved for the reason codes. If the server ends the transaction, the return code is DSM\_RC\_CHECK\_REASON\_CODE. In this case, the reason value contains more information on the cause of the abort.

### **groupLeaderObjId**

The group leader object ID that is returned when the DSM\_ACTION\_OPEN flag is used with the **dsmGroupHandler** call.

## **Return codes**

The return code numbers are provided in parentheses ( ).

*Table 31. Return codes for dsmEndTxnEx*

| <b>Return code</b>                      | <b>Explanation</b>                                                                                                                                                                                              |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_INVALID_VOTE (2011)              | The value that was specified for vote is invalid.                                                                                                                                                               |
| DSM_RC_CHECK_REASON_CODE (2302)         | The transaction was aborted. Check the reason field.                                                                                                                                                            |
| DSM_RC_ABORT_STGPOOL_COPY_CONT_NO (241) | The write to one of the copy storage pools failed, and the IBM Storage Protect storage pool option COPYCONTINUE was set to NO. The transaction terminates.                                                      |
| DSM_RC_ABORT_RETRY_SINGLE_TXN (242)     | During a simultaneous-write operation, an object in the transaction is going to a destination with different copy storage pools. End the current transaction and send each object again in its own transaction. |

## dsmGetData

The **dsmGetData** function call obtains a byte stream of data from IBM Storage Protect and places it in the caller's buffer. The application client calls **dsmGetData** when there is more data to receive from a previous **dsmGetObj** or **dsmGetData** call.

### Syntax

```
dsInt16_t dsmGetData (dsUInt32_t dsmHandle,  
DataBlk *dataBlkPtr);
```

### Parameters

#### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

#### DataBlk \*dataBlkPtr (I/O)

Points to a structure that includes both a pointer to the buffer for the data that is received and the size of the buffer. On return, this structure contains the number of bytes that is actually transferred. See [“API type definitions source files” on page 140](#) for the type definition.

### Return codes

The return code numbers are provided in parentheses ( ).

Table 32. Return codes for dsmGetData

| Return code                      | Explanation                                                                                                                                                                            |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_ABORT_INVALID_OFFSET (33) | The offset that was specified during a partial object retrieve is greater than the length of the object.                                                                               |
| DSM_RC_ABORT_INVALID_LENGTH (34) | The length that was specified during a partial object retrieve is greater than the length of the object, or the offset in addition to the length extends beyond the end of the object. |
| DSM_RC_FINISHED (121)            | Finished processing. The last buffer was received. Check numBytes for the amount of data and then call IBM Storage Protect <b>dsmEndGetObj</b> .                                       |
| DSM_RC_NULL_DATABLKPTR (2001)    | Datablock pointer is null.                                                                                                                                                             |
| DSM_RC_ZERO_BUFLen (2008)        | Buffer length is zero for datablock pointer.                                                                                                                                           |
| DSM_RC_NULL_BUFPtr (2009)        | Buffer pointer is null for datablock pointer.                                                                                                                                          |
| DSM_RC_WRONG_VERSION_PARM (2065) | The application client's API version is different from the IBM Storage Protect library version.                                                                                        |
| DSM_RC_MORE_DATA (2200)          | There is more data to get.                                                                                                                                                             |

## dsmGetBufferData

The **dsmGetBufferData** function call receives a byte stream of data from IBM Storage Protect through a buffer. After each call the application needs to copy the data and release the buffer through a call to **dsmReleaseBuffer**. If the number of buffers held by the application equals the numTsmBuffers specified in the **dsmInitEx** call, the **dsmGetBufferData** function blocks until a **dsmReleaseBuffer** is called.

## Syntax

```
dsInt16_t dsmGetBufferData (getDatatExIn_t *dsmGetBufferDataExInP,  
                           getDataExOut_t *dsmGetBufferDataExOutP) ;
```

## Parameters

### **getDataExIn\_t \* dsmGetBufferDataExInP (I)**

This structure contains the following input parameter.

#### **dsUInt32\_t dsmHandle**

The handle that identifies the session and associates it with a previous **dsmInitEx** call.

### **getDataExOut\_t \* dsmGetBufferDataExOutP (O)**

This structure contains the following output parameters.

#### **dsUInt8\_t tsmBufferHandle(0)**

The handle that identifies the buffer received.

#### **char \*dataPtr(0)**

The address to which the data was written.

#### **dsUInt32\_t numBytes(0)**

Actual number of bytes written by IBM Storage Protect.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 33. Return codes for *dsmGetBufferData*

| Return code                     | Explanation                                          |
|---------------------------------|------------------------------------------------------|
| DSM_RC_BAD_CALL_SEQUENCE (2041) | The call was not issued in the proper state.         |
| DSM_RC_OBJ_ENCRYPTED (2049)     | This function cannot be used for encrypted objects.  |
| DSM_RC_OBJ_COMPRESSED (2048)    | This function cannot be used for compressed objects. |
| DSM_RC_BUFF_ARRAY_ERROR (2045)  | A buffer array error occurred.                       |

## dsmGetNextQObj

The **dsmGetNextQObj** function call gets the next query response from a previous **dsmBeginQuery** call and places the response in the caller buffer.

The **dsmGetNextQObj** call is called one or more times. Each time the function is called, either a single query record is retrieved, or an error or a DSM\_RC\_FINISHED reason code is returned. If DSM\_RC\_FINISHED is returned, there is no more data to process. When all query data is retrieved, or if no more query data is needed, send the **dsmEndQuery** call to end the query process.

The **dataBlkPtr** parameter must point to a buffer that is defined with the **qryResp\*Data** structure type. The context in which **dsmGetNextQObj** is called determines the type of structure that is entered on the query response.

## Syntax

```
dsInt16_t dsmGetNextQObj (dsUInt32_t dsmHandle,  
                        DataBlk *dataBlkPtr);
```



## Parameters

### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **DataBlk \*dataBlkPtr (I/O)**

Points to a structure that includes both a pointer to the buffer for the data to be received and the size of the buffer. This buffer is the **qryResp\*Data** response structure. On return, this structure contains the number of bytes that is transferred. The structure that is associated with each type of query is described in the following table. For more information about the type definition of **DataBlk**, see the following topic: [“API type definitions source files”](#) on page 140.

*Table 34. DataBlk pointer structure*

| Query                 | Response structure        | Fields of special interest                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>qtArchive</b>      | <b>qryRespArchiveData</b> | <b>sizeEstimate</b><br>Contains the value that is passed on a previous <b>dsmSendObj</b> call.<br><br><b>mediaClass</b><br>Can have a value of MEDIA_FIXED if the object is on disk, or MEDIA_LIBRARY if the object is on tape.<br><br><b>clientDeduplicated</b><br>Indicates whether this object is deduplicated by the client.                                                                                                                                                                                                                                                                                                                              |
| <b>qtBackup</b>       | <b>qryRespBackupData</b>  | <b>restoreOrderExt</b><br>Is of type dsUInt16_t. Sort on this field when several objects are restored on a <b>dsmBeginGetData</b> call. An example of sorting code for this call is in the API sample, <code>dapiqry.c</code> . For a sorting example, see the following topic: <a href="#">Figure 16 on page 59</a> .<br><br><b>sizeEstimate</b><br>Contains the value that is passed on a previous <b>dsmSendObj</b> call.<br><br><b>mediaClass</b><br>Can have a value of MEDIA_FIXED if the object is on disk or MEDIA_LIBRARY if the object is on tape.<br><br><b>clientDeduplicated</b><br>Indicates whether this object is deduplicated by the client. |
| <b>qtBackupActive</b> | <b>qryARespBackupData</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>qtBackupGroups</b> | <b>qryRespBackupData</b>  | <b>dsBool_t isGroupLeader</b><br>If true, signifies this object is a group leader.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>qtOpenGroups</b>   | <b>qryRespBackupData</b>  | <b>dsBool_t isOpenGroup;</b><br>If true, signifies this group is open and not complete.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

Table 34. DataBlk pointer structure (continued)

| Query                  | Response structure                                                                                                   | Fields of special interest                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>qtFilespace</b>     | <b>qryRespFSDData</b>                                                                                                | <p><b>backStartDate</b><br/>Contains the server time stamp when the file space is updated with the <b>backStartDate</b> action.</p> <p><b>backCompleteDate</b><br/>Contains the server time stamp when the file space is updated with the <b>backCompleteDate</b> action.</p> <p><b>lastReplStartDate</b><br/>Contains the time stamp for the last time that replication was started on the server.</p> <p><b>lastReplCmpltDate</b><br/>Contains the time stamp for the last time that replication was completed, even if there was a failure.</p> <p><b>lastBackOpDateFromServer</b><br/>Contains the last store time stamp that was saved on the server.</p> <p><b>lastBackOpDateFromLocal</b><br/>Contains the last store time stamp that was saved on the client.</p> |
| <b>qtMC</b>            | <b>qryRespMCData</b><br><b>qryRespMCDetailData</b>                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>qtProxyNodeAuth</b> | <b>qryRespProxyNodeData</b><br><b>targetNodeName</b><br><b>peerNodeName</b><br><b>hlAddress</b><br><b>llAddress</b>  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>qtProxyNodePeer</b> | <b>qryRespProaxyNodeData</b><br><b>targetNodeName</b><br><b>peerNodeName</b><br><b>hlAddress</b><br><b>llAddress</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

## Return codes

The following table describes the return codes for the **dsmGetNextQObj** function call.

Table 35. Return codes for the **dsmGetNextQObj** function call

| Return code           | Return code number | Description                                                                        |
|-----------------------|--------------------|------------------------------------------------------------------------------------|
| DSM_RC_ABORT_NO_MATCH | 2                  | No match for the query was requested.                                              |
| DSM_RC_FINISHED       | 121                | Finished processing (start <b>dsmEndQuery</b> ). There is no more data to process. |

Table 35. Return codes for the **dsmGetNextQObj** function call (continued)

| Return code                | Return code number | Description                                                                                          |
|----------------------------|--------------------|------------------------------------------------------------------------------------------------------|
| DSM_RC_UNKNOWN_FORMAT      | 122                | The file that IBM Storage Protect attempted to restore or retrieve has an unknown format.            |
| DSM_RC_COMM_PROTOCOL_ERROR | 136                | Communication protocol error.                                                                        |
| DSM_RC_NULL_DATABLKPTR     | 2001               | Pointer is not pointing to a data block.                                                             |
| DSM_RC_INVALID_MCNAME      | 2025               | Invalid management class name.                                                                       |
| DSM_RC_BAD_CALL_SEQUENCE   | 2041               | The sequence of calls is invalid.                                                                    |
| DSM_RC_WRONG_VERSION_PARM  | 2065               | The version of the application client API is different from the IBM Storage Protect library version. |
| DSM_RC_MORE_DATA           | 2200               | There is more data to get.                                                                           |
| DSM_RC_BUFF_TOO_SMALL      | 2210               | Buffer is too small.                                                                                 |

## dsmGetObj

The **dsmGetObj** function call obtains the requested object data from the IBM Storage Protect data stream and places it in the caller's buffer. The **dsmGetObj** call uses the object ID to obtain the next object or partial object from the data stream.

The data for the indicated object is placed in the buffer to which **DataBlk** points. If more data is available, you must make one or more calls to **dsmGetData** to receive the remaining object data until a return code of DSM\_RC\_FINISHED is returned. Check the numBytes field in **DataBlk** to see whether any data remains in the buffer.

Objects should be asked for in the order that they were listed on the **dsmBeginGetData** call in the **dsmGetList** parameter. The exception is when the application client needs to pass over an object in the data stream to get to an object later in the list. If the object that is indicated by the object ID is not the next object in the stream, the data stream is processed until the object is located, or the stream is completed. Use this feature with care, because it might be necessary to process and discard large amounts of data to locate the requested object.

**Requirement:** If **dsmGetObj** returns a failure code (NOT FINISHED or MORE\_DATA), the session must be terminated to stop the restore operation. This is especially important when you use encryption and receive a RC\_ENC\_WRONG\_KEY. You must start a new session with the proper key.

## Syntax

```
dsInt16_t dsmGetObj (dsUInt32_t dsmHandle,
                    ObjID      *objIdP,
                    DataBlk    *dataBlkPtr);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### ObjID \*objIdP (I)

A pointer to the ID of the object to restore.

### DataBlk \*dataBlkPtr (I/O)

A pointer to the buffer where the restored data are placed.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 36. Return codes for *dsmGetObj*

| Return code                      | Explanation                                                                                                                                                                         |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_ABORT_INVALID_OFFSET (33) | The offset that is specified during a partial object retrieve is greater than the length of the object.                                                                             |
| DSM_RC_ABORT_INVALID_LENGTH (34) | The length that is specified during a partial object retrieve is greater than the length of the object, or the offset in addition to the length extends past the end of the object. |
| DSM_RC_FINISHED (121)            | Finished processing (start <b>dsmEndGetObj</b> ).                                                                                                                                   |
| DSM_RC_WRONG_VERSION_PARM (2065) | Application client's API version is different from the IBM Storage Protect library version.                                                                                         |
| DSM_RC_MORE_DATA (2200)          | There is more data to get.                                                                                                                                                          |
| RC_ENC_WRONG_KEY (4580)          | The key provided in the <b>dsmInitEx</b> call, or the saved key, does not match the key that was used to encrypt this object. Terminate the session and provide the proper key.     |

## dsmGroupHandler

The **dsmGroupHandler** function call performs an action on a logical file group depending on the input that is given. The client relates a number of individual objects together to reference and manage on the IBM Storage Protect server as a logical group.

For more information, see [“File grouping” on page 54](#).

### Syntax

```
dsInt16_t dsmGroupHandler (dsmGroupHandlerIn_t *dsmGroupHandlerInP,  
                           dsmGroupHandlerOut_t *dsmGroupHandlerOutP);
```

### Parameters

#### **dsmGroupHandlerIn\_t \*dsmGroupHandlerInP (I)**

Passes group attributes to the API.

##### **groupType**

The type of the group. Values include:

- DSM\_GROUPTYPE\_PEER - peer group

##### **actionType**

The action to be executed. Values include:

- DSM\_GROUP\_ACTION\_OPEN - creates a new group
- DSM\_GROUP\_ACTION\_CLOSE - commits and saves an open group
- DSM\_GROUP\_ACTION\_ADD - appends to a group
- DSM\_GROUP\_ACTION\_ASSIGNTO - assigns to another group
- DSM\_GROUP\_ACTION\_REMOVE - removes a member from a group

##### **memberType.**

The group type of the object. Values include:

- DSM\_MEMBertype\_LEADER - group leader
- DSM\_MEMBertype\_MEMBER - group member

**\*uniqueGroupTagP**

A unique string ID that is associated with a group.

**leaderObjId**

The Object ID for the group leader.

**\*objNameP**

A pointer to the object name of the group leader.

**memberObjList**

A list of objects to remove or assign.

**dsmGroupHandlerOut\_t \*dsmGroupHandlerOutP (O)**

Passes the address of the structure that the API completes. The structure version number is returned.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 37. Return codes for dsmGroupHandler

| Return code                                | Explanation                                                     |
|--------------------------------------------|-----------------------------------------------------------------|
| DSM_RC_ABORT_INVALID_GROUP_ACTION<br>(237) | An invalid operation was attempted on a group leader or member. |

## dsmInit

The **dsmInit** function call starts an API session and connects the client to IBM Storage Protect storage. The application client can have only one active session open at a time. To open another session with different parameters, use the **dsmTerminate** call first to end the current session.

To permit cross-node query and restore or retrieve, use the *-fromnode* and *-fromowner* string options. See [“Accessing objects across nodes and owners” on page 22](#) for more information.

## Syntax

```
dsInt16_t dsmInit (dsUInt32_t *dsmHandle,
    dsmApiVersion *dsmApiVersionP,
    char *clientNodeNameP,
    char *clientOwnerNameP,
    char *clientPasswordP,
    char *applicationType,
    char *configfile,
    char *options);
```

## Parameters

**dsUInt32\_t \*dsmHandle (O)**

The handle that identifies this initialization session and associates it with subsequent IBM Storage Protect calls.

**dsmApiVersion \*dsmApiVersionP (I)**

A pointer to the data structure identifying the version of the API that the application client is using for this session. The structure contains the values of the three constants, DSM\_API\_VERSION, DSM\_API\_RELEASE, and DSM\_API\_LEVEL, that are set in the dsmapi.h file. A previous call to **dsmQueryApiVersion** must be performed to ensure that compatibility exists between the application client API version and the version of the API library that is installed on the user's workstation.

### **char \*clientNodeNameP (I)**

This parameter is a pointer to the node for the IBM Storage Protect session. All sessions must have a node name associated with them. The constant, `DSM_MAX_NODE_LENGTH`, in the `dsmapitd.h` file sets the maximum size that is permitted for a node name.

The node name is not case-sensitive.

If this parameter is set both to NULL and *passwordaccess* is set to *prompt*, the API attempts to obtain the node name first from the options string that was passed. If it is not there, the API then attempts to obtain the node name from the configuration file or options files. If these attempts to find the node name fail, the UNIX or Linux API uses the system host name, while APIs on other operating systems return the `DSM_RC_REJECT_ID_UNKNOWN` code.

If this parameter is NULL, and the *passwordaccess* option in the `dsm.opt` file (for the Windows API) or in the `dsm.sys` file (for the UNIX or Linux API) is set to *generate*, the API uses the *nodename* option value or the system host name.

### **char \*clientOwnerNameP (I)**

This parameter is a pointer to the owner of the IBM Storage Protect session. If the operating system on which the session starts is a multi-user operating system, an owner name of NULL (the root user) has the authority to back up, archive, restore, or retrieve any objects belonging to the application, regardless of the owner of the object.

The owner name is case-sensitive.

If this parameter is NULL and the *passwordaccess* option in the `dsm.sys` file is set to *generate*, the API then uses the login user ID.

**Note:** On a multi-user operating system, it is not necessary for the owner name to match the active user ID of the session running the application.

### **char \*clientPasswordP (I)**

This parameter is a pointer to the password of the node on which the IBM Storage Protect session runs. The `DSM_MAX_VERIFIER_LENGTH` constant in the `dsmapitd.h` file sets the maximum size that is permitted for a password.

The password is not case-sensitive.

Except when the password file is first started, the value of this parameter is ignored if *passwordaccess* is set to *generate*.

### **char \*applicationType (I)**

This parameter identifies the application that is running the session. The application client defines the value.

Each time an API application client starts a session with the server, the application type (or platform) of the client is updated on the server. We recommend that the application type value contain an operating system abbreviation because this value is entered in the **platform** field on the server. The maximum string length is `DSM_MAX_PLATFORM_LENGTH`.

To see the current value of the application type, call **dsmQuerySessInfo**.

### **char \*configfile (I)**

This parameter points to a character string that contains the fully-qualified name of an API configuration file. Options specified in the API configuration file override their specification in the client options file. Options files are defined when IBM Storage Protect (client or API) is installed.

### **char \*options (I)**

Points to a character string that can contain user options such as:

- *Compressalways*
- *Servername* (UNIX or Linux only)
- *TCPServeraddr*
- *Fromnode*

- *Fromowner*
- *EnableClientEncryptKey*
- *Passwordaccess*

The application client can use the option list to override the values of these options that the configuration file sets.

The format of the options is:

1. Each option that is specified in the option list begins with a dash (-) and is followed by the option keyword.
2. The keyword, in turn, is followed by an equal sign (=) and then followed by the option parameter.
3. If the option parameter contains a blank space, enclose the parameter with single or double quotes.
4. If more than one option is specified, separate the options with blanks.

If options are NULL, values for all options are taken from the user options file or the API configuration file.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 38. Return codes for *dsmInit*

| Return code                         | Explanation                                                                                                                       |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_ABORT_SYSTEM_ERROR (1)       | The server has detected a system error and has notified the clients.                                                              |
| DSM_RC_REJECT_VERIFIER_EXPIRED (52) | Password has expired and must be updated.                                                                                         |
| DSM_RC_REJECT_ID_UNKNOWN (53)       | Could not find the node name.                                                                                                     |
| DSM_RC_AUTH_FAILURE (137)           | There was an authentication failure.                                                                                              |
| DSM_RC_NO_STARTING_DELIMITER (148)  | There is no starting delimiter in pattern.                                                                                        |
| DSM_RC_NEEDED_DIR_DELIMITER (149)   | A directory delimiter is needed immediately before and after the "match directories" meta-string ("...") and one was not located. |
| DSM_RC_NO_PASS_FILE (168)           | The password file is not available.                                                                                               |
| DSM_RC_UNMATCHED_QUOTE (177)        | An unmatched quote is in the option string.                                                                                       |
| DSM_RC_NLS_CANT_OPEN_TXT (0610)     | Unable to open the message text file.                                                                                             |
| DSM_RC_INVALID_OPT (400)            | An entry in the option string is invalid.                                                                                         |
| DSM_RC_INVALID_DS_HANDLE (2014)     | Invalid DSM handle.                                                                                                               |
| DSM_RC_NO_OWNER_REQD (2032)         | Owner parameter must be NULL when <i>passwordaccess</i> is set to <i>generate</i> .                                               |
| DSM_RC_NO_NODE_REQD (2033)          | Node parameter must be NULL when <i>passwordaccess</i> is set to <i>generate</i> .                                                |
| DSM_RC_WRONG_VERSION (2064)         | The API version for the application client has a higher value than the IBM Storage Protect version.                               |
| DSM_RC_PASSWD_TOOLONG (2103)        | The password that was specified is too long.                                                                                      |

Table 38. Return codes for *dsmInit* (continued)

| Return code                       | Explanation                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------------------|
| DSM_RC_NO_OPT_FILE (2220)         | A configuration file could not be located.                                                   |
| DSM_RC_INVALID_KEYWORD (2221)     | A keyword that was specified in an options string is invalid.                                |
| DSM_RC_PATTERN_TOO_COMPLEX (2222) | The include-exclude pattern is too complex for IBM Storage Protect to interpret.             |
| DSM_RC_NO_CLOSING_BRACKET (2223)  | There is no closing bracket in the pattern.                                                  |
| DSM_RC_INVALID_SERVER (2225)      | For a multi-user environment, the server in the system configuration file was not found.     |
| DSM_RC_NO_HOST_ADDR (2226)        | Not enough information to connect to host.                                                   |
| DSM_RC_MACHINE_SAME (2227)        | The nodename that is defined in the options file cannot be the same as the system host name. |
| DSM_RC_NO_API_CONFIGFILE (2228)   | Cannot open the configuration file.                                                          |
| DSM_RC_NO_INCLEXCL_FILE (2229)    | The include-exclude file was not found.                                                      |
| DSM_RC_NO_SYS_OR_INCLEXCL (2230)  | Either the <code>dsm.sys</code> file or the include-exclude file was not found.              |

#### Related information

[Client options file overview](#)

[Processing options](#)

## dsmInitEx

The **dsmInitEx** function call starts an API session by using the additional parameters for extended verification.

### Syntax

```
dsInt16_t dsmInitEx (dsUInt32_t *dsmHandleP,
                    dsmInitExIn_t *dsmInitExInP,
                    dsmInitExOut_t *dsmInitExOutP) ;
```

### Parameters

#### dsUInt32\_t \*dsmHandleP (O)

The handle that identifies this initialization session and associates it with subsequent IBM Storage Protect calls.

#### dsmInitExIn\_t \*dsmInitExInP

This structure contains the following input parameters:

#### dsmApiVersion \*dsmApiVersionP (I)

This parameter is a pointer to the data structure that identifies the version of the API that the application client is using for this session. The structure contains the values of the four constants, `DSM_API_VERSION`, `DSM_API_RELEASE`, `DSM_API_LEVEL`, and `DSM_API_SUBLEVEL` that are set in the `dsmapi.h` file. Call **dsmQueryApiVersionEx** and verify that the API version of the application client and the version of the API library that is installed on the user's workstation is compatible.



**char \*clientNodeNameP (I)**

This parameter is a pointer to the node for the IBM Storage Protect session. All sessions must be associated with a node name. The `DSM_MAX_NODE_LENGTH` constant in the `dsmapitd.h` file sets the maximum size for a node name.

The node name is not case-sensitive.

If this parameter is set to `NULL`, and **passwordaccess** is set to `prompt`, the API attempts to obtain the node name first from the options string that was passed. If it is not there, the API then attempts to obtain the node name from the configuration file or options files. If these attempts to find the node name fail, the UNIX or Linux API uses the system host name, while the APIs from other operating systems return `DSM_RC_REJECT_ID_UNKNOWN`.

If this parameter is `NULL`, and the **passwordaccess** option in the `dsm.opt` file (for the Windows API) or in the `dsm.sys` file (for the UNIX or Linux API) is set to `generate`, the API uses the `nodename` option value or the system host name.

**char \*clientOwnerNameP (I)**

This parameter is a pointer to the owner of the IBM Storage Protect session. If the operating system is a multi-user platform, an owner name of `NULL` (the root user) has the authority to back up, archive, restore, or retrieve any objects that belong to the application, regardless of the owner of the object.

The owner name is case-sensitive.

If this parameter is `NULL` and the **passwordaccess** option in the `dsm.sys` file is set to `generate`, the API then uses the login user ID.

**Note:** On a multi-user platform, it is not necessary for the owner name to match the active user ID of the session running the application.

**char \*clientPasswordP (I)**

A pointer to the password of the node on which the IBM Storage Protect session runs. The `DSM_MAX_VERIFIER_LENGTH` constant in the `dsmapitd.h` file sets the maximum size that is allowed for a password.

The password is not case-sensitive.

Except when the password file is first started, the value of this parameter is ignored if **passwordaccess** is set to `generate`.

**char \*userNameP;**

A pointer to the administrative user name that has client authority for this node.

**char \*userPasswordP;**

A pointer to the password for the **userName** parameter, if a value is supplied.

**char \*applicationType (I)**

Identifies the application that is running the IBM Storage Protect session. The application client identifies the value.

Each time an API application client starts a session with the server, the application type (or operating system) of the client is updated on the server. The value is entered in the **platform** field on the server. Consider using an operating system ID in the value. The maximum string length is defined in the `DSM_MAX_PLATFORM_LENGTH` constant.

To view the current value of the application type, call **dsmQuerySessInfo**.

**char \*configfile (I)**

Points to a character string that contains the fully qualified name of an API configuration file. Options that are specified in the API configuration file override their specification in the client options file. Options files are defined when IBM Storage Protect (client or API) is installed.

**char \*options (I)**

Points to a character string that can contain user options such as:

- `Compressalways`

- Servername (UNIX and Linux systems only)
- TCPServeraddr (not for UNIX systems)
- Fromnode
- Fromowner
- Passwordaccess

The application client can use the options list to override the values of these options that the configuration file sets.

Options have the following format:

1. Each option that is specified in the option list begins with a dash (-) and is followed by the option keyword.
2. The keyword is followed by an equal sign (=) and then the option parameter.
3. If the option parameter contains a blank space, enclose the parameter with single or double quotation marks.
4. If more than one option is specified, separate the options with blanks.

If options are NULL, the values for all options are taken from the user options file or the API configuration file.

#### **dirDelimiter**

The directory delimiter that is prefixed on the file space, high-level or low-level names. You must specify the **dirDelimiter** parameter only if the application overrides the system defaults. In a UNIX or Linux environment, the default is forward slash (/). In a Windows environment, the default is backslash (\).

#### **useUnicode**

A Boolean flag that indicates whether Unicode is enabled. The **useUnicode** flag must be false to achieve cross-platform interoperability between UNIX and Windows systems.

#### **bCrossPlatform**

A Boolean flag that must be set (bTrue) to achieve cross-platform interoperability between UNIX and Windows systems. When the bCrossPlatform flag is set, the API ensures that the file spaces are not Unicode and that the application does not use Unicode. A Windows application that uses Unicode is not compatible with applications that use non-Unicode encodings. The bCrossPlatform flag must not be set for a Windows application that uses Unicode.

#### **UseTsmBuffers**

Indicates whether to use buffer copy elimination.

#### **numTsmBuffers**

Number of buffers when useTsmBuffers=bTrue.

#### **bEncryptKeyEnabled**

Indicates whether encryption with application-managed key is used.

#### **encryptionPasswordP**

The encryption password.

**Restriction:** When encryptkey=save, if an encrypt key exists, the value that is specified in the **encryptionPasswordP** is ignored.

#### **dsmAppVersion \*appVersionP (I)**

This parameter is a pointer to the data structure that identifies the version information of the application that is starting an API session. The structure contains the values of the four constants, applicationVersion, applicationRelease, applicationLevel, and applicationSubLevel, which are set in the tsmapi.h file.

#### **dsmInitExOut\_t \*dsmInitExOut P**

This structure contains the output parameters.

**dsUInt32\_t \*dsmHandle (0)**

The handle that identifies this initialization session and associates it with subsequent API calls.

**infoRC**

Additional information about the return code. Check both the function return code and the value of **infoRC**. An **infoRC** value of DSM\_RC\_REJECT\_LASTSESS\_CANCELED (69), the IBM Storage Protect indicates that the administrator canceled the last session.

**Return codes**

The return code numbers are provided in parentheses ( ).

Table 39. Return codes for **dsmInitEx**

| Return code                         | Explanation                                                                                                                  |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_ABORT_SYSTEM_ERROR (1)       | The IBM Storage Protect server detected a system error and notified the clients.                                             |
| DSM_RC_REJECT_VERIFIER_EXPIRED (52) | Password expired and must be updated. The next call must be <b>dsmChangePW</b> with the handle returned on this call.        |
| DSM_RC_REJECT_ID_UNKNOWN (53)       | Cannot not find the node name.                                                                                               |
| DSM_RC_TA_COMM_DOWN (103)           | The communications link is down.                                                                                             |
| DSM_RC_AUTH_FAILURE (137)           | There was an authentication failure.                                                                                         |
| DSM_RC_NO_STARTING_DELIMITER (148)  | There is no starting delimiter in pattern.                                                                                   |
| DSM_RC_NEEDED_DIR_DELIMITER (149)   | A directory delimiter is needed immediately before and after the "match directories" meta-string ("..."), but was not found. |
| DSM_RC_NO_PASS_FILE (168)           | The password file is not available.                                                                                          |
| DSM_RC_UNMATCHED_QUOTE (177)        | An unmatched quotation mark is in the option string.                                                                         |
| DSM_RC_NLS_CANT_OPEN_TXT (0610)     | Unable to open the message text file.                                                                                        |
| DSM_RC_INVALID_OPT (2013)           | An entry in the option string is invalid.                                                                                    |
| DSM_RC_INVALID_DS_HANDLE (2014)     | Invalid DSM handle.                                                                                                          |
| DSM_RC_NO_OWNER_REQD (2032)         | Owner parameter must be NULL when <b>passwordaccess</b> is set to generate.                                                  |
| DSM_RC_NO_NODE_REQD (2033)          | Node parameter must be NULL when <b>passwordaccess</b> is set to generate.                                                   |
| DSM_RC_WRONG_VERSION (2064)         | Application client's API version has a higher value than the IBM Storage Protect version.                                    |
| DSM_RC_PASSWD_TOOLONG (2103)        | The specified password is too long.                                                                                          |
| DSM_RC_NO_OPT_FILE (2220)           | No configuration file is found.                                                                                              |
| DSM_RC_INVALID_KEYWORD (2221)       | A keyword that is specified in an options string is invalid.                                                                 |
| DSM_RC_PATTERN_TOO_COMPLEX (2222)   | Include-exclude pattern too complex to be interpreted by IBM Storage Protect.                                                |
| DSM_RC_NO_CLOSING_BRACKET (2223)    | There is no closing bracket in the pattern.                                                                                  |

Table 39. Return codes for **dsmInitEx** (continued)

| Return code                      | Explanation                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| DSM_RC_INVALID_SERVER (2225)     | For a multi-user environment, the server in the system configuration file was not found.      |
| DSM_RC_NO_HOST_ADDR (2226)       | Not enough information to connect to the host.                                                |
| DSM_RC_MACHINE_SAME (2227)       | The node name that is defined in the options file cannot be the same as the system host name. |
| DSM_RC_NO_API_CONFIGFILE (2228)  | Cannot open the configuration file.                                                           |
| DSM_RC_NO_INCLEXCL_FILE (2229)   | The include-exclude file was not found.                                                       |
| DSM_RC_NO_SYS_OR_INCLEXCL (2230) | Either the dsm.sys or the include-exclude file was not found.                                 |

#### Related information

[Client options file overview](#)

[Processing options](#)

## dsmLogEvent

The **dsmLogEvent** function call logs a user message (ANE4991 I) to the server log file, to the local error log, or to both. A structure of type **logInfo** is passed in the call. This call must be performed while at **InSession** state inside a session. Do not perform it within a send, get, or query. To retrieve messages logged on the server, use the **query actlog** command through the administrative client.

See the summary state diagram, [Figure 20 on page 66](#).

### Syntax

```
dsInt16_t dsmLogEvent
(dsUInt32_t dsmHandle,
logInfo *logInfoP);
```

### Parameters

#### dsUInt32\_t dsmHandle(I)

The handle that associates this call with a previous **dsmInitEx** call.

#### logInfo \*logInfoP (I)

Passes the message and destination. The application client is responsible for allocating storage for the structure.

The fields in the **logInfo** structure are:

#### message

The text of the message to be logged. This must be a null-ended string. The maximum length is DSM\_MAX\_RC\_MSG\_LENGTH.

#### dsmLogtype

Specifies where to log the message. Possible values include: **logServer**, **logLocal**, **logBoth**.

### Return codes

The return code numbers are provided in parentheses ().

Table 40. Return codes for *dsmLogEvent*

| Return code                   | Explanation                     |
|-------------------------------|---------------------------------|
| DSM_RC_STRING_TOO_LONG (2120) | The message string is too long. |

## dsmLogEventEx

The **dsmLogEventEx** function call logs a user message to the server log file, to the local error log, or to both. This call must be made while at an **InSession** state within a session. The call cannot be made within a send, get, or query call.

**Summary state diagram:** For an overview of the session interactions, see the summary state diagram in the following topic:

Figure 20 on page 66

The severity determines the IBM Storage Protect message number. To view messages that are logged on the server, use the **query actlog** command through the administrative client. Use the IBM Storage Protect client option, **errorlogretention**, to prune the client error log file if the application generates numerous client messages written to the client log, **dsmLogType** either **logLocal** or **logBoth**. For more information, see the IBM Storage Protect server documentation.

## Syntax

```
extern dsInt16_t DSMLINKAGE dsmLogEventEx(
    dsUInt32_t dsmHandle,
    dsmLogExIn_t *dsmLogExInP,
    dsmLogExOut_t *dsmLogExOutP
);
```

## Parameters

### dsUInt32\_t dsmHandle(I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmLogExIn\_t \*dsmLogExInP

This structure contains the input parameters.

### dsmLogSeverity severity;

This parameter is the event severity. The possible values are:

```
logSevInfo,      /* information ANE4990 */
logSevWarning,   /* warning     ANE4991 */
logSevError,     /* Error       ANE4992 */
logSevSevere     /* severe      ANE4993 */
```

### char appMsgID[8];

This parameter is a string to identify the specific application message. A suitable format is three characters that are followed by four numbers, for example: DSM0250.

### dsmLogType logType;

This parameter specifies where to direct the event. The parameter has the following possible values:

- logServer
- logLocal
- logBoth

### char \*message;

This parameter is the text of the event message to log. The text must be a null-ended string. The maximum length is **DSM\_MAX\_RC\_MSG\_LENGTH**.

**Restriction:** Messages that go to the server must be in English. Non-English messages do not display correctly.

### **dsmLogExOut\_t \*dsmLogExOutP**

This structure contains the output parameters. Currently, there are no output parameters.

## **Return codes**

The return code numbers are provided in parentheses ().

*Table 41. Return codes for dsmLogEventEx*

| Return code                   | Explanation                     |
|-------------------------------|---------------------------------|
| DSM_RC_STRING_TOO_LONG (2120) | The message string is too long. |

## **dsmQueryAccess**

The **dsmQueryAccess** function call queries the server for all access authorization rules for either backup versions or archived copies of your objects. A pointer to an array of access rules is passed in to the call, and the completed array is returned. A pointer to the number of rules is passed in to indicate how many rules are in the array.

There are no return codes that are specific to this call.

## **Syntax**

```
dsInt16_t DSMLINKAGE dsmQueryAccess
                    (dsUInt32_t      dsmHandle),
                    qryRespAccessData **accessListP,
                    dsUInt16_t      *numberOfRules) ;
```

## **Parameters**

### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **qryRespAccessData \*\*accessListP (O)**

A pointer to an array of **qryRespAccessData** elements that the API library allocates. Each element corresponds to an access rule. The number of elements in the array is returned in the **numberOfRules** parameter. The information that is returned in each **qryRespAccessData** element includes the following:

| Name              | Description                                                        |
|-------------------|--------------------------------------------------------------------|
| <b>ruleNumber</b> | The ID for the access rule. This identifies the rule for deletion. |
| <b>AccessType</b> | The backup or archive type.                                        |
| <b>Node</b>       | The node on which you gave access.                                 |
| <b>Owner</b>      | The user to whom you gave access.                                  |
| <b>objName</b>    | The high-level, or low-level file space descriptors.               |

### **dsUInt32\_t \*numberOfRules (O)**

Returns the number of rules in the **accessList** array.

## dsmQueryApiVersion

The **dsmQueryApiVersion** function call performs a query request for the API library version that the application client accesses.

All updates to the API are made in an upward-compatible format. Any application client with an API version or release less than, or equal to, the API library on the end user's workstation operates without change. Be aware before you proceed that should the **dsmQueryApiVersion** call return a version or version release older than that of the application clients, some API calls might be enhanced in a manner that is not supported by the end user's older version of the API.

The application API version number is stored in the dsmapi.h header file as constants DSM\_API\_VERSION, DSM\_API\_RELEASE, and DSM\_API\_LEVEL.

There are no return codes that are specific to this call.

### Syntax

```
void dsmQueryApiVersion (dsmApiVersion *apiVersionP);
```

### Parameters

#### dsmApiVersion \*apiVersionP (O)

This parameter is a pointer to the structure that contains the API library version, release, and level components. For example, if the library is version 1.1.0, then, after returning from the call, the fields of the structure contain the following values:

```
dsmApiVersionP->version = 1  
dsmApiVersionP->release = 1  
dsmApiVersionP->level   = 0
```

## dsmQueryApiVersionEx

The **dsmQueryApiVersionEx** function call performs a query request for the API library version that the application client accesses.

All updates to the API are made in an upward-compatible format. Any application client that has an API version or release less than or equal to the API library on the end user's workstation operates without change. See Summary of Code Changes in the README\_api\_enh file for exceptions to upward compatibility. If the **dsmQueryApiVersionEx** call returns a version or version release that is different from that of the application client, be aware before you proceed that some API calls might be enhanced in a manner that is not supported by the end user's older version of the API.

The application API version number is stored in the dsmapi.h header file as constants DSM\_API\_VERSION, DSM\_API\_RELEASE, DSM\_API\_LEVEL, and DSM\_API\_SUBLEVEL.

There are no return codes that are specific to this call.

### Syntax

```
void dsmQueryApiVersionEx (dsmApiVersionEx *apiVersionP);
```

### Parameters

#### dsmApiVersionEx \*apiVersionP (O)

This parameter is a pointer to the structure that contains the API library's version, release, level, and sublevel components. For example, if the library is version 5.5.0.0, then, after returning from the call, the fields of the structure contain the following values:

- `ApiVersionP->version = 5`
- `ApiVersionP->release = 5`
- `ApiVersionP->level = 0`
- `ApiVersionP->subLevel = 0`

## dsmQueryCliOptions

The **dsmQueryCliOptions** function call queries important option values in the user's option files. A structure of type **optStruct** is passed in the call and contains the information. This call is performed before **dsmInitEx** is called, and it determines the setup before the session.

There are no return codes that are specific to this call.

### Syntax

```
dsInt16_t dsmQueryCliOptions
(optStruct *optstructP);
```

### Parameters

#### **optStruct \*optstructP (I/O)**

This parameter passes the address of the structure that the API completes. The application client is responsible for allocating storage for the structure. On successful return, the appropriate information is entered in the fields in the structure.

The following information is returned in the **optStruct** structure:

| Name                  | Description                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>dsmiDir</b>        | The value of the environment <code>DSMI_DIR</code> variable.                                                       |
| <b>dsmiConfig</b>     | The client option file as specified by the <code>DSMI_CONFIG</code> environment variable.                          |
| <b>serverName</b>     | The name of the IBM Storage Protect server.                                                                        |
| <b>commMethod</b>     | The communication method selected. See the #defines for <code>DSM_COMM_*</code> in the <code>dsmapi.h</code> file. |
| <b>serverAddress</b>  | The address of the server that is based on the communication method.                                               |
| <b>nodeName</b>       | The client node (machine) name.                                                                                    |
| <b>compression</b>    | This field provides information regarding the compression option.                                                  |
| <b>passwordAccess</b> | The values are: <i>bTrue</i> for generate, and <i>bFalse</i> for prompt.                                           |

### Related information

[Processing options](#)

## dsmQuerySessInfo

The **dsmQuerySessInfo** function call starts a query request to IBM Storage Protect for information related to the operation of the specified session in **dsmHandle**. A structure of type **ApiSessInfo** is passed in the call, with all available session related information entered. This call is started after a successful **dsmInitEx** call.

The information that is returned in the **ApiSessInfo** structure includes the following:

- Server information: port number, date and time, and type



- Client defaults: application type, delete permissions, delimiters, and transaction limits
- Session information: login ID, and owner
- Policy data: domain, active policy set, and retention grace period

See “API type definitions source files” on page 140 for information about the content of the structure that is passed and each field within it.

## Syntax

```
dsInt16_t dsmQuerySessInfo (dsUInt32_t      dsmHandle,
                          ApiSessInfo *SessInfoP);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### ApiSessInfo \*SessInfoP (I/O)

This parameter passes the address of the structure that the API enters. The application client is responsible for allocating storage for the structure and for completing the field entries that indicate the version of the structure that is used. On successful return, the fields in the structure are completed with the appropriate information. The **adsmServerName** is the name that is given in the **define server** command on the IBM Storage Protect server. If the archiveRetentionProtection field is true, the server is enabled for retention protection.

## Return codes

The return code numbers are provided in parentheses ().

Table 42. Return codes for dsmQuerySessInfo

| Return code                      | Explanation                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------------------|
| DSM_RC_NO_SESS_BLK (2006)        | No server session block information.                                                        |
| DSM_RC_NO_POLICY_BLK (2007)      | No server policy information available.                                                     |
| DSM_RC_WRONG_VERSION_PARM (2065) | Application client's API version is different from the IBM Storage Protect library version. |

## dsmQuerySessOptions

The **dsmQuerySessOptions** function call queries important option values that are valid in the specified session in **dsmHandle**. A structure of type optStruct is passed in the call and contains the information.

This call is started after a successful **dsmInitEx** call. The values that are returned might be different from the values returned on a **dsmQueryCliOptions** call, depending on values that are passed to the **dsmInitEx** call, primarily optString, and optFile. For information about option precedence, see “Understanding configuration and options files” on page 1.

There are no return codes that are specific to this call.

## Syntax

```
dsInt16_t dsmQuerySessOptions
(dsUInt32_t      dsmHandle,
 optStruct *optstructP);
```

## Parameters

### **dsUInt32\_t dsmhandle(I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **optStruct \*optstructP (I/O)**

This parameter passes the address of the structure that the API completes. The application client is responsible for allocating storage for the structure. On successful return, the fields in the structure are completed with the appropriate information.

The information returned in the optStruct structure is:

| Name                  | Description                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------|
| <b>dsmiDir</b>        | The value of the DSMI_DIR environment variable.                                                   |
| <b>dsmiConfig</b>     | The dsm.opt file that the DSMI_CONFIG environment variable specifies.                             |
| <b>serverName</b>     | The name of the IBM Storage Protect server stanza in the options file.                            |
| <b>commMethod</b>     | The communication method that was selected. See the #defines for DSM_COMM_* in the dsmapi.h file. |
| <b>serverAddress</b>  | The address of the server that is based on the communication method.                              |
| <b>nodeName</b>       | The name of the client's node (machine).                                                          |
| <b>compression</b>    | The value of the compression option (bTrue=on and bFalse=off).                                    |
| <b>compressAlways</b> | The value of the compressalways option (bTrue=on and bFalse=off).                                 |
| <b>passwordAccess</b> | Value bTrue for generate, and bFalse for prompt.                                                  |

## Related information

[Processing options](#)

## dsmRCMsg

The **dsmRCMsg** function call obtains the message text that is associated with an API return code.

**Important:** If **dsmRCMsg** is called after a **dsmInitEx** fail, the Servername option will be ignored and an error will be printed in a log file, which is defined for the server by default.

The **msg** parameter displays the message prefix return code in parentheses ( ), followed by the message text. For example, a call to **dsmRCMsg** might return the following:

```
ANS0264E (RC2300) Only root user can execute dsmChangePW or dsmDeleteFS.
```

For some languages where characters are different in ANSI and OEM code pages, it might be necessary to convert strings from ANSI to OEM before printing them out (for example, Eastern European single-byte character sets). The following is an example:

```
dsmRCMsg(dsmHandle, rc, msgBuf);
#ifdef WIN32
#ifdef WIN64
CharToOemBuff(msgBuf, msgBuf, strlen(msgBuf));
#endif
#endif
printf("
```

## Syntax

```
dsInt16_t dsmRCMsg (dsUInt32_t dsmHandle,  
    dsInt16_t dsmRC,  
    char *msg);
```

### Parameters

#### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

#### **dsInt16\_t dsmRC (I)**

The API return code of the associated message text. The API return codes are listed in the dsmrc.h file. See [“API return codes source file: dsmrc.h” on page 131](#) for more information.

#### **char \*msg (O)**

This parameter is the message text that is associated with the return code, **dsmRC**. The caller is responsible for allocating enough space for the message text.

The maximum length for **msg** is defined as DSM\_MAX\_RC\_MSG\_LENGTH.

On platforms that have National Language Support and a choice of language message files, the API returns a message string in the national language.

### Return codes

The return code numbers are provided in parentheses ( ).

Table 43. Return codes for dsmRCMsg

| Return code                     | Explanation                                                             |
|---------------------------------|-------------------------------------------------------------------------|
| DSM_RC_NULL_MSG (2002)          | The <b>msg</b> parameter for dsmRCMsg call is a NULL pointer.           |
| DSM_RC_INVALID_RETCODE (2021)   | Return code that was passed to <b>dsmRCMsg</b> call is an invalid code. |
| DSM_RC_NLS_CANT_OPEN_TXT (0610) | Unable to open the message text file.                                   |

## dsmRegisterFS

The **dsmRegisterFS** function call registers a new file space with the IBM Storage Protect server. Register a file space first before you can back up any data to it.

Application clients should not use the same file space names that a backup-archive client would use.

- On UNIX or Linux, run the **df** command for these names.
- On Windows, these names are generally the volume labels that are associated with the different drives on your system.

## Syntax

```
dsInt16_t dsmRegisterFS (dsUInt32_t dsmHandle,  
    regFSData *regFilespaceP);
```

### Parameters

#### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### regFSData \*regFilespaceP (I)

This parameter passes the name of the file space and associated information that you need to register with the IBM Storage Protect server.

**Tip:** The *fstype* field includes the prefix, "**API:**". All file space queries display this string. For example, if the user passes *myfstype* for *fstype* in **dsmRegisterFS**, the actual value string on the server is returned as *API:myfstype* when queried. This prefix distinguishes API objects from backup-archive objects.

The usable area for **fsInfo** is now DSM\_MAX\_USER\_FSINFO\_LENGTH.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 44. Return codes for *dsmRegisterFS*

| Return code                      | Explanation                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------------------|
| DSM_RC_INVALID_FSNAME (2016)     | Invalid file space name.                                                                    |
| DSM_RC_INVALID_DRIVE_CHAR (2026) | Drive letter is not an alphabetic character.                                                |
| DSM_RC_NULL_FSNAME (2027)        | Null file space name.                                                                       |
| DSM_RC_FS_ALREADY_REGED (2062)   | File space is already registered.                                                           |
| DSM_RC_WRONG_VERSION_PARM (2065) | Application client's API version is different from the IBM Storage Protect library version. |
| DSM_RC_FSINFO_TOOLONG (2106)     | File space information is too long.                                                         |

## dsmReleaseBuffer

The **dsmReleaseBuffer** function returns a buffer to IBM Storage Protect. The application calls **dsmReleaseBuffer** after a **dsmGetDataEx** was called and the application has moved all the data out of the buffer and is ready to release it. **dsmReleaseBuffer** requires that **dsmInitEx** was called with the *UseTsmBuffers* set to *btrue* and a non-zero value was provided for *numTsmBuffers*. **dsmReleaseBuffer** should also be called if the application is about to call **dsmTerminate** and it still holds data buffers.

## dsmReleaseBufferSyntax

```
dsInt16_t dsmReleaseBuffer (releaseBufferIn_t      *dsmReleaseBufferInP,  
                             releaseBufferOut_t     *dsmReleaseBufferOutP) ;
```

## Parameters

### releaseBufferIn\_t \* dsmReleaseBufferInP (I)

This structure contains the following input parameters.

#### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

#### dsUInt8\_t tsmBufferHandle(I)

The handle that identifies this buffer.

#### char \*dataPtr(I)

The address to which the application is written.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 45. Return codes for *dsmReleaseBuffer*

| Return code              | Explanation                                            |
|--------------------------|--------------------------------------------------------|
| DSM_RC_BAD_CALL_SEQUENCE | The call was not issued in the proper state.           |
| DSM_RC_INVALID_TSMBUFFER | The handle or the value of <b>dataPtr</b> are invalid. |
| DSM_RC_BUFF_ARRAY_ERROR  | A buffer array error occurred.                         |

## dsmRenameObj

The **dsmRenameObj** function call renames the high-level or low-level object name. For backup objects, pass in the current object name and changes either for high-level or low-level object names. For archive objects, pass in the current object file space name and object ID, and changes either for high-level or low-level object names. Use this function call within **dsmBeginTxn** and **dsmEndTxn** calls.

The merge flag determines whether or not a duplicate backup object name is merged with the existing backups. If the new name corresponds to an existing object and merge is true, the current object is converted to the new name and it becomes the active version of the new name while the existing active object that had that name becomes the top most inactive copy of the object. If the new name corresponds to an existing object and merge is false, the function then returns the return code, DSM\_RC\_ABORT\_DUPLICATE\_OBJECT.

### Restrictions:

- Only the owner of the object can rename it.
- The **dsmRenameObj** function is not supported if data retention protection is enabled on the IBM Storage Protect server or if you are connected to the IBM Storage Protect for Data Retention server.

The **dsmRenameObj** function call tests for these merge conditions:

- The current **dsmObjName** object and the new high-level or low-level object must match on owner, copy group, and management class.
- The current **dsmObjName** must have been backed up more recently than the currently active object with the new name.
- There must be only an active copy of the current **dsmObjName** with no inactive copies.

## Syntax

```
dsInt16_t dsmRenameObj (dsmRenameIn_t *dsmRenameInP,  
                        dsmRenameOut_t *dsmRenameOutP);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmRenameIn\_t \*dsmRenameInP

This structure contains the input parameters.

### dsUInt8\_t repository (I);

This parameter indicates whether the file space to delete is in the backup repository or the archive repository.

**dsmObjName \*objNameP (I);**

This parameter is a pointer to the structure that contains the current file space name, high-level object name, low-level object name, and object type.

**char newHL [DSM\_MAX\_HL\_LENGTH + 1];**

This parameter specifies the new high-level name.

**char newLL [DSM\_MAX\_LL\_LENGTH + 1];**

This parameter specifies the new low-level name.

**dsBool\_t merge;**

This parameter determines whether or not a backup object is merged with duplicate named objects. The values are either true or false.

**ObjID;**

The object ID for archive objects.

**dsmRenameOut\_t \*dsmRnameOutP**

This structure contains the output parameters.

**Note:** There are no output parameters.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 46. Return codes for *dsmRenameObj*

| Return code                         | Explanation                                                                            |
|-------------------------------------|----------------------------------------------------------------------------------------|
| DSM_RC_ABORT_MERGE_ERROR (45)       | Server detected a merge error.                                                         |
| DSM_RC_ABORT_DUPLICATE_OBJECT (32)  | Object already exists and merge is false.                                              |
| DSM_RC_ABORT_NO_MATCH (2)           | Object not found.                                                                      |
| DSM_RC_REJECT_SERVER_DOWNLEVEL (58) | The IBM Storage Protect server must be at V3.7.4.0 or later for this function to work. |

## dsmRequestBuffer

The **dsmRequestBuffer** function returns a buffer to IBM Storage Protect. The application calls **dsmRequestBuffer** after a **dsmGetDataEx** was called and the application has moved all the data out of the buffer and is ready to release it.

**dsmReleaseBuffer** requires that **dsmInitEx** was called with the *UseTsmBuffers* set to *btrue* and a non-zero value was provided for *numTsmBuffers*. **dsmReleaseBuffer** should also be called if the application is about to call **dsmTerminate** and it still holds IBM Storage Protect buffers.

## Syntax

```
dsInt16_t dsmRequestBuffer (getBufferIn_t *dsmRequestBufferInP,
                           getBufferOut_t *dsmRequestBufferOutP) ;
```

## Parameters

**getBufferIn\_t \* dsmRequestBufferInP (I)**

This structure contains the following input parameter:

**dsUInt32\_t dsmHandle**

The handle that identifies the session and associates it with a previous **dsmInitEx** call.

**getBufferOut\_t \*dsmRequestBufferOut P (0)**

This structure contains the output parameters.

**dsUInt8\_t tsmBufferHandle(0)**

The handle that identifies this buffer.

**char \*dataPtr(0)**

The address to which application is written.

**dsUInt32\_t \*bufferLen(0)**

Maximum number of bytes that can be written to this buffer.

**Return codes**

The return code numbers are provided in parentheses ( ).

Table 47. Return codes for *dsmRequestBuffer*

| Return code                         | Explanation                                                                            |
|-------------------------------------|----------------------------------------------------------------------------------------|
| DSM_RC_BAD_CALL_SEQUENCE (33)       | The call was not issued in the proper state.                                           |
| DSM_RC_SENDDATA_WITH_ZERO_SIZE (34) | If the object being sent is 0 length, no calls to <b>dsmReleaseBuffer</b> are allowed. |
| DSM_RC_BUFF_ARRAY_ERROR (121)       | A valid buffer could not be obtained.                                                  |

**dsmRetentionEvent**

The **dsmRetentionEvent** function call sends a list of object IDs to the IBM Storage Protect server, with a retention event operation to be performed on these objects. Use this function call within **dsmBeginTxn** and **dsmEndTxn** calls.

**Note:** The server must be at version 5.2.2.0 or later for this function to work.

The maximum number of objects in a call is limited to the value of *maxObjPerTxn* that is returned in the *ApiSessInfo* structure from a **dsmQuerySessInfo** call.

Only an owner of an object can send an event on that object.

The following events are possible:

**eventRetentionActivate**

Can be issued only for objects that are bound to an event based management class. Sending this event activates the event for this object and the state of the retention for this object changes from DSM\_ARCH\_RETINIT\_PENDING to DSM\_ARCH\_RETINIT\_STARTED.

**eventHoldObj**

This event issues a retention or deletion hold on the object so that, until a release is issued, the object is not expired and cannot be deleted.

**eventReleaseObj**

This event can only be issued for an object that has a value of DSM\_ARCH\_HELD\_TRUE in the **objectHeld** field and removes the hold on the object resuming the original retention policy.

Before you send **dsmRetentionEvent**, send the query sequence that is described in “[Querying the IBM Storage Protect system](#)” on page 28 to obtain the information for the object. The call to **dsmGetNextQObj** returns a data structure named **qryRespArchiveData** for archive queries. This data structure contains the information that is needed for **dsmRetentionEvent**.

**Syntax**

```
extern dsInt16_t DSMLINKAGE dsmRetentionEvent(
    dsmRetentionEventIn_t      *ddsmRetentionEventInP,
```

```
dsmRetentionEventOut_t      *dsmRetentionEventOutP
);
```

## Parameters

### **dsmRetentionEventIn\_t \*dsmRetentionEventP**

This structure contains the following input parameters:

#### **dsUInt16\_t stVersion;**

This parameter indicates the structure version.

#### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

#### **dsmEventType\_t eventType (I);**

This parameter indicates the event type. See the beginning of this section for the meaning of these possible values: **eventRetentionActivate**, **eventHoldObj**, **eventReleaseObj**

#### **dsmObjList\_t objList;**

This parameter indicates a list of object IDs to signal.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 48. Return codes for *dsmRetentionEvent*

| Return code                            | Explanation                                                        |
|----------------------------------------|--------------------------------------------------------------------|
| DSM_RC_ABORT_NODE_NOT_AUTHORIZED (36)  | The node or user does not have proper authority.                   |
| DSM_RC_ABORT_TXN_LIMIT_EXCEEDED (249)  | Too many objects in the transaction.                               |
| DSM_RC_ABORT_OBJECT_ALREADY_HELD (250) | Object is already held, cannot issue another hold.                 |
| DSM_RC_REJECT_SERVER_DOWNLEVEL (58)    | The server must be at V5.2.2.0 or later for this function to work. |

## dsmSendBufferData

The **dsmSendBufferData** function call sends a byte stream of data to IBM Storage Protect through a buffer that was provided in a previous **dsmReleaseBuffer** call. The application client can pass any type of data for storage on the server. Usually this data are file data, but it is not limited to file data. You can call **dsmSendBufferData** several times, if the byte stream of data that you are sending is large. Regardless of whether the call succeeds or fails, the buffer is released.

**Restriction:** When you use the *useTsmBuffers* option, even if an object is included for compression, the object is not compressed.

## Syntax

```
dsInt16_t  dsmSendBufferData (sendBufferDataIn_t      *dsmSendBufferDataExInP,
                             sendBufferDataOut_t      *dsmSendBufferDataOutP) ;
```

## Parameters

### **sendBufferDataIn\_t \* dsmSendBufferDataInP (I)**

This structure contains the following input parameters.



**dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

**dsUInt8\_t tsmBufferHandle(I)**

The handle that identifies the buffer to send.

**char \*dataPtr(I)**

The address to which application data was written.

**dsUInt32\_t numBytes(I)**

The actual number of bytes written by the application (should always be less than the value provided in **dsmReleaseBuffer**).

## Return codes

The return code numbers are provided in parentheses ( ).

Table 49. Return codes for *dsmSendBufferData*

| Return code                     | Explanation                                                                                                      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------|
| DSM_RC_BAD_CALL_SEQUENCE (2041) | The call was not issued in the proper state.                                                                     |
| DSM_RC_INVALID_TSMBUFFER (2042) | The handle or the value of <b>dataPtr</b> are invalid.                                                           |
| DSM_RC_BUFF_ARRAY_ERROR (2045)  | A buffer array error occurred.                                                                                   |
| DSM_RC_TOO_MANY_BYTES (2043)    | The value of <b>numBytes</b> is bigger than the size of the buffer provided in the <b>dsmReleaseBuffer</b> call. |

## dsmSendData

The **dsmSendData** function call sends a byte stream of data to IBM Storage Protect through a buffer. The application client can pass any type of data for storage on the server. Usually, these data are file data, but are not limited to such. You can call **dsmSendData** several times, if the byte stream of data that you want to send is large.

**Restriction:** The application client cannot reuse the buffer that is specified in **dsmSendData** until the **dsmSendData** call returns.

**Tip:** If IBM Storage Protect returns code 157 (DSM\_RC\_WILL\_ABORT), start a call to **dsmEndSendObj** and then to **dsmEndTxn** with a vote of DSM\_VOTE\_COMMIT. The application then receives return code 2302 (DSM\_RC\_CHECK\_REASON\_CODE) and passes the reason code back to the application user. This informs the user why the server is ending the transaction.

## Syntax

```
dsInt16_t dsmSendData (dsUInt32_t    dsmHandle,
                      DataBlk *dataBlkPtr);
```

## Parameters

**dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

**DataBlk \*dataBlkPtr (I/O)**

This parameter points to a structure that includes both a pointer to the buffer from which the data are to be sent, as well as the size of the buffer. On return, this structure contains the number of bytes that is actually transferred. See [“API type definitions source files” on page 140](#) for the type definition.

## Return codes

The return code numbers are provided in parentheses ( ).

Table 50. Return codes for *dsmSendData*

| Return code                           | Explanation                                                                                 |
|---------------------------------------|---------------------------------------------------------------------------------------------|
| DSM_RC_NO_COMPRESS_MEMORY (154)       | Insufficient memory available to perform data compression or expansion.                     |
| DSM_RC_COMPRESS_GREW (155)            | During compression the compressed data grew in size compared to the original data.          |
| DSM_RC_WILL_ABORT (157)               | An unknown and unexpected error occurred, causing the transaction to halt.                  |
| DSM_RC_WRONG_VERSION_PARM (2065)      | Application client's API version is different than the IBM Storage Protect library version. |
| DSM_RC_NEEDTO_ENDTXN (2070)           | Need to end the transaction.                                                                |
| DSM_RC_OBJ_EXCLUDED (2080)            | The include-exclude list excludes the object.                                               |
| DSM_RC_OBJ_NOBCG (2081)               | The object has no backup copy group and will not be sent to the server.                     |
| DSM_RC_OBJ_NOACG (2082)               | The object has no archive copy group and is not sent to the server.                         |
| DSM_RC_SENDDATA_WITH_ZERO_SIZE (2107) | The object cannot send data with a zero byte <i>sizeEstimate</i> .                          |

## dsmSendObj

The **dsmSendObj** function call starts a request to send a single object to storage. Multiple **dsmSendObj** calls and associated **dsmSendData** calls can be made within the bounds of a transaction for performance reasons.

The **dsmSendObj** call processes the data for the object as a byte stream passed in memory buffers. The **dataBlkPtr** parameter in the **dsmSendObj** call permits the application client to either:

- Pass the data and the attributes (the attributes are passed through the **objAttrPtr**) of the object in a single call.
- Specify part of the object data through the **dsmSendObj** call and the remainder of the data through one or more **dsmSendData** calls.

Alternatively, the application client can specify only the attributes through the **dsmSendObj** call and specify the object data through one or more calls to **dsmSendData**. For this method, set **dataBlkPtr** to NULL on the **dsmSendObj** call.

**Tip:** For certain object types, byte stream data might not be associated with the data; for example, a directory entry with no extended attributes.

Before **dsmSendObj** is called, a preceding **dsmBindMC** call must be made to properly bind a management class to the object that you want to back up or archive. The API keeps this binding so that it can associate the proper management class with the object when it is sent to the server. If you permit the management class that is bound on a **dsmSendObj** call to default for an object type of directory (DSM\_OBJ\_DIRECTORY), the default might not be the default management class. Instead, the management class with the greatest retention time is used. If more than one management class exists with this retention time, the first one that is encountered is used.

Follow all object data that is sent to storage with a **dsmEndSendObj** call. If you do not have object data to send to the server, or all data was contained within the **dsmSendObj** call, start a **dsmEndSendObj**

call before you can start another **dsmSendObj** call. If multiple data sends were required through the **dsmSendData** call, the **dsmEndSendObj** follows the last send to indicate the state change.

**Tip:** If IBM Storage Protect returns code 157 (DSM\_RC\_WILL\_ABORT), start a call to **dsmEndTxn** with a vote of DSM\_VOTE\_COMMIT. The application receives return code 2302 (DSM\_RC\_CHECK\_REASON\_CODE) and passes the reason code back to the application user. This informs the user why the server is ending the transaction.

If the reason code is 11 (DSM\_RS\_ABORT\_NO\_REPOSIT\_SPACE), it is possible that the *sizeEstimate* is too small for the actual amount of data. The application needs to determine a more accurate *sizeEstimate* and send the data again.

## Syntax

```
dsInt16_t dsmSendObj (dsUInt32_t      dsmHandle,
                      dsmSendType     sendType,
                      void             *sendBuff,
                      dsmObjName       *objNameP,
                      ObjAttr         *objAttrPtr,
                      DataBlk         *dataBlkPtr);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmSendType sendType (I)

This parameter specifies the type of send that is being performed. Possible values include:

| Name                      | Description                                                                                                     |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>stBackup</b>           | A backup object that is sent to the server.                                                                     |
| <b>stArchive</b>          | An archive object that is sent to the server.                                                                   |
| <b>stBackupMountWait</b>  | A backup object for which you want the server to wait until the necessary device, such as a tape, is mounted.   |
| <b>stArchiveMountWait</b> | An archive object for which you want the server to wait until the necessary device, such as a tape, is mounted. |

**Note:** Use the **MountWait** types if there is any possibility that your application user might send data to a tape.

### void \*sendBuff (I)

This parameter is a pointer to a structure that contains other information specific to the **sendType** on the call. Currently, only a **sendType** of **stArchive** has an associated structure. This structure is called **sndArchiveData** and it contains the archive description.

### dsmObjName \*objNameP (I)

This parameter is a pointer to the structure that contains the file space name, high-level object name, low-level object name, and object type. See [“Object names and IDs” on page 20](#) for more information.

### ObjAttr \*objAttrPtr (I)

This parameter passes object attributes of interest to the application. See [“API type definitions source files” on page 140](#) for the type definition.

The attributes are:

- **owner** refers to the owner of the object. Determining whether the owner is declared to be a specific name or an empty string is important when getting the object back from IBM Storage Protect storage. See [“Accessing objects as session owner” on page 21](#) for more information.

- **sizeEstimate** is a best estimate of the total size of the data object to send to the server. Be as accurate as possible on this size, because the server uses this attribute for efficient space allocation and object placement within its storage resources.

If the size estimate that you specified is significantly smaller than the actual number of bytes that are sent, the server might have difficulty allocating enough space and end the transaction with a reason code of 11 (DSM\_RS\_ABORT\_NO\_REPOSIT\_SPACE).

**Note:** The size estimate is for the total size of the data object in bytes.

Objects with a size smaller than DSM\_MIN\_COMPRESS\_SIZE do not compress.

If your object has no bit data (only the attribute information from this call), the **sizeEstimate** should be zero.

**Note:** Starting with version 5.1.0, the copy destination within a transaction is not checked for consistency on zero-length objects.

- **objCompressed** is a Boolean value that states whether or not the object data have already been compressed.

If the object is compressed (object *compressed*=*bTrue*), IBM Storage Protect does not try to compress it again. If it is not compressed, IBM Storage Protect decides whether to compress the object, based on the values of the compression option set by the administrator and set in the API configuration sources.

If your application plans to use partial object restore or retrieve, you cannot compress the data while sending it. To enforce this, set *ObjAttr.objCompressed* to *bTrue*.

- **objInfo** saves information about the particular object.

**Restriction:** Information is not stored here automatically. When this attribute is used, you must set the attribute, *objInfoLength*, to show the length of *objInfo*.

- **mcNameP** contains the name of a management class that overrides the management class that is obtained from **dsmBindMC**.
- **disableDeduplication** is a Boolean value. When it is set to true, this object is not deduplicated by the client.

If the client-side deduplication is enabled and the object that is being sent is a directory, then the object attribute and the object name type must be configured as follows:

```
objAttrArea.disableDeduplication = bTrue;
objName.objType = DSM_OBJ_DIRECTORY;
```

### DataBlk \*dataBlkPtr (I/O)

This parameter points to a structure that includes both a pointer to the buffer of data that is to be backed up or archived and the size of that buffer. This parameter applies to **dsmSendObj** only. If you want to begin sending data on a subsequent **dsmSendData** call, rather than on the **dsmSendObj** call, set the buffer pointer in the DataBlk structure to NULL. On return, this structure contains the number of bytes that is actually transferred. See [“API type definitions source files” on page 140](#) for the type definition.

### Return codes

The return code numbers are provided in parentheses ( ).

Table 51. Return codes for *dsmSendObj*

| Return code                     | Explanation                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------|
| DSM_RC_NO_COMPRESS_MEMORY (154) | Insufficient memory available to perform data compression or expansion.             |
| DSM_RC_COMPRESS_GREW (155)      | During compression, the compressed data grew in size compared to the original data. |

Table 51. Return codes for *dsmSendObj* (continued)

| Return code                       | Explanation                                                                                 |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| DSM_RC_WILL_ABORT (157)           | An unknown and unexpected error occurred, causing the transaction to be halted.             |
| DSM_RC_TL_NOBCG (184)             | The management class for this file has no valid backup copy group.                          |
| DSM_RC_TL_NOACG (186)             | The management class for this file does not have a valid copy group for the send type.      |
| DSM_RC_NULL_OBJNAME (2000)        | Null object name.                                                                           |
| DSM_RC_NULL_OBJATTRPTR (2004)     | Null object attribute pointer.                                                              |
| DSM_RC_INVALID_OBJTYPE (2010)     | Invalid object type.                                                                        |
| DSM_RC_INVALID_OBJOWNER (2019)    | Invalid object owner.                                                                       |
| DSM_RC_INVALID_SENDTYPE (2022)    | Invalid send type.                                                                          |
| DSM_RC_WILDCHAR_NOTALLOWED (2050) | Wildcard characters not allowed.                                                            |
| DSM_RC_FS_NOT_REGISTERED (2061)   | File space not registered.                                                                  |
| DSM_RC_WRONG_VERSION_PARM (2065)  | Application client's API version is different from the IBM Storage Protect library version. |
| DSM_RC_NEEDTO_ENDTXN (2070)       | Need to end transaction.                                                                    |
| DSM_RC_OBJ_EXCLUDED (2080)        | The include-exclude list excluded the object.                                               |
| DSM_RC_OBJ_NOBCG (2081)           | The object has no backup copy group, and it is not sent to the server.                      |
| DSM_RC_OBJ_NOACG (2082)           | The object has no archive copy group, and it is not sent to the server.                     |
| DSM_RC_DESC_TOOLONG (2100)        | Description is too long.                                                                    |
| DSM_RC_OBJINFO_TOOLONG (2101)     | Object information is too long.                                                             |
| DSM_RC_HL_TOOLONG (2102)          | High-level qualifier is too long.                                                           |
| DSM_RC_FILESPACE_TOOLONG (2104)   | File space name is too long.                                                                |
| DSM_RC_LL_TOOLONG (2105)          | Low-level qualifier is too long.                                                            |
| DSM_RC_NEEDTO_CALL_BINDMC (2301)  | <b>dsmBindMC</b> must be called first.                                                      |

## dsmSetAccess

The **dsmSetAccess** function call gives other users or nodes access to backup versions or archived copies of your objects, access to all your objects, or access to a selective set. When you give access to another user, that user can query, restore, or retrieve your files. This command supports wildcards for the following fields: *fs*, *hl*, *ll*, *node*, *owner*.

**Note:** You cannot give access to both backup versions and archive copies by using a single command. You must specify either backup or archive.

## Syntax

```
dsInt16_t DSMLINKAGE dsmSetAccess
(
    dsUInt32_t      dsmHandle,
    dsmSetAccessType accessType,
    dsmObjName      *objNameP,
    char            *node,
    char            *owner);
```

## Parameters

### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

### dsmAccessType accessType (I)

This parameter specifies the type of objects for which you want to give access. Possible values include:

| Name             | Description                                                 |
|------------------|-------------------------------------------------------------|
| <b>atBackup</b>  | Specifies that access is being set to backup objects.       |
| <b>atArchive</b> | Specifies that the access is being set for archive objects. |

### dsmObjName \*objNameP (I)

This parameter is a pointer to the structure that contains the file space name, the high-level object name, and the low-level object name.

**Note:** To specify all file spaces, use an asterisk (\*) for the file space name.

### char \*node (I)

This parameter is a pointer to the node name for which access is given. For any node, specify an asterisk (\*).

### char \*owner (I)

This parameter is a pointer to the user name on the node to which you gave access. For all users, specify an asterisk (\*).

## Return codes

The return code numbers are provided in parentheses ( ).

Table 52. Return codes for *dsmSetAccess*

| Return code                       | Explanation                                       |
|-----------------------------------|---------------------------------------------------|
| DSM_RC_INVALID_ACCESS_TYPE (2110) | Invalid access type specified.                    |
| DSM_RC_FILE_SPACE_NOT_FOUND (124) | Specified file space was not found on the server. |
| DSM_RC_QUERY_COMM_FAILURE (2111)  | Communication error during server query.          |
| DSM_RC_NO_FILES_BACKUP (2112)     | No files were backed up for this file space.      |
| DSM_RC_NO_FILES_ARCHIVE (2113)    | No files were archived for this file space.       |
| DSM_RC_INVALID_SETACCESS (2114)   | Invalid formulation of set access.                |

## dsmSetUp

The **dsmSetUp** function call overwrites environment variable values. Call **dsmSetUp** before **dsmInitEx**. The values that were passed in the **envSetUp** structure overwrite any existing environment variables or defaults. If you specify NULL for a field, values are taken from the environment. If you do not set a value, the values are taken from the defaults.

### Requirements:

1. If you use **dsmSetUp**, always call **dsmTerminate** before **dsmCleanUp**.
2. API instrumentation can only be activated if the testflag INSTRUMENT: API is set in the configuration file and the **dsmSetUp** or **dsmCleanUp** calls are used in the application.

## Syntax

```
dsInt16_t DSMLINKAGE dsmSetUp
(dsBool_t mtFlag,
 envSetUp *envSetUpP);
```

## Parameters

### dsBool\_t mtFlag (I)

This parameter specifies if the API will be used in a single thread, or a multithread mode. Values include:

```
DSM_SINGLETHREAD
DSM_MULTITHREAD
```

**Requirement:** The multithread flag must be on for LAN-free data transfer to occur.

### envSetUp \*envSetUpP(I)

This parameter is a pointer to the structure that holds the overwrite values. Specify NULL if you do not want to override existing environment variables. The fields in the **envSetUp** structure include:

| Name                         | Description                                                                                                                                                                                          |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>dsmiDir</b>               | A fully-qualified directory path that contains a message file on UNIX or Linux. It also specifies the dsm.sys directories.                                                                           |
| <b>dsmiConfig</b>            | The fully-qualified name of the client options file.                                                                                                                                                 |
| <b>dsmiLog</b>               | The fully-qualified path of the error log directory.                                                                                                                                                 |
| <b>argv</b>                  | Pass the argv[0] name of the calling program if the application must run with authorized user authority. See <a href="#">“Controlling access to password files” on page 18</a> for more information. |
| <b>logName</b>               | The file name for an error log if the application does not use dserror.log.                                                                                                                          |
| <b>inclExclCaseSensitive</b> | Indicates whether include/exclude rules are case-sensitive or case-insensitive. This parameter can be used on Windows only, it is ignored elsewhere.                                                 |

## Return codes

The return code numbers are provided in parentheses ( ).

Table 53. Return codes for dsmSetUp

| Return code                | Explanation                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------------------|
| DSM_RC_ACCESS_DENIED (106) | Access to the specified file or directory is denied.                                                      |
| DSM_RC_INVALID_OPT (0400)  | An invalid option was found.                                                                              |
| DSM_RC_NO_HOST_ADDR (0405) | The TCPSERVERADDRESS for this server is not defined in the server name stanza in the system options file. |
| DSM_RC_NO_OPT_FILE (0406)  | The options file specified by filename cannot be found.                                                   |

Table 53. Return codes for *dsmSetUp* (continued)

| Return code                        | Explanation                                                                                                             |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| DSM_RC_MACHINE_SAME (0408)         | The NODENAME defined in the options file cannot be the same as the system <i>HostName</i> .                             |
| DSM_RC_INVALID_SERVER (0409)       | The system options file does not contain the SERVERNAME option.                                                         |
| DSM_RC_INVALID_KEYWORD (0410)      | An invalid option keyword was found in the <b>dsmInitEx</b> configuration file, the option string, dsm.sys, or dsm.opt. |
| DSM_RC_PATTERN_TOO_COMPLEX (0411)  | The include or exclude pattern issued is too complex to be accurately interpreted by IBM Storage Protect.               |
| DSM_RC_NO_CLOSING_BRACKET (0412)   | The include or exclude pattern is incorrectly constructed. The closing bracket is missing.                              |
| DSM_RC_NLS_CANT_OPEN_TXT (0610)    | The system is unable to open the message text file.                                                                     |
| DSM_RC_NLS_INVALID_CNTL_REC (0612) | The system is unable to use the message text file.                                                                      |
| DSM_RC_NOT_ADSM_AUTHORIZED (0927)  | You must be the authorized user to have multithreading and <i>passwordaccess</i> generate.                              |
| DSM_RC_NO_INCLEXCL_FILE (2229)     | The include-exclude file was not found.                                                                                 |
| DSM_RC_NO_SYS_OR_INCLEXCL (2230)   | Either the dsm.sys or the include-exclude file was not found.                                                           |

## dsmTerminate

The **dsmTerminate** function call ends a session with the IBM Storage Protect server and cleans up the IBM Storage Protect environment.

### Syntax

There are no return codes that are specific for this call.

```
dsInt16_t dsmTerminate (dsUInt32_t dsmHandle);
```

### Parameters

#### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

## dsmUpdateFS

The **dsmUpdateFS** function call updates a file space in IBM Storage Protect storage. This update ensures that the administrator has a current record of your file space.

### Syntax

```
dsInt16_t dsmUpdateFS (dsUInt32_t      dsmHandle,
char          *fs,
dsmFSUpd      *fsUpdP,
dsUInt32_t     fsUpdAct);
```



## Parameters

### **dsUInt32\_t dsmHandle (I)**

The handle that associates this call with a previous **dsmInitEx** call.

### **char \*fs (I)**

This parameter is a pointer to the file space name.

### **dsmFSUpd \*fsUpdP (I)**

This parameter is a pointer to the structure that has the correct fields for the update that you want. Complete only those fields that need updating.

### **dsUInt32\_t fsUpdAct (I)**

A 2-byte bit map that indicates which of the fields to update. The bit masks have the following values:

- DSM\_FSUPD\_FSTYPE
- DSM\_FSUPD\_FSINFO

**Tip:** For Windows operating systems, the drive letter value from **dsmDOSAttrib** is also updated when **FSINFO** is selected.

- DSM\_FSUPD\_OCCUPANCY
- DSM\_FSUPD\_CAPACITY
- DSM\_FSUPD\_BACKSTARTDATE
- DSM\_FSUPD\_BACKCOMPLETEDATE

For a description of these bit masks, see the DSM\_FSUPD definitions in the following topic: [“API type definitions source files” on page 140](#).

## Return codes

The following table lists return codes for the **dsmUpdateFS** function call.

*Table 54. Return codes for dsmUpdateFS*

| Return code                    | Return code number | Description                                                                                          |
|--------------------------------|--------------------|------------------------------------------------------------------------------------------------------|
| DSM_RC_FS_NOT_REGISTERED       | 2061               | File space name is not registered.                                                                   |
| DSM_RC_WRONG_VERSION_PARTITION | 2065               | The API version of the application client is different from the IBM Storage Protect library version. |
| DSM_RC_FSINFO_TOOLONG          | 2106               | File space information is too long.                                                                  |

## dsmUpdateObj

The **dsmUpdateObj** function call updates the meta information associated with an active backup or archive object already on the server. The application bit data is not affected. To update an object, you must give a specific non-wildcard name. To update an archived object, set the **dsmSendType** to **stArchive**. Only the latest named archive object is updated.

You can only start the **dsmUpdateObj** call in the session state; it cannot be called inside a transaction because it performs its own transaction. And, you can update only one object at a time.

**Restriction:** On a UNIX or Linux operating system, if you change the owner field, you cannot query or restore the object unless you are the root user.

## Syntax

```
dsInt16_t dsmUpdateObj
(dsUInt32_t dsmHandle,
 dsmSendType sendType,
 void *sendBuff,
 dsmObjName *objNameP,
 ObjAttr *objAttrPtr, /* objInfo */
 dsUInt16_t objUpdAct); /* action bit vector */
```

## Parameters

The field descriptions are the same as those in **dsmSendObj**, with the following exceptions:

### **dsmObjName \*objNameP (I)**

You cannot use a wildcard.

### **ObjAttr \*objAttrPtr (I)**

The **objCompressed** field is ignored for this call.

Other differences are:

- **owner**. If you specify a new **owner** field, the owner changes.
- **sizeEstimate**. If you specify a non-zero value it should be the actual amount of data sent, in bytes. The value is stored in the IBM Storage Protect metadata for future use.
- **objInfo**. This attribute contains the new information to be placed in the **objInfo** field. Set the **objInfoLength** to the length of the new **objInfo**.

### **dsUInt16\_t objUpdAct**

The bit masks and possible actions for **objUpdAct** are:

#### **DSM\_BACKUPD\_MC**

Updates the management class for the object.

#### **DSM\_BACKUPD\_OBJINFO**

Updates **objInfo**, **objInfoLength**, and **sizeEstimate**.

#### **DSM\_BACKUPD\_OWNER**

Updates the owner of the object.

#### **DSM\_ARCHUPD\_DESCR**

Updates the **Description** field. Enter the value for the new description through the **SendBuff** parameter. See the sample program for proper use.

#### **DSM\_ARCHUPD\_OBJINFO**

Updates **objInfo**, **objInfoLength**, and **sizeEstimate**.

#### **DSM\_ARCHUPD\_OWNER**

Updates the owner of the object.

## Return codes

The return code numbers are provided in parentheses ().

Table 55. Return codes for *dsmUpdateObj*

| Return code                     | Explanation                   |
|---------------------------------|-------------------------------|
| DSM_RC_INVALID_ACTION (2232)    | Invalid action.               |
| DSM_RC_FS_NOT_REGISTERED (2061) | File space not registered.    |
| DSM_RC_BAD_CALL_SEQUENCE (2041) | Sequence of calls is invalid. |

Table 55. Return codes for *dsmUpdateObj* (continued)

| Return code                       | Explanation                          |
|-----------------------------------|--------------------------------------|
| DSM_RC_WILDCARD_NOTALLOWED (2050) | Wildcard characters are not allowed. |
| DSM_RC_ABORT_NO_MATCH (2)         | Previous query does not match.       |

## dsmUpdateObjEx

The **dsmUpdateObjEx** function call updates the meta information that is associated with an active backup or archive object that is on the server. The application bit data is not affected. To update an object, you must specify a non-wildcard name, or you can specify the object ID to update a specific archived object. You cannot use wildcard characters when specifying the name. To update a backup object, set the **dsmSendType** parameter to **stBackup**. To update an archived object, set the **dsmSendType** parameter to **stArchive**.

You can only start the **dsmUpdateObjEx** call in the session state; it cannot be called inside a transaction because it performs its own transaction. You can update only one object at a time.

**Restriction:** On a UNIX or Linux operating system, if you change the owner field, you cannot query or restore the object unless you are the root user. Only the current active version of a backup object can be updated.

## Syntax

```
dsInt16_t dsmUpdateObjEx
(dsmUpdateObjExIn_t *dsmUpdateObjExInP,
 dsmUpdateObjExOut_t *dsmUpdateObjExOutP);
```

## Parameters

### dsmUpdateObjExIn\_t \*dsmUpdateObjExInP

This structure contains the following input parameters:

#### dsUInt16\_t stVersion (I)

The current version of the structure that is used.

#### dsUInt32\_t dsmHandle (I)

The handle that associates this call with a previous **dsmInitEx** call.

#### dsmSendType sendType (I)

The type of send that is being performed. The value can be:

##### **stBackup**

A backup object that is sent to the server.

##### **stArchive**

An archive object that is sent to the server.

#### dsmObjName \*objNameP (I)

A pointer to the structure that contains the filespace name, high-level object name, low-level object name, and object type. You cannot use a wildcard.

#### ObjAttr \*objAttrPtr (I)

Passes object attributes to the application. The values that are updated depend on the flags in the **objUpdAct** field. The **objCompressed** attribute is ignored for this call.

The attributes are:

- **owner** changes the owner if a new name is entered.

- **sizeEstimate** is the actual amount of data that is sent in bytes. The value is stored in the IBM Storage Protect meta data for future use.
- **objCompressed** is a Boolean value that states whether or not the object data have already been compressed.
- **objInfo** is an attribute that contains the new information to be placed in the **objInfo** field. Set the **objInfoLength** to the length of the new **objInfo**.
- **mcNameP** contains the name of a management class that overrides the management class that is obtained from **dsmBindMC**.

#### **dsUInt32\_t objUpdAct**

Specifies the bit masks and actions for **objUpdAct** are:

##### **DSM\_BACKUPD\_MC**

Updates the management class for the object.

##### **DSM\_BACKUPD\_OBJINFO**

Updates the information object (**objInfo**), the length of the information object (**objInfoLength**), and the amount of data that is sent (**sizeEstimate**) for the backup object.

##### **DSM\_BACKUPD\_OWNER**

Updates the owner for the backup object.

##### **DSM\_ARCHUPD\_DESCR**

Updates the **Description** field for the archive object. Enter the value for the new description through the **sendBuff** parameter.

##### **DSM\_ARCHUPD\_OBJINFO**

Updates the information object (**objInfo**), the length of the information object (**objInfoLength**), and the amount of data that is sent (**sizeEstimate**) for the archive object.

##### **DSM\_ARCHUPD\_OWNER**

Updates the owner of the archive object.

#### **ObjID archObjId**

Specifies the unique object ID for a specific archive object. Because multiple archive objects can have the same name, this parameter identifies a specific one. You can obtain the object ID by using a query archive call.

#### **dsmUpdateObjExOut\_t \*dsmUpdateObjExOutP**

This structure contains the output parameter:

#### **dsUInt16\_t stVersion (I)**

The current version of the structure that is used.

## **Return codes**

The return code numbers are provided in parentheses ( ) in the following table.

*Table 56. Return codes for dsmUpdateObjEx*

| <b>Return code</b>                | <b>Explanation</b>                   |
|-----------------------------------|--------------------------------------|
| DSM_RC_INVALID_ACTION (2012)      | Invalid action.                      |
| DSM_RC_FS_NOT_REGISTERED (2061)   | File space not registered.           |
| DSM_RC_BAD_CALL_SEQUENCE (2041)   | Sequence of calls is invalid.        |
| DSM_RC_WILDCHAR_NOTALLOWED (2050) | Wildcard characters are not allowed. |

Table 56. Return codes for *dsmUpdateObjEx* (continued)

| Return code               | Explanation                    |
|---------------------------|--------------------------------|
| DSM_RC_ABORT_NO_MATCH (2) | Previous query does not match. |

## API return codes source file: `dsmrc.h`

The `dsmrc.h` header file contains all return codes that the API can return to an application.

The information that is provided here contains a point-in-time copy of the `dsmrc.h` file that is distributed with the API. View the file in the API distribution package for the latest version.

```

/*****
* IBM Storage Protect
* API Client Component
*
* IBM Confidential
* (IBM Confidential-Restricted when combined with the Aggregated OCO
* source modules for this program)
*
* OCO Source Materials
*
* 5648-020 (C) Copyright IBM Corporation 1993, 2016
*****/

/*****/
/* Header File Name:  dsmrc.h
/*
/*
/* Descriptive-name:  Return codes from IBM Storage Protect APIs
*****/
#ifndef _H_DSMRC
#define _H_DSMRC

#ifndef DSMAPILIB

#ifndef _H_ANSMACH
typedef int RetCode ;
#endif

#endif

#define DSM_RC_SUCCESSFUL          0 /* successful completion */
#define DSM_RC_OK                  0 /* successful completion */

#define DSM_RC_UNSUCCESSFUL        -1 /* unsuccessful completion */

/* dsmEndTxn reason code */
#define DSM_RS_ABORT_SYSTEM_ERROR      1
#define DSM_RS_ABORT_NO_MATCH          2
#define DSM_RS_ABORT_BY_CLIENT         3
#define DSM_RS_ABORT_ACTIVE_NOT_FOUND  4
#define DSM_RS_ABORT_NO_DATA           5
#define DSM_RS_ABORT_BAD_VERIFIER      6
#define DSM_RS_ABORT_NODE_IN_USE       7
#define DSM_RS_ABORT_EXPDATE_TOO_LOW  8
#define DSM_RS_ABORT_DATA_OFFLINE     9
#define DSM_RS_ABORT_EXCLUDED_BY_SIZE 10
#define DSM_RS_ABORT_NO_STO_SPACE_SKIP 11
#define DSM_RS_ABORT_NO_REPOSIT_SPACE  DSM_RS_ABORT_NO_STO_SPACE_SKIP
#define DSM_RS_ABORT_MOUNT_NOT_POSSIBLE 12
#define DSM_RS_ABORT_SIZEESTIMATE_EXCEED 13
#define DSM_RS_ABORT_DATA_UNAVAILABLE  14
#define DSM_RS_ABORT_RETRY              15
#define DSM_RS_ABORT_NO_LOG_SPACE       16
#define DSM_RS_ABORT_NO_DB_SPACE        17
#define DSM_RS_ABORT_NO_MEMORY          18

#define DSM_RS_ABORT_FS_NOT_DEFINED     20
#define DSM_RS_ABORT_NODE_ALREADY_DEFED 21
#define DSM_RS_ABORT_NO_DEFAULT_DOMAIN  22
#define DSM_RS_ABORT_INVALID_NODENAME   23
#define DSM_RS_ABORT_INVALID_POL_BIND   24
#define DSM_RS_ABORT_DEST_NOT_DEFINED   25
#define DSM_RS_ABORT_WAIT_FOR_SPACE     26

```

```

#define DSM_RS_ABORT_NOT_AUTHORIZED 27
#define DSM_RS_ABORT_RULE_ALREADY_DEFED 28
#define DSM_RS_ABORT_NO_STOR_SPACE_STOP 29

#define DSM_RS_ABORT_LICENSE_VIOLATION 30
#define DSM_RS_ABORT_EXTOBJID_ALREADY_EXISTS 31
#define DSM_RS_ABORT_DUPLICATE_OBJECT 32

#define DSM_RS_ABORT_INVALID_OFFSET 33 /* Partial Object Retrieve */
#define DSM_RS_ABORT_INVALID_LENGTH 34 /* Partial Object Retrieve */
#define DSM_RS_ABORT_STRING_ERROR 35
#define DSM_RS_ABORT_NODE_NOT_AUTHORIZED 36
#define DSM_RS_ABORT_RESTART_NOT_POSSIBLE 37
#define DSM_RS_ABORT_RESTORE_IN_PROGRESS 38
#define DSM_RS_ABORT_SYNTAX_ERROR 39

#define DSM_RS_ABORT_DATA_SKIPPED 40
#define DSM_RS_ABORT_EXCEED_MAX_MP 41
#define DSM_RS_ABORT_NO_OBJSET_MATCH 42
#define DSM_RS_ABORT_PVR_ERROR 43
#define DSM_RS_ABORT_BAD_RECOGToken 44
#define DSM_RS_ABORT_MERGE_ERROR 45
#define DSM_RS_ABORT_FSRENAME_ERROR 46
#define DSM_RS_ABORT_INVALID_OPERATION 47
#define DSM_RS_ABORT_STGPOOL_UNDEFINED 48
#define DSM_RS_ABORT_INVALID_DATA_FORMAT 49
#define DSM_RS_ABORT_DATAMOVER_UNDEFINED 50

#define DSM_RS_ABORT_INVALID_MOVER_TYPE 231
#define DSM_RS_ABORT_ITEM_IN_USE 232
#define DSM_RS_ABORT_LOCK_CONFLICT 233
#define DSM_RS_ABORT_SRV_PLUGIN_COMM_ERROR 234
#define DSM_RS_ABORT_SRV_PLUGIN_OS_ERROR 235
#define DSM_RS_ABORT_CRC_FAILED 236
#define DSM_RS_ABORT_INVALID_GROUP_ACTION 237
#define DSM_RS_ABORT_DISK_UNDEFINED 238
#define DSM_RS_ABORT_BAD_DESTINATION 239
#define DSM_RS_ABORT_DATAMOVER_NOT_AVAILABLE 240
#define DSM_RS_ABORT_STGPOOL_COPY_CONT_NO 241
#define DSM_RS_ABORT_RETRY_SINGLE_TXN 242
#define DSM_RS_ABORT_TOC_CREATION_FAIL 243
#define DSM_RS_ABORT_TOC_LOAD_FAIL 244
#define DSM_RS_ABORT_PATH_RESTRICTED 245
#define DSM_RS_ABORT_NO_LANFREE_SCRATCH 246
#define DSM_RS_ABORT_INSERT_NOT_ALLOWED 247
#define DSM_RS_ABORT_DELETE_NOT_ALLOWED 248
#define DSM_RS_ABORT_TXN_LIMIT_EXCEEDED 249
#define DSM_RS_ABORT_OBJECT_ALREADY_HELD 250
#define DSM_RS_ABORT_INVALID_CHUNK_REFERENCE 254
#define DSM_RS_ABORT_DESTINATION_NOT_DEDUP 255
#define DSM_RS_ABORT_DESTINATION_POOL_CHANGED 257
#define DSM_RS_ABORT_NOT_ROOT 258

/* RETURN CODE */

#define DSM_RC_ABORT_SYSTEM_ERROR DSM_RS_ABORT_SYSTEM_ERROR
#define DSM_RC_ABORT_NO_MATCH DSM_RS_ABORT_NO_MATCH
#define DSM_RC_ABORT_BY_CLIENT DSM_RS_ABORT_BY_CLIENT
#define DSM_RC_ABORT_ACTIVE_NOT_FOUND DSM_RS_ABORT_ACTIVE_NOT_FOUND
#define DSM_RC_ABORT_NO_DATA DSM_RS_ABORT_NO_DATA
#define DSM_RC_ABORT_BAD_VERIFIER DSM_RS_ABORT_BAD_VERIFIER
#define DSM_RC_ABORT_NODE_IN_USE DSM_RS_ABORT_NODE_IN_USE
#define DSM_RC_ABORT_EXPIRATE_TOO_LOW DSM_RS_ABORT_EXPIRATE_TOO_LOW
#define DSM_RC_ABORT_DATA_OFFLINE DSM_RS_ABORT_DATA_OFFLINE
#define DSM_RC_ABORT_EXCLUDED_BY_SIZE DSM_RS_ABORT_EXCLUDED_BY_SIZE

#define DSM_RC_ABORT_NO_REPOSIT_SPACE DSM_RS_ABORT_NO_STO_SPACE_SKIP
#define DSM_RC_ABORT_NO_STO_SPACE_SKIP DSM_RS_ABORT_NO_STO_SPACE_SKIP

#define DSM_RC_ABORT_MOUNT_NOT_POSSIBLE DSM_RS_ABORT_MOUNT_NOT_POSSIBLE
#define DSM_RC_ABORT_SIZEESTIMATE_EXCEED DSM_RS_ABORT_SIZEESTIMATE_EXCEED
#define DSM_RC_ABORT_DATA_UNAVAILABLE DSM_RS_ABORT_DATA_UNAVAILABLE
#define DSM_RC_ABORT_RETRY DSM_RS_ABORT_RETRY
#define DSM_RC_ABORT_NO_LOG_SPACE DSM_RS_ABORT_NO_LOG_SPACE
#define DSM_RC_ABORT_NO_DB_SPACE DSM_RS_ABORT_NO_DB_SPACE
#define DSM_RC_ABORT_NO_MEMORY DSM_RS_ABORT_NO_MEMORY

#define DSM_RC_ABORT_FS_NOT_DEFINED DSM_RS_ABORT_FS_NOT_DEFINED
#define DSM_RC_ABORT_NODE_ALREADY_DEFED DSM_RS_ABORT_NODE_ALREADY_DEFED
#define DSM_RC_ABORT_NO_DEFAULT_DOMAIN DSM_RS_ABORT_NO_DEFAULT_DOMAIN
#define DSM_RC_ABORT_INVALID_NODENAME DSM_RS_ABORT_INVALID_NODENAME

```

```

#define DSM_RC_ABORT_INVALID_POL_BIND DSM_RS_ABORT_INVALID_POL_BIND
#define DSM_RC_ABORT_DEST_NOT_DEFINED DSM_RS_ABORT_DEST_NOT_DEFINED
#define DSM_RC_ABORT_WAIT_FOR_SPACE DSM_RS_ABORT_WAIT_FOR_SPACE
#define DSM_RC_ABORT_NOT_AUTHORIZED DSM_RS_ABORT_NOT_AUTHORIZED
#define DSM_RC_ABORT_RULE_ALREADY_DEFED DSM_RS_ABORT_RULE_ALREADY_DEFED
#define DSM_RC_ABORT_NO_STOR_SPACE_STOP DSM_RS_ABORT_NO_STOR_SPACE_STOP

#define DSM_RC_ABORT_LICENSE_VIOLATION DSM_RS_ABORT_LICENSE_VIOLATION
#define DSM_RC_ABORT_EXTOBJID_ALREADY_EXISTS DSM_RS_ABORT_EXTOBJID_ALREADY_EXISTS
#define DSM_RC_ABORT_DUPLICATE_OBJECT DSM_RS_ABORT_DUPLICATE_OBJECT

#define DSM_RC_ABORT_INVALID_OFFSET DSM_RS_ABORT_INVALID_OFFSET
#define DSM_RC_ABORT_INVALID_LENGTH DSM_RS_ABORT_INVALID_LENGTH

#define DSM_RC_ABORT_STRING_ERROR DSM_RS_ABORT_STRING_ERROR
#define DSM_RC_ABORT_NODE_NOT_AUTHORIZED DSM_RS_ABORT_NODE_NOT_AUTHORIZED
#define DSM_RC_ABORT_RESTART_NOT_POSSIBLE DSM_RS_ABORT_RESTART_NOT_POSSIBLE
#define DSM_RC_ABORT_RESTORE_IN_PROGRESS DSM_RS_ABORT_RESTORE_IN_PROGRESS
#define DSM_RC_ABORT_SYNTAX_ERROR DSM_RS_ABORT_SYNTAX_ERROR

#define DSM_RC_ABORT_DATA_SKIPPED DSM_RS_ABORT_DATA_SKIPPED
#define DSM_RC_ABORT_EXCEED_MAX_MP DSM_RS_ABORT_EXCEED_MAX_MP
#define DSM_RC_ABORT_NO_OBJSET_MATCH DSM_RS_ABORT_NO_OBJSET_MATCH
#define DSM_RC_ABORT_PVR_ERROR DSM_RS_ABORT_PVR_ERROR
#define DSM_RC_ABORT_BAD_RECOGTOKEN DSM_RS_ABORT_BAD_RECOGTOKEN
#define DSM_RC_ABORT_MERGE_ERROR DSM_RS_ABORT_MERGE_ERROR
#define DSM_RC_ABORT_FSRENAME_ERROR DSM_RS_ABORT_FSRENAME_ERROR
#define DSM_RC_ABORT_INVALID_OPERATION DSM_RS_ABORT_INVALID_OPERATION
#define DSM_RC_ABORT_STGPOOL_UNDEFINED DSM_RS_ABORT_STGPOOL_UNDEFINED
#define DSM_RC_ABORT_INVALID_DATA_FORMAT DSM_RS_ABORT_INVALID_DATA_FORMAT
#define DSM_RC_ABORT_DATAMOVER_UNDEFINED DSM_RS_ABORT_DATAMOVER_UNDEFINED

#define DSM_RC_ABORT_INVALID_MOVER_TYPE DSM_RS_ABORT_INVALID_MOVER_TYPE
#define DSM_RC_ABORT_ITEM_IN_USE DSM_RS_ABORT_ITEM_IN_USE
#define DSM_RC_ABORT_LOCK_CONFLICT DSM_RS_ABORT_LOCK_CONFLICT
#define DSM_RC_ABORT_SRV_PLUGIN_COMM_ERROR DSM_RS_ABORT_SRV_PLUGIN_COMM_ERROR
#define DSM_RC_ABORT_SRV_PLUGIN_OS_ERROR DSM_RS_ABORT_SRV_PLUGIN_OS_ERROR
#define DSM_RC_ABORT_CRC_FAILED DSM_RS_ABORT_CRC_FAILED
#define DSM_RC_ABORT_INVALID_GROUP_ACTION DSM_RS_ABORT_INVALID_GROUP_ACTION
#define DSM_RC_ABORT_DISK_UNDEFINED DSM_RS_ABORT_DISK_UNDEFINED
#define DSM_RC_ABORT_BAD_DESTINATION DSM_RS_ABORT_BAD_DESTINATION
#define DSM_RC_ABORT_DATAMOVER_NOT_AVAILABLE DSM_RS_ABORT_DATAMOVER_NOT_AVAILABLE
#define DSM_RC_ABORT_STGPOOL_COPY_CONT_NO DSM_RS_ABORT_STGPOOL_COPY_CONT_NO
#define DSM_RC_ABORT_RETRY_SINGLE_TXN DSM_RS_ABORT_RETRY_SINGLE_TXN
#define DSM_RC_ABORT_TOC_CREATION_FAIL DSM_RS_ABORT_TOC_CREATION_FAIL
#define DSM_RC_ABORT_TOC_LOAD_FAIL DSM_RS_ABORT_TOC_LOAD_FAIL
#define DSM_RC_ABORT_PATH_RESTRICTED DSM_RS_ABORT_PATH_RESTRICTED
#define DSM_RC_ABORT_NO_LANFREE_SCRATCH DSM_RS_ABORT_NO_LANFREE_SCRATCH
#define DSM_RC_ABORT_INSERT_NOT_ALLOWED DSM_RS_ABORT_INSERT_NOT_ALLOWED
#define DSM_RC_ABORT_DELETE_NOT_ALLOWED DSM_RS_ABORT_DELETE_NOT_ALLOWED
#define DSM_RC_ABORT_TXN_LIMIT_EXCEEDED DSM_RS_ABORT_TXN_LIMIT_EXCEEDED
#define DSM_RC_ABORT_OBJECT_ALREADY_HELD DSM_RS_ABORT_OBJECT_ALREADY_HELD
#define DSM_RC_ABORT_INVALID_CHUNK_REFERENCE DSM_RS_ABORT_INVALID_CHUNK_REFERENCE
#define DSM_RC_ABORT_DESTINATION_NOT_DEDUP DSM_RS_ABORT_DESTINATION_NOT_DEDUP
#define DSM_RC_ABORT_DESTINATION_POOL_CHANGED DSM_RS_ABORT_DESTINATION_POOL_CHANGED
#define DSM_RC_ABORT_NOT_ROOT DSM_RS_ABORT_NOT_ROOT

#define DSM_RC_ABORT_CERTIFICATE_NOT_FOUND DSM_RS_ABORT_CERTIFICATE_NOT_FOUND

/* Definitions for server signon reject codes */
/* These error codes are in the range (51 to 99) inclusive. */
#define DSM_RC_REJECT_NO_RESOURCES 51
#define DSM_RC_REJECT_VERIFIER_EXPIRED 52
#define DSM_RC_REJECT_ID_UNKNOWN 53
#define DSM_RC_REJECT_DUPLICATE_ID 54
#define DSM_RC_REJECT_SERVER_DISABLED 55
#define DSM_RC_REJECT_CLOSED_REGISTER 56
#define DSM_RC_REJECT_CLIENT_DOWNLEVEL 57
#define DSM_RC_REJECT_SERVER_DOWNLEVEL 58
#define DSM_RC_REJECT_ID_IN_USE 59
#define DSM_RC_REJECT_ID_LOCKED 61
#define DSM_RC_SIGNONREJECT_LICENSE_MAX 62
#define DSM_RC_REJECT_NO_MEMORY 63
#define DSM_RC_REJECT_NO_DB_SPACE 64
#define DSM_RC_REJECT_NO_LOG_SPACE 65
#define DSM_RC_REJECT_INTERNAL_ERROR 66
#define DSM_RC_SIGNONREJECT_INVALID_CLI 67 /* client type not licensed */
#define DSM_RC_CLIENT_NOT_ARCHRETPROT 68
#define DSM_RC_REJECT_LASTSESS_CANCELED 69
#define DSM_RC_REJECT_UNICODE_NOT_ALLOWED 70
#define DSM_RC_REJECT_NOT_AUTHORIZED 71
#define DSM_RC_REJECT_TOKEN_TIMEOUT 72

```

```

#define DSM_RC_REJECT_INVALID_NODE_TYPE      73
#define DSM_RC_REJECT_INVALID_SESSIONINIT    74
#define DSM_RC_REJECT_WRONG_PORT            75
#define DSM_RC_CLIENT_NOT_SPMRETPROT        79

#define DSM_RC_USER_ABORT                    101 /* processing aborted by user */
#define DSM_RC_NO_MEMORY                     102 /* no RAM left to complete request */
#define DSM_RC_TA_COMM_DOWN                  2021 /* no longer used */
#define DSM_RC_FILE_NOT_FOUND                104 /* specified file not found */
#define DSM_RC_PATH_NOT_FOUND                105 /* specified path doesn't exist */
#define DSM_RC_ACCESS_DENIED                 106 /* denied due to improper permission */
#define DSM_RC_NO_HANDLES                    107 /* no more file handles available */
#define DSM_RC_FILE_EXISTS                   108 /* file already exists */
#define DSM_RC_INVALID_PARM                  109 /* invalid parameter passed. CRITICAL */
#define DSM_RC_INVALID_HANDLE                110 /* invalid file handle passed */
#define DSM_RC_DISK_FULL                     111 /* out of disk space */
#define DSM_RC_PROTOCOL_VIOLATION            113 /* call protocol violation. CRITICAL */
#define DSM_RC_UNKNOWN_ERROR                 114 /* unknown system error. CRITICAL */
#define DSM_RC_UNEXPECTED_ERROR              115 /* unexpected error. CRITICAL */
#define DSM_RC_FILE_BEING_EXECUTED           116 /* No write is allowed */
#define DSM_RC_DIR_NO_SPACE                  117 /* directory can't be expanded */
#define DSM_RC_LOOPED_SYM_LINK               118 /* too many symbolic links were
                                                    encountered in translating path. */
#define DSM_RC_FILE_NAME_TOO_LONG            119 /* file name too long */
#define DSM_RC_FILE_SPACE_LOCKED             120 /* filespace is locked by the system */
#define DSM_RC_FINISHED                      121 /* finished processing */
#define DSM_RC_UNKNOWN_FORMAT                122 /* unknown format */
#define DSM_RC_NO_AUTHORIZATION              123 /* server response when the client has
                                                    no authorization to read another
                                                    host's owner backup/archive data */
#define DSM_RC_FILE_SPACE_NOT_FOUND           124 /* specified file space not found */
#define DSM_RC_TXN_ABORTED                   125 /* transaction aborted */
#define DSM_RC_SUBDIR_AS_FILE                 126 /* Subdirectory name exists as file */
#define DSM_RC_PROCESS_NO_SPACE              127 /* process has no more disk space. */
#define DSM_RC_PATH_TOO_LONG                 128 /* a directory path being build became
                                                    too long */
#define DSM_RC_NOT_COMPRESSED                129 /* file thought to be compressed is
                                                    actually not */
#define DSM_RC_TOO_MANY_BITS                 130 /* file was compressed using more bits
                                                    then the expander can handle */
#define DSM_RC_SYSTEM_ERROR                  131 /* internal system error */
#define DSM_RC_NO_SERVER_RESOURCES            132 /* server out of resources. */
#define DSM_RC_FS_NOT_KNOWN                  133 /* the file space is not known by the
                                                    server */
#define DSM_RC_NO_LEADING_DIRSEP              134 /* no leading directory separator */
#define DSM_RC_WILDCARD_DIR                  135 /* wildcard character in directory
                                                    path when not allowed */
#define DSM_RC_COMM_PROTOCOL_ERROR           136 /* communications protocol error */
#define DSM_RC_AUTH_FAILURE                  137 /* authentication failure */
#define DSM_RC_TA_NOT_VALID                  138 /* TA not a root and/or SUID program */
#define DSM_RC_KILLED                        139 /* process killed. */

#define DSM_RC_RETRY                         143 /* retry same operation again */

#define DSM_RC_WOULD_BLOCK                    145 /* operation would cause the system to
                                                    block waiting for input. */
#define DSM_RC_TOO_SMALL                     146 /* area for compiled pattern small */
#define DSM_RC_UNCLOSED                      147 /* no closing bracket in pattern */
#define DSM_RC_NO_STARTING_DELIMITER          148 /* pattern has to start with
                                                    directory delimiter */
#define DSM_RC_NEEDED_DIR_DELIMITER           149 /* a directory delimiter is needed
                                                    immediately before and after the
                                                    "match directories" metastring
                                                    ("...") and one wasn't found */
#define DSM_RC_UNKNOWN_FILE_DATA_TYPE         150 /* structured file data type is
                                                    unknown */
#define DSM_RC_BUFFER_OVERFLOW                151 /* data buffer overflow */

#define DSM_RC_NO_COMPRESS_MEMORY             154 /* Compress/Expand out of memory */
#define DSM_RC_COMPRESS_GREW                 155 /* Compression grew */
#define DSM_RC_INV_COMM_METHOD               156 /* Invalid comm method specified */
#define DSM_RC_WILL_ABORT                     157 /* Transaction will be aborted */
#define DSM_RC_FS_WRITE_LOCKED               158 /* File space is write locked */
#define DSM_RC_SKIPPED_BY_USER                159 /* User wanted file skipped in the
                                                    case of ABORT_DATA_OFFLINE */
#define DSM_RC_TA_NOT_FOUND                   160 /* TA not found in it's directory */
#define DSM_RC_TA_ACCESS_DENIED              161 /* Access to TA is denied */
#define DSM_RC_FS_NOT_READY                   162 /* File space not ready */
#define DSM_RC_FS_IS_BAD                     163 /* File space is bad */

```



```

#define DSM_RC_FIO_ERROR 164 /* File input/output error */
#define DSM_RC_WRITE_FAILURE 165 /* Error writing to file */
#define DSM_RC_OVER_FILE_SIZE_LIMIT 166 /* File over system/user limit */
#define DSM_RC_CANNOT_MAKE 167 /* Could not create file/directory,
could be a bad name */
#define DSM_RC_NO_PASS_FILE 168 /* password file needed and user is
not root */
#define DSM_RC_VERFILE_OLD 169 /* password stored locally doesn't
match the one at the host */
#define DSM_RC_INPUT_ERROR 173 /* unable to read keyboard input */
#define DSM_RC_REJECT_PLATFORM_MISMATCH 174 /* Platform name doesn't match
up with what the server says
is the platform for the client */
#define DSM_RC_TL_NOT_FILE_OWNER 175 /* User trying to backup a file is not
the file's owner. */
#define DSM_RC_COMPRESSED_DATA_CORRUPTED 176 /* Compressed data is corrupted */
#define DSM_RC_UNMATCHED_QUOTE 177 /* missing starting or ending quote */

#define DSM_RC_SIGNON_FAILOVER_MODE 178 /* Failed over to the replication server,
running in failover mode */
#define DSM_RC_FAILOVER_MODE_FUNC_BLOCKED 179 /* function is blocked because
session is in failover mode */

/*-----*/
/* Return codes 180-199 are reserved for Policy Set handling */
/*-----*/
#define DSM_RC_PS_MULTBCG 181 /* Multiple backup copy groups in 1 MC*/
#define DSM_RC_PS_MULTACG 182 /* Multiple arch. copy groups in 1 MC*/
#define DSM_RC_PS_NODFLTMC 183 /* Default MC name not in policy set */
#define DSM_RC_TL_NOBCG 184 /* Backup req, no backup copy group */
#define DSM_RC_TL_EXCLUDED 185 /* Backup req, excl. by in/ex filter */
#define DSM_RC_TL_NOACG 186 /* Archive req, no archive copy group */
#define DSM_RC_PS_INVALID_ARCHMC 187 /* Invalid MC name in archive override*/
#define DSM_RC_NO_PS_DATA 188 /* No policy set data on the server */
#define DSM_RC_PS_INVALID_DIRMC 189 /* Invalid directory MC specified in
the options file. */
#define DSM_RC_PS_NO_CG_IN_DIR_MC 190 /* No backup copy group in directory MC.
Must specify an MC using DirMC
option. */

#define DSM_RC_WIN32_UNSUPPORTED_FILE_TYPE 280 /* File is not of
Win32 type FILE_TYPE_DISK */

/*-----*/
/* Return codes for the Trusted Communication Agent */
/*-----*/
#define DSM_RC_TCA_NOT_ROOT 161 /* Access to TA is denied */
#define DSM_RC_TCA_ATTACH_SHR_MEM_ERR 200 /* Error attaching shared memory */
#define DSM_RC_TCA_SHR_MEM_BLOCK_ERR 200 /* Shared memory block error */
#define DSM_RC_TCA_SHR_MEM_IN_USE 200 /* Shared memory block error */
#define DSM_RC_TCA_SHARED_MEMORY_ERROR 200 /* Shared memory block error */
#define DSM_RC_TCA_SEGMENT_MISMATCH 200 /* Shared memory block error */
#define DSM_RC_TCA_FORK_FAILED 292 /* Error forking off TCA process */
#define DSM_RC_TCA_DIED 294 /* TCA died unexpectedly */
#define DSM_RC_TCA_INVALID_REQUEST 295 /* Invalid request sent to TCA */
#define DSM_RC_TCA_SEMGET_ERROR 297 /* Error getting semaphores */
#define DSM_RC_TCA_SEM_OP_ERROR 298 /* Error in semaphore set or wait */
#define DSM_RC_TCA_NOT_ALLOWED 299 /* TCA not allowed (multi thread) */

/*-----*/
/* 400-430 for options */
/*-----*/
#define DSM_RC_INVALID_OPT 400 /* invalid option */
#define DSM_RC_NO_HOST_ADDR 405 /* Not enuf info to connect server */
#define DSM_RC_NO_OPT_FILE 406 /* No default user configuration file*/
#define DSM_RC_MACHINE_SAME 408 /* -MACHINENAME same as real name */
#define DSM_RC_INVALID_SERVER 409 /* Invalid server name from client */
#define DSM_RC_INVALID_KEYWORD 410 /* Invalid option keyword */
#define DSM_RC_PATTERN_TOO_COMPLEX 411 /* Can't match Include/Exclude entry*/
#define DSM_RC_NO_CLOSING_BRACKET 412 /* Missing closing bracket inc/excl */
#define DSM_RC_OPT_CLIENT_NOT_ACCEPTING 417/* Client doesn't accept this option
from the server */
#define DSM_RC_OPT_CLIENT_DOES_NOT_WANT 418/* Client doesn't want this value
from the server */
#define DSM_RC_OPT_NO_INCLEXCL_FILE 419 /* inclexcl file not found */
#define DSM_RC_OPT_OPEN_FAILURE 420 /* can't open file */
#define DSM_RC_OPT_INV_NODENAME 421/* used for Windows if nodename=local
machine when CLUSTERMODE=YES */
#define DSM_RC_OPT_NODENAME_INVALID 423/* generic invalid nodename */
#define DSM_RC_OPT_ERRORLOG_CONFLICT 424/* both logmax & retention specified */
#define DSM_RC_OPT_SCHEDLOG_CONFLICT 425/* both logmax & retention specified */
#define DSM_RC_CANNOT_OPEN_TRACEFILE 426/* cannot open trace file */

```

```

#define DSM_RC_CANNOT_OPEN_LOGFILE 427/* cannot open error log file */
#define DSM_RC_OPT_SESSINIT_LF_CONFLICT 428/* both sessioninit=server and
enablelanfree=yes are specified*/
#define DSM_RC_OPT_OPTION_IGNORE 429/* option will be ignored */
#define DSM_RC_OPT_DEDUP_CONFLICT 430/* cannot open error log file */
#define DSM_RC_OPT_HSMLOG_CONFLICT 431/* both logmax & retention specified */

/*-----*/
/* 600 to 610 for volume label codes */
/*-----*/
#define DSM_RC_DUP_LABEL 600 /* duplicate volume label found */
#define DSM_RC_NO_LABEL 601 /* drive has no label */

/*-----*/
/* Return codes for message file processing */
/*-----*/
#define DSM_RC_NLS_CANT_OPEN_TXT 610 /* error trying to open msg txt file */
#define DSM_RC_NLS_CANT_READ_HDR 611 /* error trying to read header */
#define DSM_RC_NLS_INVALID_CNTL_REC 612 /* invalid control record */
#define DSM_RC_NLS_INVALID_DATE_FMT 613 /* invalid default date format */
#define DSM_RC_NLS_INVALID_TIME_FMT 614 /* invalid default time format */
#define DSM_RC_NLS_INVALID_NUM_FMT 615 /* invalid default number format */

/*-----*/
/* Return codes 620-630 are reserved for log message return codes */
/*-----*/
#define DSM_RC_LOG_CANT_BE_OPENED 620 /* error trying to open error log */
#define DSM_RC_LOG_ERROR_WRITING_TO_LOG 621 /* error occurred writing to
log file */
#define DSM_RC_LOG_NOT_SPECIFIED 622 /* no error log file was specified */

/*-----*/
/* Return codes 900-999 IBM STORAGE PROTECT CLIENT ONLY */
/*-----*/
#define DSM_RC_NOT_ADSM_AUTHORIZED 927 /* Must be ADSM authorized to perform*/
/* action : root user or pwd auth */
#define DSM_RC_REJECT_USERID_UNKNOWN 940 /* userid unknown on server */
#define DSM_RC_FILE_IS_SYMLINK 959 /* errorlog or trace is a symbolic
link */
*/

#define DSM_RC_DIRECT_STORAGE_AGENT_UNSUPPORTED 961 /* Direct connection to SA not supported */
#define DSM_RC_FS_NAMESPACE_DOWNLEVEL 963 /* Long namespace has been removed from
from the Netware volume */
#define DSM_RC_CONTINUE_NEW_CONSUMER 972 /* Continue processing using a new consumer */
#define DSM_RC_CONTINUE_NEW_CONSUMER_NODEDUP 973 /* Continue processing using a new consumer no dedup*/
#define DSM_RC_CONTINUE_NEW_CONSUMER_NOCOMPRESS 976 /* Continue processing using a new consumer no
compression */

#define DSM_RC_SERVER_SUPPORTS_FUNC 994 /* the server supports this function */
#define DSM_RC_SERVER_AND_SA_SUPPORT_FUNC 995 /* Both server and SA support func */
#define DSM_RC_SERVER_DOWNLEVEL_FUNC 996 /* The server is downlevel for func */
#define DSM_RC_STORAGEAGENT_DOWNLEVEL 997 /* the storage agent is downlevel */
#define DSM_RC_SERVER_AND_SA_DOWNLEVEL 998 /* both server and SA downlevel */

/* TCP/IP error codes */
#define DSM_RC_TCPIP_FAILURE -50 /* TCP/IP communications failure */
#define DSM_RC_CONN_TIMEDOUT -51 /* TCP/IP connection attempt timedout */
#define DSM_RC_CONN_REFUSED -52 /* TCP/IP connection refused by host */
#define DSM_RC_BAD_HOST_NAME -53 /* TCP/IP invalid host name specified */
#define DSM_RC_NETWORK_UNREACHABLE -54 /* TCP/IP host name unreachable */
#define DSM_RC_WINSOCK_MISSING -55 /* TCP/IP WINSOCK.DLL missing */
#define DSM_RC_TCPIP_DLL_LOADFAILURE -56 /* Error from LoadLibrary */
#define DSM_RC_TCPIP_LOADFAILURE -57 /* Error from GetProcAddress */
#define DSM_RC_TCPIP_USER_ABORT -58 /* User aborted while in TCP/IP layer */

/*-----*/
/* Return codes (-71)-(-90) are reserved for CommTSM error codes */
/*-----*/
#define DSM_RC_TSM_FAILURE -71 /* IBM Storage Protect comm failure */
#define DSM_RC_TSM_ABORT -72 /* Session aborted abnormally */

/*comm3270 error codes - no longer used*/
#define DSM_RC_COMM_TIMEOUT 2021 /* no longer used */
#define DSM_RC_EMULATOR_INACTIVE 2021 /* no longer used */
#define DSM_RC_BAD_HOST_ID 2021 /* no longer used */
#define DSM_RC_HOST_SESS_BUSY 2021 /* no longer used */
#define DSM_RC_3270_CONNECT_FAILURE 2021 /* no longer used */

```

```

#define DSM_RC_NO_ACS3ELKE_DLL      2021 /* no longer used */
#define DSM_RC_EMULATOR_ERROR      2021 /* no longer used */
#define DSM_RC_EMULATOR_BACKLEVEL  2021 /* no longer used */
#define DSM_RC_CKSUM_FAILURE         2021 /* no longer used */

/* The following Return codes are for EHLLAPI for Windows */
#define DSM_RC_3270COMMErrors_DLL    2021 /* no longer used */
#define DSM_RC_3270COMMErrors_GetProc 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_DLL     2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_GetProc 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_HostConnect 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_AllocBuff 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_SendKey 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_PacketChk 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_ChkSum 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_HostTimeOut 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_Send 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_Recv 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_General 2021 /* no longer used */
#define DSM_RC_PC3270_MISSING_DLL    2021 /* no longer used */
#define DSM_RC_3270COMM_MISSING_DLL  2021 /* no longer used */

/* NETBIOS error codes */
#define DSM_RC_NETB_ERROR             -151 /* Could not add node to LAN */
#define DSM_RC_NETB_NO_DLL            -152 /* The ACSNETB.DLL could not be loaded */
#define DSM_RC_NETB_LAN_ERR          -155 /* LAN error detected */
#define DSM_RC_NETB_NAME_ERR         -158 /* Netbios error on Add Name */
#define DSM_RC_NETB_TIMEOUT          -159 /* Netbios send timeout */
#define DSM_RC_NETB_NOTINST          -160 /* Netbios not installed - DOS */
#define DSM_RC_NETB_REBOOT           -161 /* Netbios config err - reboot DOS */

/* Named Pipe error codes */
#define DSM_RC_NP_ERROR               -190

/* CPIC error codes */
#define DSM_RC_CPIC_ALLOCATE_FAILURE  2021 /* no longer used */
#define DSM_RC_CPIC_TYPE_MISMATCH     2021 /* no longer used */
#define DSM_RC_CPIC_PIP_NOT_SPECIFY_ERR 2021 /* no longer used */
#define DSM_RC_CPIC_SECURITY_NOT_VALID 2021 /* no longer used */
#define DSM_RC_CPIC_SYNC_LVL_NO_SUPPORT 2021 /* no longer used */
#define DSM_RC_CPIC_TPN_NOT_RECOGNIZED 2021 /* no longer used */
#define DSM_RC_CPIC_TP_ERROR          2021 /* no longer used */
#define DSM_RC_CPIC_PARAMETER_ERROR    2021 /* no longer used */
#define DSM_RC_CPIC_PROD_SPECIFIC_ERR  2021 /* no longer used */
#define DSM_RC_CPIC_PROGRAM_ERROR      2021 /* no longer used */
#define DSM_RC_CPIC_RESOURCE_ERROR     2021 /* no longer used */
#define DSM_RC_CPIC_DEALLOCATE_ERROR   2021 /* no longer used */
#define DSM_RC_CPIC_SVC_ERROR          2021 /* no longer used */
#define DSM_RC_CPIC_PROGRAM_STATE_CHECK 2021 /* no longer used */
#define DSM_RC_CPIC_PROGRAM_PARAM_CHECK 2021 /* no longer used */
#define DSM_RC_CPIC_UNSUCCESSFUL       2021 /* no longer used */
#define DSM_RC_UNKNOWN_CPIC_PROBLEM    2021 /* no longer used */
#define DSM_RC_CPIC_MISSING_LU         2021 /* no longer used */
#define DSM_RC_CPIC_MISSING_TP         2021 /* no longer used */
#define DSM_RC_CPIC_SNA6000_LOAD_FAIL  2021 /* no longer used */
#define DSM_RC_CPIC_STARTUP_FAILURE     2021 /* no longer used */

/*-----*/
/* Return codes -300 to -307 are reserved for IPX/SPX communications */
/*-----*/
#define DSM_RC_TLI_ERROR              2021 /* no longer used */
#define DSM_RC_IPXSPX_FAILURE         2021 /* no longer used */
#define DSM_RC_TLI_DLL_MISSING        2021 /* no longer used */
#define DSM_RC_DLL_LOADFAILURE        2021 /* no longer used */
#define DSM_RC_DLL_FUNCTION_LOADFAILURE 2021 /* no longer used */
#define DSM_RC_IPXCONN_REFUSED        2021 /* no longer used */
#define DSM_RC_IPXCONN_TIMEOUT        2021 /* no longer used */
#define DSM_RC_IPXADDR_UNREACHABLE    2021 /* no longer used */
#define DSM_RC_CPIC_MISSING_DLL       2021 /* no longer used */
#define DSM_RC_CPIC_DLL_LOADFAILURE   2021 /* no longer used */
#define DSM_RC_CPIC_FUNC_LOADFAILURE  2021 /* no longer used */

/*=== Shared Memory Protocol error codes ===*/
#define DSM_RC_SHM_TCPIP_FAILURE      -450
#define DSM_RC_SHM_FAILURE            -451
#define DSM_RC_SHM_NOTAUTH            -452

#define DSM_RC_NULL_OBJNAME           2000 /* Object name pointer is NULL */
#define DSM_RC_NULL_DATA_BLKPTR       2001 /* dataBlkPtr is NULL */
#define DSM_RC_NULL_MSG               2002 /* msg parm in dsmRCMsg is NULL */

```

```

#define DSM_RC_NULL_OBJATTRPTR      2004 /* Object Attr Pointer is NULL      */
#define DSM_RC_NO_SESS_BLK           2006 /* no server session info          */
#define DSM_RC_NO_POLICY_BLK         2007 /* no policy hdr info              */
#define DSM_RC_ZERO_BUFLEN           2008 /* bufferLen is zero for dataBlkPtr */
#define DSM_RC_NULL_BUFPTR           2009 /* bufferPtr is NULL for dataBlkPtr */

#define DSM_RC_INVALID_OBJTYPE       2010 /* invalid object type             */
#define DSM_RC_INVALID_VOTE          2011 /* invalid vote                    */
#define DSM_RC_INVALID_ACTION        2012 /* invalid action                  */
#define DSM_RC_INVALID_DS_HANDLE     2014 /* invalid ADSM handle            */
#define DSM_RC_INVALID_REPOS         2015 /* invalid value for repository    */
#define DSM_RC_INVALID_FSNAME        2016 /* fs should start with dir delim  */
#define DSM_RC_INVALID_OBJNAME       2017 /* invalid full path name         */
#define DSM_RC_INVALID_LLNAME        2018 /* ll should start with dir delim  */
#define DSM_RC_INVALID_OBJOWNER      2019 /* invalid object owner name       */
#define DSM_RC_INVALID_ACTYPE        2020 /* invalid action type            */
#define DSM_RC_INVALID_RETCODE       2021 /* dsmRC in dsmRCMsg is invalid   */
#define DSM_RC_INVALID_SENDTYPE      2022 /* invalid send type              */
#define DSM_RC_INVALID_PARAMETER     2023 /* invalid parameter              */
#define DSM_RC_INVALID_OBJSTATE      2024 /* active, inactive, or any match? */
#define DSM_RC_INVALID_MCNAM        2025 /* Mgmt class name not found      */
#define DSM_RC_INVALID_DRIVE_CHAR    2026 /* Drive letter is not alphabet   */
#define DSM_RC_NULL_FSNAME           2027 /* Filespace name is NULL         */
#define DSM_RC_INVALID_HLNAME        2028 /* hl should start with dir delim  */

#define DSM_RC_NUMOBJ_EXCEED         2029 /* BeginGetData num objs exceeded */

#define DSM_RC_NEWPW_REQD            2030 /* new password is required        */
#define DSM_RC_OLDPW_REQD            2031 /* old password is required        */
#define DSM_RC_NO_OWNER_REQD         2032 /* owner not allowed. Allow default */
#define DSM_RC_NO_NODE_REQD          2033 /* node not allowed w/ pw=generate */
#define DSM_RC_KEY_MISSING            2034 /* key file can't be found        */
#define DSM_RC_KEY_BAD               2035 /* content of key file is bad     */

#define DSM_RC_BAD_CALL_SEQUENCE     2041 /* Sequence of DSM calls not allowed */
#define DSM_RC_INVALID_TSMBUFFER     2042 /* invalid value for tsmbuffhandle or dataPtr */
#define DSM_RC_TOO_MANY_BYTES        2043 /* too many bytes copied to buffer */
#define DSM_RC_MUST_RELEASE_BUFFER    2044 /* cant exit app needs to release buffers */
#define DSM_RC_BUFF_ARRAY_ERROR       2045 /* internal buff array error      */
#define DSM_RC_INVALID_DATABLK       2046 /* using tsmbuff datablk should be null */
#define DSM_RC_ENCR_NOT_ALLOWED       2047 /* when using tsmbuffers encryption not allowed */
#define DSM_RC_OBJ_COMPRESSED        2048 /* Can't restore using tsmBuff on compressed object */
#define DSM_RC_OBJ_ENCRYPTED          2049 /* Cant restore using tsmbuff an encr obj */
#define DSM_RC_WILDCHAR_NOTALLOWED   2050 /* Wild card not allowed for hl,ll */
#define DSM_RC_POR_NOT_ALLOWED        2051 /* Can't use partial object restore with tsmBuffers */
#define DSM_RC_NO_ENCRYPTION_KEY      2052 /* Encryption key not found*/
#define DSM_RC_ENCR_CONFLICT          2053 /* mutually exclusive options */

#define DSM_RC_FSNAME_NOTFOUND        2060 /* Filespace name not found        */
#define DSM_RC_FS_NOT_REGISTERED      2061 /* Filespace name not registered    */
#define DSM_RC_FS_ALREADY_REGED       2062 /* Filespace already registered     */
#define DSM_RC_OBJID_NOTFOUND         2063 /* No object id to restore         */
#define DSM_RC_WRONG_VERSION          2064 /* Wrong level of code             */
#define DSM_RC_WRONG_VERSION_PARM     2065 /* Wrong level of parameter struct */

#define DSM_RC_NEEDTO_ENDTXN          2070 /* Need to call dsmEndTxn          */

#define DSM_RC_OBJ_EXCLUDED           2080 /* Object is excluded by MC         */
#define DSM_RC_OBJ_NOBCG              2081 /* Object has no backup copy group  */
#define DSM_RC_OBJ_NOACG              2082 /* Object has no archive copy group */

#define DSM_RC_APISYSTEM_ERROR        2090 /* API internal error              */

#define DSM_RC_DESC_TOOLONG           2100 /* description is too long          */
#define DSM_RC_OBJINFO_TOOLONG        2101 /* object attr objinfo too long    */
#define DSM_RC_HL_TOOLONG             2102 /* High level qualifier is too long */
#define DSM_RC_PASSWD_TOOLONG         2103 /* password is too long            */
#define DSM_RC_FILESPACE_TOOLONG      2104 /* filespace name is too long      */
#define DSM_RC_LL_TOOLONG             2105 /* Low level qualifier is too long  */
#define DSM_RC_FSINFO_TOOLONG         2106 /* filespace length is too big     */
#define DSM_RC_SENDDATA_WITH_ZERO_SIZE 2107 /* send data w/ zero est           */

/*=== new return codes for dsmaccess ===*/
#define DSM_RC_INVALID_ACCESS_TYPE    2110 /* invalid access type             */
#define DSM_RC_QUERY_COMM_FAILURE     2111 /* communication error during query */
#define DSM_RC_NO_FILES_BACKUP        2112 /* No backed up files for this fs   */
#define DSM_RC_NO_FILES_ARCHIVE       2113 /* No archived files for this fs    */
#define DSM_RC_INVALID_SETACCESS      2114 /* invalid set access format        */

/*=== new return codes for dsmaccess ===*/

```

```

#define DSM_RC_STRING_TOO_LONG      2120 /* String parameter too long */
#define DSM_RC_MORE_DATA             2200 /* There are more data to restore */
#define DSM_RC_BUFF_TOO_SMALL       2210 /* DataBlk buffer too small for qry */
#define DSM_RC_NO_API_CONFIGFILE     2228 /*specified API config file not found*/
#define DSM_RC_NO_INCLEXCL_FILE      2229 /* specified inclexcl file not found*/
#define DSM_RC_NO_SYS_OR_INCLEXCL    2230 /* either dsm.sys or inclexcl file
                                           specified in dsm.sys not found */
#define DSM_RC_REJECT_NO_POR_SUPPORT 2231 /* server doesn't have POR support*/

#define DSM_RC_NEED_ROOT             2300 /* API caller must be root */
#define DSM_RC_NEEDTO_CALL_BINDMC    2301 /* dsmBindMC must be called first */
#define DSM_RC_CHECK_REASON_CODE     2302 /* check reason code from dsmEndTxn */
#define DSM_RC_NEEDTO_ENDTXN_DEDUP_SIZE_EXCEEDED 2303 /* max dedup bytes exceeded */

/*=== return codes 2400 - 2410 used by lic file see agentrc.h ===*/

/*=== return codes 2410 - 2430 used by Oracle agent see agentrc.h ===*/

#define DSM_RC_ENC_WRONG_KEY         4580 /* the key provided is incorrect */
#define DSM_RC_ENC_NOT_AUTHORIZED    4582 /* user is not allowed to decrypt */
#define DSM_RC_ENC_TYPE_UNKNOWN      4584 /* encryption type unknown */

/*=====
Return codes (4600)-(4624) are reserved for clustering
=====*/
#define DSM_RC_CLUSTER_INFO_LIBRARY_NOT_LOADED 4600
#define DSM_RC_CLUSTER_LIBRARY_INVALID         4601
#define DSM_RC_CLUSTER_LIBRARY_NOT_LOADED     4602
#define DSM_RC_CLUSTER_NOT_MEMBER_OF_CLUSTER  4603
#define DSM_RC_CLUSTER_NOT_ENABLED            4604
#define DSM_RC_CLUSTER_NOT_SUPPORTED          4605
#define DSM_RC_CLUSTER_UNKNOWN_ERROR          4606

/*=====
Return codes (5200)-(5600) are reserved for new Server ABORT codes (dsmcomm.h)
=====*/
#define DSM_RS_ABORT_CERTIFICATE_NOT_FOUND 5200

/*=====
Return codes (5701)-(5749) are reserved for proxy
=====*/
#define DSM_RC_PROXY_REJECT_NO_RESOURCES      5702
#define DSM_RC_PROXY_REJECT_DUPLICATE_ID      5705
#define DSM_RC_PROXY_REJECT_ID_IN_USE         5710
#define DSM_RC_PROXY_REJECT_INTERNAL_ERROR    5717
#define DSM_RC_PROXY_REJECT_NOT_AUTHORIZED    5722
#define DSM_RC_PROXY_INVALID_FROMNODE         5746
#define DSM_RC_PROXY_INVALID_SERVERFREE       5747
#define DSM_RC_PROXY_INVALID_CLUSTER          5748
#define DSM_RC_PROXY_INVALID_FUNCTION          5749

/*=====
Return codes 5801 - 5849 are reserved for cryptography/security
=====*/
#define DSM_RC_CRYPTO_ICC_ERROR               5801
#define DSM_RC_CRYPTO_ICC_CANNOT_LOAD         5802
#define DSM_RC_SSL_NOT_SUPPORTED              5803
#define DSM_RC_SSL_INIT_FAILED                5804
#define DSM_RC_SSL_KEYFILE_OPEN_FAILED        5805
#define DSM_RC_SSL_KEYFILE_BAD_PASSWORD       5806
#define DSM_RC_SSL_BAD_CERTIFICATE            5807

/*=====
Return codes 6300 - 6399 are reserved for client-side deduplication
=====*/
#define DSM_RC_DIGEST_VALIDATION_ERROR        6300 /* End-to-end digest validation err */
#define DSM_RC_DATA_FINGERPRINT_ERROR         6301 /* Failure in Rabin fingerprinting */
#define DSM_RC_DATA_DEDUP_ERROR               6302 /* Error converting data into chunks */

#endif /* _H_DSMRC */

```

## Related information

[API return codes](#)

[illegible]



```

/*-----+
| Definitions for "member type" field in tsmGroupHandlerIn_t |
+-----*/

#define DSM_MEMBERTYPE_LEADER      0x01  /* group leader */
#define DSM_MEMBERTYPE_MEMBER     0x02  /* group member */

/*-----+
| Definitions for "operation type" field in tsmGroupHandlerIn_t |
+-----*/
#define DSM_GROUP_ACTION_BEGIN      0x01
#define DSM_GROUP_ACTION_OPEN      0x02 /* create new group */
#define DSM_GROUP_ACTION_CLOSE     0x03 /* commit and save an open group */
#define DSM_GROUP_ACTION_ADD       0x04 /* Append to a group */
#define DSM_GROUP_ACTION_ASSIGNTO  0x05 /* Assign to a another group */
#define DSM_GROUP_ACTION_REMOVE    0x06 /* remove a member from a group */

/*-----+
| Values for copySer in DetailCG structures for Query Mgmt Class response |
+-----*/
#define Copy_Serial_Static         1  /*Copy Serialization Static */
#define Copy_Serial_Shared_Static  2  /*Copy Serialization Shared Static*/
#define Copy_Serial_Shared_Dynamic 3  /*Copy Serialization Shared Dynamic*/
#define Copy_Serial_Dynamic        4  /*Copy Serialization Dynamic */

/*-----+
| Values for copyMode in DetailCG structures for Query Mgmt Class response |
+-----*/
#define Copy_Mode_Modified         1  /*Copy Mode Modified */
#define Copy_Mode_Absolute         2  /*Copy Mode Absolute */

/*-----+
| Values for objState in qryBackupData structure |
+-----*/
#define DSM_ACTIVE                 0x01 /* query only active objects */
#define DSM_INACTIVE               0x02 /* query only inactive objects */
#define DSM_ANY_MATCH              0xFF /* query all backup objects */

/*-----+
| Boundary values for dsmDate.year field in qryArchiveData structure |
+-----*/
#define DATE_MINUS_INFINITE        0x0000 /* lowest boundary */
#define DATE_PLUS_INFINITE         0xFFFF /* highest upper boundary */

/*-----+
| Bits masks for update action parameter on dsmUpdateFS() |
+-----*/
#define DSM_FSUPD_FSTYPE           ((unsigned) 0x00000002)
#define DSM_FSUPD_FSINFO           ((unsigned) 0x00000004)
#define DSM_FSUPD_BACKSTARTDATE    ((unsigned) 0x00000008)
#define DSM_FSUPD_BACKCOMPLETEDATE ((unsigned) 0x00000010)
#define DSM_FSUPD_OCCUPANCY        ((unsigned) 0x00000020)
#define DSM_FSUPD_CAPACITY         ((unsigned) 0x00000040)
#define DSM_FSUPD_RESERVED1        ((unsigned) 0x00000100)

/*-----+
| Bits mask for backup update action parameter on dsmUpdateObj() |
+-----*/
#define DSM_BACKUPD_OWNER          ((unsigned) 0x00000001)
#define DSM_BACKUPD_OBJINFO        ((unsigned) 0x00000002)
#define DSM_BACKUPD_MC              ((unsigned) 0x00000004)

#define DSM_ARCHUPD_OWNER          ((unsigned) 0x00000001)
#define DSM_ARCHUPD_OBJINFO        ((unsigned) 0x00000002)
#define DSM_ARCHUPD_DESCR          ((unsigned) 0x00000004)

/*-----+
| Values for repository parameter on dsmDeleteFS() |
+-----*/
#define DSM_ARCHIVE_REP            0x0A /* archive repository */
#define DSM_BACKUP_REP             0x0B /* backup repository */
#define DSM_REPOS_ALL              0x01 /* all repository types */

/*-----+
| Values for vote parameter on dsmEndTxn() |
+-----*/
#define DSM_VOTE_COMMIT            1  /* commit current transaction */
#define DSM_VOTE_ABORT             2  /* roll back current transaction */

/*-----+
| Values for various flags returned in ApiSessInfo structure. |
+-----*/

```



```

+-----*/
/* Client compression field codes */
#define COMPRESS_YES 1 /* client must compress data */
#define COMPRESS_NO 2 /* client must NOT compress data */
#define COMPRESS_CD 3 /* client determined */

/* Archive delete permission codes. */
#define ARCHDEL_YES 1 /* archive delete allowed */
#define ARCHDEL_NO 2 /* archive delete NOT allowed */

/* Backup delete permission codes. */
#define BACKDEL_YES 1 /* backup delete allowed */
#define BACKDEL_NO 2 /* backup delete NOT allowed */

/*-----+
| Values for various flags returned in optStruct structure. |
+-----*/
#define DSM_PASSWD_GENERATE 1
#define DSM_PASSWD_PROMPT 0

#define DSM_COMM_TCP 1 /* tcpip */
#define DSM_COMM_NAMEDPIPE 2 /* Named pipes */
#define DSM_COMM_SHM 3 /* Shared Memory */

/* obsolete commmethods */
#define DSM_COMM_PVM_IUCV 12
#define DSM_COMM_3270 12
#define DSM_COMM_IUCV 12
#define DSM_COMM_PWSCS 12
#define DSM_COMM_SNA_LU6_2 12
#define DSM_COMM_IPXSPX 12 /* For IPX/SPX support */
#define DSM_COMM_NETBIOS 12 /* NETBIOS */
#define DSM_COMM_400COMM 12
#define DSM_COMM_CLIO 12 /* CLIO/S */

/*-----+
| Values for userNameAuthorities in dsmInitEx for future use |
+-----*/
#define DSM_USERAUTH_NONE ((dsInt16_t)0x0000)
#define DSM_USERAUTH_ACCESS ((dsInt16_t)0x0001)
#define DSM_USERAUTH_OWNER ((dsInt16_t)0x0002)
#define DSM_USERAUTH_POLICY ((dsInt16_t)0x0004)
#define DSM_USERAUTH_SYSTEM ((dsInt16_t)0x0008)

/*-----+
| Values for encryptionType on dsmEndSendObjEx, queryResp |
+-----*/
#define DSM_ENCRYPT_NO ((dsUInt8_t)0x00)
#define DSM_ENCRYPT_USER ((dsUInt8_t)0x01)
#define DSM_ENCRYPT_CLIENTENCRKEY ((dsUInt8_t)0x02)
#define DSM_ENCRYPT_DES_56BIT ((dsUInt8_t)0x04)
#define DSM_ENCRYPT_AES_128BIT ((dsUInt8_t)0x08)
#define DSM_ENCRYPT_AES_256BIT ((dsUInt8_t)0x10)

/*-----+
| Definitions for mediaClass field. |
+-----*/
/*
 * The following constants define a hierarchy of media access classes.
 * Lower numbers indicate media which can supply faster access to data.
 */

/* Fixed: represents the class of on-line, fixed media (such as
hard disks). */
#define MEDIA_FIXED 0x10

/* Library: represents the class of mountable media accessible
through a mechanical mounting device. */
#define MEDIA_LIBRARY 0x20

/* future use */
#define MEDIA_NETWORK 0x30

/* future use */
#define MEDIA_SHELF 0x40

/* future use */
#define MEDIA_OFFSITE 0x50

/* future use */
#define MEDIA_UNAVAILABLE 0xF0

```

```

/*-----+
| Type definition for partial object data for dsmBeginGetData() |
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;          /* Structure version */
    dsStruct64_t  partialObjOffset;    /* offset into object to begin reading */
    dsStruct64_t  partialObjLength;    /* amount of object to read */
} PartialObjData ;                  /* partial object data */

#define PartialObjDataVersion 1 /*

/*-----+
| Type definition for date structure |
+-----*/
typedef struct
{
    dsUInt16_t    year;                /* year, 16-bit integer (e.g., 1990) */
    dsUInt8_t     month;               /* month, 8-bit integer (1 - 12) */
    dsUInt8_t     day;                /* day, 8-bit integer (1 - 31) */
    dsUInt8_t     hour;               /* hour, 8-bit integer (0 - 23) */
    dsUInt8_t     minute;             /* minute, 8-bit integer (0 - 59) */
    dsUInt8_t     second;             /* second, b-bit integer (0 - 59) */
} dsmDate ;

/*-----+
| Type definition for Object ID on dsmGetObj() and in dsmGetList structure |
+-----*/
typedef dsStruct64_t  ObjID ;

/*-----+
| Type definition for dsmQueryBuff on dsmBeginQuery() |
+-----*/
typedef void dsmQueryBuff ;

/*-----+
| Type definition for dsmGetType parameter on dsmBeginGetData() |
+-----*/
typedef enum
{
    gtBackup = 0x00,                /* Backup processing type */
    gtArchive                /* Archive processing type */
} dsmGetType ;

/*-----+
| Type definition for dsmQueryType parameter on dsmBeginQuery() |
+-----*/
typedef enum
{
    qtArchive = 0x00,                /* Archive query type */
    qtBackup,                        /* Backup query type */
    qtBackupActive,                  /* Fast query for active backup files */
    qtFilespace,                    /* Filespace query type */
    qtMC,                            /* Mgmt. class query type */
    qtReserved1,                    /* future use */
    qtReserved2,                    /* future use */
    qtReserved3,                    /* future use */
    qtReserved4,                    /* future use */
    qtBackupGroups,                 /* group leaders in a specific fs */
    qtOpenGroups,                   /* Open groups in a specific fs */
    qtReserved5,                    /* future use */
    qtProxyNodeAuth,                /* nodes that his node can proxy to */
    qtProxyNodePeer,                /* Peer nodes with the same target */
    qtReserved6,                    /* future use */
    qtReserved7,                    /* future use */
    qtReserved8,                    /* future use */
} dsmQueryType ;

/*-----+
| Type definition sendType parameter on dsmBindMC() and dsmSendObj() |
+-----*/
typedef enum
{
    stBackup = 0x00,                /* Backup processing type */
    stArchive,                        /* Archive processing type */
    stBackupMountWait,                /* Backup processing with mountwait on */
    stArchiveMountWait                /* Archive processing with mountwait on */
} dsmSendType ;

/*-----+
| Type definition for delType parameter on dsmDeleteObj() |

```

```

+-----*/
typedef enum
{
    dtArchive = 0x00,                /* Archive delete type */
    dtBackup,                        /* Backup delete (deactivate) type */
    dtBackupID                       /* Backup delete (remove) type */
}dsmDelType ;

/*-----+
| Type definition sendType parameter on dsmSetAccess() |
+-----*/
typedef enum
{
    atBackup = 0x00,                /* Backup processing type */
    atArchive                       /* Archive processing type */
}dsmAccessType;

/*-----+
| Type definition for API Version on dsmInit() and dsmQueryApiVersion() |
+-----*/
typedef struct
{
    dsUint16_t version;             /* API version */
    dsUint16_t release;            /* API release */
    dsUint16_t level;              /* API level */
}dsmApiVersion;

/*-----+
| Type definition for API Version on dsmInit() and dsmQueryApiVersion() |
+-----*/
typedef struct
{
    dsUint16_t stVersion;           /* Structure version */
    dsUint16_t version;            /* API version */
    dsUint16_t release;            /* API release */
    dsUint16_t level;             /* API level */
    dsUint16_t subLevel;          /* API sub level */
    dsmBool_t unicode;            /* API unicode? */
}dsmApiVersionEx;

#define apiVersionExVer    2

/*-----+
| Type definition for Application Version on dsmInit() |
+-----*/
typedef struct
{
    dsUint16_t stVersion;           /* Structure version */
    dsUint16_t applicationVersion; /* application version number */
    dsUint16_t applicationRelease; /* application release number */
    dsUint16_t applicationLevel;   /* application level number */
    dsUint16_t applicationSubLevel; /* application sub level number */
} dsmAppVersion;

#define appVersionVer    1

/*-----+
| Type definition for object name used on BindMC, Send, Delete, Query |
+-----*/
typedef struct S_dsmObjName
{
    char fs[DSM_MAX_FSNAME_LENGTH + 1] ; /* Filespace name */
    char hl[DSM_MAX_HL_LENGTH + 1] ; /* High level name */
    char ll[DSM_MAX_LL_LENGTH + 1] ; /* Low level name */
    dsUint8_t objType; /* for object type values, see defines above */
}dsmObjName;

/*-----+
| Type definition for Backup delete info on dsmDeleteObj() |
+-----*/
typedef struct
{
    dsUint16_t stVersion ; /* structure version */
    dsmObjName *objNameP ; /* object name */
    dsUint32_t copyGroup ; /* copy group */
}delBack ;

#define delBackVersion    1

+-----*/

```

```

| Type definition for Archive delete info on dsmDeleteObj()
+-----+
typedef struct
{
    dsUint16_t      stVersion ;           /* structure version      */
    dsStruct64_t    objId ;              /* object ID              */
}delArch ;

#define delArchVersion 1

/*-----+
| Type definition for Backup ID delete info on dsmDeleteObj()
+-----+
typedef struct
{
    dsUint16_t      stVersion ;           /* structure version      */
    dsStruct64_t    objId ;              /* object ID              */
}delBackID;

#define delBackIDVersion 1

/*-----+
| Type definition for delete info on dsmDeleteObj()
+-----+
typedef union
{
    delBack    backInfo ;
    delArch    archInfo ;
    delBackID  backIDInfo ;
}dsmDelInfo ;

/*-----+
| Type definition for Object Attribute parameter on dsmSendObj()
+-----+
typedef struct
{
    dsUint16_t      stVersion;           /* Structure version */
    char            owner[DSM_MAX_OWNER_LENGTH + 1]; /* object owner */
    dsStruct64_t    sizeEstimate;        /* Size estimate in bytes of the object */
    dsmBool_t       objCompressed;       /* Is object already compressed? */
    dsUint16_t      objInfoLength;       /* length of object-dependent info */
    char            *objInfo;           /* object-dependent info */
    char            *mcNameP;           /* mgmnt class name for override */
    dsmBool_t       disableDeduplication; /* force no dedup for this object */
    dsmBool_t       useExtObjInfo;       /* use ext obj info up to 1536 */
}ObjAttr;

#define ObjAttrVersion 4

/*-----+
| Type definition for mcBindKey returned on dsmBindMC()
+-----+
typedef struct
{
    dsUint16_t      stVersion;           /* structure version      */
    char            mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* Name of mc bound to object. */
                                           /* True/false */
    dsmBool_t       backup_cg_exists;    /* True/false */
    dsmBool_t       archive_cg_exists;   /* True/false */
    char            backup_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1]; /* Backup copy dest. name */
                                           /* True/false */
    char            archive_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1]; /* Arch copy dest.name */
}mcBindKey;

#define mcBindKeyVersion 1

/*-----+
| Type definition for object list on dsmBeginGetData()
+-----+
typedef struct
{
    dsUint16_t      stVersion ;           /* structure version      */
    dsUint32_t      numObjId ;           /* number of object IDs in the list */
    ObjID           *objId ;            /* list of object IDs to restore */
    PartialObjData  *partialObjData;    /* list of partial obj data info */
}dsmGetList ;

#define dsmGetListVersion 2 /* default if not using Partial Obj data */

```

```

#define dsmGetListPORVersion 3 /* version if using Partial Obj data */

/*-----+
| Type definition for DataBlk used to Get or Send data |
+-----*/
typedef struct
{
    dsUInt16_t stVersion; /* structure version */
    dsUInt32_t bufferLen; /* Length of buffer passed below */
    dsUInt32_t numBytes; /* Actual number of bytes read from */
    /* or written to the buffer */
    char *bufferPtr; /* Data buffer */
    dsUInt32_t numBytesCompressed; /* on send actual bytes compressed */
    dsUInt16_t reserved; /* for future use */
}DataBlk;

#define DataBlkVersion 3

/*-----+
| Type definition for Mgmt Class queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct S_qryMCData
{
    dsUInt16_t stVersion; /* structure version */
    char *mcName; /* Mgmt class name */
    dsMBool_t mcDetail; /* single name to get one or empty string to get all*/
    /* Want details or not? */
}qryMCData;

#define qryMCDataVersion 1

/*=== values for RETINIT ===*/
#define ARCH_RETINIT_CREATE 0
#define ARCH_RETINIT_EVENT 1

/*-----+
| Type definition for Archive Copy Group details on Query MC response |
+-----*/
typedef struct S_archDetailCG
{
    char cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUInt16_t frequency; /* Copy (archive) frequency */
    dsUInt16_t retainVers; /* Retain version */
    dsUInt8_t copySer; /* for copy serialization values, see defines */
    dsUInt8_t copyMode; /* for copy mode values, see defines above */
    char destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsMBool_t blAnFreeDest; /* Destination has lan free path? */
    dsMBool_t reserved; /* Not currently used */
    dsUInt8_t retainInit; /* possible values see above */
    dsUInt16_t retainMin; /* if retInit is EVENT num of days */
    dsMBool_t bDeduplicate; /* destination has dedup enabled */
}archDetailCG;

/*-----+
| Type definition for Backup Copy Group details on Query MC response |
+-----*/
typedef struct S_backupDetailCG
{
    char cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUInt16_t frequency; /* Backup frequency */
    dsUInt16_t verDataExst; /* Versions data exists */
    dsUInt16_t verDataDltd; /* Versions data deleted */
    dsUInt16_t retXtraVers; /* Retain extra versions */
    dsUInt16_t retOnlyVers; /* Retain only versions */
    dsUInt8_t copySer; /* for copy serialization values, see defines */
    dsUInt8_t copyMode; /* for copy mode values, see defines above */
    char destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsMBool_t blAnFreeDest; /* Destination has lan free path? */
    dsMBool_t reserved; /* Not currently used */
    dsMBool_t bDeduplicate; /* destination has dedup enabled */
}backupDetailCG;

/*-----+
| Type definition for Query Mgmt Class detail response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryRespMCDetailData
{
    dsUInt16_t stVersion; /* structure version */
    char mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char mcDesc[DSM_MAX_MC_DESCR_LENGTH + 1]; /*mc description */

```

```

    archDetailCG    archDet;                /* Archive copy group detail */
    backupDetailCG  backupDet;              /* Backup copy group detail */
}qryRespMCDetailData;

#define qryRespMCDetailDataVersion 4

/*-----+
| Type definition for Query Mgmt Class summary response on dsmGetNextQObj()|
+-----*/
typedef struct S_qryRespMCDData
{
    dsUInt16_t      stVersion;                /* structure version */
    char            mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char            mcDesc[DSM_MAX_MC_DESCR_LENGTH + 1]; /* mc description */
}qryRespMCDData;

#define qryRespMCDDataVersion 1

/*-----+
| Type definition for Archive queryBuffer on dsmBeginQuery()|
+-----*/
typedef struct S_qryArchiveData
{
    dsUInt16_t      stVersion;                /* structure version */
    dsmObjName      *objName;                /* Full dsm name of object */
    char            *owner;                  /* owner name */
    /* for maximum date boundaries, see defines above */
    dsmDate          insDateLowerBound;      /* low bound archive insert date */
    dsmDate          insDateUpperBound;      /* hi bound archive insert date */
    dsmDate          expDateLowerBound;      /* low bound expiration date */
    dsmDate          expDateUpperBound;      /* hi bound expiration date */
    char            *descr;                  /* archive description */
} qryArchiveData;

#define qryArchiveDataVersion 1

/*=== values for retentionInitiated field ===*/
#define DSM_ARCH_RETINIT_UNKNOWN 0 /* ret init is unknown (down-level srv) */
#define DSM_ARCH_RETINIT_STARTED 1 /* retention clock is started */
#define DSM_ARCH_RETINIT_PENDING 2 /* retention clock is not started */

/*=== Values for objHeld ===*/
#define DSM_ARCH_HELD_UNKNOWN 0 /* unknown hold status (down-level srv) */
#define DSM_ARCH_HELD_FALSE 1 /* object is NOT in a delete hold state */
#define DSM_ARCH_HELD_TRUE 2 /* object is in a delete hold state */

/*-----+
| Type definition for Query Archive response on dsmGetNextQObj()|
+-----*/
typedef struct S_qryRespArchiveData
{
    dsUInt16_t      stVersion;                /* structure version */
    dsmObjName      objName;                /* Filespace name qualifier */
    dsUInt32_t      copyGroup;              /* copy group number */
    char            mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char            owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsStruct64_t    objId;                  /* Unique copy id */
    dsStruct64_t    reserved;               /* backward compatability */
    dsUInt8_t       mediaClass;             /* media access class */
    dsmDate          insDate;               /* archive insertion date */
    dsmDate          expDate;               /* expiration date for object */
    char            descr[DSM_MAX_DESCR_LENGTH + 1]; /* archive description */
    dsUInt16_t      objInfoLen;             /* length of object-dependent info */
    char            reservedObjInfo[DSM_MAX_OBGINFO_LENGTH]; /*object-dependent info */
    dsUInt160_t     restoreOrderExt;        /* restore order */
    dsStruct64_t     sizeEstimate;           /* size estimate stored by user */
    dsUInt8_t       compressType;           /* Compression flag */
    dsUInt8_t       retentionInitiated;     /* object waiting on retention event */
    dsUInt8_t       objHeld; /*object is on retention "hold" see values above */
    dsUInt8_t       encryptionType;         /* type of encryption */
    dsBool_t        clientDeduplicated;     /* obj deduplicated by API */
    char            objInfo[DSM_MAX_EXT_OBGINFO_LENGTH]; /*object-dependent info */
    char            compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
}qryRespArchiveData;

#define qryRespArchiveDataVersion 7

```

```

/*-----+
| Type definition for Archive sendBuff parameter on dsmSendObj()
+-----*/
typedef struct S_sndArchiveData
{
    dsUInt16_t    stVersion;           /* structure version */
    char          *descr;              /* archive description */
}sndArchiveData;

#define sndArchiveDataVersion 1

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery()
+-----*/
typedef struct S_qryBackupData
{
    dsUInt16_t    stVersion;           /* structure version */
    dsmObjName    *objName;            /* full dsm name of object */
    char          *owner;              /* owner name */
    dsUInt8_t     objState;            /* object state selector */
    dsmDate       pitDate;             /* Date value for point in time restore */
                                           /* for possible values, see defines above */
}qryBackupData;

#define qryBackupDataVersion 2

typedef struct
{
    dsUInt8_t     reserved1;
    dsStruct64_t  reserved2;
} reservedInfo_t;           /* for future use */

/*-----+
| Type definition for Query Backup response on dsmGetNextQObj()
+-----*/
typedef struct S_qryRespBackupData
{
    dsUInt16_t    stVersion;           /* structure version */
    dsmObjName    objName;             /* full dsm name of object */
    dsUInt32_t    copyGroup;           /* copy group number */
    char          mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char          owner[DSM_MAX_OWNER_LENGTH + 1];   /* owner name */
    dsStruct64_t  objId;               /* Unique object id */
    dsStruct64_t  reserved;            /* backward compatability */
    dsUInt8_t     mediaClass;          /* media access class */
    dsUInt8_t     objState;            /* Obj state, active, etc. */
    dsmDate       insDate;             /* backup insertion date */
    dsmDate       expDate;             /* expiration date for object */
    dsUInt16_t    objInfoLen;          /* length of object-dependent info*/
    char          reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    dsUInt160_t   restoreOrderExt;     /* restore order */
    dsStruct64_t  sizeEstimate;        /* size estimate stored by user */
    dsStruct64_t  baseObjId;
    dsUInt16_t    baseObjInfoLen;      /* length of base object-dependent info*/
    dsUInt8_t     baseObjInfo[DSM_MAX_OBJINFO_LENGTH]; /* base object-dependent info */
    dsUInt160_t   baseRestoreOrder;    /* restore order */
    dsUInt32_t    fsID;
    dsUInt8_t     compressType;
    dsmBool_t     isGroupLeader;
    dsmBool_t     isOpenGroup;
    dsUInt8_t     reserved1;           /* for future use */
    dsmBool_t     reserved2;           /* for future use */
    dsUInt16_t    reserved3;           /* for future use */
    reservedInfo_t *reserved4;         /* for future use */
    dsUInt8_t     encryptionType;      /* type of encryption */
    dsmBool_t     clientDeduplicated;  /* obj deduplicated by API*/
    char          objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
    char          compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
}qryRespBackupData;

#define qryRespBackupDataVersion 8

/*-----+
| Type definition for Active Backup queryBuffer on dsmBeginQuery()
|
| Notes: For the active backup query, only the fs (filesystem) and objType
|        fields of objName need be set. objType can only be set to
|        DSM_OBJ_FILE or DSM_OBJ_DIRECTORY. DSM_OBJ_ANY_TYPE will not
|        find a match on the query.
+-----*/
typedef struct S_qryABackupData
{

```

```

    dsUint16_t    stVersion;                                /* structure version */
    dsmObjName    *objName;                                /* Only fs and objtype used */
}qryABackupData;

#define qryABackupDataVersion 1

/*-----+
| Type definition for Query Active Backup response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryARespBackupData
{
    dsUint16_t    stVersion;                                /* structure version */
    dsmObjName    objName;                                /* full dsm name of object */
    dsUint32_t    copyGroup;                               /* copy group number */
    char          mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /*management class name*/
    char          owner[DSM_MAX_OWNER_LENGTH + 1];      /* owner name */
    dsmDate       insDate;                                  /* backup insertion date */
    dsUint16_t    objInfolen;                               /* length of object-dependent info*/
    char          reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    char          objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
}qryARespBackupData;

#define qryARespBackupDataVersion 2

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct qryBackupGroups
{
    dsUint16_t    stVersion;                                /* structure version */
    dsUint8_t     groupType;
    char          *fsName;
    char          *owner;
    dsStruct64_t  groupLeaderObjId;
    dsUint8_t     objType;
    dsmBool_t     noRestoreOrder;
    dsmBool_t     noGroupInfo;
    char          *hl;
}qryBackupGroups;

#define qryBackupGroupsVersion 3

/*-----+
| Type definition for proxynode queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct qryProxyNodeData
{
    dsUint16_t    stVersion;                                /* structure version */
    char          *targetNodeName;                          /* target node name */
}qryProxyNodeData;

#define qryProxyNodeDataVersion 1

/*-----+
| Type definition for qryRespProxyNodeData parameter used on dsmGetNextQObj() |
+-----*/
typedef struct
{
    dsUint16_t    stVersion ;                               /* structure version */
    char          targetNodeName[DSM_MAX_ID_LENGTH+1]; /* target node name */
    char          peerNodeName[DSM_MAX_ID_LENGTH+1]; /* Peer node name */
    char          hlAddress[DSM_MAX_ID_LENGTH+1]; /* peer hlAddress */
    char          llAddress[DSM_MAX_ID_LENGTH+1]; /* peer hlAddress */
}qryRespProxyNodeData;

#define qryRespProxyNodeDataVersion 1

/*-----+
| Type definition for WINNT and OS/2 Filespace attributes |
+-----*/
typedef struct
{
    char          driveLetter ;                               /* drive letter for filespace */
    dsUint16_t    fsInfoLength;                               /* fsInfo length used */
    char          fsInfo[DSM_MAX_FSINFO_LENGTH]; /*caller-determined data */
}dsmDosFSAttrib ;

/*-----+
| Type definition for UNIX Filespace attributes |

```



```

+-----*/
typedef struct
{
    dsUInt16_t    fsInfoLength;          /* fsInfo length used          */
    char          fsInfo[DSM_MAX_FSINFO_LENGTH]; /* caller-determined data */
} dsmUnixFSAttrib ;

/*-----+
| Type definition for NetWare Filespace attributes |
+-----*/
typedef dsmUnixFSAttrib dsmNetwareFSAttrib;

/*-----+
| Type definition for Filespace attributes on all Filespace calls |
+-----*/
typedef union
{
    dsmNetwareFSAttrib    netwareFSAttr;
    dsmUnixFSAttrib       unixFSAttr ;
    dsmDosFSAttrib        dosFSAttr ;
} dsmFSAttr ;

/*-----+
| Type definition for fsUpd parameter on dsmUpdateFS() |
+-----*/
typedef struct S_dsmFSUpd
{
    dsUInt16_t    stVersion ;          /* structure version          */
    char          *fsType ;           /* filespace type             */
    dsStruct64_t   occupancy ;         /* occupancy estimate         */
    dsStruct64_t   capacity ;         /* capacity estimate          */
    dsmFSAttr      fsAttr ;           /* platform specific attributes */
} dsmFSUpd ;

#define dsmFSUpdVersion 1

/*-----+
| Type definition for Filespace queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct S_qryFSData
{
    dsUInt16_t    stVersion;          /* structure version */
    char          *fsName;           /* File space name */
} qryFSData;

#define qryFSDataVersion 1

/*-----+
| Type definition for Query Filespace response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryRespFSData
{
    dsUInt16_t    stVersion;          /* structure version */
    char          fsName[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name */
    char          fsType[DSM_MAX_FSTYPE_LENGTH + 1]; /* Filespace type */
    dsStruct64_t   occupancy;          /* Occupancy est. in bytes */
    dsStruct64_t   capacity;           /* Capacity est. in bytes */
    dsmFSAttr      fsAttr ;           /* platform specific attributes */
    dsmDate        backStartDate;      /* start backup date */
    dsmDate        backCompleteDate;   /* end backup Date */
    dsmDate        reserved1;          /* For future use */
    dsmDate        lastReplStartDate;   /* The last time replication was started */
    dsmDate        lastReplCmpltDate;  /* The last time replication completed */
    /* (could have had a failure, */
    /* but it still completes) */
    dsmDate        lastBackOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server */
    dsmDate        lastArchOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server */
    dsmDate        lastSpMgOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server */
    dsmDate        lastBackOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local */
    dsmDate        lastArchOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local */
    dsmDate        lastSpMgOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local */
    dsInt32_t      failOverWriteDelay; /* Minutes for client to wait before allowed */
    /* to store to this Repl srvr, Specail codes: */
    /* NO_ACCESS(-1), ACCESS_RDONLY (-2) */
} qryRespFSData;

```

```

#define qryRespFSDataVersion 4

/*-----+
| Type definition for regFilespace parameter on dsmRegisterFS()
+-----*/
typedef struct S_regFSData
{
    dsUInt16_t    stVersion;                /* structure version */
    char          *fsName;                  /* Filespace name */
    char          *fsType;                  /* Filespace type */
    dsStruct64_t  occupancy;                 /* Occupancy est. in bytes. */
    dsStruct64_t  capacity;                 /* Capacity est. in bytes. */
    dsmFSAttr     fsAttr;                   /* platform specific attributes */
}regFSData;

#define regFSDataVersion 1

/*-----+
| Type definition for dedupType used in apisessInfo
+-----*/
typedef enum
{
    dedupServerOnly= 0x00,                 /* dedup only done on server */
    dedupClientOrServer                    /* dedup can be done on client or server */
}dsmDedupType ;

/*-----+
| Type definition for fail over configuration and status
+-----*/
typedef enum
{
    failOvrNotConfigured = 0x00,
    failOvrConfigured,
    failOvrConnectedToReplServer
}dsmFailOvrCfgType ;

/*-----+
| Type definition for session info response on dsmQuerySessionInfo()
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;                /* Structure version */
    /*-----+
    /* Server information
    +-----*/
    char          serverHost[DSM_MAX_SERVERNAME_LENGTH+1]; /* Network host name of DSM server */
    dsUInt16_t    serverPort;               /* Server comm port on host */
    dsmDate       serverDate;               /* Server's date/time */
    char          serverType[DSM_MAX_SERVERTYPE_LENGTH+1]; /* Server's execution platform */
    dsUInt16_t    serverVer;                /* Server's version number */
    dsUInt16_t    serverRel;                /* Server's release number */
    dsUInt16_t    serverLev;                /* Server's level number */
    dsUInt16_t    serverSubLev;             /* Server's sublevel number */
    /*-----+
    /* Client Defaults
    +-----*/
    char          nodeType[DSM_MAX_PLATFORM_LENGTH+1]; /*node/application type*/
    char          fsdelim;                  /* File space delimiter */
    char          hldelim;                  /* Delimiter betw highlev & lowlev */
    dsUInt8_t     compression;              /* Compression flag */
    dsUInt8_t     archDel;                  /* Archive delete permission */
    dsUInt8_t     backDel;                  /* Backup delete permission */
    dsUInt32_t    maxBytesPerTxn;           /* for future use */
    dsUInt16_t    maxObjPerTxn;            /* The max objects allowed in a txn */
    /*-----+
    /* Session Information
    +-----*/
    char          id[DSM_MAX_ID_LENGTH+1]; /* Sign-in id node name */
    char          owner[DSM_MAX_OWNER_LENGTH+1]; /* Sign-in owner */
    /* (for multi-user platforms) */
    char          confFile[DSM_PATH_MAX + DSM_NAME_MAX +1];
    /* len is platform dep
    /* dsInit name of appl config file
    dsUInt8_t     opNoTrace;                 /* dsInit option - NoTrace = 1 */
    /*-----+
    /* Policy Data
    +-----*/
    char          domainName[DSM_MAX_DOMAIN_LENGTH+1]; /* Domain name */
    char          policySetName[DSM_MAX_PS_NAME_LENGTH+1];
    /* Active policy set name

```

```

dsmDate      polActDate;          /* Policy set activation date      */
char          dfltMCName[DSM_MAX_MC_NAME_LENGTH+1]; /* Default Mgmt Class */
dsUint16_t    gpBackRetn;         /* Grace-period backup retention */
dsUint16_t    gpArchRetn;         /* Grace-period archive retention */
char          adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* adsm server name */
dsmBool_t     archiveRetentionProtection; /* is server Retention protection enabled */
dsStruct64_t  maxBytesPerTxn_64;   /* for future use */
dsmBool_t     lanFreeEnabled;      /* lan free option is set */
dsmDedupType  dedupType;           /* server or clientOrServer */
char          accessNode[DSM_MAX_ID_LENGTH+1]; /* as node node name */

/*-----*/
/*          Replication and fail over information          */
/*-----*/
dsmFailOvrCfgType failOverCfgType; /* status of fail over */
char          replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
char          homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
char          replServerHost[DSM_MAX_SERVERNAME_LENGTH+1]; /* Network host name of DSM server */
dsInt32_t     replServerPort;      /* Server comm port on host */

}ApiSessInfo;

#define ApiSessInfoVersion 6

/*-----+
| Type definition for Query options response on dsmQueryCliOptions()
| and dsmQuerySessOptions()
+-----*/

typedef struct
{
    char          dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char          dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char          serverName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsInt16_t     commMethod;
    char          serverAddress[DSM_MAX_SERVER_ADDRESS];
    char          nodeName[DSM_MAX_NODE_LENGTH+1];
    dsmBool_t     compression;
    dsmBool_t     compressalways;
    dsmBool_t     passwordAccess;
}optStruct;

/*-----+
| Type definition for LogType used in logInfo
+-----*/
typedef enum
{
    logServer = 0x00,          /* log msg only to server */
    logLocal,                  /* log msg only to local error log */
    logBoth,                   /* log msg to server and to local error log */
    logNone
}dsmLogType;

/*-----+
| Type definition for logInfo parameter used on dsmLogEvent()
+-----*/

typedef struct
{
    char          *message; /* text of message to be logged */
    dsmLogType    logType;  /* log type : local, server, both */
}logInfo;

/*-----+
| Type definition for qryRespAccessData parameter used on dsmQueryAccess()
+-----*/

typedef struct
{
    dsUint16_t     stVersion; /* structure version */
    char          node[DSM_MAX_ID_LENGTH+1]; /* node name */
    char          owner[DSM_MAX_OWNER_LENGTH+1]; /* owner */
    dsmObjName     objName; /* object name */
    dsmAccessType  accessType; /* archive or backup */
    dsUint32_t     ruleNumber; /* Access rule id */
}qryRespAccessData;

#define qryRespAccessDataVersion 1

/*-----+

```

```

| Type definition for envSetUp parameter on dsmSetUp()
+-----*/
typedef struct S_envSetUp
{
    dsUInt16_t      stVersion; /* structure version */
    char            dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char            dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char            dsmiLog[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char            **argv; /* for executables name argv[0] */
    char            logName[DSM_NAME_MAX +1];
    dsmBool_t       reserved1; /* for future use */
    dsmBool_t       reserved2; /* for future use */
}envSetUp;

#define envSetUpVersion 4

/*-----+
| Type definition for dsmInitExIn_t
+-----*/
typedef struct dsmInitExIn_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsmApiVersionEx *apiVersionEx;
    char            *clientNodeNameP;
    char            *clientOwnerNameP;
    char            *clientPasswordP;
    char            *userNameP;
    char            *userPasswordP;
    char            *applicationTypeP;
    char            *configfile;
    char            *options;
    char            dirDelimiter;
    dsmBool_t       useUnicode;
    dsmBool_t       bCrossPlatform;
    dsmBool_t       bService;
    dsmBool_t       bEncryptKeyEnabled;
    char            *encryptionPasswordP;
    dsmBool_t       useTsmBuffers;
    dsUInt8_t       numTsmBuffers;
    dsmAppVersion   *appVersionP;
}dsmInitExIn_t;

#define dsmInitExInVersion 5

/*-----+
| Type definition for dsmInitExOut_t
+-----*/
typedef struct dsmInitExOut_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsInt16_t       userNameAuthorities;
    dsInt16_t       infoRC; /* error return code if encountered */
    char            adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsUInt16_t      serverVer; /* Server's version number */
    dsUInt16_t      serverRel; /* Server's release number */
    dsUInt16_t      serverLev; /* Server's level number */
    dsUInt16_t      serverSubLev; /* Server's sublevel number */

    dsmBool_t       bIsFailOverMode; /* true if failover has occurred */
    char            replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
    char            homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
}dsmInitExOut_t;

#define dsmInitExOutVersion 3

/*-----+
| Type definition for LogType used in logInfo
+-----*/
typedef enum
{
    logSevInfo = 0x00, /* information ANE4991 */
    logSevWarning, /* warning ANE4992 */
    logSevError, /* Error ANE4993 */
    logSevSevere, /* severe ANE4994 */
    logSevLicense, /* license ANE4995 */
    logSevTryBuy /* try Buy ANE4996 */
}dsmLogSeverity ;

/*-----+
| Type definition for dsmLogExIn_t

```

```

+-----*/
typedef struct dsmLogExIn_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsmLogSeverity  severity;
    char            appMsgID[8];
    dsmLogType      logType; /* log type : local, server, both */
    char            *message; /* text of message to be logged */
    char            appName[DSM_MAX_PLATFORM_LENGTH];
    char            osPlatform[DSM_MAX_PLATFORM_LENGTH];
    char            appVersion[DSM_MAX_PLATFORM_LENGTH];
}dsmLogExIn_t;

#define dsmLogExInVersion 2

/*-----+
| Type definition for dsmLogExOut_t
+-----*/
typedef struct dsmLogExOut_t
{
    dsUInt16_t      stVersion; /* structure version */
}dsmLogExOut_t;

#define dsmLogExOutVersion 1

/*-----+
| Type definition for dsmRenameIn_t
+-----*/
typedef struct dsmRenameIn_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsUInt32_t      dsmHandle; /* handle for session */
    dsUInt8_t       repository; /* Backup or Archive */
    dsmObjNameP     *objNameP; /* object name */
    char            newHl[DSM_MAX_HL_LENGTH + 1]; /* new High level name */
    char            newLl[DSM_MAX_LL_LENGTH + 1]; /* new Low level name */
    dsmBool_t       merge; /* merge into existing name*/
    ObjID           objId; /* objId for Archive */
}dsmRenameIn_t;

#define dsmRenameInVersion 1

/*-----+
| Type definition for dsmRenameOut_t
+-----*/
typedef struct dsmRenameOut_t
{
    dsUInt16_t      stVersion; /* structure version */
}dsmRenameOut_t;

#define dsmRenameOutVersion 1

/*-----+
| Type definition for dsmEndSendObjExIn_t
+-----*/
typedef struct dsmEndSendObjExIn_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsUInt32_t      dsmHandle; /* handle for session */
}dsmEndSendObjExIn_t;

#define dsmEndSendObjExInVersion 1

/*-----+
| Type definition for dsmEndSendObjExOut_t
+-----*/
typedef struct dsmEndSendObjExOut_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsStruct64_t     totalBytesSent; /* total bytes read from app */
    dsmBool_t        objCompressed; /* was object compressed */
    dsStruct64_t     totalCompressSize; /* total size after compress */
    dsStruct64_t     totalLFBytesSent; /* total bytes sent Lan Free */
    dsUInt8_t        encryptionType; /* type of encryption used */
    dsmBool_t        objDeduplicated; /* was object processed for dist. data dedup */
    dsStruct64_t     totalDedupSize; /* total size after de-dup */
}dsmEndSendObjExOut_t;

#define dsmEndSendObjExOutVersion 3

/*-----+
| Type definition for dsmGroupHandlerIn_t
+-----*/

```

```

typedef struct dsmGroupHandlerIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* handle for session */
    dsUInt8_t       groupType;          /* Type of group */
    dsUInt8_t       actionType;         /* Type of group operation */
    dsUInt8_t       memberType;         /* Type of member: Leader or member */
    dsStruct64_t    leaderObjId;        /* OBJID of the groupleader when manipulating a member */
    char            *uniqueGroupTagP;   /* Unique group identifier */
    dsmObjName      *objNameP;          /* group leader object name */
    dsmGetList      memberObjList;      /* list of objects to remove, assign */
}dsmGroupHandlerIn_t;

#define dsmGroupHandlerInVersion 1

/*-----+
| Type definition for dsmGroupHandlerExOut_t
+-----*/
typedef struct dsmGroupHandlerOut_t
{
    dsUInt16_t      stVersion;          /* structure version */
}dsmGroupHandlerOut_t;

#define dsmGroupHandlerOutVersion 1

/*-----+
| Type definition for dsmEndTxnExIn_t
+-----*/
typedef struct dsmEndTxnExIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* handle for session */
    dsUInt8_t       vote;
}dsmEndTxnExIn_t;

#define dsmEndTxnExInVersion 1

/*-----+
| Type definition for dsmEndTxnExOut_t
+-----*/
typedef struct dsmEndTxnExOut_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt16_t      reason;             /* reason code */
    dsStruct64_t    groupLeaderObjId;   /* groupLeader obj id returned on */
                                         /* DSM_ACTION_OPEN */
    dsUInt8_t       reserved1;          /* future use */
    dsUInt16_t      reserved2;          /* future use */
}dsmEndTxnExOut_t;

#define dsmEndTxnExOutVersion 1

/*-----+
| Type definition for dsmEndGetDataExIn_t
+-----*/
typedef struct dsmEndGetDataExIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* handle for session */
}dsmEndGetDataExIn_t;

#define dsmEndGetDataExInVersion 1

/*-----+
| Type definition for dsmEndGetDataExOut_t
+-----*/
typedef struct dsmEndGetDataExOut_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt16_t      reason;             /* reason code */
    dsStruct64_t    totalLFBBytesRecv; /* total lan free bytes recieved */
}dsmEndGetDataExOut_t;

#define dsmEndGetDataExOutVersion 1

/*-----+
| Type definition for object list on dsmRetentionEvent()
+-----*/
typedef struct dsmObjList
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      numObjId;           /* number of object IDs in the list */
}

```

```

ObjID      *objId;          /* list of object IDs to signal */
}dsmObjList_t ;

#define dsmObjlistVersion 1

/*-----+
| Type definition eventType used on dsmRetentionEvent |
+-----*/
typedef enum
{
    eventRetentionActivate = 0x00, /* signal the server that the event has occurred */
    eventHoldObj,                /* suspend delete/expire of the object */
    eventReleaseObj              /* Resume normal delete/expire processing */
}dsmEventType_t;

/*-----+
| Type definition for on dsmRetentionEvent() |
+-----*/
typedef struct dsmRetentionEventIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
    dsmEventType_t  eventType;          /* Event type */
    dsmObjList_t    objList;           /* object ID */
}dsmRetentionEventIn_t;

#define dsmRetentionEventInVersion 1

/*-----+
| Type definition for on dsmRetentionEvent() |
+-----*/
typedef struct dsmRetentionEventOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */
}dsmRetentionEventOut_t;

#define dsmRetentionEventOutVersion 1

/*-----+
| Type definition for on dsmRequestBuffer() |
+-----*/
typedef struct requestBufferIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
}requestBufferIn_t;

#define requestBufferInVersion 1

/*-----+
| Type definition for on dsmRequestBuffer() |
+-----*/
typedef struct requestBufferOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */
    dsUInt8_t       tsmBufferHandle;     /* handle to tsm Data buffer */
    char            *dataPtr;            /* Address to write data to */
    dsUInt32_t      bufferLen;           /* Max length of data to be written */
}requestBufferOut_t;

#define requestBufferOutVersion 1

/*-----+
| Type definition for on dsmReleaseBuffer() |
+-----*/
typedef struct releaseBufferIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
    dsUInt8_t       tsmBufferHandle;     /* handle to tsm Data buffer */
    char            *dataPtr;            /* Address to write data to */
}releaseBufferIn_t;

#define releaseBufferInVersion 1

/*-----+
| Type definition for on dsmReleaseBuffer() |
+-----*/
typedef struct releaseBufferOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */

```

```

}releaseBufferOut_t;

#define releaseBufferOutVersion 1

/*-----+
| Type definition for on dsmGetBufferData() |
+-----*/
typedef struct getBufferDataIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
}getBufferDataIn_t;

#define getBufferDataInVersion 1

/*-----+
| Type definition for on dsmGetBufferData() |
+-----*/
typedef struct getBufferDataOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */
    dsUInt8_t       tsmBufferHandle;      /* handle to tsm Data buffer */
    char            *dataPtr;             /* Address of actual data to read */
    dsUInt32_t      numBytes;             /* Actual number of bytes to read from dataPtr*/
}getBufferDataOut_t;

#define getBufferDataOutVersion 1

/*-----+
| Type definition for on dsmSendBufferData() |
+-----*/
typedef struct sendBufferDataIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
    dsUInt8_t       tsmBufferHandle;      /* handle to tsm Data buffer */
    char            *dataPtr;             /* Address of actual data to send */
    dsUInt32_t      numBytes;             /* Actual number of bytes to send from dataPtr*/
}sendBufferDataIn_t;

#define sendBufferDataInVersion 1

/*-----+
| Type definition for on dsmSendBufferData() |
+-----*/
typedef struct sendBufferDataOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */
}sendBufferDataOut_t;

#define sendBufferDataOutVersion 1

/*-----+
| Type definition for dsmUpdateObjExIn_t |
+-----*/
typedef struct dsmUpdateObjExIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
    dsmSendType     sendType;           /* send type back/arch */
    char            *descrP;            /* archive description */
    dsmObjName       *objNameP;         /* objName */
    ObjAttr          *objAttrPtr;       /* attribute */
    dsUInt32_t       objUpdAct;          /* update action */
    ObjID            archObjId;          /* objId for archive */
}dsmUpdateObjExIn_t;

#define dsmUpdateObjExInVersion 1

/*-----+
| Type definition for dsmUpdateObjExOut_t |
+-----*/
typedef struct dsmUpdateObjExOut_t
{
    dsUInt16_t      stVersion;          /* structure version */
}dsmUpdateObjExOut_t;

#define dsmUpdateObjExOutVersion 1

#if (_OPSYS_TYPE == DS_WINNT) && !defined(_WIN64)
#pragma pack()
#endif

```



```

#ifdef _MAC
#pragma options align=reset
#endif
#endif /* _H_DSMAPI_TD */

```

## tsmapitd.h

```

/*****
* IBM Storage Protect
* API Client Component
*
* IBM Confidential
* (IBM Confidential-Restricted when combined with the Aggregated OCO
* source modules for this program)
*
* OCO Source Materials
*
* 5648-020 (C) Copyright IBM Corporation 1993, 2018
*****/

/*****
* Header File Name: tsmapitd.h
*
* Environment:
* **** This is a platform-independent source file **
*
* ****
*
* Design Notes: This file contains basic data types and constants
* includable by all client source files. The constants
* within this file should be set properly for the
* particular machine and operating system on which the
* client software is to be run.
*
* Platform specific definitions are included in dsmapips.h
*
* Descriptive-name: Definitions for IBM Storage Protect API constants
*-----*/

#ifndef _H_TSMAPITD
#define _H_TSMAPITD

/*=== set the structure alignment to pack the structures ===*/
#if _OPSYS_TYPE == DS_WINNT
#ifdef _WIN64
#pragma pack(8)
#else
#pragma pack(1)
#endif
#endif

#ifdef _MAC
#pragma options align = packed
#endif

/*=====
Win32 applications using the tsm interface must use the
-DUNICODE flag during compilation.
=====*/
#if _OPSYS_TYPE == DS_WINNT && !defined(DSMAPILIB)
#ifndef UNICODE
#error "Win32 applications using the TSM interface MUST be compiled with the -DUNICODE flag"
#endif
#endif

/*=====
Mac OS X applications using the tsm interface must use the
-DUNICODE flag during compilation.
=====*/
#if _OPSYS_TYPE == DS_MACOS && !defined(DSMAPILIB)
#ifndef UNICODE
#error "Mac OS X applications using the TSM interface MUST be compiled with the -DUNICODE flag"
#endif

```

```

#endif

/*-----+
| Type definition for dsmGetType parameter on tsmBeginGetData() |
+-----*/
typedef enum
{
    gtTsmBackup = 0x00,          /* Backup processing type */
    gtTsmArchive          /* Archive processing type */
} tsmGetType ;

/*-----+
| Type definition for dsmQueryType parameter on tsmBeginQuery() |
+-----*/
typedef enum
{
    qtTsmArchive = 0x00,          /* Archive query type */
    qtTsmBackup,          /* Backup query type */
    qtTsmBackupActive,     /* Fast query for active backup files */
    qtTsmFilespace,        /* Filespace query type */
    qtTsmMC,               /* Mgmt. class query type */
    qtTsmReserved1,        /* future use */
    qtTsmReserved2,        /* future use */
    qtTsmReserved3,        /* future use */
    qtTsmReserved4,        /* future use */
    qtTsmBackupGroups,     /* All group leaders in a specific filesystem */
    qtTsmOpenGroups,       /* All group members associated with a leader */
    qtTsmReserved5,        /* future use */
    qtTsmProxyNodeAuth,    /* nodes that this node can proxy to */
    qtTsmProxyNodePeer,    /* peer nodes under this target node */
    qtTsmReserved6,        /* future use */
    qtTsmReserved7,        /* future use */
    qtTsmReserved8         /* future use */
} tsmQueryType ;

/*-----+
| Type definition sendType parameter on tsmBindMC() and tsmSendObj() |
+-----*/
typedef enum
{
    stTsmBackup = 0x00,          /* Backup processing type */
    stTsmArchive,          /* Archive processing type */
    stTsmBackupMountWait,       /* Backup processing with mountwait on */
    stTsmArchiveMountWait      /* Archive processing with mountwait on */
} tsmSendType ;

/*-----+
| Type definition for delType parameter on tsmDeleteObj() |
+-----*/
typedef enum
{
    dtTsmArchive = 0x00,          /* Archive delete type */
    dtTsmBackup,          /* Backup delete (deactivate) type */
    dtTsmBackupID         /* Backup delete (remove) type */
} tsmDelType ;

/*-----+
| Type definition sendType parameter on tsmSetAccess() |
+-----*/
typedef enum
{
    atTsmBackup = 0x00,          /* Backup processing type */
    atTsmArchive          /* Archive processing type */
} tsmAccessType;

/*-----+
| Type definition for Overwrite parameter on tsmSendObj() |
+-----*/
typedef enum
{
    owIGNORE = 0x00,
    owYES,
    owNO
} tsmOwType;

/*-----+
| Type definition for API Version on tsmInit() and tsmQueryApiVersion() |
+-----*/
typedef struct
{
    dsUint16_t stVersion;        /* Structure version */
}

```

```

    dsUInt16_t version;          /* API version          */
    dsUInt16_t release;          /* API release         */
    dsUInt16_t level;            /* API level           */
    dsUInt16_t subLevel;         /* API sub level       */
    dsmBool_t  unicode;          /* API unicode?        */
} tsmApiVersionEx;

#define tsmApiVersionExVer      2

/*-----+
| Type definition for Application Version on tsmInit()
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;          /* Structure version    */
    dsUInt16_t    applicationVersion; /* application version number */
    dsUInt16_t    applicationRelease; /* application release number */
    dsUInt16_t    applicationLevel;   /* application level number */
    dsUInt16_t    applicationSubLevel; /* application sub level number */
} tsmAppVersion;

#define tsmAppVersionVer        1

/*-----+
| Type definition for object name used on BindMC, Send, Delete, Query
+-----*/

typedef struct tsmObjName
{
    dsChar_t      fs[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name */
    dsChar_t      hl[DSM_MAX_HL_LENGTH + 1];     /* High level name */
    dsChar_t      ll[DSM_MAX_LL_LENGTH + 1];     /* Low level name */
    dsUInt8_t     objType; /* for object type values, see defines above */
    dsChar_t      dirDelimiter;
} tsmObjName;

/*-----+
| Type definition for Backup delete info on dsmDeleteObj()
+-----*/
typedef struct tsmDelBack
{
    dsUInt16_t     stVersion; /* structure version */
    tsmObjName     *objNameP; /* object name       */
    dsUInt32_t     copyGroup; /* copy group        */
} tsmDelBack;

#define tsmDelBackVersion      1

/*-----+
| Type definition for Archive delete info on dsmDeleteObj()
+-----*/
typedef struct
{
    dsUInt16_t     stVersion; /* structure version */
    dsStruct64_t   objId;     /* object ID         */
} tsmDelArch;

#define tsmDelArchVersion      1

/*-----+
| Type definition for Backup ID delete info on dsmDeleteObj()
+-----*/
typedef struct
{
    dsUInt16_t     stVersion; /* structure version */
    dsStruct64_t   objId;     /* object ID         */
} tsmDelBackID;

#define tsmDelBackIDVersion    1

/*-----+
| Type definition for delete info on dsmDeleteObj()
+-----*/
typedef union
{
    tsmDelBack     backInfo;
    tsmDelArch     archInfo;
    tsmDelBackID   backIDInfo;
} tsmDelInfo;

```

```

/*-----+
| Type definition for Object Attribute parameter on dsmSendObj() |
+-----*/
typedef struct tsmObjAttr
{
    dsUuint16_t    stVersion;                /* Structure version */
    dsChar_t       owner[DSM_MAX_OWNER_LENGTH + 1]; /* object owner */
    dsStruct64_t   sizeEstimate;             /* Size estimate in bytes of the object */
    dsmBool_t      objCompressed;           /* Is object already compressed? */
    dsUuint16_t    objInfoLength;           /* length of object-dependent info */
    char           *objInfo;                /* object-dependent info byte buffer */
    dsChar_t       *mcNameP;                /* mgmnt class name for override */
    tsmOwType       reserved1;              /* for future use */
    tsmOwType       reserved2;              /* for future use */
    dsmBool_t       disableDeduplication;    /* force no dedup for this object */
    dsmBool_t       useExtObjInfo;          /* use ext objinfo up to 1536 */
} tsmObjAttr;

#define tsmObjAttrVersion 5

/*-----+
| Type definition for mcBindKey returned on dsmBindMC() |
+-----*/
typedef struct tsmMcBindKey
{
    dsUuint16_t    stVersion;                /* structure version */
    dsChar_t       mcName[DSM_MAX_MC_NAME_LENGTH + 1];
    /* Name of mc bound to object. */
    dsmBool_t       backup_cg_exists;        /* True/false */
    dsmBool_t       archive_cg_exists;       /* True/false */
    dsChar_t       backup_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1];
    /* Backup copy dest. name */
    dsChar_t       archive_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1];
    /* Arch copy dest.name */
} tsmMcBindKey;

#define tsmMcBindKeyVersion 1

/*-----+
| Type definition for Mgmt Class queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct tsmQryMCData
{
    dsUuint16_t    stVersion;                /* structure version */
    dsChar_t       *mcName;                 /* Mgmt class name */
    /* single name to get one or empty string to get all */
    dsmBool_t       mcDetail;               /* Want details or not? */
} tsmQryMCData;

#define tsmQryMCDataVersion 1

/*-----+
| Type definition for Archive Copy Group details on Query MC response |
+-----*/
typedef struct tsmArchDetailCG
{
    dsChar_t       cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUuint16_t     frequency;                        /* Copy (archive) frequency */
    dsUuint16_t     retainVers;                       /* Retain version */
    dsUuint8_t      copySer;                          /* for copy serialization values, see defines */
    dsUuint8_t      copyMode;                         /* for copy mode values, see defines above */
    dsChar_t       destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsmBool_t       bLanFreeDest;                     /* Destination has lan free path? */
    dsmBool_t       reserved;                         /* Not currently used */
    dsUuint8_t      retainInit;                       /* possible values see dsmapi.h */
    dsUuint16_t     retainMin;                       /* if retInit is EVENT num of days */
    dsmBool_t       bDeduplicate;                     /* destination has dedup enabled */
} tsmArchDetailCG;

/*-----+
| Type definition for Backup Copy Group details on Query MC response |
+-----*/
typedef struct tsmBackupDetailCG
{
    dsChar_t       cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUuint16_t     frequency;                        /* Backup frequency */
    dsUuint16_t     verDataExst;                     /* Versions data exists */
    dsUuint16_t     verDataDltd;                     /* Versions data deleted */
    dsUuint16_t     retXtraVers;                     /* Retain extra versions */
    dsUuint16_t     retOnlyVers;                     /* Retain only versions */
} tsmBackupDetailCG;

```

```

    dsUInt8_t    copySer;          /* for copy serialization values, see defines */
    dsUInt8_t    copyMode;         /* for copy mode values, see defines above */
    dsChar_t     destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsmBool_t    bLanFreeDest;     /* Destination has lan free path? */
    dsmBool_t    reserved;         /* Not currently used */
    dsmBool_t    bDeduplicate;     /* destination has dedup enabled */
} tsmBackupDetailCG;

/*-----+
| Type definition for Query Mgmt Class detail response on dsmGetNextQObj()|
+-----*/
typedef struct tsmQryRespMCDetailData
{
    dsUInt16_t    stVersion;        /* structure version */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    dsChar_t      mcDesc[DSM_MAX_MC_DESCR_LENGTH + 1]; /*mc description */
    archDetailCG  archDet;          /* Archive copy group detail */
    backupDetailCG backupDet;       /* Backup copy group detail */
} tsmQryRespMCDetailData;

#define tsmQryRespMCDetailDataVersion 4

/*-----+
| Type definition for Query Mgmt Class summary response on dsmGetNextQObj()|
+-----*/
typedef struct tsmQryRespMCData
{
    dsUInt16_t    stVersion;        /* structure version */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    dsChar_t      mcDesc[DSM_MAX_MC_DESCR_LENGTH + 1]; /* mc description */
} tsmQryRespMCData;

#define tsmQryRespMCDataVersion 1

/*-----+
| Type definition for Archive queryBuffer on tsmBeginQuery()|
+-----*/
typedef struct tsmQryArchiveData
{
    dsUInt16_t    stVersion;        /* structure version */
    tsmObjName    *objName;         /* Full dsm name of object */
    dsChar_t      *owner;           /* owner name */
    /* for maximum date boundaries, see defines above */
    dsmDate       insDateLowerBound; /* low bound archive insert date */
    dsmDate       insDateUpperBound; /* hi bound archive insert date */
    dsmDate       expDateLowerBound; /* low bound expiration date */
    dsmDate       expDateUpperBound; /* hi bound expiration date */
    dsChar_t      *descr;           /* archive description */
} tsmQryArchiveData;

#define tsmQryArchiveDataVersion 1

/*-----+
| Type definition for Query Archive response on dsmGetNextQObj()|
+-----*/
typedef struct tsmQryRespArchiveData
{
    dsUInt16_t    stVersion;        /* structure version */
    tsmObjName    objName;          /* Filespace name qualifier */
    dsUInt32_t    copyGroup;        /* copy group number */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsStruct64_t  objId;            /* Unique copy id */
    dsStruct64_t  reserved;         /* backward compatability */
    dsUInt8_t     mediaClass;       /* media access class */
    dsmDate       insDate;          /* archive insertion date */
    dsmDate       expDate;          /* expiration date for object */
    dsChar_t      descr[DSM_MAX_DESCR_LENGTH + 1]; /* archive description */
    dsUInt16_t    objInfoLen;       /* length of object-dependent info*/
    dsUInt8_t     reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    dsUInt160_t   restoreOrderExt; /* restore order */
    dsStruct64_t  sizeEstimate;     /* size estimate stored by user*/
    dsUInt8_t     compressType;     /* Compression flag */
    dsUInt8_t     retentionInitiated; /* object waiting on retention event */
    dsUInt8_t     objHeld;          /* object is on "hold" see dsmapi.h for values */
    dsUInt8_t     encryptionType;  /* type of encryption */
    dsmBool_t     clientDeduplicated; /* obj deduplicated by API*/
    dsUInt8_t     objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
    dsChar_t      compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
} tsmQryRespArchiveData;

```

```

#define tsmQryRespArchiveDataVersion 7

/*-----+
| Type definition for Archive sendBuff parameter on dsmSendObj()
+-----*/
typedef struct tsmSndArchiveData
{
    dsUInt16_t    stVersion;          /* structure version */
    dsChar_t      *descr;             /* archive description */
} tsmSndArchiveData;

#define tsmSndArchiveDataVersion 1

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery()
+-----*/
typedef struct tsmQryBackupData
{
    dsUInt16_t    stVersion;          /* structure version */
    tsmObjName    *objName;           /* full dsm name of object */
    dsChar_t      *owner;             /* owner name */
    dsUInt8_t     objState;           /* object state selector */
    dsmDate       pitDate;            /* Date value for point in time restore */
    /* for possible values, see defines above */
    dsUInt32_t    reserved1;
    dsUInt32_t    reserved2;
} tsmQryBackupData;

#define tsmQryBackupDataVersion 3

/*-----+
| Type definition for Query Backup response on dsmGetNextQObj()
+-----*/
typedef struct tsmQryRespBackupData
{
    dsUInt16_t    stVersion;          /* structure version */
    tsmObjName    objName;            /* full dsm name of object */
    dsUInt32_t    copyGroup;          /* copy group number */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsStruct64_t  objId;              /* Unique object id */
    dsStruct64_t  reserved;           /* backward compatability */
    dsUInt8_t     mediaClass;         /* media access class */
    dsUInt8_t     objState;           /* Obj state, active, etc. */
    dsmDate       insDate;            /* backup insertion date */
    dsmDate       expDate;            /* expiration date for object */
    dsUInt16_t    objInfolen;         /* length of object-dependent info */
    dsUInt8_t     reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    dsUInt160_t   restoreOrderExt;    /* restore order */
    dsStruct64_t  sizeEstimate;       /* size estimate stored by user */
    dsStruct64_t  baseObjId;
    dsUInt16_t    baseObjInfolen;     /* length of base object-dependent info */
    dsUInt8_t     baseObjInfo[DSM_MAX_OBJINFO_LENGTH]; /* base object-dependent info */
    dsUInt160_t   baseRestoreOrder;  /* restore order */
    dsUInt32_t    fsID;
    dsUInt8_t     compressType;
    dsmBool_t     isGroupLeader;
    dsmBool_t     isOpenGroup;
    dsUInt8_t     reserved1;          /* for future use */
    dsmBool_t     reserved2;          /* for future use */
    dsUInt16_t    reserved3;          /* for future use */
    reservedInfo_t *reserved4;        /* for future use */
    dsUInt8_t     encryptionType;     /* type of encryption */
    dsmBool_t     clientDeduplicated; /* obj deduplicated by API */
    dsUInt8_t     objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
    dsChar_t      compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
} tsmQryRespBackupData;

#define tsmQryRespBackupDataVersion 8

/*-----+
| Type definition for Active Backup queryBuffer on dsmBeginQuery()
|
| Notes: For the active backup query, only the fs (filesystem) and objType
|        fields of objName need be set.  objType can only be set to
|        DSM_OBJ_FILE or DSM_OBJ_DIRECTORY.  DSM_OBJ_ANY_TYPE will not
|        find a match on the query.
+-----*/
typedef struct tsmQryABackupData
{
    dsUInt16_t    stVersion;          /* structure version */
    tsmObjName    *objName;           /* Only fs and objtype used */
} tsmQryABackupData;

```

```

#define tsmQryABackupDataVersion 1

/*-----+
| Type definition for Query Active Backup response on dsmGetNextQObj() |
+-----*/
typedef struct tsmQryARespBackupData
{
    dsUInt16_t    stVersion;                /* structure version */
    tsmObjName    objName;                 /* full dsm name of object */
    dsUInt32_t    copyGroup;               /* copy group number */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /*management class name*/
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsmDate       insDate;                 /* backup insertion date */
    dsUInt16_t    objInfolen;              /* length of object-dependent info*/
    dsUInt8_t     reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    dsUInt8_t     objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
} tsmQryARespBackupData;

#define tsmQryARespBackupDataVersion 2

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct tsmQryBackupGroups
{
    dsUInt16_t    stVersion;                /* structure version */
    dsUInt8_t     groupType;
    dsChar_t      *fsName;
    dsChar_t      *owner;
    dsStruct64_t  groupLeaderObjId;
    dsUInt8_t     objType;
    dsUInt32_t     reserved1;
    dsUInt32_t     reserved2;
    dsmBool_t     noRestoreOrder;
    dsmBool_t     noGroupInfo;
    dsChar_t      *hl;
} tsmQryBackupGroups;

#define tsmQryBackupGroupsVersion 4

/*-----+
| Type definition for proxynode queryBuffer on tsmBeginQuery() |
+-----*/
typedef struct tsmQryProxyNodeData
{
    dsUInt16_t    stVersion;                /* structure version */
    dsChar_t      *targetNodeName;         /* target node name */
} tsmQryProxyNodeData;

#define tsmQryProxyNodeDataVersion 1

/*-----+
| Type definition for qryRespProxyNodeData parameter used on tsmGetNextQObj() |
+-----*/
typedef struct tsmQryRespProxyNodeData
{
    dsUInt16_t    stVersion;                /* structure version */
    dsChar_t      targetNodeName[DSM_MAX_ID_LENGTH+1]; /* target node name */
    dsChar_t      peerNodeName[DSM_MAX_ID_LENGTH+1]; /* peer node name */
    dsChar_t      hlAddress[DSM_MAX_ID_LENGTH+1]; /* peer hlAddress */
    dsChar_t      llAddress[DSM_MAX_ID_LENGTH+1]; /* peer llAddress */
} tsmQryRespProxyNodeData;

#define tsmQryRespProxyNodeDataVersion 1

/*-----+
| Type definition for WINNT and OS/2 Filespace attributes |
+-----*/
typedef struct tsmDosFSAttrib
{
    osChar_t      driveLetter;             /* drive letter for filespace */
    dsUInt16_t    fsInfoLength;            /* fsInfo length used */
    osChar_t      fsInfo[DSM_MAX_FSINFO_LENGTH]; /*caller-determined data */
} tsmDosFSAttrib;

/*-----+
| Type definition for UNIX Filespace attributes |
+-----*/
typedef struct tsmUnixFSAttrib
{

```

```

    dsUInt16_t    fsInfoLength;          /* fsInfo length used          */
    osChar_t      fsInfo[DSM_MAX_FSINFO_LENGTH]; /* caller-determined data */
} tsmUnixFSAttr;

/*-----+
| Type definition for NetWare Filespace attributes |
+-----*/
typedef tsmUnixFSAttr tsmNetwareFSAttr;

/*-----+
| Type definition for Filespace attributes on all Filespace calls |
+-----*/
typedef union
{
    tsmNetwareFSAttr    netwareFSAttr;
    tsmUnixFSAttr       unixFSAttr;
    tsmDosFSAttr        dosFSAttr;
} tsmFSAttr;

/*-----+
| Type definition for fsUpd parameter on dsmUpdateFS() |
+-----*/
typedef struct    tsmFSUpd
{
    dsUInt16_t      stVersion;          /* structure version          */
    dsChar_t        *fsType;            /* filespace type             */
    dsStruct64_t     occupancy;          /* occupancy estimate         */
    dsStruct64_t     capacity;           /* capacity estimate          */
    tsmFSAttr        fsAttr;            /* platform specific attributes */
} tsmFSUpd;

#define tsmFSUpdVersion 1

/*-----+
| Type definition for Filespace queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct tsmQryFSData
{
    dsUInt16_t      stVersion;          /* structure version */
    dsChar_t        *fsName;            /* File space name */
} tsmQryFSData;

#define tsmQryFSDataVersion 1

/*-----+
| Type definition for Query Filespace response on dsmGetNextQObj() |
+-----*/
typedef struct tsmQryRespFSData
{
    dsUInt16_t      stVersion;          /* structure version          */
    dsChar_t        fsName[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name          */
    dsChar_t        fsType[DSM_MAX_FSTYPE_LENGTH + 1]; /* Filespace type           */
    dsStruct64_t     occupancy;          /* Occupancy est. in bytes.  */
    dsStruct64_t     capacity;           /* Capacity est. in bytes.    */
    tsmFSAttr        fsAttr;            /* platform specific attributes */
    dsmDate          backStartDate;      /* start backup date          */
    dsmDate          backCompleteDate;   /* end backup Date            */
    dsmDate          reserved1;          /* For future use             */
    dsmBool_t        bIsUnicode;
    dsUInt32_t       fsID;
    dsmDate          lastReplStartDate;   /* The last time replication was started */
    dsmDate          lastReplCmpltDate;   /* The last time replication completed   */
    /* (could have had a failure,          */
    /* but it still completes)             */
    dsmDate          lastBackOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server                  */
    dsmDate          lastArchOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server                  */
    dsmDate          lastSpMgOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server                  */
    dsmDate          lastBackOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local                   */
    dsmDate          lastArchOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local                   */
    dsmDate          lastSpMgOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local                   */
    dsInt32_t        failOverWriteDelay; /* Minutes for client to wait before allowed */
    /* to store to this Repl srvr, Specail codes: */
    /* NO_ACCESS(-1), ACCESS_RDONLY (-2)          */
} tsmQryRespFSData;

#define tsmQryRespFSDataVersion 5

```



```

/*-----+
| Type definition for regFilespace parameter on dsmRegisterFS()
+-----*/
typedef struct tsmRegFSData
{
    dsUInt16_t    stVersion;                /* structure version */
    dsChar_t      *fsName;                  /* Filespace name */
    dsChar_t      *fsType;                  /* Filespace type */
    dsStruct64_t  occupancy;                /* Occupancy est. in bytes. */
    dsStruct64_t  capacity;                /* Capacity est. in bytes. */
    tsmFSAttr     fsAttr;                  /* platform specific attributes */
} tsmRegFSData;

#define tsmRegFSDataVersion 1

/*-----+
| Type definition for session info response on dsmQuerySessionInfo()
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;                /* Structure version */
    /*-----*/
    /* Server information */
    /*-----*/
    dsChar_t      serverHost[DSM_MAX_SERVERNAME_LENGTH+1];
    /* Network host name of DSM server */
    dsUInt16_t    serverPort;                /* Server comm port on host */
    dsmDate       serverDate;                /* Server's date/time */
    dsChar_t      serverType[DSM_MAX_SERVERTYPE_LENGTH+1];
    /* Server's execution platform */
    dsUInt16_t    serverVer;                /* Server's version number */
    dsUInt16_t    serverRel;                /* Server's release number */
    dsUInt16_t    serverLev;                /* Server's level number */
    dsUInt16_t    serverSubLev;            /* Server's sublevel number */
    /*-----*/
    /* Client Defaults */
    /*-----*/
    dsChar_t      nodeType[DSM_MAX_PLATFORM_LENGTH+1]; /*node/application type*/
    dsChar_t      fsdelim;                  /* File space delimiter */
    dsChar_t      hldelim;                  /* Delimiter betw highlev & lowlev */
    dsUInt8_t     compression;              /* Compression flag */
    dsUInt8_t     archDel;                  /* Archive delete permission */
    dsUInt8_t     backDel;                  /* Backup delete permission */
    dsUInt32_t    maxBytesPerTxn;           /* for future use */
    dsUInt16_t    maxObjPerTxn;            /* The max objects allowed in a txn */
    /*-----*/
    /* Session Information */
    /*-----*/
    dsChar_t      id[DSM_MAX_ID_LENGTH+1]; /* Sign-in id node name */
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH+1]; /* Sign-in owner */
    /* (for multi-user platforms) */
    dsChar_t      confFile[DSM_PATH_MAX + DSM_NAME_MAX +1];
    /* len is platform dep */
    /* dsInit name of appl config file */
    dsUInt8_t     opNoTrace;                /* dsInit option - NoTrace = 1 */
    /*-----*/
    /* Policy Data */
    /*-----*/
    dsChar_t      domainName[DSM_MAX_DOMAIN_LENGTH+1]; /* Domain name */
    dsChar_t      policySetName[DSM_MAX_PS_NAME_LENGTH+1];
    /* Active policy set name */
    dsmDate       polActDate;                /* Policy set activation date */
    dsChar_t      dfltMCName[DSM_MAX_MC_NAME_LENGTH+1]; /* Default Mgmt Class */
    dsUInt16_t    gpBackRetn;                /* Grace-period backup retention */
    dsUInt16_t    gpArchRetn;                /* Grace-period archive retention */
    dsChar_t      adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* adsm server name */
    dsmBool_t     archiveRetentionProtection; /* is server Retention protection enabled */
    dsUInt64_t    maxBytesPerTxn_64;        /* for future use */
    dsmBool_t     lanFreeEnabled;            /* lan free option is set */
    dsmDedupType  dedupType;                /* server or clientOrServer */
    dsChar_t      accessNode[DSM_MAX_ID_LENGTH+1]; /* as node node name */
    /*-----*/
    /* Replication and fail over information */
    /*-----*/
    dsmFail0vrCfgType fail0verCfgType; /* status of fail over */
    dsChar_t      replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
    dsChar_t      homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
    dsChar_t      replServerHost[DSM_MAX_SERVERNAME_LENGTH+1]; /* Network host name of DSM server */
    dsInt32_t     replServerPort;           /* Server comm port on host */
}

```

```

} tsmApiSessInfo;

#define tsmApiSessInfoVersion 6

/*-----+
| Type definition for Query options response on dsmQueryCliOptions()
| and dsmQuerySessOptions()
+-----*/

typedef struct
{
    dsUInt16_t    stVersion;
    dsChar_t      dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t      dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t      serverName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsInt16_t      commMethod;
    dsChar_t      serverAddress[DSM_MAX_SERVER_ADDRESS];
    dsChar_t      nodeName[DSM_MAX_NODE_LENGTH+1];
    dsmBool_t      compression;
    dsmBool_t      compressalways;
    dsmBool_t      passwordAccess;
} tsmOptStruct ;

#define tsmOptStructVersion 1

/*-----+
| Type definition for qryRespAccessData parameter used on dsmQueryAccess()
+-----*/

typedef struct
{
    dsUInt16_t      stVersion ;                /* structure version */
    dsChar_t         node[DSM_MAX_ID_LENGTH+1]; /* node name */
    dsChar_t         owner[DSM_MAX_OWNER_LENGTH+1]; /* owner */
    tsmObjName       objName ;                 /* object name */
    dsmAccessType    accessType;               /* archive or backup */
    dsUInt32_t       ruleNumber ;              /* Access rule id */
} tsmQryRespAccessData;

#define tsmQryRespAccessDataVersion 1

/*-----+
| Type definition for envSetUp parameter on dsmSetUp()
+-----*/

typedef struct tsmEnvSetUp
{
    dsUInt16_t      stVersion;                /* structure version */
    dsChar_t         dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t         dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t         dsmiLog[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char             **argv; /* for executables name argv[0] */
    dsChar_t         logName[DSM_NAME_MAX +1];
    dsmBool_t         reserved1; /* for future use */
    dsmBool_t         reserved2; /* for future use */
} tsmEnvSetUp;

#define tsmEnvSetUpVersion 4

/*-----+
| Type definition for dsmInitExIn_t
+-----*/

typedef struct tsmInitExIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    tsmApiVersionEx *apiVersionExp;
    dsChar_t        *clientNodeNameP;
    dsChar_t        *clientOwnerNameP;
    dsChar_t        *clientPasswordP;
    dsChar_t        *userNameP;
    dsChar_t        *userPasswordP;
    dsChar_t        *applicationTypeP;
    dsChar_t        *configfile;
    dsChar_t        *options;
    dsChar_t        dirDelimiter;
    dsmBool_t        useUnicode;
    dsmBool_t        bCrossPlatform;
    dsmBool_t        bService;
    dsmBool_t        bEncryptKeyEnabled;
    dsChar_t        *encryptionPasswordP;
    dsmBool_t        useTsmBuffers;
    dsUInt8_t        numTsmBuffers;

```

```

    tsmAppVersion      appVersionP;
} tsmInitExIn_t;

#define tsmInitExInVersion 5

/*-----+
| Type definition for dsmInitExOut_t
+-----*/
typedef struct tsmInitExOut_t
{
    dsUint16_t          stVersion; /* structure version */
    dsInt16_t           userNameAuthorities;
    dsInt16_t           infoRC; /* error return code if encountered */
    /* adsm server name */
    dsChar_t            adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsUint16_t          serverVer; /* Server's version number */
    dsUint16_t          serverRel; /* Server's release number */
    dsUint16_t          serverLev; /* Server's level number */
    dsUint16_t          serverSubLev; /* Server's sublevel number */
    dsMBool_t           bIsFailOverMode; /* true if failover has occurred */
    dsChar_t            replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
    dsChar_t            homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
} tsmInitExOut_t;

#define tsmInitExOutVersion 3

/*-----+
| Type definition for dsmLogExIn_t
+-----*/
typedef struct tsmLogExIn_t
{
    dsUint16_t          stVersion; /* structure version */
    dsMLogSeverity_t    severity;
    dsChar_t            appMsgID[8];
    dsMLogType_t        logType; /* log type : local, server, both */
    dsChar_t            *message; /* text of message to be logged */
    dsChar_t            appName[DSM_MAX_PLATFORM_LENGTH];
    dsChar_t            osPlatform[DSM_MAX_PLATFORM_LENGTH];
    dsChar_t            appVersion[DSM_MAX_PLATFORM_LENGTH];
} tsmLogExIn_t;

#define tsmLogExInVersion 2

/*-----+
| Type definition for dsmLogExOut_t
+-----*/
typedef struct tsmLogExOut_t
{
    dsUint16_t          stVersion; /* structure version */
} tsmLogExOut_t;

#define tsmLogExOutVersion 1

/*-----+
| Type definition for dsmRenameIn_t
+-----*/
typedef struct tsmRenameIn_t
{
    dsUint16_t          stVersion; /* structure version */
    dsUint32_t          tsmHandle; /* handle for session */
    dsUint8_t           repository; /* Backup or Archive */
    tsmObjName_t        *objNameP; /* object name */
    dsChar_t            newHl[DSM_MAX_HL_LENGTH + 1]; /* new High level name */
    dsChar_t            newLl[DSM_MAX_LL_LENGTH + 1]; /* new Low level name */
    dsMBool_t           merge; /* merge into existing name */
    ObjID_t             objId; /* objId for Archive */
} tsmRenameIn_t;

#define tsmRenameInVersion 1

/*-----+
| Type definition for dsmRenameOut_t
+-----*/
typedef struct tsmRenameOut_t
{
    dsUint16_t          stVersion; /* structure version */
} tsmRenameOut_t;

#define tsmRenameOutVersion 1

/*-----+

```

```

| Type definition for tsmEndSendObjExIn_t
+-----*/
typedef struct tsmEndSendObjExIn_t
{
    dsUint16_t      stVersion;                /* structure version */
    dsUint32_t      tsmHandle;                /* handle for session */
} tsmEndSendObjExIn_t;

#define tsmEndSendObjExInVersion 1

/*-----+
| Type definition for dsmEndSendObjExOut_t
+-----*/
typedef struct tsmEndSendObjExOut_t
{
    dsUint16_t      stVersion;                /* structure version */
    dsStruct64_t     totalBytesSent;           /* total bytes read from app */
    dsmBool_t       objCompressed;            /* was object compressed */
    dsStruct64_t     totalCompressSize;        /* total size after compress */
    dsStruct64_t     totalLFBytesSent;        /* total bytes sent Lan Free */
    dsUint8_t       encryptionType;          /* type of encryption used */
    dsmBool_t       objDeduplicated;          /* was object processed for dist. data dedup */
    dsStruct64_t     totalDedupSize;          /* total size after de-dup */
} tsmEndSendObjExOut_t;

#define tsmEndSendObjExOutVersion 3

/*-----+
| Type definition for tsmGroupHandlerIn_t
+-----*/
typedef struct tsmGroupHandlerIn_t
{
    dsUint16_t      stVersion;                /* structure version */
    dsUint32_t      tsmHandle;                /* handle for session */
    dsUint8_t       groupType;                /* Type of group */
    dsUint8_t       actionType;               /* Type of group operation */
    dsUint8_t       memberType;              /* Type of member: Leader or member */
    dsStruct64_t     leaderObjId;             /* OBJID of the groupleader */
    dsChar_t        *uniqueGroupTagP;         /* Unique group identifier */
    tsmObjName       *objNameP;              /* group leader object name */
    dsmGetList       memberObjList;          /* list of objects to remove, assign */
} tsmGroupHandlerIn_t;

#define tsmGroupHandlerInVersion 1

/*-----+
| Type definition for tsmGroupHandlerExOut_t
+-----*/
typedef struct tsmGroupHandlerOut_t
{
    dsUint16_t      stVersion;                /* structure version */
} tsmGroupHandlerOut_t;

#define tsmGroupHandlerOutVersion 1

/*-----+
| Type definition for tsmEndTxnExIn_t
+-----*/
typedef struct tsmEndTxnExIn_t
{
    dsUint16_t      stVersion;                /* structure version */
    dsUint32_t      tsmHandle;                /* handle for session */
    dsUint8_t       vote;
} tsmEndTxnExIn_t;

#define tsmEndTxnExInVersion 1

/*-----+
| Type definition for tsmEndTxnExOut_t
+-----*/
typedef struct tsmEndTxnExOut_t
{
    dsUint16_t      stVersion;                /* structure version */
    dsUint16_t      reason;                   /* reason code */
    dsStruct64_t     groupLeaderObjId;        /* groupLeader obj id returned on */
    /* DSM_ACTION_OPEN */
    dsUint8_t       reserved1;                /* future use */
    dsUint16_t      reserved2;                /* future use */
} tsmEndTxnExOut_t;

#define tsmEndTxnExOutVersion 1

```

```

/*-----+
| Type definition for tsmEndGetDataExIn_t
+-----*/
typedef struct tsmEndGetDataExIn_t
{
    dsUint16_t    stVersion;    /* structure version */
    dsUint32_t    tsmHandle;    /* handle for session */
}tsmEndGetDataExIn_t;

#define tsmEndGetDataExInVersion 1

/*-----+
| Type definition for tsmEndGetDataExOut_t
+-----*/
typedef struct tsmEndGetDataExOut_t
{
    dsUint16_t    stVersion;    /* structure version */
    dsUint16_t    reason;        /* reason code */
    dsStruct64_t  totalLFBytesRecv; /* total lan free bytes recieved */
}tsmEndGetDataExOut_t;

#define tsmEndGetDataExOutVersion 1

/*-----+
| Type definition for on tsmRetentionEvent()
+-----*/
typedef struct tsmRetentionEventIn_t
{
    dsUint16_t    stVersion;    /* structure version */
    dsUint32_t    tsmHandle;    /* session Handle */
    dsmEventType_t eventType;    /* Event type */
    dsmObjList_t  objList;      /* object ID */
}tsmRetentionEventIn_t;

#define tsmRetentionEventInVersion 1

/*-----+
| Type definition for on tsmRetentionEvent()
+-----*/
typedef struct tsmRetentionEventOut_t
{
    dsUint16_t    stVersion;    /* structure version */
}tsmRetentionEventOut_t;

#define tsmRetentionEventOutVersion 1

/*-----+
| Type definition for tsmUpdateObjExIn_t
+-----*/
typedef struct tsmUpdateObjExIn_t
{
    dsUint16_t    stVersion;    /* structure version */
    dsUint32_t    tsmHandle;    /* session Handle */
    tsmSendType_t sendType;    /* send type back/arch */
    dsChar_t      *descrP;      /* archive description */
    tsmObjName_t  *objNameP;    /* objName */
    tsmObjAttr_t  *objAttrPtr;  /* attribute */
    dsUint32_t    objUpdAct;    /* update action */
    ObjID_t       archObjId;    /* objId for archive */
}tsmUpdateObjExIn_t;

#define tsmUpdateObjExInVersion 1

/*-----+
| Type definition for tsmUpdateObjExOut_t
+-----*/
typedef struct tsmUpdateObjExOut_t
{
    dsUint16_t    stVersion;    /* structure version */
}tsmUpdateObjExOut_t;

#define tsmUpdateObjExOutVersion 1

#if _OPSYS_TYPE == DS_WINNT
#pragma pack()
#endif

#ifdef _MAC
#pragma options align = reset
#endif
#endif /* _H_TSMAPITD */

```

## dsmapips.h

```
/*
 * IBM Storage Protect
 * API Client Component
 *
 * IBM Confidential
 * (IBM Confidential-Restricted when combined with the Aggregated OCO
 * source modules for this program)
 *
 * OCO Source Materials
 *
 * 5648-020 (C) Copyright IBM Corporation 1993, 2016
 */

/*
 * Header File Name: dsmapips.h
 *
 * Environment:
 * ** This is a platform-specific source file **
 * ** versioned for Windows NT **
 *
 * Design Notes: This file includes platform dependent definitions
 *
 * Descriptive-name: Definitions for IBM Storage Protect typedefs and LINKAGE
 *-----*/

#ifndef _H_DSMAPIPS
#define _H_DSMAPIPS

#ifndef _WIN64
#pragma pack(1)
#endif

/*
 * T Y P E D E F S
 */

/* new typedef file for Version 3 */

#if !defined(DSMAPILIB) || defined (XOPEN_BUILD)

/* support for linkage */
#include <windows.h>
#define DSMLINKAGE WINAPI

#define DS_WINNT 22
#define _OPSYS_TYPE DS_WINNT

typedef signed char dsInt8_t;
typedef unsigned char dsUInt8_t;
typedef signed short dsInt16_t;
typedef unsigned short dsUInt16_t;
typedef signed long dsInt32_t;
typedef unsigned long dsUInt32_t;

/*=== Character and string types ===*/
#ifdef UNICODE
typedef wchar_t dsChar_t;
#define dsTEXT(x) L##x
#else
typedef char dsChar_t;
#define dsTEXT(x) x
#endif /* !UNICODE */

/*=== Common typedefs and defines derived from dsChar_t ===*/
typedef dsChar_t *dsString_t;

/* added for the extended restore order */
typedef struct
```

```

{
    dsUint32_t top;
    dsUint32_t hi_hi;
    dsUint32_t hi_lo;
    dsUint32_t lo_hi;
    dsUint32_t lo_lo;
} dsUint160_t ;

#if defined(_LONG_LONG)
    typedef __int64          dsInt64_t;
    typedef unsigned __int64 dsUint64_t;
    /*=== A "true" unsigned 64-bit integer ===*/
    typedef __int64          dsLongLong_t;
#else
    typedef struct tagUINT64_t
    {
        dsUint32_t hi;          /* Most significant 32 bits. */
        dsUint32_t lo;          /* Least significant 32 bits. */
    } dsUint64_t;
#endif

/*-----+
| Type definition for bool_t |
+-----*/
/*
 * Had to create a Boolean type that didn't clash with any other predefined
 * version in any operating system or windowing system.
 */
typedef enum
{
    dsmFalse = 0x00,
    dsmTrue  = 0x01
} dsmBool_t ;

/*=== for backward compatability ===*/
#define uint8      dsUint8_t
#define int8       dsInt8_t
#define uint16     dsUint16_t
#define int16      dsInt16_t
#define uint32     dsUint32_t
#define int32      dsInt32_t
#define uint64     dsStruct64_t
#define bool_t     dsBool_t
#define dsBool_t   dsmBool_t
#define bTrue      dsmTrue
#define bFalse     dsmFalse

typedef struct
{
    dsUint32_t hi;          /* Most significant 32 bits. */
    dsUint32_t lo;          /* Least significant 32 bits. */
} dsStruct64_t ;

#endif /* DSMAPILIB */

#ifndef _WIN64
#pragma pack()
#endif
#endif /* _H_DSMAPIPS */

```

## release.h

```

/*****
 * IBM Storage Protect
 * Common Source Component
 *
 * (C) Copyright IBM Corporation 1993,2018
 *****/

/*****
 * Header File Name: release.h
 *
 * Environment: ****
 *               ** This is a platform-independent source file **
 *               ****
 *****/

```

```

* Design Notes:      This file contains the common information about
*                    the actual version.release.level.sublevel
*
* Descriptive-name:  Definitions for IBM Storage Protect version
*
* Note: This file should contain no LOG or CMVC information. It is
*       shipped with the API code.
*
*-----*/

#ifndef _H_RELEASE
#define _H_RELEASE

#define COMMON_VERSION      8
#define COMMON_RELEASE      1
#define COMMON_LEVEL        13
#define COMMON_SUBLEVEL     0
#define COMMON_DRIVER       dsTEXT("")

#define COMMON_VERSIONTXT "8.1.13.0"

#define SHIPYEARTXT ""
#define SHIPYEARTXTW dsTEXT("2021")
#define TSMPRODTXT "IBM Storage Protect"

/*=====
The following string definitions are used for VERSION information
and should not be converted to dsTEXT or osTEXT. They are used
only at link time.

These are also used when the Jar file is built on Unix. See the
the perl script tools/unx/mzbuild/createReleaseJava
=====*/
#define COMMON_VERSION_STR "8"
#define COMMON_RELEASE_STR "1"
#define COMMON_LEVEL_STR "13"
#define COMMON_SUBLEVEL_STR "0"
#define COMMON_DRIVER_STR ""

/*=== product names definitions ===*/
#define COMMON_NAME_DFDSM 1
#define COMMON_NAME_ADSM 2
#define COMMON_NAME_TSM 3
#define COMMON_NAME_ITSM 4
#define COMMON_NAME COMMON_NAME_ITSM

/*=====
Internal version, release, and level (build) version. This
should be unique for every version+release+ptf of a product.
This information is recorded in the file attributes and data
stream for diagnostic purposes.
NOTE: DO NOT MODIFY THESE VALUES. YOU CAN ONLY ADD NEW ENTRIES!
=====*/
#define COMMON_BUILD_TSM_510 1
#define COMMON_BUILD_TSM_511 2
#define COMMON_BUILD_TSM_515 3
#define COMMON_BUILD_TSM_516 4
#define COMMON_BUILD_TSM_520 5
#define COMMON_BUILD_TSM_522 6
#define COMMON_BUILD_TSM_517 7
#define COMMON_BUILD_TSM_523 8
#define COMMON_BUILD_TSM_530 9
#define COMMON_BUILD_TSM_524 10
#define COMMON_BUILD_TSM_532 11
#define COMMON_BUILD_TSM_533 12
#define COMMON_BUILD_TSM_525 13
#define COMMON_BUILD_TSM_534 14
#define COMMON_BUILD_TSM_540 15
#define COMMON_BUILD_TSM_535 16
#define COMMON_BUILD_TSM_541 17
#define COMMON_BUILD_TSM_550 18
#define COMMON_BUILD_TSM_542 19
#define COMMON_BUILD_TSM_551 20
#define COMMON_BUILD_TSM_610 21
#define COMMON_BUILD_TSM_552 22
#define COMMON_BUILD_TSM_611 23
#define COMMON_BUILD_TSM_543 24
#define COMMON_BUILD_TSM_620 25
#define COMMON_BUILD_TSM_612 26
#define COMMON_BUILD_TSM_553 27
#define COMMON_BUILD_TSM_613 28
#define COMMON_BUILD_TSM_621 29

```



```

#define COMMON_BUILD_TSM_622 30
#define COMMON_BUILD_TSM_614 31
#define COMMON_BUILD_TSM_623 32
#define COMMON_BUILD_TSM_630 33
#define COMMON_BUILD_TSM_615 34
#define COMMON_BUILD_TSM_624 35
#define COMMON_BUILD_TSM_631 36
#define COMMON_BUILD_TSM_640 37
#define COMMON_BUILD_TSM_710 38
#define COMMON_BUILD_TSM_625 39
#define COMMON_BUILD_TSM_641 40
#define COMMON_BUILD_TSM_711 41
#define COMMON_BUILD_TSM_712 42
#define COMMON_BUILD_TSM_713 43
#define COMMON_BUILD_TSM_714 44
#define COMMON_BUILD_TSM_720 45
#define COMMON_BUILD_TSM_721 46
#define COMMON_BUILD_TSM_642 47
#define COMMON_BUILD_TSM_643 48
#define COMMON_BUILD_TSM_715 49
#define COMMON_BUILD_TSM_716 50
#define COMMON_BUILD_TSM_810 51
#define COMMON_BUILD_TSM_811 52
#define COMMON_BUILD_TSM_812 53
#define COMMON_BUILD_TSM_718 54
#define COMMON_BUILD_TSM_814 55
#define COMMON_BUILD_TSM_816 56
#define COMMON_BUILD_TSM_817 57
#define COMMON_BUILD_TSM_818 58
#define COMMON_BUILD_TSM_819 59
#define COMMON_BUILD_TSM_8110 60
#define COMMON_BUILD_TSM_8111 61
#define COMMON_BUILD_TSM_8112 62
#define COMMON_BUILD_TSM_8113 63
#define COMMON_BUILD COMMON_BUILD_TSM_8113

/*=== define VRL as an Int for bitmap version compares ===*/
static const int VRL_712 = 712;
static const int VRL_713 = 713;
static const int VRL_714 = 714;
static const int VRL_715 = 715;
static const int VRL_716 = 716;
static const int VRL_718 = 718;
static const int VRL_810 = 810;
static const int VRL_811 = 811;
static const int VRL_812 = 812;
static const int VRL_814 = 814;
static const int VRL_816 = 816;
static const int VRL_817 = 817;
static const int VRL_818 = 818;
static const int VRL_819 = 819;
static const int VRL_8110 = 8110;
static const int VRL_8111 = 8111;
static const int VRL_8112 = 8112;
static const int VRL_8113 = 8113;

#define TDP4VE_PLATFORM_STRING_MBCS "TDP VMware"
#define TDP4VE_PLATFORM_STRING dsTEXT("TDP VMware")

#define TDP4HYPERV_PLATFORM_STRING_MBCS "TDP HyperV"
#define TDP4HYPERV_PLATFORM_STRING dsTEXT("TDP HyperV")

#endif /* _H_RELEASE */

```

## API function definitions source file

This appendix contains the `dsmapifp.h` header file, so you can see the function definitions for the API.

**Note:** **DSMLINKAGE** is defined differently for each operating system. See the definitions in the `dsmapips.h` file for your specific operating system.

The information that is provided here contains a point-in-time copy of the files that are distributed with the API. View the files in the API distribution package for the latest version.

## dsmapifp.h

```

/*****
* IBM Storage Protect
* API Client Component
*
* IBM Confidential
* (IBM Confidential-Restricted when combined with the Aggregated OCO
* source modules for this program)
*
* OCO Source Materials
*
* 5648-020 (C) Copyright IBM Corporation 1993, 2016
*****/

/*****/
/* Header File Name: dsmapifp.h */
/*
/* Descriptive-name: IBM Storage Protect API function prototypes */
/*****/
#ifndef _H_DSMAPIFP
#define _H_DSMAPIFP

#ifdef __cplusplus
extern "C" {
#endif

#ifdef DYNALOAD_DSMAPI

/* function will be dynamically loaded */
#include "dsmapidl.h"

#else

/* functions will be implicitly loaded from library */

/*****
/*
/* P U B L I C F U N C T I O N S
/*
*****/

extern dsInt16_t DSMLINKAGE dsmBeginGetData(
    dsUInt32_t
    dsBool_t
    dsmGetType
    dsmGetList
    dsmHandle,
    mountWait,
    getType,
    *dsmGetObjListP
);

extern dsInt16_t DSMLINKAGE dsmBeginQuery(
    dsUInt32_t
    dsmQueryType
    dsmQueryBuff
    dsmHandle,
    queryType,
    *queryBuffer
);

extern dsInt16_t DSMLINKAGE dsmBeginTxn(
    dsUInt32_t
    dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmBindMC(
    dsUInt32_t
    dsmObjName
    dsmSendType
    mcBindKey
    dsmHandle,
    *objNameP,
    sendType,
    *mcBindKeyP
);

extern dsInt16_t DSMLINKAGE dsmChangePW(
    dsUInt32_t
    char
    char
    dsmHandle,
    *oldPW,
    *newPW
);

extern dsInt16_t DSMLINKAGE dsmCleanUp(
    dsBool_t
    mtFlag
);

extern dsInt16_t DSMLINKAGE dsmDeleteAccess(
    dsUInt32_t
    dsUInt32_t
    dsmHandle,
    ruleNum
);

```

```

extern dsInt16_t DSMLINKAGE dsmDeleteObj(
    dsUInt32_t dsmHandle,
    dsmDelType delType,
    dsmDelInfo delInfo
);

extern dsInt16_t DSMLINKAGE dsmDeleteFS(
    dsInt16_t dsmHandle,
    char *fsName,
    dsUInt8_t repository
);

extern dsInt16_t DSMLINKAGE dsmEndGetData(
    dsInt16_t dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndGetDataEx(
    dsInt16_t dsmEndGetDataExIn_t *dsmEndGetDataExInP,
    dsInt16_t dsmEndGetDataExOut_t *dsmEndGetDataExOutP
);

extern dsInt16_t DSMLINKAGE dsmEndGetObj(
    dsInt16_t dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndQuery(
    dsInt16_t dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndSendObj(
    dsInt16_t dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndSendObjEx(
    dsInt16_t dsmEndSendObjExIn_t *dsmEndSendObjExInP,
    dsInt16_t dsmEndSendObjExOut_t *dsmEndSendObjExOutP
);

extern dsInt16_t DSMLINKAGE dsmEndTxnEx(
    dsInt16_t dsmEndTxnExIn_t *dsmEndTxnExInP,
    dsInt16_t dsmEndTxnExOut_t *dsmEndTxnExOutP
);

extern dsInt16_t DSMLINKAGE dsmEndTxn(
    dsInt16_t dsmHandle,
    dsUInt8_t vote,
    dsUInt16_t *reason
);

extern dsInt16_t DSMLINKAGE dsmGetData(
    dsInt16_t dsmHandle,
    dsUInt32_t DataBlk,
    *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmGetBufferData(
    dsInt16_t getBufferDataIn_t *dsmGetBufferDataInP,
    dsInt16_t getBufferDataOut_t *dsmGetBufferDataOutP
);

extern dsInt16_t DSMLINKAGE dsmGetNextQObj(
    dsInt16_t dsmHandle,
    dsUInt32_t DataBlk,
    *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmGetObj(
    dsInt16_t dsmHandle,
    dsUInt32_t ObjID,
    dsInt16_t *objIdP,
    dsInt16_t *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmGroupHandler(
    dsInt16_t dsmGroupHandlerIn_t *dsmGroupHandlerInP,
    dsInt16_t dsmGroupHandlerOut_t *dsmGroupHandlerOutP
);

extern dsInt16_t DSMLINKAGE dsmInit(
    dsInt16_t *dsmHandle,
    dsInt16_t *dsmApiVersionP,
    char *clientNodeNameP,
    char *clientOwnerNameP,
    char *clientPasswordP,
    char *applicationType,

```

```

    char
    char
);
extern dsInt16_t DSMLINKAGE dsmInitEx(
    dsUInt32_t *dsmHandleP,
    dsmInitExIn_t *dsmInitExInP,
    dsmInitExOut_t *dsmInitExOutP
);

extern dsInt16_t DSMLINKAGE dsmLogEvent(
    dsUInt32_t dsmHandle,
    logInfo *lopInfoP
);

extern dsInt16_t DSMLINKAGE dsmLogEventEx(
    dsUInt32_t dsmHandle,
    dsmLogExIn_t *dsmLogExInP,
    dsmLogExOut_t *dsmLogExOutP
);

extern dsInt16_t DSMLINKAGE dsmQueryAccess(
    dsUInt32_t dsmHandle,
    qryRespAccessData **accessListP,
    dsUInt16_t *numberOfRules
);

extern void DSMLINKAGE dsmQueryApiVersion(
    dsmApiVersion *apiVersionP
);

extern void DSMLINKAGE dsmQueryApiVersionEx(
    dsmApiVersionEx *apiVersionP
);

extern dsInt16_t DSMLINKAGE dsmQueryCliOptions(
    optStruct *optstructP
);

extern dsInt16_t DSMLINKAGE dsmQuerySessInfo(
    dsUInt32_t dsmHandle,
    ApiSessInfo *SessInfoP
);

extern dsInt16_t DSMLINKAGE dsmQuerySessOptions(
    dsUInt32_t dsmHandle,
    optStruct *optstructP
);

extern dsInt16_t DSMLINKAGE dsmRCMsg(
    dsUInt32_t dsmHandle,
    dsInt16_t dsmRC,
    char *msg
);

extern dsInt16_t DSMLINKAGE dsmRegisterFS(
    dsUInt32_t dsmHandle,
    regFSData *regFilespaceP
);

extern dsInt16_t DSMLINKAGE dsmReleaseBuffer(
    dsInt16_t releaseBufferIn_t,
    dsInt16_t releaseBufferOut_t,
    *dsmReleaseBufferInP,
    *dsmReleaseBufferOutP
);

extern dsInt16_t DSMLINKAGE dsmRenameObj(
    dsInt16_t dsmRenameIn_t,
    dsInt16_t dsmRenameOut_t,
    *dsmRenameInP,
    *dsmRenameOutP
);

extern dsInt16_t DSMLINKAGE dsmRequestBuffer(
    dsInt16_t requestBufferIn_t,
    dsInt16_t requestBufferOut_t,
    *dsmRequestBufferInP,
    *dsmRequestBufferOutP
);

extern dsInt16_t DSMLINKAGE dsmRetentionEvent(
    dsInt16_t dsmRetentionEventIn_t,
    dsInt16_t dsmRetentionEventOut_t,
    *dsmRetentionEventInP,
    *dsmRetentionEventOutP
);

extern dsInt16_t DSMLINKAGE dsmSendBufferData(

```

```

        sendBufferDataIn_t      *dsmSendBufferDataInP,
        sendBufferDataOut_t     *dsmSendBufferDataOutP
    );

extern dsInt16_t DSMLINKAGE     dsmSendData(
    dsUInt32_t
    DataBlk
    ) ;

extern dsInt16_t DSMLINKAGE     dsmSendObj(
    dsUInt32_t
    dsmSendType
    void
    dsmObjName
    ObjAttr
    DataBlk
    );

extern dsInt16_t DSMLINKAGE     dsmSetAccess(
    dsUInt32_t
    dsmAccessType
    dsmObjName
    char
    char
    );

extern dsInt16_t DSMLINKAGE     dsmSetUp(
    dsBool_t
    envSetUp
    );

extern dsInt16_t DSMLINKAGE     dsmTerminate(
    dsUInt32_t
    );

extern dsInt16_t DSMLINKAGE     dsmUpdateFS(
    dsUInt32_t
    char
    dsmFSUpd
    dsUInt32_t
    );

extern dsInt16_t DSMLINKAGE     dsmUpdateObj(
    dsUInt32_t
    dsmSendType
    void
    dsmObjName
    ObjAttr
    dsUInt32_t
    );

extern dsInt16_t DSMLINKAGE     dsmUpdateObjEx(
    dsmUpdateObjExIn_t
    dsmUpdateObjExOut_t
    );

#endif /* ifdef DYNALOAD */

#ifdef __cplusplus
}
#endif

#endif /* _H_DSMAPIFP */

```

## tsmapifp.h

This section contains the function definitions for the API. It is a copy of the tsmapifp.h header file.

**Note:** **DSMLINKAGE** is defined differently for each operating system. See the definitions in the tsmapifs.h file for your specific operating system.

```

/*****
* IBM Storage Protect
* API Client Component
*
* IBM Confidential
*****/

```

```

* (IBM Confidential-Restricted when combined with the Aggregated OCO *
* source modules for this program) *
* *
* OCO Source Materials *
* *
* 5648-020 (C) Copyright IBM Corporation 1993, 2016 *
*****/

/*****/
/* Header File Name: tsmapifp.h */
/*
/* Descriptive-name: IBM Storage Protect API function prototypes */
/*****/
#ifndef _H_TSMAPIFP
#define _H_TSMAPIFP

#if defined(__cplusplus)
extern "C" {
#endif

#ifdef DYNALOAD_DSMAPI

/* function will be dynamically loaded */
#include "dsmapidl.h"

#else

/* functions will be implicitly loaded from library */

/*=====*/
/*P U B L I C   F U N C T I O N S                               */
/*=====*/

typedef void tsmQueryBuff;

extern dsInt16_t DSMLINKAGE tsmBeginGetData(
    dsUInt32_t tsmHandle,
    dsBool_t mountWait,
    tsmGetType,
    dsmGetList *dsmGetObjListP
);

extern dsInt16_t DSMLINKAGE tsmBeginQuery(
    dsUInt32_t tsmHandle,
    tsmQueryType queryType,
    tsmQueryBuff *queryBuffer
);

extern dsInt16_t DSMLINKAGE tsmBeginTxn(
    dsUInt32_t tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmBindMC(
    dsUInt32_t tsmHandle,
    tsmObjName *objNameP,
    tsmSendType sendType,
    tsmMcBindKey *mcBindKeyP
);

extern dsInt16_t DSMLINKAGE tsmChangePW(
    dsUInt32_t tsmHandle,
    dsChar_t *oldPW,
    dsChar_t *newPW
);

extern dsInt16_t DSMLINKAGE tsmCleanUp(
    dsBool_t mtFlag
);

extern dsInt16_t DSMLINKAGE tsmDeleteAccess(
    dsUInt32_t tsmHandle,
    dsUInt32_t ruleNum
);

extern dsInt16_t DSMLINKAGE tsmDeleteObj(
    dsUInt32_t tsmHandle,
    tsmDelType delType,
    tsmDelInfo delInfo
);

```

```

extern dsInt16_t DSMLINKAGE tsmDeleteFS(
    dsUInt32_t tsmHandle,
    dsChar_t *fsName,
    dsUInt8_t repository
);

extern dsInt16_t DSMLINKAGE tsmEndGetData(
    dsUInt32_t tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmEndGetDataEx(
    tsmEndGetDataExIn_t *tsmEndGetDataExInP,
    tsmEndGetDataExOut_t *tsmEndGetDataExOutP
);

extern dsInt16_t DSMLINKAGE tsmEndGetObj(
    dsUInt32_t tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmEndQuery(
    dsUInt32_t tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmEndSendObj(
    dsUInt32_t tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmEndSendObjEx(
    tsmEndSendObjExIn_t *tsmEndSendObjExInP,
    tsmEndSendObjExOut_t *tsmEndSendObjExOutP
);

extern dsInt16_t DSMLINKAGE tsmEndTxn(
    dsUInt32_t tsmHandle,
    dsUInt8_t vote,
    dsUInt16_t *reason
);

extern dsInt16_t DSMLINKAGE tsmEndTxnEx(
    tsmEndTxnExIn_t *tsmEndTxnExInP,
    tsmEndTxnExOut_t *tsmEndTxnExOutP
);

extern dsInt16_t DSMLINKAGE tsmGetData(
    dsUInt32_t tsmHandle,
    DataBlk*dataBlkPtr
);

extern dsInt16_t DSMLINKAGE tsmGetBufferData(
    dsInt16_t getBufferDataIn_t *tsmGetBufferDataInP,
    dsInt16_t getBufferDataOut_t *tsmGetBufferDataOutP
);

extern dsInt16_t DSMLINKAGE tsmGetNextQObj(
    dsUInt32_t tsmHandle,
    DataBlk*dataBlkPtr
);

extern dsInt16_t DSMLINKAGE tsmGetObj(
    dsUInt32_t tsmHandle,
    ObjID *objIdP,
    DataBlk *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE tsmGroupHandler(
    tsmGroupHandlerIn_t *tsmGroupHandlerInP,
    tsmGroupHandlerOut_t *tsmGroupHandlerOutP
);

extern dsInt16_t DSMLINKAGE tsmInitEx(
    dsUInt32_t *tsmHandleP,
    tsmInitExIn_t *tsmInitExInP,
    tsmInitExOut_t *tsmInitExOutP
);

extern dsInt16_t DSMLINKAGE tsmLogEventEx(
    dsUInt32_t tsmHandle,
    tsmLogExIn_t *tsmLogExInP,
    tsmLogExOut_t *tsmLogExOutP
);

```

```

extern dsInt16_t DSMLINKAGE tsmQueryAccess(
    dsUInt32_t tsmQryRespAccessData
    dsUInt16_t *accessListP,
    *numberOfRules
);

extern void DSMLINKAGE tsmQueryApiVersionEx(
    *apiVersionP
);

extern dsInt16_t DSMLINKAGE tsmQueryCliOptions(
    tsmOptStruct *optstructP
);

extern dsInt16_t DSMLINKAGE tsmQuerySessInfo(
    dsUInt32_t tsmApiSessInfo
    *SessInfoP
);

extern dsInt16_t DSMLINKAGE tsmQuerySessOptions(
    dsUInt32_t tsmOptStruct
    *optstructP
);

extern dsInt16_t DSMLINKAGE tsmRCMsg(
    dsUInt32_t tsmHandle,
    dsInt16_t tsmRC,
    dsChar_t *msg
);

extern dsInt16_t DSMLINKAGE tsmRegisterFS(
    dsUInt32_t tsmHandle,
    tsmRegFSData *regFilespaceP
);

extern dsInt16_t DSMLINKAGE tsmReleaseBuffer(
    releaseBufferIn_t *tsmReleaseBufferInP,
    releaseBufferOut_t *tsmReleaseBufferOutP
);

extern dsInt16_t DSMLINKAGE tsmRenameObj(
    tsmRenameIn_t *tsmRenameInP,
    tsmRenameOut_t *tsmRenameOutP
);

extern dsInt16_t DSMLINKAGE tsmRequestBuffer(
    requestBufferIn_t *tsmRequestBufferInP,
    requestBufferOut_t *tsmRequestBufferOutP
);

extern dsInt16_t DSMLINKAGE tsmRetentionEvent(
    tsmRetentionEventIn_t *tsmRetentionEventInP,
    tsmRetentionEventOut_t *tsmRetentionEventOutP
);

extern dsInt16_t DSMLINKAGE tsmSendBufferData(
    sendBufferDataIn_t *tsmSendBufferDataInP,
    sendBufferDataOut_t *tsmSendBufferDataOutP
);

extern dsInt16_t DSMLINKAGE tsmSendData(
    dsUInt32_t tsmHandle,
    DataBlk *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE tsmSendObj(
    dsUInt32_t tsmHandle,
    tsmSendType sendType,
    void *sendBuff,
    tsmObjName *objNameP,
    tsmObjAttr *objAttrPtr,
    DataBlk *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE tsmSetAccess(
    dsUInt32_t tsmHandle,
    tsmAccessType accessType,
    tsmObjName *objNameP,
    dsChar_t *node,
    dsChar_t *owner
);

```



```

extern dsInt16_t DSMLINKAGE tsmSetUp(
    dsBool_t      mtFlag,
    tsmEnvSetUp   *envSetUpP
);

extern dsInt16_t DSMLINKAGE tsmTerminate(
    dsUInt32_t    tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmUpdateFS(
    dsUInt32_t    tsmHandle,
    dsChar_t      *fs,
    tsmFSUpd      *fsUpdP,
    dsUInt32_t    fsUpdAct
);

extern dsInt16_t DSMLINKAGE tsmUpdateObj(
    dsUInt32_t    tsmHandle,
    tsmSendType   sendType,
    void          *sendBuff,
    tsmObjName    *objNameP,
    tsmObjAttr    *objAttrPtr,
    dsUInt32_t    objUpdAct
);

extern dsInt16_t DSMLINKAGE tsmUpdateObjEx(
    tsmUpdateObjExIn_t    *tsmUpdateObjExInP,
    tsmUpdateObjExOut_t   *tsmUpdateObjExOutP
);

#endif /* ifdef DYNALOAD */

#ifdef __cplusplus
}
#endif

#endif /* _H_TSMAPIFP */

```



## Appendix A. API return codes source file: dsmrc.h

The dsmrc.h header file contains all return codes that the API can return to an application.

The information that is provided here contains a point-in-time copy of the dsmrc.h file that is distributed with the API. View the file in the API distribution package for the latest version.

```

/*****
 * IBM Storage Protect
 * API Client Component
 *
 * IBM Confidential
 * (IBM Confidential-Restricted when combined with the Aggregated OCO
 * source modules for this program)
 *
 * OCO Source Materials
 *
 * 5648-020 (C) Copyright IBM Corporation 1993, 2016
 *****/

/*****
/* Header File Name: dsmrc.h
/*
/*
/* Descriptive-name: Return codes from IBM Storage Protect APIs
*****/
#ifndef _H_DSMRC
#define _H_DSMRC

#ifndef DSMAPILIB

#ifndef _H_ANSMACH
typedef int RetCode ;
#endif

#endif

#define DSM_RC_SUCCESSFUL          0 /* successful completion */
#define DSM_RC_OK                  0 /* successful completion */

#define DSM_RC_UNSUCCESSFUL        -1 /* unsuccessful completion */

/* dsmEndTxn reason code */
#define DSM_RS_ABORT_SYSTEM_ERROR      1
#define DSM_RS_ABORT_NO_MATCH          2
#define DSM_RS_ABORT_BY_CLIENT         3
#define DSM_RS_ABORT_ACTIVE_NOT_FOUND  4
#define DSM_RS_ABORT_NO_DATA           5
#define DSM_RS_ABORT_BAD_VERIFIER      6
#define DSM_RS_ABORT_NODE_IN_USE       7
#define DSM_RS_ABORT_EXPDATE_TOO_LOW   8
#define DSM_RS_ABORT_DATA_OFFLINE      9
#define DSM_RS_ABORT_EXCLUDED_BY_SIZE  10
#define DSM_RS_ABORT_NO_STO_SPACE_SKIP 11
#define DSM_RS_ABORT_NO_REPOSIT_SPACE   DSM_RS_ABORT_NO_STO_SPACE_SKIP
#define DSM_RS_ABORT_MOUNT_NOT_POSSIBLE 12
#define DSM_RS_ABORT_SIZEESTIMATE_EXCEED 13
#define DSM_RS_ABORT_DATA_UNAVAILABLE  14
#define DSM_RS_ABORT_RETRY              15
#define DSM_RS_ABORT_NO_LOG_SPACE       16
#define DSM_RS_ABORT_NO_DB_SPACE        17
#define DSM_RS_ABORT_NO_MEMORY          18

#define DSM_RS_ABORT_FS_NOT_DEFINED     20
#define DSM_RS_ABORT_NODE_ALREADY_DEFED 21
#define DSM_RS_ABORT_NO_DEFAULT_DOMAIN 22
#define DSM_RS_ABORT_INVALID_NODENAME   23
#define DSM_RS_ABORT_INVALID_POL_BIND   24
#define DSM_RS_ABORT_DEST_NOT_DEFINED   25
#define DSM_RS_ABORT_WAIT_FOR_SPACE     26
#define DSM_RS_ABORT_NOT_AUTHORIZED     27
#define DSM_RS_ABORT_RULE_ALREADY_DEFED 28
#define DSM_RS_ABORT_NO_STOR_SPACE_STOP 29

#define DSM_RS_ABORT_LICENSE_VIOLATION  30

```

```

#define DSM_RS_ABORT_EXTOBJID_ALREADY_EXISTS 31
#define DSM_RS_ABORT_DUPLICATE_OBJECT 32

#define DSM_RS_ABORT_INVALID_OFFSET 33 /* Partial Object Retrieve */
#define DSM_RS_ABORT_INVALID_LENGTH 34 /* Partial Object Retrieve */
#define DSM_RS_ABORT_STRING_ERROR 35
#define DSM_RS_ABORT_NODE_NOT_AUTHORIZED 36
#define DSM_RS_ABORT_RESTART_NOT_POSSIBLE 37
#define DSM_RS_ABORT_RESTORE_IN_PROGRESS 38
#define DSM_RS_ABORT_SYNTAX_ERROR 39

#define DSM_RS_ABORT_DATA_SKIPPED 40
#define DSM_RS_ABORT_EXCEED_MAX_MP 41
#define DSM_RS_ABORT_NO_OBJSET_MATCH 42
#define DSM_RS_ABORT_PVR_ERROR 43
#define DSM_RS_ABORT_BAD_RECOGToken 44
#define DSM_RS_ABORT_MERGE_ERROR 45
#define DSM_RS_ABORT_FSRENAME_ERROR 46
#define DSM_RS_ABORT_INVALID_OPERATION 47
#define DSM_RS_ABORT_STGPOOL_UNDEFINED 48
#define DSM_RS_ABORT_INVALID_DATA_FORMAT 49
#define DSM_RS_ABORT_DATAMOVER_UNDEFINED 50

#define DSM_RS_ABORT_INVALID_MOVER_TYPE 231
#define DSM_RS_ABORT_ITEM_IN_USE 232
#define DSM_RS_ABORT_LOCK_CONFLICT 233
#define DSM_RS_ABORT_SRV_PLUGIN_COMM_ERROR 234
#define DSM_RS_ABORT_SRV_PLUGIN_OS_ERROR 235
#define DSM_RS_ABORT_CRC_FAILED 236
#define DSM_RS_ABORT_INVALID_GROUP_ACTION 237
#define DSM_RS_ABORT_DISK_UNDEFINED 238
#define DSM_RS_ABORT_BAD_DESTINATION 239
#define DSM_RS_ABORT_DATAMOVER_NOT_AVAILABLE 240
#define DSM_RS_ABORT_STGPOOL_COPY_CONT_NO 241
#define DSM_RS_ABORT_RETRY_SINGLE_TXN 242
#define DSM_RS_ABORT_TOC_CREATION_FAIL 243
#define DSM_RS_ABORT_TOC_LOAD_FAIL 244
#define DSM_RS_ABORT_PATH_RESTRICTed 245
#define DSM_RS_ABORT_NO_LANFREE_SCRATCH 246
#define DSM_RS_ABORT_INSERT_NOT_ALLOWED 247
#define DSM_RS_ABORT_DELETE_NOT_ALLOWED 248
#define DSM_RS_ABORT_TXN_LIMIT_EXCEEDED 249
#define DSM_RS_ABORT_OBJECT_ALREADY_HELD 250
#define DSM_RS_ABORT_INVALID_CHUNK_REFERENCE 254
#define DSM_RS_ABORT_DESTINATION_NOT_DEDUP 255
#define DSM_RS_ABORT_DESTINATION_POOL_CHANGED 257
#define DSM_RS_ABORT_NOT_ROOT 258

/* RETURN CODE */

#define DSM_RC_ABORT_SYSTEM_ERROR DSM_RS_ABORT_SYSTEM_ERROR
#define DSM_RC_ABORT_NO_MATCH DSM_RS_ABORT_NO_MATCH
#define DSM_RC_ABORT_BY_CLIENT DSM_RS_ABORT_BY_CLIENT
#define DSM_RC_ABORT_ACTIVE_NOT_FOUND DSM_RS_ABORT_ACTIVE_NOT_FOUND
#define DSM_RC_ABORT_NO_DATA DSM_RS_ABORT_NO_DATA
#define DSM_RC_ABORT_BAD_VERIFIER DSM_RS_ABORT_BAD_VERIFIER
#define DSM_RC_ABORT_NODE_IN_USE DSM_RS_ABORT_NODE_IN_USE
#define DSM_RC_ABORT_EXPDATE_TOO_LOW DSM_RS_ABORT_EXPDATE_TOO_LOW
#define DSM_RC_ABORT_DATA_OFFLINE DSM_RS_ABORT_DATA_OFFLINE
#define DSM_RC_ABORT_EXCLUDED_BY_SIZE DSM_RS_ABORT_EXCLUDED_BY_SIZE

#define DSM_RC_ABORT_NO_REPOSIT_SPACE DSM_RS_ABORT_NO_STO_SPACE_SKIP
#define DSM_RC_ABORT_NO_STO_SPACE_SKIP DSM_RS_ABORT_NO_STO_SPACE_SKIP

#define DSM_RC_ABORT_MOUNT_NOT_POSSIBLE DSM_RS_ABORT_MOUNT_NOT_POSSIBLE
#define DSM_RC_ABORT_SIZEESTIMATE_EXCEED DSM_RS_ABORT_SIZEESTIMATE_EXCEED
#define DSM_RC_ABORT_DATA_UNAVAILABLE DSM_RS_ABORT_DATA_UNAVAILABLE
#define DSM_RC_ABORT_RETRY DSM_RS_ABORT_RETRY
#define DSM_RC_ABORT_NO_LOG_SPACE DSM_RS_ABORT_NO_LOG_SPACE
#define DSM_RC_ABORT_NO_DB_SPACE DSM_RS_ABORT_NO_DB_SPACE
#define DSM_RC_ABORT_NO_MEMORY DSM_RS_ABORT_NO_MEMORY

#define DSM_RC_ABORT_FS_NOT_DEFINED DSM_RS_ABORT_FS_NOT_DEFINED
#define DSM_RC_ABORT_NODE_ALREADY_DEFED DSM_RS_ABORT_NODE_ALREADY_DEFED
#define DSM_RC_ABORT_NO_DEFAULT_DOMAIN DSM_RS_ABORT_NO_DEFAULT_DOMAIN
#define DSM_RC_ABORT_INVALID_NODENAME DSM_RS_ABORT_INVALID_NODENAME
#define DSM_RC_ABORT_INVALID_POL_BIND DSM_RS_ABORT_INVALID_POL_BIND
#define DSM_RC_ABORT_DEST_NOT_DEFINED DSM_RS_ABORT_DEST_NOT_DEFINED
#define DSM_RC_ABORT_WAIT_FOR_SPACE DSM_RS_ABORT_WAIT_FOR_SPACE
#define DSM_RC_ABORT_NOT_AUTHORIZED DSM_RS_ABORT_NOT_AUTHORIZED
#define DSM_RC_ABORT_RULE_ALREADY_DEFED DSM_RS_ABORT_RULE_ALREADY_DEFED

```

```

#define DSM_RC_ABORT_NO_STOR_SPACE_STOP DSM_RS_ABORT_NO_STOR_SPACE_STOP

#define DSM_RC_ABORT_LICENSE_VIOLATION DSM_RS_ABORT_LICENSE_VIOLATION
#define DSM_RC_ABORT_EXTOBJID_ALREADY_EXISTS DSM_RS_ABORT_EXTOBJID_ALREADY_EXISTS
#define DSM_RC_ABORT_DUPLICATE_OBJECT DSM_RS_ABORT_DUPLICATE_OBJECT

#define DSM_RC_ABORT_INVALID_OFFSET DSM_RS_ABORT_INVALID_OFFSET
#define DSM_RC_ABORT_INVALID_LENGTH DSM_RS_ABORT_INVALID_LENGTH

#define DSM_RC_ABORT_STRING_ERROR DSM_RS_ABORT_STRING_ERROR
#define DSM_RC_ABORT_NODE_NOT_AUTHORIZED DSM_RS_ABORT_NODE_NOT_AUTHORIZED
#define DSM_RC_ABORT_RESTART_NOT_POSSIBLE DSM_RS_ABORT_RESTART_NOT_POSSIBLE
#define DSM_RC_ABORT_RESTORE_IN_PROGRESS DSM_RS_ABORT_RESTORE_IN_PROGRESS
#define DSM_RC_ABORT_SYNTAX_ERROR DSM_RS_ABORT_SYNTAX_ERROR

#define DSM_RC_ABORT_DATA_SKIPPED DSM_RS_ABORT_DATA_SKIPPED
#define DSM_RC_ABORT_EXCEED_MAX_MP DSM_RS_ABORT_EXCEED_MAX_MP
#define DSM_RC_ABORT_NO_OBJSET_MATCH DSM_RS_ABORT_NO_OBJSET_MATCH
#define DSM_RC_ABORT_PVR_ERROR DSM_RS_ABORT_PVR_ERROR
#define DSM_RC_ABORT_BAD_RECOGTOKEN DSM_RS_ABORT_BAD_RECOGTOKEN
#define DSM_RC_ABORT_MERGE_ERROR DSM_RS_ABORT_MERGE_ERROR
#define DSM_RC_ABORT_FSRENAME_ERROR DSM_RS_ABORT_FSRENAME_ERROR
#define DSM_RC_ABORT_INVALID_OPERATION DSM_RS_ABORT_INVALID_OPERATION
#define DSM_RC_ABORT_STGPOOL_UNDEFINED DSM_RS_ABORT_STGPOOL_UNDEFINED
#define DSM_RC_ABORT_INVALID_DATA_FORMAT DSM_RS_ABORT_INVALID_DATA_FORMAT
#define DSM_RC_ABORT_DATAMOVER_UNDEFINED DSM_RS_ABORT_DATAMOVER_UNDEFINED

#define DSM_RC_ABORT_INVALID_MOVER_TYPE DSM_RS_ABORT_INVALID_MOVER_TYPE
#define DSM_RC_ABORT_ITEM_IN_USE DSM_RS_ABORT_ITEM_IN_USE
#define DSM_RC_ABORT_LOCK_CONFLICT DSM_RS_ABORT_LOCK_CONFLICT
#define DSM_RC_ABORT_SRV_PLUGIN_COMM_ERROR DSM_RS_ABORT_SRV_PLUGIN_COMM_ERROR
#define DSM_RC_ABORT_SRV_PLUGIN_OS_ERROR DSM_RS_ABORT_SRV_PLUGIN_OS_ERROR
#define DSM_RC_ABORT_CRC_FAILED DSM_RS_ABORT_CRC_FAILED
#define DSM_RC_ABORT_INVALID_GROUP_ACTION DSM_RS_ABORT_INVALID_GROUP_ACTION
#define DSM_RC_ABORT_DISK_UNDEFINED DSM_RS_ABORT_DISK_UNDEFINED
#define DSM_RC_ABORT_BAD_DESTINATION DSM_RS_ABORT_BAD_DESTINATION
#define DSM_RC_ABORT_DATAMOVER_NOT_AVAILABLE DSM_RS_ABORT_DATAMOVER_NOT_AVAILABLE
#define DSM_RC_ABORT_STGPOOL_COPY_CONT_NO DSM_RS_ABORT_STGPOOL_COPY_CONT_NO
#define DSM_RC_ABORT_RETRY_SINGLE_TXN DSM_RS_ABORT_RETRY_SINGLE_TXN
#define DSM_RC_ABORT_TOC_CREATION_FAIL DSM_RS_ABORT_TOC_CREATION_FAIL
#define DSM_RC_ABORT_TOC_LOAD_FAIL DSM_RS_ABORT_TOC_LOAD_FAIL
#define DSM_RC_ABORT_PATH_RESTRICTED DSM_RS_ABORT_PATH_RESTRICTED
#define DSM_RC_ABORT_NO_LANFREE_SCRATCH DSM_RS_ABORT_NO_LANFREE_SCRATCH
#define DSM_RC_ABORT_INSERT_NOT_ALLOWED DSM_RS_ABORT_INSERT_NOT_ALLOWED
#define DSM_RC_ABORT_DELETE_NOT_ALLOWED DSM_RS_ABORT_DELETE_NOT_ALLOWED
#define DSM_RC_ABORT_TXN_LIMIT_EXCEEDED DSM_RS_ABORT_TXN_LIMIT_EXCEEDED
#define DSM_RC_ABORT_OBJECT_ALREADY_HELD DSM_RS_ABORT_OBJECT_ALREADY_HELD
#define DSM_RC_ABORT_INVALID_CHUNK_REFERENCE DSM_RS_ABORT_INVALID_CHUNK_REFERENCE
#define DSM_RC_ABORT_DESTINATION_NOT_DEDUP DSM_RS_ABORT_DESTINATION_NOT_DEDUP
#define DSM_RC_ABORT_DESTINATION_POOL_CHANGED DSM_RS_ABORT_DESTINATION_POOL_CHANGED
#define DSM_RC_ABORT_NOT_ROOT DSM_RS_ABORT_NOT_ROOT

#define DSM_RC_ABORT_CERTIFICATE_NOT_FOUND DSM_RS_ABORT_CERTIFICATE_NOT_FOUND

/* Definitions for server signon reject codes */
/* These error codes are in the range (51 to 99) inclusive. */
#define DSM_RC_REJECT_NO_RESOURCES 51
#define DSM_RC_REJECT_VERIFIER_EXPIRED 52
#define DSM_RC_REJECT_ID_UNKNOWN 53
#define DSM_RC_REJECT_DUPLICATE_ID 54
#define DSM_RC_REJECT_SERVER_DISABLED 55
#define DSM_RC_REJECT_CLOSED_REGISTER 56
#define DSM_RC_REJECT_CLIENT_DOWNLEVEL 57
#define DSM_RC_REJECT_SERVER_DOWNLEVEL 58
#define DSM_RC_REJECT_ID_IN_USE 59
#define DSM_RC_REJECT_ID_LOCKED 61
#define DSM_RC_SIGNONREJECT_LICENSE_MAX 62
#define DSM_RC_REJECT_NO_MEMORY 63
#define DSM_RC_REJECT_NO_DB_SPACE 64
#define DSM_RC_REJECT_NO_LOG_SPACE 65
#define DSM_RC_REJECT_INTERNAL_ERROR 66
#define DSM_RC_SIGNONREJECT_INVALID_CLI 67 /* client type not licensed */
#define DSM_RC_CLIENT_NOT_ARCHRETPROT 68
#define DSM_RC_REJECT_LASTSESS_CANCELED 69
#define DSM_RC_REJECT_UNICODE_NOT_ALLOWED 70
#define DSM_RC_REJECT_NOT_AUTHORIZED 71
#define DSM_RC_REJECT_TOKEN_TIMEOUT 72
#define DSM_RC_REJECT_INVALID_NODE_TYPE 73
#define DSM_RC_REJECT_INVALID_SESSIONINIT 74
#define DSM_RC_REJECT_WRONG_PORT 75
#define DSM_RC_CLIENT_NOT_SPMRETPROT 79

```

```

#define DSM_RC_USER_ABORT 101 /* processing aborted by user */
#define DSM_RC_NO_MEMORY 102 /* no RAM left to complete request */
#define DSM_RC_TA_COMM_DOWN 2021 /* no longer used */
#define DSM_RC_FILE_NOT_FOUND 104 /* specified file not found */
#define DSM_RC_PATH_NOT_FOUND 105 /* specified path doesn't exist */
#define DSM_RC_ACCESS_DENIED 106 /* denied due to improper permission */
#define DSM_RC_NO_HANDLES 107 /* no more file handles available */
#define DSM_RC_FILE_EXISTS 108 /* file already exists */
#define DSM_RC_INVALID_PARM 109 /* invalid parameter passed. CRITICAL */
#define DSM_RC_INVALID_HANDLE 110 /* invalid file handle passed */
#define DSM_RC_DISK_FULL 111 /* out of disk space */
#define DSM_RC_PROTOCOL_VIOLATION 113 /* call protocol violation. CRITICAL */
#define DSM_RC_UNKNOWN_ERROR 114 /* unknown system error. CRITICAL */
#define DSM_RC_UNEXPECTED_ERROR 115 /* unexpected error. CRITICAL */
#define DSM_RC_FILE_BEING_EXECUTED 116 /* No write is allowed */
#define DSM_RC_DIR_NO_SPACE 117 /* directory can't be expanded */
#define DSM_RC_LOOPED_SYM_LINK 118 /* too many symbolic links were
encountered in translating path. */
#define DSM_RC_FILE_NAME_TOO_LONG 119 /* file name too long */
#define DSM_RC_FILE_SPACE_LOCKED 120 /* filespace is locked by the system */
#define DSM_RC_FINISHED 121 /* finished processing */
#define DSM_RC_UNKNOWN_FORMAT 122 /* unknown format */
#define DSM_RC_NO_AUTHORIZATION 123 /* server response when the client has
no authorization to read another
host's owner backup/archive data */
#define DSM_RC_FILE_SPACE_NOT_FOUND 124 /* specified file space not found */
#define DSM_RC_TXN_ABORTED 125 /* transaction aborted */
#define DSM_RC_SUBDIR_AS_FILE 126 /* Subdirectory name exists as file */
#define DSM_RC_PROCESS_NO_SPACE 127 /* process has no more disk space. */
#define DSM_RC_PATH_TOO_LONG 128 /* a directory path being build became
too long */
#define DSM_RC_NOT_COMPRESSED 129 /* file thought to be compressed is
actually not */
#define DSM_RC_TOO_MANY_BITS 130 /* file was compressed using more bits
then the expander can handle */
#define DSM_RC_SYSTEM_ERROR 131 /* internal system error */
#define DSM_RC_NO_SERVER_RESOURCES 132 /* server out of resources. */
#define DSM_RC_FS_NOT_KNOWN 133 /* the file space is not known by the
server */
#define DSM_RC_NO_LEADING_DIRSEP 134 /* no leading directory separator */
#define DSM_RC_WILDCARD_DIR 135 /* wildcard character in directory
path when not allowed */
#define DSM_RC_COMM_PROTOCOL_ERROR 136 /* communications protocol error */
#define DSM_RC_AUTH_FAILURE 137 /* authentication failure */
#define DSM_RC_TA_NOT_VALID 138 /* TA not a root and/or SUID program */
#define DSM_RC_KILLED 139 /* process killed. */

#define DSM_RC_RETRY 143 /* retry same operation again */

#define DSM_RC_WOULD_BLOCK 145 /* operation would cause the system to
block waiting for input. */
#define DSM_RC_TOO_SMALL 146 /* area for compiled pattern small */
#define DSM_RC_UNCLOSED 147 /* no closing bracket in pattern */
#define DSM_RC_NO_STARTING_DELIMITER 148 /* pattern has to start with
directory delimiter */
#define DSM_RC_NEEDED_DIR_DELIMITER 149 /* a directory delimiter is needed
immediately before and after the
"match directories" metastring
("...") and one wasn't found */
#define DSM_RC_UNKNOWN_FILE_DATA_TYPE 150 /* structured file data type is
unknown */
#define DSM_RC_BUFFER_OVERFLOW 151 /* data buffer overflow */

#define DSM_RC_NO_COMPRESS_MEMORY 154 /* Compress/Expand out of memory */
#define DSM_RC_COMPRESS_GREW 155 /* Compression grew */
#define DSM_RC_INV_COMM_METHOD 156 /* Invalid comm method specified */
#define DSM_RC_WILL_ABORT 157 /* Transaction will be aborted */
#define DSM_RC_FS_WRITE_LOCKED 158 /* File space is write locked */
#define DSM_RC_SKIPPED_BY_USER 159 /* User wanted file skipped in the
case of ABORT_DATA_OFFLINE */
#define DSM_RC_TA_NOT_FOUND 160 /* TA not found in it's directory */
#define DSM_RC_TA_ACCESS_DENIED 161 /* Access to TA is denied */
#define DSM_RC_FS_NOT_READY 162 /* File space not ready */
#define DSM_RC_FS_IS_BAD 163 /* File space is bad */
#define DSM_RC_FIO_ERROR 164 /* File input/output error */
#define DSM_RC_WRITE_FAILURE 165 /* Error writing to file */
#define DSM_RC_OVER_FILE_SIZE_LIMIT 166 /* File over system/user limit */
#define DSM_RC_CANNOT_MAKE 167 /* Could not create file/directory,
could be a bad name */

```

```

#define DSM_RC_NO_PASS_FILE          168 /* password file needed and user is
                                         not root */
#define DSM_RC_VERFILE_OLD           169 /* password stored locally doesn't
                                         match the one at the host */
#define DSM_RC_INPUT_ERROR           173 /* unable to read keyboard input */
#define DSM_RC_REJECT_PLATFORM_MISMATCH 174 /* Platform name doesn't match
                                         up with what the server says
                                         is the platform for the client */
#define DSM_RC_TL_NOT_FILE_OWNER     175 /* User trying to backup a file is not
                                         the file's owner. */
#define DSM_RC_COMPRESSED_DATA_CORRUPTED 176 /* Compressed data is corrupted */
#define DSM_RC_UNMATCHED_QUOTE       177 /* missing starting or ending quote */

#define DSM_RC_SIGNON_FAILOVER_MODE 178 /* Failed over to the replication server,
                                         running in failover mode */
#define DSM_RC_FAILOVER_MODE_FUNC_BLOCKED 179 /* function is blocked because
                                         session is in failover mode */

/*-----*/
/* Return codes 180-199 are reserved for Policy Set handling */
/*-----*/
#define DSM_RC_PS_MULTBCG           181 /* Multiple backup copy groups in 1 MC*/
#define DSM_RC_PS_MULTACG           182 /* Multiple arch. copy groups in 1 MC*/
#define DSM_RC_PS_NODFLTMC          183 /* Default MC name not in policy set */
#define DSM_RC_TL_NOBCG             184 /* Backup req, no backup copy group */
#define DSM_RC_TL_EXCLUDED          185 /* Backup req, excl. by in/ex filter */
#define DSM_RC_TL_NOACG             186 /* Archive req, no archive copy group */
#define DSM_RC_PS_INVALID_ARCHMC    187 /* Invalid MC name in archive override*/
#define DSM_RC_NO_PS_DATA           188 /* No policy set data on the server */
#define DSM_RC_PS_INVALID_DIRMC     189 /* Invalid directory MC specified in
                                         the options file. */
#define DSM_RC_PS_NO_CG_IN_DIR_MC   190 /* No backup copy group in directory MC.
                                         Must specify an MC using DirMC
                                         option. */

#define DSM_RC_WIN32_UNSUPPORTED_FILE_TYPE 280 /* File is not of
                                         Win32 type FILE_TYPE_DISK */

/*-----*/
/* Return codes for the Trusted Communication Agent */
/*-----*/
#define DSM_RC_TCA_NOT_ROOT         161 /* Access to TA is denied */
#define DSM_RC_TCA_ATTACH_SHR_MEM_ERR 200 /* Error attaching shared memory */
#define DSM_RC_TCA_SHR_MEM_BLOCK_ERR 200 /* Shared memory block error */
#define DSM_RC_TCA_SHR_MEM_IN_USE    200 /* Shared memory block error */
#define DSM_RC_TCA_SHARED_MEMORY_ERROR 200 /* Shared memory block error */
#define DSM_RC_TCA_SEGMENT_MISMATCH 200 /* Shared memory block error */
#define DSM_RC_TCA_FORK_FAILED       292 /* Error forking off TCA process */
#define DSM_RC_TCA_DIED              294 /* TCA died unexpectedly */
#define DSM_RC_TCA_INVALID_REQUEST   295 /* Invalid request sent to TCA */
#define DSM_RC_TCA_SEMGET_ERROR       297 /* Error getting semaphores */
#define DSM_RC_TCA_SEM_OP_ERROR       298 /* Error in semaphore set or wait */
#define DSM_RC_TCA_NOT_ALLOWED       299 /* TCA not allowed (multi thread) */

/*-----*/
/* 400-430 for options */
/*-----*/
#define DSM_RC_INVALID_OPT           400 /* invalid option */
#define DSM_RC_NO_HOST_ADDR          405 /* Not enuf info to connect server */
#define DSM_RC_NO_OPT_FILE           406 /* No default user configuration file*/
#define DSM_RC_MACHINE_SAME          408 /* -MACHINENAME same as real name */
#define DSM_RC_INVALID_SERVER        409 /* Invalid server name from client */
#define DSM_RC_INVALID_KEYWORD       410 /* Invalid option keyword */
#define DSM_RC_PATTERN_TOO_COMPLEX   411 /* Can't match Include/Exclude entry*/
#define DSM_RC_NO_CLOSING_BRACKET     412 /* Missing closing bracket inc/excl */
#define DSM_RC_OPT_CLIENT_NOT_ACCEPTING 417/* Client doesn't accept this option
                                         from the server */
#define DSM_RC_OPT_CLIENT_DOES_NOT_WANT 418/* Client doesn't want this value
                                         from the server */
#define DSM_RC_OPT_NO_INCLEXCL_FILE  419 /* inclexcl file not found */
#define DSM_RC_OPT_OPEN_FAILURE       420 /* can't open file */
#define DSM_RC_OPT_INV_NODENAME       421/* used for Windows if nodename=local
                                         machine when CLUSTERNODE=YES */
#define DSM_RC_OPT_NODENAME_INVALID   423/* generic invalid nodename */
#define DSM_RC_OPT_ERRORLOG_CONFLICT  424/* both logmax & retention specified */
#define DSM_RC_OPT_SCHEDLOG_CONFLICT  425/* both logmax & retention specified */
#define DSM_RC_CANNOT_OPEN_TRACEFILE 426/* cannot open trace file */
#define DSM_RC_CANNOT_OPEN_LOGFILE    427/* cannot open error log file */
#define DSM_RC_OPT_SESSINIT_LF_CONFLICT 428/* both sessioninit=server and
                                         enablelanfree=yes are specified*/
#define DSM_RC_OPT_OPTION_IGNORE      429/* option will be ignored */
#define DSM_RC_OPT_DEDUP_CONFLICT     430/* cannot open error log file */

```

```

#define DSM_RC_OPT_HSMLOG_CONFLICT 431 /* both logmax & retention specified */

/*-----*/
/* 600 to 610 for volume label codes */
/*-----*/
#define DSM_RC_DUP_LABEL 600 /* duplicate volume label found */
#define DSM_RC_NO_LABEL 601 /* drive has no label */

/*-----*/
/* Return codes for message file processing */
/*-----*/
#define DSM_RC_NLS_CANT_OPEN_TXT 610 /* error trying to open msg txt file */
#define DSM_RC_NLS_CANT_READ_HDR 611 /* error trying to read header */
#define DSM_RC_NLS_INVALID_CNTL_REC 612 /* invalid control record */
#define DSM_RC_NLS_INVALID_DATE_FMT 613 /* invalid default date format */
#define DSM_RC_NLS_INVALID_TIME_FMT 614 /* invalid default time format */
#define DSM_RC_NLS_INVALID_NUM_FMT 615 /* invalid default number format */

/*-----*/
/* Return codes 620-630 are reserved for log message return codes */
/*-----*/
#define DSM_RC_LOG_CANT_BE_OPENED 620 /* error trying to open error log */
#define DSM_RC_LOG_ERROR_WRITING_TO_LOG 621 /* error occurred writing to
log file */
#define DSM_RC_LOG_NOT_SPECIFIED 622 /* no error log file was specified */

/*-----*/
/* Return codes 900-999 IBM STORAGE PROTECT CLIENT ONLY */
/*-----*/
#define DSM_RC_NOT_ADSM_AUTHORIZED 927 /* Must be ADSM authorized to perform */
/* action : root user or pwd auth */
#define DSM_RC_REJECT_USERID_UNKNOWN 940 /* userid unknown on server */
#define DSM_RC_FILE_IS_SYMLINK 959 /* errorlog or trace is a symbolic
link */
*/

#define DSM_RC_DIRECT_STORAGE_AGENT_UNSUPPORTED 961 /* Direct connection to SA not supported */
#define DSM_RC_FS_NAMESPACE_DOWNLEVEL 963 /* Long namespace has been removed from
from the Netware volume */
#define DSM_RC_CONTINUE_NEW_CONSUMER 972 /* Continue processing using a new consumer */
#define DSM_RC_CONTINUE_NEW_CONSUMER_NODUP 973 /* Continue processing using a new consumer no dedup */
#define DSM_RC_CONTINUE_NEW_CONSUMER_NOCOMPRESS 976 /* Continue processing using a new consumer no
compression */

#define DSM_RC_SERVER_SUPPORTS_FUNC 994 /* the server supports this function */
#define DSM_RC_SERVER_AND_SA_SUPPORT_FUNC 995 /* Both server and SA support func */
#define DSM_RC_SERVER_DOWNLEVEL_FUNC 996 /* The server is downlevel for func */
#define DSM_RC_STORAGEAGENT_DOWNLEVEL 997 /* the storage agent is downlevel */
#define DSM_RC_SERVER_AND_SA_DOWNLEVEL 998 /* both server and SA downlevel */

/* TCP/IP error codes */
#define DSM_RC_TCPIP_FAILURE -50 /* TCP/IP communications failure */
#define DSM_RC_CONN_TIMEDOUT -51 /* TCP/IP connection attempt timedout */
#define DSM_RC_CONN_REFUSED -52 /* TCP/IP connection refused by host */
#define DSM_RC_BAD_HOST_NAME -53 /* TCP/IP invalid host name specified */
#define DSM_RC_NETWORK_UNREACHABLE -54 /* TCP/IP host name unreachable */
#define DSM_RC_WINSOCK_MISSING -55 /* TCP/IP WINSOCK.DLL missing */
#define DSM_RC_TCPIP_DLL_LOADFAILURE -56 /* Error from LoadLibrary */
#define DSM_RC_TCPIP_LOADFAILURE -57 /* Error from GetProcAddress */
#define DSM_RC_TCPIP_USER_ABORT -58 /* User aborted while in TCP/IP layer */

/*-----*/
/* Return codes (-71)-(-90) are reserved for CommTSM error codes */
/*-----*/
#define DSM_RC_TSM_FAILURE -71 /* IBM Storage Protect comm failure */
#define DSM_RC_TSM_ABORT -72 /* Session aborted abnormally */

/*comm3270 error codes - no longer used*/
#define DSM_RC_COMM_TIMEOUT 2021 /* no longer used */
#define DSM_RC_EMULATOR_INACTIVE 2021 /* no longer used */
#define DSM_RC_BAD_HOST_ID 2021 /* no longer used */
#define DSM_RC_HOST_SESS_BUSY 2021 /* no longer used */
#define DSM_RC_3270_CONNECT_FAILURE 2021 /* no longer used */
#define DSM_RC_NO_ACS3ELKE_DLL 2021 /* no longer used */
#define DSM_RC_EMULATOR_ERROR 2021 /* no longer used */
#define DSM_RC_EMULATOR_BACKLEVEL 2021 /* no longer used */
#define DSM_RC_CKSUM_FAILURE 2021 /* no longer used */

```



```

/* The following Return codes are for EHLLAPI for Windows */
#define DSM_RC_3270COMMErrors_DLL 2021 /* no longer used */
#define DSM_RC_3270COMMErrors_GetProc 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_DLL 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_GetProc 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_HostConnect 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_AllocBuff 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_SendKey 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_PacketChk 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_ChkSum 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_HostTimeout 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_Send 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_Recv 2021 /* no longer used */
#define DSM_RC_EHLLAPIErrors_General 2021 /* no longer used */
#define DSM_RC_PC3270_MISSING_DLL 2021 /* no longer used */
#define DSM_RC_3270COMM_MISSING_DLL 2021 /* no longer used */

/* NETBIOS error codes */
#define DSM_RC_NETB_ERROR -151 /* Could not add node to LAN */
#define DSM_RC_NETB_NO_DLL -152 /* The ACSNETB.DLL could not be loaded */
#define DSM_RC_NETB_LAN_ERR -155 /* LAN error detected */
#define DSM_RC_NETB_NAME_ERR -158 /* Netbios error on Add Name */
#define DSM_RC_NETB_TIMEOUT -159 /* Netbios send timeout */
#define DSM_RC_NETB_NOTINST -160 /* Netbios not installed - DOS */
#define DSM_RC_NETB_REBOOT -161 /* Netbios config err - reboot DOS */

/* Named Pipe error codes */
#define DSM_RC_NP_ERROR -190

/* CPIC error codes */
#define DSM_RC_CPIC_ALLOCATE_FAILURE 2021 /* no longer used */
#define DSM_RC_CPIC_TYPE_MISMATCH 2021 /* no longer used */
#define DSM_RC_CPIC_PIPE_NOT_SPECIFY_ERR 2021 /* no longer used */
#define DSM_RC_CPIC_SECURITY_NOT_VALID 2021 /* no longer used */
#define DSM_RC_CPIC_SYNC_LVL_NO_SUPPORT 2021 /* no longer used */
#define DSM_RC_CPIC_TPN_NOT_RECOGNIZED 2021 /* no longer used */
#define DSM_RC_CPIC_TP_ERROR 2021 /* no longer used */
#define DSM_RC_CPIC_PARAMETER_ERROR 2021 /* no longer used */
#define DSM_RC_CPIC_PROD_SPECIFIC_ERR 2021 /* no longer used */
#define DSM_RC_CPIC_PROGRAM_ERROR 2021 /* no longer used */
#define DSM_RC_CPIC_RESOURCE_ERROR 2021 /* no longer used */
#define DSM_RC_CPIC_DEALLOCATE_ERROR 2021 /* no longer used */
#define DSM_RC_CPIC_SVC_ERROR 2021 /* no longer used */
#define DSM_RC_CPIC_PROGRAM_STATE_CHECK 2021 /* no longer used */
#define DSM_RC_CPIC_PROGRAM_PARAM_CHECK 2021 /* no longer used */
#define DSM_RC_CPIC_UNSUCCESSFUL 2021 /* no longer used */
#define DSM_RC_UNKNOWN_CPIC_PROBLEM 2021 /* no longer used */
#define DSM_RC_CPIC_MISSING_LU 2021 /* no longer used */
#define DSM_RC_CPIC_MISSING_TP 2021 /* no longer used */
#define DSM_RC_CPIC_SNA6000_LOAD_FAIL 2021 /* no longer used */
#define DSM_RC_CPIC_STARTUP_FAILURE 2021 /* no longer used */

/*-----*/
/* Return codes -300 to -307 are reserved for IPX/SPX communications */
/*-----*/
#define DSM_RC_TLI_ERROR 2021 /* no longer used */
#define DSM_RC_IPXSPX_FAILURE 2021 /* no longer used */
#define DSM_RC_TLI_DLL_MISSING 2021 /* no longer used */
#define DSM_RC_DLL_LOADFAILURE 2021 /* no longer used */
#define DSM_RC_DLL_FUNCTION_LOADFAILURE 2021 /* no longer used */
#define DSM_RC_IPXCONN_REFUSED 2021 /* no longer used */
#define DSM_RC_IPXCONN_TIMEOUT 2021 /* no longer used */
#define DSM_RC_IPXADDR_UNREACHABLE 2021 /* no longer used */
#define DSM_RC_CPIC_MISSING_DLL 2021 /* no longer used */
#define DSM_RC_CPIC_DLL_LOADFAILURE 2021 /* no longer used */
#define DSM_RC_CPIC_FUNC_LOADFAILURE 2021 /* no longer used */

/*=== Shared Memory Protocol error codes ===*/
#define DSM_RC_SHM_TCPIP_FAILURE -450
#define DSM_RC_SHM_FAILURE -451
#define DSM_RC_SHM_NOTAUTH -452

#define DSM_RC_NULL_OBJNAME 2000 /* Object name pointer is NULL */
#define DSM_RC_NULL_DATA_BLKPTR 2001 /* dataBlkPtr is NULL */
#define DSM_RC_NULL_MSG 2002 /* msg parm in dsmRCMsg is NULL */

#define DSM_RC_NULL_OBJATTRPTR 2004 /* Object Attr Pointer is NULL */

#define DSM_RC_NO_SESS_BLK 2006 /* no server session info */
#define DSM_RC_NO_POLICY_BLK 2007 /* no policy hdr info */

```

```

#define DSM_RC_ZERO_BUFLLEN      2008 /* bufferLen is zero for dataBlkPtr */
#define DSM_RC_NULL_BUFPTR      2009 /* bufferPtr is NULL for dataBlkPtr */

#define DSM_RC_INVALID_OBJTYPE   2010 /* invalid object type */
#define DSM_RC_INVALID_VOTE     2011 /* invalid vote */
#define DSM_RC_INVALID_ACTION   2012 /* invalid action */
#define DSM_RC_INVALID_DS_HANDLE 2014 /* invalid ADSM handle */
#define DSM_RC_INVALID_REPOS    2015 /* invalid value for repository */
#define DSM_RC_INVALID_FSNAME   2016 /* fs should start with dir delim */
#define DSM_RC_INVALID_OBJNAME  2017 /* invalid full path name */
#define DSM_RC_INVALID_LLNAME   2018 /* ll should start with dir delim */
#define DSM_RC_INVALID_OBJOWNER 2019 /* invalid object owner name */
#define DSM_RC_INVALID_ACTYPE   2020 /* invalid action type */
#define DSM_RC_INVALID_RETCODE  2021 /* dsmRC in dsmRCMsg is invalid */
#define DSM_RC_INVALID_SENDTYPE 2022 /* invalid send type */
#define DSM_RC_INVALID_PARAMETER 2023 /* invalid parameter */
#define DSM_RC_INVALID_OBJSTATE 2024 /* active, inactive, or any match? */
#define DSM_RC_INVALID_MCNAME   2025 /* Mgmt class name not found */
#define DSM_RC_INVALID_DRIVE_CHAR 2026 /* Drive letter is not alphabet */
#define DSM_RC_NULL_FSNAME     2027 /* Filespace name is NULL */
#define DSM_RC_INVALID_HLNAME   2028 /* hl should start with dir delim */

#define DSM_RC_NUMOBJ_EXCEED    2029 /* BeginGetData num objs exceeded */

#define DSM_RC_NEWPW_REQD      2030 /* new password is required */
#define DSM_RC_OLDPW_REQD     2031 /* old password is required */
#define DSM_RC_NO_OWNER_REQD   2032 /* owner not allowed. Allow default */
#define DSM_RC_NO_NODE_REQD    2033 /* node not allowed w/ pw=generate */
#define DSM_RC_KEY_MISSING     2034 /* key file can't be found */
#define DSM_RC_KEY_BAD        2035 /* content of key file is bad */

#define DSM_RC_BAD_CALL_SEQUENCE 2041 /* Sequence of DSM calls not allowed */
#define DSM_RC_INVALID_TSMBUFFER 2042 /* invalid value for tsmbuffhandle or dataPtr */
#define DSM_RC_TOO_MANY_BYTES  2043 /* too many bytes copied to buffer */
#define DSM_RC_MUST_RELEASE_BUFFER 2044 /* cant exit app needs to release buffers */
#define DSM_RC_BUFF_ARRAY_ERROR 2045 /* internal buff array error */
#define DSM_RC_INVALID_DATA_BLK 2046 /* using tsmbuff datablk should be null */
#define DSM_RC_ENCR_NOT_ALLOWED 2047 /* when using tsmbuffers encryption not allowed */
#define DSM_RC_OBJ_COMPRESSED   2048 /* Can't restore using tsmBuff on compressed object */
#define DSM_RC_OBJ_ENCRYPTED     2049 /* Cant restore using tsmbuff an encr obj */
#define DSM_RC_WILDCHAR_NOTALLOWED 2050 /* Wild card not allowed for hl,ll */
#define DSM_RC_POR_NOT_ALLOWED  2051 /* Can't use partial object restore with tsmBuffers */
#define DSM_RC_NO_ENCRYPTION_KEY 2052 /* Encryption key not found */
#define DSM_RC_ENCR_CONFLICT    2053 /* mutually exclusive options */

#define DSM_RC_FSNAME_NOTFOUND  2060 /* Filespace name not found */
#define DSM_RC_FS_NOT_REGISTERED 2061 /* Filespace name not registered */
#define DSM_RC_FS_ALREADY_REGED  2062 /* Filespace already registered */
#define DSM_RC_OBJID_NOTFOUND    2063 /* No object id to restore */
#define DSM_RC_WRONG_VERSION     2064 /* Wrong level of code */
#define DSM_RC_WRONG_VERSION_PARM 2065 /* Wrong level of parameter struct */

#define DSM_RC_NEEDTO_ENDTXN    2070 /* Need to call dsmEndTxn */

#define DSM_RC_OBJ_EXCLUDED     2080 /* Object is excluded by MC */
#define DSM_RC_OBJ_NOBCG       2081 /* Object has no backup copy group */
#define DSM_RC_OBJ_NOACG       2082 /* Object has no archive copy group */

#define DSM_RC_APISYSTEM_ERROR  2090 /* API internal error */

#define DSM_RC_DESC_TOOLONG     2100 /* description is too long */
#define DSM_RC_OBJINFO_TOOLONG  2101 /* object attr objinfo too long */
#define DSM_RC_HL_TOOLONG       2102 /* High level qualifier is too long */
#define DSM_RC_PASSWD_TOOLONG   2103 /* password is too long */
#define DSM_RC_FILESPACE_TOOLONG 2104 /* filespace name is too long */
#define DSM_RC_LL_TOOLONG       2105 /* Low level qualifier is too long */
#define DSM_RC_FSINFO_TOOLONG   2106 /* filespace length is too big */
#define DSM_RC_SENDDATA_WITH_ZERO_SIZE 2107 /* send data w/ zero est */

/*==== new return codes for dsmaaccess ====*/
#define DSM_RC_INVALID_ACCESS_TYPE 2110 /* invalid access type */
#define DSM_RC_QUERY_COMM_FAILURE 2111 /* communication error during query */
#define DSM_RC_NO_FILES_BACKUP    2112 /* No backed up files for this fs */
#define DSM_RC_NO_FILES_ARCHIVE   2113 /* No archived files for this fs */
#define DSM_RC_INVALID_SETACCESS  2114 /* invalid set access format */

/*==== new return codes for dsmaaccess ====*/
#define DSM_RC_STRING_TOO_LONG    2120 /* String parameter too long */

#define DSM_RC_MORE_DATA          2200 /* There are more data to restore */

#define DSM_RC_BUFF_TOO_SMALL     2210 /* DataBlk buffer too small for qry */

```

```

#define DSM_RC_NO_API_CONFIGFILE 2228 /*specified API config file not found*/
#define DSM_RC_NO_INCLEXCL_FILE 2229 /* specified inclexcl file not found*/
#define DSM_RC_NO_SYS_OR_INCLEXCL 2230 /* either dsm.sys or inclexcl file
specified in dsm.sys not found */
#define DSM_RC_REJECT_NO_POR_SUPPORT 2231 /* server doesn't have POR support*/

#define DSM_RC_NEED_ROOT 2300 /* API caller must be root */
#define DSM_RC_NEEDTO_CALL_BINDMC 2301 /* dsmBindMC must be called first */
#define DSM_RC_CHECK_REASON_CODE 2302 /* check reason code from dsmEndTxn */
#define DSM_RC_NEEDTO_ENDTXN_DEDUP_SIZE_EXCEEDED 2303 /* max dedup bytes exceeded */

/*=== return codes 2400 - 2410 used by lic file see agentrc.h ===*/

/*=== return codes 2410 - 2430 used by Oracle agent see agentrc.h ===*/

#define DSM_RC_ENC_WRONG_KEY 4580 /* the key provided is incorrect */
#define DSM_RC_ENC_NOT_AUTHORIZED 4582 /* user is not allowed to decrypt */
#define DSM_RC_ENC_TYPE_UNKNOWN 4584 /* encryption type unknown */

/*=====
Return codes (4600)-(4624) are reserved for clustering
=====*/
#define DSM_RC_CLUSTER_INFO_LIBRARY_NOT_LOADED 4600
#define DSM_RC_CLUSTER_LIBRARY_INVALID 4601
#define DSM_RC_CLUSTER_LIBRARY_NOT_LOADED 4602
#define DSM_RC_CLUSTER_NOT_MEMBER_OF_CLUSTER 4603
#define DSM_RC_CLUSTER_NOT_ENABLED 4604
#define DSM_RC_CLUSTER_NOT_SUPPORTED 4605
#define DSM_RC_CLUSTER_UNKNOWN_ERROR 4606

/*=====
Return codes (5200)-(5600) are reserved for new Server ABORT codes (dsmcomm.h)
=====*/
#define DSM_RS_ABORT_CERTIFICATE_NOT_FOUND 5200

/*=====
Return codes (5701)-(5749) are reserved for proxy
=====*/
#define DSM_RC_PROXY_REJECT_NO_RESOURCES 5702
#define DSM_RC_PROXY_REJECT_DUPLICATE_ID 5705
#define DSM_RC_PROXY_REJECT_ID_IN_USE 5710
#define DSM_RC_PROXY_REJECT_INTERNAL_ERROR 5717
#define DSM_RC_PROXY_REJECT_NOT_AUTHORIZED 5722
#define DSM_RC_PROXY_INVALID_FROMNODE 5746
#define DSM_RC_PROXY_INVALID_SERVERFREE 5747
#define DSM_RC_PROXY_INVALID_CLUSTER 5748
#define DSM_RC_PROXY_INVALID_FUNCTION 5749

/*=====
Return codes 5801 - 5849 are reserved for cryptography/security
=====*/

#define DSM_RC_CRYPT0_ICC_ERROR 5801
#define DSM_RC_CRYPT0_ICC_CANNOT_LOAD 5802
#define DSM_RC_SSL_NOT_SUPPORTED 5803
#define DSM_RC_SSL_INIT_FAILED 5804
#define DSM_RC_SSL_KEYFILE_OPEN_FAILED 5805
#define DSM_RC_SSL_KEYFILE_BAD_PASSWORD 5806
#define DSM_RC_SSL_BAD_CERTIFICATE 5807

/*=====
Return codes 6300 - 6399 are reserved for client-side deduplication
=====*/
#define DSM_RC_DIGEST_VALIDATION_ERROR 6300 /* End-to-end digest validation err */
#define DSM_RC_DATA_FINGERPRINT_ERROR 6301 /* Failure in Rabin fingerprinting */
#define DSM_RC_DATA_DEDUP_ERROR 6302 /* Error converting data into chunks */

#endif /* _H_DSMRC */

```

## Related information

### [API return codes](#)



## Appendix B. API type definitions source files

This appendix contains structure definitions, type definitions, and constants for the API.

- The first header files, `dsmapi.h` and `tsmapi.h`, illustrate the definitions that are common to all operating systems.
- The second header file, `dsmapi.h`, provides an example of definitions that are specific to a particular operating system; in this example, the Windows platform.
- The third header file, `release.h`, includes the version and release information.

The information that is provided here contains a point-in-time copy of the files that are distributed with the API. View the files in the API distribution package for the latest version.

### `dsmapi.h`

```
/*
 * IBM Storage Protect
 * API Client Component
 *
 * IBM Confidential
 * (IBM Confidential-Restricted when combined with the Aggregated OCO
 * source modules for this program)
 *
 * OCO Source Materials
 *
 * 5648-020 (C) Copyright IBM Corporation 1993, 2018
 */

/*
 * Header File Name: dsmapi.h
 *
 * Environment:
 * ** This is a platform-independent source file **
 *
 * Design Notes:
 * This file contains basic data types and constants
 * includable by all client source files. The constants
 * within this file should be set properly for the
 * particular machine and operating system on which the
 * client software is to be run.
 *
 * Platform specific definitions are included in dsmapi.h
 *
 * Descriptive-name: Definitions for IBM Storage Protect API constants
 */

#ifndef _H_DSMAPITD
#define _H_DSMAPITD

#include "dsmapi.h" /* Platform specific definitions*/
#include "release.h"

/*== set the structure alignment to pack the structures ==*/
#if (_OPSYS_TYPE == DS_WINNT) && !defined(_WIN64)
#pragma pack(1)
#endif

#ifdef _MAC
/*=====
 choices are:
 http://developer.apple.com/documentation/DeveloperTools/Conceptual/PowerPCRuntime/Data/
 chapter_2_section_3.html
#pragma option align=<mode>
 where <mode> is power, mac68k, natural, or packed.
=====*/
#pragma options align=packed
#endif

typedef char osChar_t;
```



```

#define DSM GROUPTYPE_RESERVED1    0x01    /* for future use */
#define DSM GROUPTYPE_PEER          0x02    /* Peer group */
#define DSM GROUPTYPE_RESERVED2    0x03    /* for future use */

/*-----+
| Definitions for "member type" field in tsmGroupHandlerIn_t |
+-----*/

#define DSM_MEMBERTYPE_LEADER      0x01    /* group leader */
#define DSM_MEMBERTYPE_MEMBER      0x02    /* group member */

/*-----+
| Definitions for "operation type" field in tsmGroupHandlerIn_t |
+-----*/
#define DSM_GROUP_ACTION_BEGIN      0x01
#define DSM_GROUP_ACTION_OPEN      0x02    /* create new group */
#define DSM_GROUP_ACTION_CLOSE      0x03    /* commit and save an open group */
#define DSM_GROUP_ACTION_ADD        0x04    /* Append to a group */
#define DSM_GROUP_ACTION_ASSIGNTO   0x05    /* Assign to a another group */
#define DSM_GROUP_ACTION_REMOVE     0x06    /* remove a member from a group */

/*-----+
| Values for copySer in DetailCG structures for Query Mgmt Class response |
+-----*/
#define Copy_Serial_Static          1    /*Copy Serialization Static */
#define Copy_Serial_Shared_Static   2    /*Copy Serialization Shared Static*/
#define Copy_Serial_Shared_Dynamic  3    /*Copy Serialization Shared Dynamic*/
#define Copy_Serial_Dynamic         4    /*Copy Serialization Dynamic */

/*-----+
| Values for copyMode in DetailCG structures for Query Mgmt Class response |
+-----*/
#define Copy_Mode_Modified          1    /*Copy Mode Modified */
#define Copy_Mode_Absolute          2    /*Copy Mode Absolute */

/*-----+
| Values for objState in qryBackupData structure |
+-----*/
#define DSM_ACTIVE                  0x01    /* query only active objects */
#define DSM_INACTIVE                0x02    /* query only inactive objects */
#define DSM_ANY_MATCH               0xFF    /* query all backup objects */

/*-----+
| Boundary values for dsmDate.year field in qryArchiveData structure |
+-----*/
#define DATE_MINUS_INFINITE         0x0000    /* lowest boundary */
#define DATE_PLUS_INFINITE          0xFFFF    /* highest upper boundary */

/*-----+
| Bits masks for update action parameter on dsmUpdateFS() |
+-----*/
#define DSM_FSUPD_FSTYPE             ((unsigned) 0x00000002)
#define DSM_FSUPD_FSINFO             ((unsigned) 0x00000004)
#define DSM_FSUPD_BACKSTARTDATE      ((unsigned) 0x00000008)
#define DSM_FSUPD_BACKCOMPLETEDATE  ((unsigned) 0x00000010)
#define DSM_FSUPD_OCCUPANCY          ((unsigned) 0x00000020)
#define DSM_FSUPD_CAPACITY           ((unsigned) 0x00000040)
#define DSM_FSUPD_RESERVED1         ((unsigned) 0x00000100)

/*-----+
| Bits mask for backup update action parameter on dsmUpdateObj() |
+-----*/
#define DSM_BACKUPD_OWNER            ((unsigned) 0x00000001)
#define DSM_BACKUPD_OBJINFO          ((unsigned) 0x00000002)
#define DSM_BACKUPD_MC               ((unsigned) 0x00000004)

#define DSM_ARCHUPD_OWNER            ((unsigned) 0x00000001)
#define DSM_ARCHUPD_OBJINFO          ((unsigned) 0x00000002)
#define DSM_ARCHUPD_DESCR            ((unsigned) 0x00000004)

/*-----+
| Values for repository parameter on dsmDeleteFS() |
+-----*/
#define DSM_ARCHIVE_REP              0x0A    /* archive repository */
#define DSM_BACKUP_REP               0x0B    /* backup repository */
#define DSM_REPOS_ALL                0x01    /* all repository types */

/*-----+
| Values for vote parameter on dsmEndTxn() |
+-----*/
#define DSM_VOTE_COMMIT              1    /* commit current transaction */
#define DSM_VOTE_ABORT               2    /* roll back current transaction */

```

```

/*-----+
| Values for various flags returned in ApiSessInfo structure.          |
+-----*/
/* Client compression field codes */
#define COMPRESS_YES      1      /* client must compress data      */
#define COMPRESS_NO       2      /* client must NOT compress data   */
#define COMPRESS_CD       3      /* client determined               */

/* Archive delete permission codes. */
#define ARCHDEL_YES       1      /* archive delete allowed          */
#define ARCHDEL_NO        2      /* archive delete NOT allowed      */

/* Backup delete permission codes. */
#define BACKDEL_YES       1      /* backup delete allowed           */
#define BACKDEL_NO        2      /* backup delete NOT allowed       */

/*-----+
| Values for various flags returned in optStruct structure.          |
+-----*/
#define DSM_PASSWD_GENERATE 1
#define DSM_PASSWD_PROMPT  0

#define DSM_COMM_TCP        1      /* tcpip                */
#define DSM_COMM_NAMEDPIPE  2      /* Named pipes          */
#define DSM_COMM_SHM        3      /* Shared Memory        */

/* obsolete commmethods */
#define DSM_COMM_PVM_IUCV    12
#define DSM_COMM_3270        12
#define DSM_COMM_IUCV        12
#define DSM_COMM_PWSCS       12
#define DSM_COMM_SNA_LU6_2   12
#define DSM_COMM_IPXSPX      12      /* For IPX/SPX support */
#define DSM_COMM_NETBIOS     12      /* NETBIOS             */
#define DSM_COMM_400COMM      12
#define DSM_COMM_CLIO        12      /* CLIO/S              */

/*-----+
| Values for userNameAuthorities in dsmInitEx for future use        |
+-----*/
#define DSM_USERAUTH_NONE    ((dsInt16_t)0x0000)
#define DSM_USERAUTH_ACCESS  ((dsInt16_t)0x0001)
#define DSM_USERAUTH_OWNER   ((dsInt16_t)0x0002)
#define DSM_USERAUTH_POLICY  ((dsInt16_t)0x0004)
#define DSM_USERAUTH_SYSTEM  ((dsInt16_t)0x0008)

/*-----+
| Values for encryptionType on dsmEndSendObjEx, queryResp          |
+-----*/
#define DSM_ENCRYPT_NO        ((dsUInt8_t)0x00)
#define DSM_ENCRYPT_USER      ((dsUInt8_t)0x01)
#define DSM_ENCRYPT_CLIENTENCRKEY ((dsUInt8_t)0x02)
#define DSM_ENCRYPT_DES_56BIT ((dsUInt8_t)0x04)
#define DSM_ENCRYPT_AES_128BIT ((dsUInt8_t)0x08)
#define DSM_ENCRYPT_AES_256BIT ((dsUInt8_t)0x10)

/*-----+
| Definitions for mediaClass field.                                  |
+-----*/
/*
 * The following constants define a hierarchy of media access classes.
 * Lower numbers indicate media which can supply faster access to data.
 */

/* Fixed: represents the class of on-line, fixed media (such as
   hard disks). */
#define MEDIA_FIXED          0x10

/* Library: represents the class of mountable media accessible
   through a mechanical mounting device. */
#define MEDIA_LIBRARY        0x20

/* future use */
#define MEDIA_NETWORK        0x30

/* future use */
#define MEDIA_SHELF          0x40

/* future use */
#define MEDIA_OFFSITE        0x50

```



```

/* future use */
#define MEDIA_UNAVAILABLE    0xF0

/*-----+
| Type definition for partial object data for dsmBeginGetData() |
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;          /* Structure version */
    dsStruct64_t  partialObjOffset;    /* offset into object to begin reading */
    dsStruct64_t  partialObjLength;    /* amount of object to read */
} PartialObjData ;                  /* partial object data */

#define PartialObjDataVersion 1 /*

/*-----+
| Type definition for date structure |
+-----*/
typedef struct
{
    dsUInt16_t    year;                /* year, 16-bit integer (e.g., 1990) */
    dsUInt8_t     month;               /* month, 8-bit integer (1 - 12) */
    dsUInt8_t     day;                 /* day, 8-bit integer (1 - 31) */
    dsUInt8_t     hour;                /* hour, 8-bit integer (0 - 23) */
    dsUInt8_t     minute;              /* minute, 8-bit integer (0 - 59) */
    dsUInt8_t     second;              /* second, b-bit integer (0 - 59) */
} dsmDate ;

/*-----+
| Type definition for Object ID on dsmGetObj() and in dsmGetList structure |
+-----*/
typedef dsStruct64_t  ObjID ;

/*-----+
| Type definition for dsmQueryBuff on dsmBeginQuery() |
+-----*/
typedef void dsmQueryBuff ;

/*-----+
| Type definition for dsmGetType parameter on dsmBeginGetData() |
+-----*/
typedef enum
{
    gtBackup = 0x00,                  /* Backup processing type */
    gtArchive                               /* Archive processing type */
} dsmGetType ;

/*-----+
| Type definition for dsmQueryType parameter on dsmBeginQuery() |
+-----*/
typedef enum
{
    qtArchive = 0x00,                  /* Archive query type */
    qtBackup,                          /* Backup query type */
    qtBackupActive,                    /* Fast query for active backup files */
    qtFilespace,                      /* Filespace query type */
    qtMC,                              /* Mgmt. class query type */
    qtReserved1,                      /* future use */
    qtReserved2,                      /* future use */
    qtReserved3,                      /* future use */
    qtReserved4,                      /* future use */
    qtBackupGroups,                   /* group leaders in a specific fs */
    qtOpenGroups,                     /* Open groups in a specific fs */
    qtReserved5,                      /* future use */
    qtProxyNodeAuth,                  /* nodes that his node can proxy to */
    qtProxyNodePeer,                  /* Peer nodes with the same target */
    qtReserved6,                      /* future use */
    qtReserved7,                      /* future use */
    qtReserved8,                      /* future use */
} dsmQueryType ;

/*-----+
| Type definition sendType parameter on dsmBindMC() and dsmSendObj() |
+-----*/
typedef enum
{
    stBackup = 0x00,                  /* Backup processing type */
    stArchive,                        /* Archive processing type */
    stBackupMountWait,                /* Backup processing with mountwait on */
    stArchiveMountWait                /* Archive processing with mountwait on */
} dsmSendType ;

```

```

/*-----+
| Type definition for delType parameter on dsmDeleteObj() |
+-----*/
typedef enum
{
    dtArchive = 0x00,                /* Archive delete type */
    dtBackup,                        /* Backup delete (deactivate) type */
    dtBackupID,                      /* Backup delete (remove) type */
}dsmDelType ;

/*-----+
| Type definition sendType parameter on dsmSetAccess() |
+-----*/
typedef enum
{
    atBackup = 0x00,                /* Backup processing type */
    atArchive,                      /* Archive processing type */
}dsmAccessType;

/*-----+
| Type definition for API Version on dsmInit() and dsmQueryApiVersion() |
+-----*/
typedef struct
{
    dsUint16_t version;             /* API version */
    dsUint16_t release;            /* API release */
    dsUint16_t level;              /* API level */
}dsmApiVersion;

/*-----+
| Type definition for API Version on dsmInit() and dsmQueryApiVersion() |
+-----*/
typedef struct
{
    dsUint16_t stVersion;           /* Structure version */
    dsUint16_t version;            /* API version */
    dsUint16_t release;            /* API release */
    dsUint16_t level;              /* API level */
    dsUint16_t subLevel;           /* API sub level */
    dsmBool_t unicode;             /* API unicode? */
}dsmApiVersionEx;

#define apiVersionExVer    2

/*-----+
| Type definition for Application Version on dsmInit() |
+-----*/
typedef struct
{
    dsUint16_t stVersion;           /* Structure version */
    dsUint16_t applicationVersion; /* application version number */
    dsUint16_t applicationRelease; /* application release number */
    dsUint16_t applicationLevel;   /* application level number */
    dsUint16_t applicationSubLevel; /* application sub level number */
} dsmAppVersion;

#define appVersionVer    1

/*-----+
| Type definition for object name used on BindMC, Send, Delete, Query |
+-----*/
typedef struct S_dsmObjName
{
    char fs[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name */
    char hl[DSM_MAX_HL_LENGTH + 1];     /* High level name */
    char ll[DSM_MAX_LL_LENGTH + 1];     /* Low level name */
    dsUint8_t objType;                  /* for object type values, see defines above */
}dsmObjName;

/*-----+
| Type definition for Backup delete info on dsmDeleteObj() |
+-----*/
typedef struct
{
    dsUint16_t stVersion;             /* structure version */
    dsmObjName *objNameP;             /* object name */
    dsUint32_t copyGroup;             /* copy group */
}delBack ;

```

```

#define delBackVersion 1

/*-----+
| Type definition for Archive delete info on dsmDeleteObj() |
+-----*/
typedef struct
{
    dsUInt16_t      stVersion ;           /* structure version */
    dsStruct64_t     objId ;              /* object ID */
}delArch ;

#define delArchVersion 1

/*-----+
| Type definition for Backup ID delete info on dsmDeleteObj() |
+-----*/
typedef struct
{
    dsUInt16_t      stVersion ;           /* structure version */
    dsStruct64_t     objId ;              /* object ID */
}delBackID;

#define delBackIDVersion 1

/*-----+
| Type definition for delete info on dsmDeleteObj() |
+-----*/
typedef union
{
    delBack    backInfo ;
    delArch    archInfo ;
    delBackID  backIDInfo ;
}dsmDelInfo ;

/*-----+
| Type definition for Object Attribute parameter on dsmSendObj() |
+-----*/
typedef struct
{
    dsUInt16_t      stVersion;           /* Structure version */
    char            owner[DSM_MAX_OWNER_LENGTH + 1]; /* object owner */
    dsStruct64_t     sizeEstimate;        /* Size estimate in bytes of the object */
    dsmBool_t        objCompressed;       /* Is object already compressed? */
    dsUInt16_t        objInfoLength;      /* length of object-dependent info */
    char            *objInfo;             /* object-dependent info */
    char            *mcNameP;             /* mgmnt class name for override */
    dsmBool_t        disableDeduplication; /* force no dedup for this object */
    dsmBool_t        useExtObjInfo;       /* use ext obj info up to 1536 */
}ObjAttr;

#define ObjAttrVersion 4

/*-----+
| Type definition for mcBindKey returned on dsmBindMC() |
+-----*/
typedef struct
{
    dsUInt16_t      stVersion;           /* structure version */
    char            mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* Name of mc bound to object. */
                                                    /* True/false */
    dsmBool_t        backup_cg_exists;    /* True/false */
    dsmBool_t        archive_cg_exists;   /* True/false */
    char            backup_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1]; /* Backup copy dest. name */
                                                    /* Arch copy dest.name */
    char            archive_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1];
}mcBindKey;

#define mcBindKeyVersion 1

/*-----+
| Type definition for object list on dsmBeginGetData() |
+-----*/
typedef struct
{
    dsUInt16_t      stVersion ;           /* structure version */
    dsUInt32_t       numObjId ;           /* number of object IDs in the list */
    ObjID            *objId ;             /* list of object IDs to restore*/
    PartialObjData   *partialObjData;     /*list of partial obj data info */
}

```

```

}dsmGetList ;

#define dsmGetListVersion      2  /* default if not using Partial Obj data */
#define dsmGetListPORVersion   3  /* version if using Partial Obj data */

/*-----+
| Type definition for DataBlk used to Get or Send data |
+-----*/
typedef struct
{
    dsUint16_t stVersion ;           /* structure version */
    dsUint32_t bufferLen;           /* Length of buffer passed below */
    dsUint32_t numBytes;            /* Actual number of bytes read from */
                                   /* or written to the buffer */
    char *bufferPtr;                /* Data buffer */
    dsUint32_t numBytesCompressed;  /* on send actual bytes compressed */
    dsUint16_t reserved;            /* for future use */
}DataBlk;

#define DataBlkVersion 3

/*-----+
| Type definition for Mgmt Class queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct S_qryMCData
{
    dsUint16_t stVersion;           /* structure version */
    char *mcName;                   /* Mgmt class name */
    dsmBool_t mcDetail;             /* single name to get one or empty string to get all*/
                                   /* Want details or not? */
}qryMCData;

#define qryMCDataVersion 1

/*=== values for RETINIT ===*/
#define ARCH_RETINIT_CREATE 0
#define ARCH_RETINIT_EVENT 1

/*-----+
| Type definition for Archive Copy Group details on Query MC response |
+-----*/
typedef struct S_archDetailCG
{
    char cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUint16_t frequency;                    /* Copy (archive) frequency */
    dsUint16_t retainVers;                   /* Retain version */
    dsUint8_t copySer;                       /* for copy serialization values, see defines */
    dsUint8_t copyMode;                      /* for copy mode values, see defines above */
    char destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsmBool_t bLanFreeDest;                  /* Destination has lan free path? */
    dsmBool_t reserved;                      /* Not currently used */
    dsUint8_t retainInit;                    /* possible values see above */
    dsUint16_t retainMin;                    /* if retInit is EVENT num of days */
    dsmBool_t bDeduplicate;                  /* destination has dedup enabled */
}archDetailCG;

/*-----+
| Type definition for Backup Copy Group details on Query MC response |
+-----*/
typedef struct S_backupDetailCG
{
    char cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUint16_t frequency;                    /* Backup frequency */
    dsUint16_t verDataExst;                  /* Versions data exists */
    dsUint16_t verDataDltd;                  /* Versions data deleted */
    dsUint16_t retXtraVers;                  /* Retain extra versions */
    dsUint16_t retOnlyVers;                  /* Retain only versions */
    dsUint8_t copySer;                       /* for copy serialization values, see defines */
    dsUint8_t copyMode;                      /* for copy mode values, see defines above */
    char destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsmBool_t bLanFreeDest;                  /* Destination has lan free path? */
    dsmBool_t reserved;                      /* Not currently used */
    dsmBool_t bDeduplicate;                  /* destination has dedup enabled */
}backupDetailCG;

/*-----+
| Type definition for Query Mgmt Class detail response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryRespMCDetailData
{

```

```

    dsUInt16_t    stVersion; /* structure version */
    char          mName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char          mDesc[DSM_MAX_MC_DESCR_LENGTH + 1]; /*mc description */
    archDetailCG  archDet; /* Archive copy group detail */
    backupDetailCG backupDet; /* Backup copy group detail */
}qryRespMCDetailData;

#define qryRespMCDetailDataVersion 4

/*-----+
| Type definition for Query Mgmt Class summary response on dsmGetNextQObj()|
+-----*/
typedef struct S_qryRespMCData
{
    dsUInt16_t    stVersion; /* structure version */
    char          mName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char          mDesc[DSM_MAX_MC_DESCR_LENGTH + 1]; /* mc description */
}qryRespMCData;

#define qryRespMCDataVersion 1

/*-----+
| Type definition for Archive queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct S_qryArchiveData
{
    dsUInt16_t    stVersion; /* structure version */
    dsmObjName    *objName; /* Full dsm name of object */
    char          *owner; /* owner name */
    /* for maximum date boundaries, see defines above */
    dsmDate       insDateLowerBound; /* low bound archive insert date */
    dsmDate       insDateUpperBound; /* hi bound archive insert date */
    dsmDate       expDateLowerBound; /* low bound expiration date */
    dsmDate       expDateUpperBound; /* hi bound expiration date */
    char          *descr; /* archive description */
} qryArchiveData;

#define qryArchiveDataVersion 1

/*=== values for retentionInitiated field ===*/
#define DSM_ARCH_RETINIT_UNKNOWN 0 /* ret init is unknown (down-level srv) */
#define DSM_ARCH_RETINIT_STARTED 1 /* retention clock is started */
#define DSM_ARCH_RETINIT_PENDING 2 /* retention clock is not started */

/*=== Values for objHeld ===*/
#define DSM_ARCH_HELD_UNKNOWN 0 /* unknown hold status (down-level srv) */
#define DSM_ARCH_HELD_FALSE 1 /* object is NOT in a delete hold state */
#define DSM_ARCH_HELD_TRUE 2 /* object is in a delete hold state */

/*-----+
| Type definition for Query Archive response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryRespArchiveData
{
    dsUInt16_t    stVersion; /* structure version */
    dsmObjName    objName; /* Filespace name qualifier */
    dsUInt32_t    copyGroup; /* copy group number */
    char          mName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char          owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsStruct64_t  objId; /* Unique copy id */
    dsStruct64_t  reserved; /* backward compatability */
    dsUInt8_t     mediaClass; /* media access class */
    dsmDate       insDate; /* archive insertion date */
    dsmDate       expDate; /* expiration date for object */
    char          descr[DSM_MAX_DESCR_LENGTH + 1]; /* archive description */
    dsUInt16_t    objInfolen; /* length of object-dependent info*/
    char          reservedObjInfo[DSM_MAX_OBGINFO_LENGTH]; /*object-dependent info */
    dsUInt160_t   restoreOrderExt; /* restore order */
    dsStruct64_t  sizeEstimate; /* size estimate stored by user*/
    dsUInt8_t     compressType; /* Compression flag*/
    dsUInt8_t     retentionInitiated; /* object waiting on retention event*/
    dsUInt8_t     objHeld; /*object is on retention "hold" see values above*/
    dsUInt8_t     encryptionType; /* type of encryption */
    dsmBool_t     clientDeduplicated; /* obj deduplicated by API*/
    char          objInfo[DSM_MAX_EXT_OBGINFO_LENGTH]; /*object-dependent info */
    char          compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
}qryRespArchiveData;

```

```

#define qryRespArchiveDataVersion 7

/*-----+
| Type definition for Archive sendBuff parameter on dsmSendObj()
+-----*/
typedef struct S_sndArchiveData
{
    dsUuint16_t    stVersion;           /* structure version */
    char           *descr;             /* archive description */
}sndArchiveData;

#define sndArchiveDataVersion 1

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery()
+-----*/
typedef struct S_qryBackupData
{
    dsUuint16_t    stVersion;           /* structure version */
    dsmObjName     *objName;           /* full dsm name of object */
    char           *owner;             /* owner name */
    dsUuint8_t     objState;           /* object state selector */
    dsmDate        pitDate;            /* Date value for point in time restore */
                                           /* for possible values, see defines above */
}qryBackupData;

#define qryBackupDataVersion 2

typedef struct
{
    dsUuint8_t     reserved1;
    dsStruct64_t   reserved2;
} reservedInfo_t;           /* for future use */

/*-----+
| Type definition for Query Backup response on dsmGetNextQObj()
+-----*/
typedef struct S_qryRespBackupData
{
    dsUuint16_t    stVersion;           /* structure version */
    dsmObjName     objName;             /* full dsm name of object */
    dsUuint32_t    copyGroup;          /* copy group number */
    char           mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    char           owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsStruct64_t   objId;              /* Unique object id */
    dsStruct64_t   reserved;           /* backward compatability */
    dsUuint8_t     mediaClass;         /* media access class */
    dsUuint8_t     objState;           /* Obj state, active, etc. */
    dsmDate        insDate;            /* backup insertion date */
    dsmDate        expDate;            /* expiration date for object */
    dsUuint16_t    objInfolen;         /* length of object-dependent info*/
    char           reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    dsUuint160_t   restoreOrderExt;    /* restore order */
    dsStruct64_t   sizeEstimate;       /* size estimate stored by user */
    dsStruct64_t   baseObjId;
    dsUuint16_t    baseObjInfolen;     /* length of base object-dependent info*/
    dsUuint8_t     baseObjInfo[DSM_MAX_OBJINFO_LENGTH]; /* base object-dependent info */
    dsUuint160_t   baseRestoreOrder;   /* restore order */
    dsUuint32_t    fsID;
    dsUuint8_t     compressType;
    dsmBool_t      isGroupLeader;
    dsmBool_t      isOpenGroup;
    dsUuint8_t     reserved1;          /* for future use */
    dsmBool_t      reserved2;          /* for future use */
    dsUuint16_t    reserved3;          /* for future use */
    reservedInfo_t *reserved4;         /* for future use */
    dsUuint8_t     encryptionType;     /* type of encryption */
    dsmBool_t      clientDeduplicated; /* obj deduplicated by API*/
    char           objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
    char           compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
}qryRespBackupData;

#define qryRespBackupDataVersion 8

/*-----+
| Type definition for Active Backup queryBuffer on dsmBeginQuery()
|
| Notes: For the active backup query, only the fs (filesystem) and objType
|        fields of objName need be set.  objType can only be set to
|        DSM_OBJ_FILE or DSM_OBJ_DIRECTORY.  DSM_OBJ_ANY_TYPE will not
|        find a match on the query.
|
+-----*/

```

```

+-----*/
typedef struct S_qryABackupData
{
    dsUuint16_t    stVersion;                /* structure version */
    dsmObjName     *objName;                /* Only fs and objtype used */
}qryABackupData;

#define qryABackupDataVersion 1

/*-----+
| Type definition for Query Active Backup response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryARespBackupData
{
    dsUuint16_t    stVersion;                /* structure version */
    dsmObjName     objName;                /* full dsm name of object */
    dsUuint32_t    copyGroup;              /* copy group number */
    char           mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /*management class name*/
    char           owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsmDate        insDate;                /* backup insertion date */
    dsUuint16_t    objInfoLen;             /* length of object-dependent info*/
    char           reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    char           objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
}qryARespBackupData;

#define qryARespBackupDataVersion 2

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct qryBackupGroups
{
    dsUuint16_t    stVersion;                /* structure version */
    dsUuint8_t     groupType;
    char           *fsName;
    char           *owner;
    dsStruct64_t   groupLeaderObjId;
    dsUuint8_t     objType;
    dsmBool_t      noRestoreOrder;
    dsmBool_t      noGroupInfo;
    char           *hl;
}qryBackupGroups;

#define qryBackupGroupsVersion 3

/*-----+
| Type definition for proxynode queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct qryProxyNodeData
{
    dsUuint16_t    stVersion;                /* structure version */
    char           *targetNodeName;         /* target node name */
}qryProxyNodeData;

#define qryProxyNodeDataVersion 1

/*-----+
| Type definition for qryRespProxyNodeData parameter used on dsmGetNextQObj() |
+-----*/
typedef struct
{
    dsUuint16_t    stVersion ;                /* structure version */
    char           targetNodeName[DSM_MAX_ID_LENGTH+1]; /* target node name */
    char           peerNodeName[DSM_MAX_ID_LENGTH+1]; /* Peer node name */
    char           hlAddress[DSM_MAX_ID_LENGTH+1]; /* peer hlAddress */
    char           llAddress[DSM_MAX_ID_LENGTH+1]; /* peer hlAddress */
}qryRespProxyNodeData;

#define qryRespProxyNodeDataVersion 1

/*-----+
| Type definition for WINNT and OS/2 Filespace attributes |
+-----*/
typedef struct
{
    char           driveLetter ;                /* drive letter for filesystem */
    dsUuint16_t    fsInfoLength;                /* fsInfo length used */
    char           fsInfo[DSM_MAX_FSINFO_LENGTH]; /*caller-determined data */
}dsmDosFSAttrib ;

```

```

/*-----+
| Type definition for UNIX Filespace attributes |
+-----*/
typedef struct
{
    dsUInt16_t    fsInfoLength;          /* fsInfo length used */
    char          fsInfo[DSM_MAX_FSINFO_LENGTH]; /* caller-determined data */
} dsmUnixFSAttr ;

/*-----+
| Type definition for NetWare Filespace attributes |
+-----*/
typedef dsmUnixFSAttr dsmNetwareFSAttr;

/*-----+
| Type definition for Filespace attributes on all Filespace calls |
+-----*/
typedef union
{
    dsmNetwareFSAttr    netwareFSAttr;
    dsmUnixFSAttr       unixFSAttr ;
    dsmDosFSAttr        dosFSAttr ;
} dsmFSAttr ;

/*-----+
| Type definition for fsUpd parameter on dsmUpdateFS() |
+-----*/
typedef struct S_dsmFSUpd
{
    dsUInt16_t    stVersion ;          /* structure version */
    char          *fsType ;            /* filespace type */
    dsStruct64_t  occupancy ;          /* occupancy estimate */
    dsStruct64_t  capacity ;           /* capacity estimate */
    dsmFSAttr     fsAttr ;             /* platform specific attributes */
} dsmFSUpd ;

#define dsmFSUpdVersion 1

/*-----+
| Type definition for Filespace queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct S_qryFSData
{
    dsUInt16_t    stVersion;           /* structure version */
    char          *fsName;             /* File space name */
} qryFSData;

#define qryFSDataVersion 1

/*-----+
| Type definition for Query Filespace response on dsmGetNextQObj() |
+-----*/
typedef struct S_qryRespFSData
{
    dsUInt16_t    stVersion;           /* structure version */
    char          fsName[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name */
    char          fsType[DSM_MAX_FSTYPE_LENGTH + 1]; /* Filespace type */
    dsStruct64_t  occupancy;            /* Occupancy est. in bytes */
    dsStruct64_t  capacity;             /* Capacity est. in bytes */
    dsmFSAttr     fsAttr ;              /* platform specific attributes */
    dsmDate       backStartDate;        /* start backup date */
    dsmDate       backCompleteDate;     /* end backup Date */
    dsmDate       reserved1;            /* For future use */
    dsmDate       lastReplStartDate;    /* The last time replication was started */
    dsmDate       lastReplCmpltDate;    /* The last time replication completed */
    /* (could have had a failure, */
    /* but it still completes) */
    dsmDate       lastBackOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server */
    dsmDate       lastArchOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server */
    dsmDate       lastSpMgOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server */
    dsmDate       lastBackOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local */
    dsmDate       lastArchOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local */
    dsmDate       lastSpMgOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local */
    dsInt32_t     failOverWriteDelay;   /* Minutes for client to wait before allowed */
    /* to store to this Repl svr, Specail codes: */

```



```

/* NO_ACCESS(-1), ACCESS_RDONLY (-2) */
}qryRespFSData;

#define qryRespFSDataVersion 4

/*-----+
| Type definition for regFilespace parameter on dsmRegisterFS()
+-----*/
typedef struct S_regFSData
{
    dsUInt16_t    stVersion;                /* structure version */
    char          *fsName;                  /* Filespace name */
    char          *fsType;                  /* Filespace type */
    dsStruct64_t  occupancy;                /* Occupancy est. in bytes. */
    dsStruct64_t  capacity;                 /* Capacity est. in bytes. */
    dsmFSAttr     fsAttr;                   /* platform specific attributes */
}regFSData;

#define regFSDataVersion 1

/*-----+
| Type definition for dedupType used in apisessInfo
+-----*/
typedef enum
{
    dedupServerOnly= 0x00,                 /* dedup only done on server */
    dedupClientOrServer                    /* dedup can be done on client or server */
}dsmDedupType ;

/*-----+
| Type definition for fail over configuration and status
+-----*/
typedef enum
{
    failOvrNotConfigured = 0x00,
    failOvrConfigured,
    failOvrConnectedToReplServer
}dsmFailOvrCfgType ;

/*-----+
| Type definition for session info response on dsmQuerySessionInfo()
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;                /* Structure version */
    /*-----+
    /* Server information
    +-----*/
    char          serverHost[DSM_MAX_SERVERNAME_LENGTH+1];
    /* Network host name of DSM server */
    dsUInt16_t    serverPort;               /* Server comm port on host */
    dsmDate       serverDate;               /* Server's date/time */
    char          serverType[DSM_MAX_SERVERTYPE_LENGTH+1];
    /* Server's execution platform */
    dsUInt16_t    serverVer;                 /* Server's version number */
    dsUInt16_t    serverRel;                 /* Server's release number */
    dsUInt16_t    serverLev;                 /* Server's level number */
    dsUInt16_t    serverSubLev;              /* Server's sublevel number */
    /*-----+
    /* Client Defaults
    +-----*/
    char          nodeType[DSM_MAX_PLATFORM_LENGTH+1]; /*node/application type*/
    char          fsdelim;                   /* File space delimiter */
    char          hldelim;                   /* Delimiter betw highlev & lowlev */
    dsUInt8_t     compression;               /* Compression flag */
    dsUInt8_t     archDel;                   /* Archive delete permission */
    dsUInt8_t     backDel;                   /* Backup delete permission */
    dsUInt32_t    maxBytesPerTxn;            /* for future use */
    dsUInt16_t    maxObjPerTxn;              /* The max objects allowed in a txn */
    /*-----+
    /* Session Information
    +-----*/
    char          id[DSM_MAX_ID_LENGTH+1];   /* Sign-in id node name */
    char          owner[DSM_MAX_OWNER_LENGTH+1]; /* Sign-in owner */
    /* (for multi-user platforms) */
    char          confFile[DSM_PATH_MAX + DSM_NAME_MAX +1];
    /* len is platform dep */
    /* dsInit name of appl config file */
    dsUInt8_t     opNoTrace;                 /* dsInit option - NoTrace = 1 */
    /*-----+
    /* Policy Data
    +-----*/

```

```

char        domainName[DSM_MAX_DOMAIN_LENGTH+1]; /* Domain name          */
char        policySetName[DSM_MAX_PS_NAME_LENGTH+1];
            /* Active policy set name          */
dsmDate     polActDate; /* Policy set activation date      */
char        dfltMCName[DSM_MAX_MC_NAME_LENGTH+1]; /* Default Mgmt Class */
dsUint16_t  gpBackRetn; /* Grace-period backup retention   */
dsUint16_t  gpArchRetn; /* Grace-period archive retention  */
char        adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* adsm server name */
dsmBool_t   archiveRetentionProtection; /* is server Retention protection enabled */
dsStruct64_t maxBytesPerTxn_64; /* for future use */
dsmBool_t   lanFreeEnabled; /* lan free option is set */
dsmDedupType dedupType; /* server or clientOrServer */
char        accessNode[DSM_MAX_ID_LENGTH+1]; /* as node node name */

/*-----*/
/* Replication and fail over information */
/*-----*/
dsmFailOvrCfgType failOverCfgType; /* status of fail over */
char        replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
char        homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
char        replServerHost[DSM_MAX_SERVERNAME_LENGTH+1]; /* Network host name of DSM server */
dsInt32_t   replServerPort; /* Server comm port on host */

}ApiSessInfo;

#define ApiSessInfoVersion 6

/*-----+
| Type definition for Query options response on dsmQueryCliOptions()
| and dsmQuerySessOptions()
|-----*/

typedef struct
{
    char        dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char        dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char        serverName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsInt16_t   commMethod;
    char        serverAddress[DSM_MAX_SERVER_ADDRESS];
    char        nodeName[DSM_MAX_NODE_LENGTH+1];
    dsmBool_t   compression;
    dsmBool_t   compressalways;
    dsmBool_t   passwordAccess;
}optStruct;

/*-----+
| Type definition for LogType used in logInfo
|-----*/
typedef enum
{
    logServer = 0x00, /* log msg only to server */
    logLocal, /* log msg only to local error log */
    logBoth, /* log msg to server and to local error log */
    logNone
}dsmLogType;

/*-----+
| Type definition for logInfo parameter used on dsmLogEvent()
|-----*/

typedef struct
{
    char        *message; /* text of message to be logged */
    dsmLogType  logType; /* log type : local, server, both */
}logInfo;

/*-----+
| Type definition for qryRespAccessData parameter used on dsmQueryAccess()
|-----*/

typedef struct
{
    dsUint16_t   stVersion; /* structure version */
    char        node[DSM_MAX_ID_LENGTH+1]; /* node name */
    char        owner[DSM_MAX_OWNER_LENGTH+1]; /* owner */
    dsmObjName  objName; /* object name */
    dsmAccessType  accessType; /* archive or backup */
    dsUint32_t  ruleNumber; /* Access rule id */
}qryRespAccessData;

```

```

#define qryRespAccessDataVersion 1

/*-----+
| Type definition for envSetUp parameter on dsmSetUp()
+-----*/
typedef struct S_envSetUp
{
    dsUInt16_t      stVersion; /* structure version */
    char            dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char            dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char            dsmiLog[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char            **argv; /* for executables name argv[0] */
    char            logName[DSM_NAME_MAX +1];
    dsmBool_t       reserved1; /* for future use */
    dsmBool_t       reserved2; /* for future use */
}envSetUp;

#define envSetUpVersion 4

/*-----+
| Type definition for dsmInitExIn_t
+-----*/
typedef struct dsmInitExIn_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsmApiVersionEx *apiVersionExP;
    char            *clientNodeNameP;
    char            *clientOwnerNameP;
    char            *clientPasswordP;
    char            *userNameP;
    char            *userPasswordP;
    char            *applicationTypeP;
    char            *configfile;
    char            *options;
    char            dirDelimiter;
    dsmBool_t       useUnicode;
    dsmBool_t       bCrossPlatform;
    dsmBool_t       bService;
    dsmBool_t       bEncryptKeyEnabled;
    char            *encryptionPasswordP;
    dsmBool_t       useTsmBuffers;
    dsUInt8_t       numTsmBuffers;
    dsmAppVersion   *appVersionP;
}dsmInitExIn_t;

#define dsmInitExInVersion 5

/*-----+
| Type definition for dsmInitExOut_t
+-----*/
typedef struct dsmInitExOut_t
{
    dsUInt16_t      stVersion; /* structure version */
    dsInt16_t       userNameAuthorities;
    dsInt16_t       infoRC; /* error return code if encountered */
    char            adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsUInt16_t      serverVer; /* Server's version number */
    dsUInt16_t      serverRel; /* Server's release number */
    dsUInt16_t      serverLev; /* Server's level number */
    dsUInt16_t      serverSubLev; /* Server's sublevel number */

    dsmBool_t       bIsFailOverMode; /* true if failover has occurred */
    char            replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
    char            homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
}dsmInitExOut_t;

#define dsmInitExOutVersion 3

/*-----+
| Type definition for LogType used in logInfo
+-----*/
typedef enum
{
    logSevInfo = 0x00, /* information ANE4991 */
    logSevWarning, /* warning ANE4992 */
    logSevError, /* Error ANE4993 */
    logSevSevere, /* severe ANE4994 */
    logSevLicense, /* License ANE4995 */
    logSevTryBuy /* try Buy ANE4996 */
}dsmLogSeverity ;

```

```

/*-----+
| Type definition for dsmLogExIn_t
+-----*/
typedef struct dsmLogExIn_t
{
    dsUint16_t      stVersion; /* structure version */
    dsmLogSeverity  severity;
    char            appMsgID[8];
    dsmLogType      logType; /* log type : local, server, both */
    char            *message; /* text of message to be logged */
    char            appName[DSM_MAX_PLATFORM_LENGTH];
    char            osPlatform[DSM_MAX_PLATFORM_LENGTH];
    char            appVersion[DSM_MAX_PLATFORM_LENGTH];
}dsmLogExIn_t;

#define dsmLogExInVersion 2

/*-----+
| Type definition for dsmLogExOut_t
+-----*/
typedef struct dsmLogExOut_t
{
    dsUint16_t      stVersion; /* structure version */
}dsmLogExOut_t;

#define dsmLogExOutVersion 1

/*-----+
| Type definition for dsmRenameIn_t
+-----*/
typedef struct dsmRenameIn_t
{
    dsUint16_t      stVersion; /* structure version */
    dsUint32_t      dsmHandle; /* handle for session */
    dsUint8_t       repository; /* Backup or Archive */
    dsmObjNameP     *objNameP; /* object name */
    char            newHl[DSM_MAX_HL_LENGTH + 1]; /* new High level name */
    char            newLl[DSM_MAX_LL_LENGTH + 1]; /* new Low level name */
    dsmBool_t       merge; /* merge into existing name*/
    ObjID           objId; /* objId for Archive */
}dsmRenameIn_t;

#define dsmRenameInVersion 1

/*-----+
| Type definition for dsmRenameOut_t
+-----*/
typedef struct dsmRenameOut_t
{
    dsUint16_t      stVersion; /* structure version */
}dsmRenameOut_t;

#define dsmRenameOutVersion 1

/*-----+
| Type definition for dsmEndSendObjExIn_t
+-----*/
typedef struct dsmEndSendObjExIn_t
{
    dsUint16_t      stVersion; /* structure version */
    dsUint32_t      dsmHandle; /* handle for session */
}dsmEndSendObjExIn_t;

#define dsmEndSendObjExInVersion 1

/*-----+
| Type definition for dsmEndSendObjExOut_t
+-----*/
typedef struct dsmEndSendObjExOut_t
{
    dsUint16_t      stVersion; /* structure version */
    dsStruct64_t     totalBytesSent; /* total bytes read from app */
    dsmBool_t        objCompressed; /* was object compressed */
    dsStruct64_t     totalCompressSize; /* total size after compress */
    dsStruct64_t     totalLFBytesSent; /* total bytes sent Lan Free */
    dsUint8_t        encryptionType; /* type of encryption used */
    dsmBool_t        objDeduplicated; /* was object processed for dist. data dedup */
    dsStruct64_t     totalDedupSize; /* total size after de-dup */
}dsmEndSendObjExOut_t;

#define dsmEndSendObjExOutVersion 3

```

```

/*-----+
| Type definition for dsmGroupHandlerIn_t
+-----*/
typedef struct dsmGroupHandlerIn_t
{
    dsUuint16_t      stVersion;          /* structure version */
    dsUuint32_t      dsmHandle;          /* handle for session */
    dsUuint8_t       groupType;          /* Type of group */
    dsUuint8_t       actionType;         /* Type of group operation */
    dsUuint8_t       memberType;         /* Type of member: Leader or member */
    dsStruct64_t     leaderObjId;        /* OBJID of the groupleader when manipulating a member */
    char             *uniqueGroupTagP;    /* Unique group identifier */
    dsmObjName       *objNameP;          /* group leader object name */
    dsmGetList       memberObjList;      /* list of objects to remove, assign */
}dsmGroupHandlerIn_t;

#define dsmGroupHandlerInVersion 1

/*-----+
| Type definition for dsmGroupHandlerExOut_t
+-----*/
typedef struct dsmGroupHandlerOut_t
{
    dsUuint16_t      stVersion;          /* structure version */
}dsmGroupHandlerOut_t;

#define dsmGroupHandlerOutVersion 1

/*-----+
| Type definition for dsmEndTxnExIn_t
+-----*/
typedef struct dsmEndTxnExIn_t
{
    dsUuint16_t      stVersion;          /* structure version */
    dsUuint32_t      dsmHandle;          /* handle for session */
    dsUuint8_t       vote;
}dsmEndTxnExIn_t;

#define dsmEndTxnExInVersion 1

/*-----+
| Type definition for dsmEndTxnExOut_t
+-----*/
typedef struct dsmEndTxnExOut_t
{
    dsUuint16_t      stVersion;          /* structure version */
    dsUuint16_t      reason;             /* reason code */
    dsStruct64_t     groupLeaderObjId;    /* groupLeader obj id returned on */
                                         /* DSM_ACTION_OPEN */
    dsUuint8_t       reserved1;          /* future use */
    dsUuint16_t      reserved2;          /* future use */
}dsmEndTxnExOut_t;

#define dsmEndTxnExOutVersion 1

/*-----+
| Type definition for dsmEndGetDataExIn_t
+-----*/
typedef struct dsmEndGetDataExIn_t
{
    dsUuint16_t      stVersion;          /* structure version */
    dsUuint32_t      dsmHandle;          /* handle for session */
}dsmEndGetDataExIn_t;

#define dsmEndGetDataExInVersion 1

/*-----+
| Type definition for dsmEndGetDataExOut_t
+-----*/
typedef struct dsmEndGetDataExOut_t
{
    dsUuint16_t      stVersion;          /* structure version */
    dsUuint16_t      reason;             /* reason code */
    dsStruct64_t     totalLFBytesRecv;    /* total lan free bytes recieved */
}dsmEndGetDataExOut_t;

#define dsmEndGetDataExOutVersion 1

/*-----+
| Type definition for object list on dsmRetentionEvent()
+-----*/
typedef struct dsmObjList

```

```

{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      numObjId;           /* number of object IDs in the list */
    ObjID           *objId;             /* list of object IDs to signal */
}dsmObjList_t ;

#define dsmObjlistVersion 1

/*-----+
| Type definition eventType used on dsmRetentionEvent |
+-----*/
typedef enum
{
    eventRetentionActivate = 0x00,      /* signal the server that the event has occurred */
    eventHoldObj,                      /* suspend delete/expire of the object */
    eventReleaseObj                    /* Resume normal delete/expire processing */
}dsmEventType_t;

/*-----+
| Type definition for on dsmRetentionEvent() |
+-----*/
typedef struct dsmRetentionEventIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
    dsmEventType_t  eventType;          /* Event type */
    dsmObjList_t    objList;           /* object ID */
}dsmRetentionEventIn_t;

#define dsmRetentionEventInVersion 1

/*-----+
| Type definition for on dsmRetentionEvent() |
+-----*/
typedef struct dsmRetentionEventOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */
}dsmRetentionEventOut_t;

#define dsmRetentionEventOutVersion 1

/*-----+
| Type definition for on dsmRequestBuffer() |
+-----*/
typedef struct requestBufferIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
}requestBufferIn_t;

#define requestBufferInVersion 1

/*-----+
| Type definition for on dsmRequestBuffer() |
+-----*/
typedef struct requestBufferOut_t
{
    dsUInt16_t      stVersion ;          /* structure version */
    dsUInt8_t       tsmBufferHandle;     /* handle to tsm Data buffer */
    char            *dataPtr;            /* Address to write data to */
    dsUInt32_t      bufferLen;           /* Max length of data to be written */
}requestBufferOut_t;

#define requestBufferOutVersion 1

/*-----+
| Type definition for on dsmReleaseBuffer() |
+-----*/
typedef struct releaseBufferIn_t
{
    dsUInt16_t      stVersion;          /* structure version */
    dsUInt32_t      dsmHandle;          /* session Handle */
    dsUInt8_t       tsmBufferHandle;     /* handle to tsm Data buffer */
    char            *dataPtr;            /* Address to write data to */
}releaseBufferIn_t;

#define releaseBufferInVersion 1

/*-----+
| Type definition for on dsmReleaseBuffer() |
+-----*/

```

```

typedef struct releaseBufferOut_t
{
    dsUInt16_t      stVersion ;                /* structure version */
}releaseBufferOut_t;

#define releaseBufferOutVersion 1

/*-----+
| Type definition for on dsmGetBufferData()      |
+-----*/
typedef struct getBufferDataIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt32_t      dsmHandle;                /* session Handle */
}getBufferDataIn_t;

#define getBufferDataInVersion 1

/*-----+
| Type definition for on dsmGetBufferData()      |
+-----*/
typedef struct getBufferDataOut_t
{
    dsUInt16_t      stVersion ;                /* structure version */
    dsUInt8_t       tsmBufferHandle;          /* handle to tsm Data buffer */
    char            *dataPtr;                 /* Address of actual data to read */
    dsUInt32_t      numBytes;                 /* Actual number of bytes to read from dataPtr*/
}getBufferDataOut_t;

#define getBufferDataOutVersion 1

/*-----+
| Type definition for on dsmSendBufferData()      |
+-----*/
typedef struct sendBufferDataIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt32_t      dsmHandle;                /* session Handle */
    dsUInt8_t       tsmBufferHandle;          /* handle to tsm Data buffer */
    char            *dataPtr;                 /* Address of actual data to send */
    dsUInt32_t      numBytes;                 /* Actual number of bytes to send from dataPtr*/
}sendBufferDataIn_t;

#define sendBufferDataInVersion 1

/*-----+
| Type definition for on dsmSendBufferData()      |
+-----*/
typedef struct sendBufferDataOut_t
{
    dsUInt16_t      stVersion ;                /* structure version */
}sendBufferDataOut_t;

#define sendBufferDataOutVersion 1

/*-----+
| Type definition for dsmUpdateObjExIn_t          |
+-----*/
typedef struct dsmUpdateObjExIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt32_t      dsmHandle;                /* session Handle */
    dsmSendType     sendType;                 /* send type back/arch */
    char            *descrP;                  /* archive description */
    dsmObjName       *objNameP;               /* objName */
    ObjAttr          *objAttrPtr;             /* attribute */
    dsUInt32_t       objUpdAct;                /* update action */
    ObjID            archObjId;               /* objId for archive */
}dsmUpdateObjExIn_t;

#define dsmUpdateObjExInVersion 1

/*-----+
| Type definition for dsmUpdateObjExOut_t          |
+-----*/
typedef struct dsmUpdateObjExOut_t
{
    dsUInt16_t      stVersion;                /* structure version */
}dsmUpdateObjExOut_t;

#define dsmUpdateObjExOutVersion 1

```

```

#if (_OPSYS_TYPE == DS_WINNT) && !defined(_WIN64)
#pragma pack()
#endif

#ifdef _MAC
#pragma options align=reset
#endif
#endif /* _H_DSMAPI_TD */

```

## tsmapitd.h

```

/*****
* IBM Storage Protect
* API Client Component
*
* IBM Confidential
* (IBM Confidential-Restricted when combined with the Aggregated OCO
* source modules for this program)
*
* OCO Source Materials
*
* 5648-020 (C) Copyright IBM Corporation 1993, 2018
*****/

/*****
* Header File Name: tsmapitd.h
*
* Environment:
* ** This is a platform-independent source file **
*
*
* Design Notes: This file contains basic data types and constants
* includable by all client source files. The constants
* within this file should be set properly for the
* particular machine and operating system on which the
* client software is to be run.
*
* Platform specific definitions are included in dsmapips.h
*
* Descriptive-name: Definitions for IBM Storage Protect API constants
*-----*/

#ifndef _H_TSMAPITD
#define _H_TSMAPITD

/*=== set the structure alignment to pack the structures ===*/
#if _OPSYS_TYPE == DS_WINNT
#ifdef _WIN64
#pragma pack(8)
#else
#pragma pack(1)
#endif
#endif

#ifdef _MAC
#pragma options align = packed
#endif

/*=====
Win32 applications using the tsm interface must use the
-DUNICODE flag during compilation.
=====*/
#if _OPSYS_TYPE == DS_WINNT && !defined(DSMAPILIB)
#ifndef UNICODE
#error "Win32 applications using the TSM interface MUST be compiled with the -DUNICODE flag"
#endif
#endif

/*=====
Mac OS X applications using the tsm interface must use the
-DUNICODE flag during compilation.
=====*/
#if _OPSYS_TYPE == DS_MACOS && !defined(DSMAPILIB)

```



```

#ifndef UNICODE
#error "Mac OS X applications using the TSM interface MUST be compiled with the -DUNICODE flag"
#endif
#endif

/*-----+
| Type definition for dsmGetType parameter on tsmBeginGetData() |
+-----*/
typedef enum
{
    gtTsmBackup = 0x00,          /* Backup processing type */
    gtTsmArchive          /* Archive processing type */
} tsmGetType ;

/*-----+
| Type definition for dsmQueryType parameter on tsmBeginQuery() |
+-----*/
typedef enum
{
    qtTsmArchive = 0x00,          /* Archive query type */
    qtTsmBackup,          /* Backup query type */
    qtTsmBackupActive,      /* Fast query for active backup files */
    qtTsmFilespace,          /* Filespace query type */
    qtTsmMC,              /* Mgmt. class query type */
    qtTsmReserved1,          /* future use */
    qtTsmReserved2,          /* future use */
    qtTsmReserved3,          /* future use */
    qtTsmReserved4,          /* future use */
    qtTsmBackupGroups,      /* All group leaders in a specific filesystem */
    qtTsmOpenGroups,        /* All group members associated with a leader */
    qtTsmReserved5,          /* future use */
    qtTsmProxyNodeAuth,      /* nodes that this node can proxy to */
    qtTsmProxyNodePeer,      /* peer nodes under this target node */
    qtTsmReserved6,          /* future use */
    qtTsmReserved7,          /* future use */
    qtTsmReserved8,          /* future use */
} tsmQueryType ;

/*-----+
| Type definition sendType parameter on tsmBindMC() and tsmSendObj() |
+-----*/
typedef enum
{
    stTsmBackup = 0x00,          /* Backup processing type */
    stTsmArchive,          /* Archive processing type */
    stTsmBackupMountWait,      /* Backup processing with mountwait on */
    stTsmArchiveMountWait      /* Archive processing with mountwait on */
} tsmSendType ;

/*-----+
| Type definition for delType parameter on tsmDeleteObj() |
+-----*/
typedef enum
{
    dtTsmArchive = 0x00,          /* Archive delete type */
    dtTsmBackup,          /* Backup delete (deactivate) type */
    dtTsmBackupID          /* Backup delete (remove) type */
} tsmDelType ;

/*-----+
| Type definition sendType parameter on tsmSetAccess() |
+-----*/
typedef enum
{
    atTsmBackup = 0x00,          /* Backup processing type */
    atTsmArchive          /* Archive processing type */
} tsmAccessType;

/*-----+
| Type definition for Overwrite parameter on tsmSendObj() |
+-----*/
typedef enum
{
    owIGNORE = 0x00,
    owYES,
    owNO
} tsmOwType;

/*-----+
| Type definition for API Version on tsmInit() and tsmQueryApiVersion() |
+-----*/

```

```

typedef struct
{
    dsUInt16_t stVersion;      /* Structure version          */
    dsUInt16_t version;        /* API version                */
    dsUInt16_t release;        /* API release                */
    dsUInt16_t level;          /* API level                  */
    dsUInt16_t subLevel;       /* API sub level              */
    dsmBool_t  unicode;        /* API unicode?               */
} tsmApiVersionEx;

#define tsmApiVersionExVer      2

/*-----+
| Type definition for Application Version on tsmInit()
+-----*/
typedef struct
{
    dsUInt16_t stVersion;      /* Structure version          */
    dsUInt16_t applicationVersion; /* application version number */
    dsUInt16_t applicationRelease; /* application release number */
    dsUInt16_t applicationLevel;  /* application level number   */
    dsUInt16_t applicationSubLevel; /* application sub level number */
} tsmAppVersion;

#define tsmAppVersionVer      1

/*-----+
| Type definition for object name used on BindMC, Send, Delete, Query
+-----*/
typedef struct tsmObjName
{
    dsChar_t fs[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name */
    dsChar_t hl[DSM_MAX_HL_LENGTH + 1];     /* High level name */
    dsChar_t ll[DSM_MAX_LL_LENGTH + 1];     /* Low level name */
    dsUInt8_t objType; /* for object type values, see defines above */
    dsChar_t dirDelimiter;
} tsmObjName;

/*-----+
| Type definition for Backup delete info on dsmDeleteObj()
+-----*/
typedef struct tsmDelBack
{
    dsUInt16_t stVersion; /* structure version */
    tsmObjName *objNameP; /* object name */
    dsUInt32_t copyGroup; /* copy group */
} tsmDelBack;

#define tsmDelBackVersion      1

/*-----+
| Type definition for Archive delete info on dsmDeleteObj()
+-----*/
typedef struct
{
    dsUInt16_t stVersion; /* structure version */
    dsStruct64_t objId; /* object ID */
} tsmDelArch;

#define tsmDelArchVersion      1

/*-----+
| Type definition for Backup ID delete info on dsmDeleteObj()
+-----*/
typedef struct
{
    dsUInt16_t stVersion; /* structure version */
    dsStruct64_t objId; /* object ID */
} tsmDelBackID;

#define tsmDelBackIDVersion      1

/*-----+
| Type definition for delete info on dsmDeleteObj()
+-----*/
typedef union
{
    tsmDelBack backInfo;
    tsmDelArch archInfo;
}

```

```

    tsmDelBackID backIDInfo;
} tsmDelInfo ;

/*-----+
| Type definition for Object Attribute parameter on dsmSendObj()
+-----*/
typedef struct tsmObjAttr
{
    dsUint16_t    stVersion;                /* Structure version */
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH + 1]; /* object owner */
    dsStruct64_t  sizeEstimate;             /* Size estimate in bytes of the object */
    dsmBool_t     objCompressed;            /* Is object already compressed? */
    dsUint16_t    objInfoLength;           /* length of object-dependent info */
    char          *objInfo;                /* object-dependent info byte buffer */
    dsChar_t      *mcNameP;                /* mgmnt class name for override */
    tsmOwType      reserved1;              /* for future use */
    tsmOwType      reserved2;              /* for future use */
    dsmBool_t      disableDeduplication;    /* force no dedup for this object */
    dsmBool_t      useExtObjInfo;          /* use ext objinfo up to 1536 */
} tsmObjAttr;

#define tsmObjAttrVersion 5

/*-----+
| Type definition for mcBindKey returned on dsmBindMC()
+-----*/
typedef struct tsmMcBindKey
{
    dsUint16_t    stVersion;                /* structure version */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1];
    /* Name of mc bound to object. */
    dsmBool_t      backup_cg_exists;        /* True/false */
    dsmBool_t      archive_cg_exists;       /* True/false */
    dsChar_t      backup_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1];
    /* Backup copy dest. name */
    dsChar_t      archive_copy_dest[DSM_MAX_CG_DEST_LENGTH + 1];
    /* Arch copy dest.name */
} tsmMcBindKey;

#define tsmMcBindKeyVersion 1

/*-----+
| Type definition for Mgmt Class queryBuffer on dsmBeginQuery()
+-----*/
typedef struct tsmQryMCData
{
    dsUint16_t    stVersion;                /* structure version */
    dsChar_t      *mcName;                 /* Mgmt class name */
    /* single name to get one or empty string to get all */
    dsmBool_t      mcDetail;               /* Want details or not? */
} tsmQryMCData;

#define tsmQryMCDataVersion 1

/*-----+
| Type definition for Archive Copy Group details on Query MC response
+-----*/
typedef struct tsmArchDetailCG
{
    dsChar_t      cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUint16_t    frequency;                        /* Copy (archive) frequency */
    dsUint16_t    retainVers;                       /* Retain version */
    dsUint8_t     copySer;                          /* for copy serialization values, see defines */
    dsUint8_t     copyMode;                         /* for copy mode values, see defines above */
    dsChar_t      destName[DSM_MAX_CG_DEST_LENGTH + 1]; /* Copy dest name */
    dsmBool_t     blanFreeDest;                     /* Destination has lan free path? */
    dsmBool_t     reserved;                         /* Not currently used */
    dsUint8_t     retainInit;                       /* possible values see dsmapi.h */
    dsUint16_t    retainMin;                        /* if retInit is EVENT num of days */
    dsmBool_t     bDeduplicate;                     /* destination has dedup enabled */
} tsmArchDetailCG;

/*-----+
| Type definition for Backup Copy Group details on Query MC response
+-----*/
typedef struct tsmBackupDetailCG
{
    dsChar_t      cgName[DSM_MAX_CG_NAME_LENGTH + 1]; /* Copy group name */
    dsUint16_t    frequency;                        /* Backup frequency */
    dsUint16_t    verDataExst;                     /* Versions data exists */

```

```

    dsUInt16_t    verDataDltd;                                /* Versions data deleted */
    dsUInt16_t    retXtraVers;                                /* Retain extra versions */
    dsUInt16_t    retOnlyVers;                                /* Retain only versions */
    dsUInt8_t     copySer;                                    /* for copy serialization values, see defines */
    dsUInt8_t     copyMode;                                   /* for copy mode values, see defines above */
    dsChar_t      destName[DSM_MAX.CG_DEST_LENGTH + 1];      /* Copy dest name */
    dsmBool_t     bLanFreeDest;                               /* Destination has lan free path? */
    dsmBool_t     reserved;                                   /* Not currently used */
    dsmBool_t     bDeduplicate;                               /* destination has dedup enabled */
} tsmBackupDetailCG;

/*-----+
| Type definition for Query Mgmt Class detail response on dsmGetNextQObj()|
+-----*/
typedef struct tsmQryRespMCDetailData
{
    dsUInt16_t     stVersion;                                /* structure version */
    dsChar_t       mcName[DSM_MAX_MC_NAME_LENGTH + 1];       /* mc name */
    dsChar_t       mcDesc[DSM_MAX_MC_DESCR_LENGTH + 1];      /*mc description */
    archDetailCG   archDet;                                   /* Archive copy group detail */
    backupDetailCG backupDet;                                 /* Backup copy group detail */
} tsmQryRespMCDetailData;

#define tsmQryRespMCDetailDataVersion 4

/*-----+
| Type definition for Query Mgmt Class summary response on dsmGetNextQObj()|
+-----*/
typedef struct tsmQryRespMCData
{
    dsUInt16_t     stVersion;                                /* structure version */
    dsChar_t       mcName[DSM_MAX_MC_NAME_LENGTH + 1];       /* mc name */
    dsChar_t       mcDesc[DSM_MAX_MC_DESCR_LENGTH + 1];      /* mc description */
} tsmQryRespMCData;

#define tsmQryRespMCDataVersion 1

/*-----+
| Type definition for Archive queryBuffer on tsmBeginQuery()|
+-----*/
typedef struct tsmQryArchiveData
{
    dsUInt16_t     stVersion;                                /* structure version */
    tsmObjName     *objName;                                /* Full dsm name of object */
    dsChar_t       *owner;                                   /* owner name */
    /* for maximum date boundaries, see defines above */
    dsmDate        insDateLowerBound;                        /* low bound archive insert date */
    dsmDate        insDateUpperBound;                        /* hi bound archive insert date */
    dsmDate        expDateLowerBound;                        /* low bound expiration date */
    dsmDate        expDateUpperBound;                        /* hi bound expiration date */
    dsChar_t       *descr;                                   /* archive description */
} tsmQryArchiveData;

#define tsmQryArchiveDataVersion 1

/*-----+
| Type definition for Query Archive response on dsmGetNextQObj()|
+-----*/
typedef struct tsmQryRespArchiveData
{
    dsUInt16_t     stVersion;                                /* structure version */
    tsmObjName     objName;                                  /* Filespace name qualifier */
    dsUInt32_t     copyGroup;                                /* copy group number */
    dsChar_t       mcName[DSM_MAX_MC_NAME_LENGTH + 1];       /* mc name */
    dsChar_t       owner[DSM_MAX_OWNER_LENGTH + 1];          /* owner name */
    dsStruct64_t   objId;                                     /* Unique copy id */
    dsStruct64_t   reserved;                                  /* backward compatability */
    dsUInt8_t      mediaClass;                                /* media access class */
    dsmDate        insDate;                                   /* archive insertion date */
    dsmDate        expDate;                                   /* expiration date for object */
    dsChar_t       descr[DSM_MAX_DESCR_LENGTH + 1];          /* archive description */
    dsUInt16_t     objInfoLen;                                /* length of object-dependent info */
    dsUInt8_t      reservedObjInfo[DSM_MAX_OBJINFO_LENGTH];  /*object-dependent info */
    dsUInt160_t    restoreOrderExt;                           /* restore order */
    dsStruct64_t   sizeEstimate;                              /* size estimate stored by user */
    dsUInt8_t      compressType;                              /* Compression flag */
    dsUInt8_t      retentionInitiated; /* object waiting on retention event */
    dsUInt8_t      objHeld; /* object is on "hold" see dsmapi.h for values */
    dsUInt8_t      encryptionType;                           /* type of encryption */
    dsmBool_t      clientDeduplicated;                        /* obj deduplicated by API */
    dsUInt8_t      objInfo[DSM_MAX_EXT_OBJINFO_LENGTH];      /*object-dependent info */

```

```

    dsChar_t        compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
} tsmQryRespArchiveData;

#define tsmQryRespArchiveDataVersion 7

/*-----+
| Type definition for Archive sendBuff parameter on dsmSendObj()
+-----*/
typedef struct tsmSndArchiveData
{
    dsUInt16_t    stVersion;          /* structure version */
    dsChar_t      *descr;             /* archive description */
} tsmSndArchiveData;

#define tsmSndArchiveDataVersion 1

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery()
+-----*/
typedef struct tsmQryBackupData
{
    dsUInt16_t    stVersion;          /* structure version */
    tsmObjName    *objName;          /* full dsm name of object */
    dsChar_t      *owner;            /* owner name */
    dsUInt8_t     objState;          /* object state selector */
    dsmDate       pitDate;           /* Date value for point in time restore */
    /* for possible values, see defines above */
    dsUInt32_t    reserved1;
    dsUInt32_t    reserved2;
} tsmQryBackupData;

#define tsmQryBackupDataVersion 3

/*-----+
| Type definition for Query Backup response on dsmGetNextQObj()
+-----*/
typedef struct tsmQryRespBackupData
{
    dsUInt16_t    stVersion;          /* structure version */
    tsmObjName    objName;            /* full dsm name of object */
    dsUInt32_t    copyGroup;          /* copy group number */
    dsChar_t      mcName[DSM_MAX_MC_NAME_LENGTH + 1]; /* mc name */
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH + 1]; /* owner name */
    dsStruct64_t  objId;              /* Unique object id */
    dsStruct64_t  reserved;           /* backward compatability */
    dsUInt8_t     mediaClass;         /* media access class */
    dsUInt8_t     objState;           /* Obj state, active, etc. */
    dsmDate       insDate;            /* backup insertion date */
    dsmDate       expDate;            /* expiration date for object */
    dsUInt16_t    objInfolen;         /* length of object-dependent info */
    dsUInt8_t     reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /*object-dependent info */
    dsUInt160_t   restoreOrderExt;    /* restore order */
    dsStruct64_t  sizeEstimate;       /* size estimate stored by user */
    dsStruct64_t  baseObjId;
    dsUInt16_t    baseObjInfolen;     /* length of base object-dependent info */
    dsUInt8_t     baseObjInfo[DSM_MAX_OBJINFO_LENGTH]; /* base object-dependent info */
    dsUInt160_t   baseRestoreOrder;   /* restore order */
    dsUInt32_t    fsID;
    dsUInt8_t     compressType;
    dsmBool_t     isGroupLeader;
    dsmBool_t     isOpenGroup;
    dsUInt8_t     reserved1;          /* for future use */
    dsmBool_t     reserved2;          /* for future use */
    dsUInt16_t    reserved3;          /* for future use */
    reservedInfo_t *reserved4;        /* for future use */
    dsUInt8_t     encryptionType;     /* type of encryption */
    dsmBool_t     clientDeduplicated; /* obj deduplicated by API */
    dsUInt8_t     objInfo[DSM_MAX_EXT_OBJINFO_LENGTH]; /*object-dependent info */
    dsChar_t      compressAlg[DSM_MAX_COMPRESSTYPE_LENGTH + 1]; /* compression algorithm name */
} tsmQryRespBackupData;

#define tsmQryRespBackupDataVersion 8

/*-----+
| Type definition for Active Backup queryBuffer on dsmBeginQuery()
|
| Notes: For the active backup query, only the fs (filesystem) and objType
|        fields of objName need be set.  objType can only be set to
|        DSM_OBJ_FILE or DSM_OBJ_DIRECTORY.  DSM_OBJ_ANY_TYPE will not
|        find a match on the query.
+-----*/
typedef struct tsmQryABackupData
{

```

```

    dsUInt16_t      stVersion;                                /* structure version */
    tsmObjName      *objName;                                /* Only fs and objtype used */
} tsmQryABackupData;

#define tsmQryABackupDataVersion 1

/*-----+
| Type definition for Query Active Backup response on dsmGetNextQObj() |
+-----*/
typedef struct tsmQryARespBackupData
{
    dsUInt16_t      stVersion;                                /* structure version */
    tsmObjName      objName;                                  /* full dsm name of object */
    dsUInt32_t      copyGroup;                                /* copy group number */
    dsChar_t        mcName[DSM_MAX_MC_NAME_LENGTH + 1];      /* management class name */
    dsChar_t        owner[DSM_MAX_OWNER_LENGTH + 1];          /* owner name */
    dsmDate         insDate;                                   /* backup insertion date */
    dsUInt16_t      objInfolen;                                /* length of object-dependent info */
    dsUInt8_t        reservedObjInfo[DSM_MAX_OBJINFO_LENGTH]; /* object-dependent info */
    dsUInt8_t        objInfo[DSM_MAX_EXT_OBJINFO_LENGTH];     /* object-dependent info */
} tsmQryARespBackupData;

#define tsmQryARespBackupDataVersion 2

/*-----+
| Type definition for Backup queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct tsmQryBackupGroups
{
    dsUInt16_t      stVersion;                                /* structure version */
    dsUInt8_t        groupType;
    dsChar_t        *fsName;
    dsChar_t        *owner;
    dsStruct64_t     groupLeaderObjId;
    dsUInt8_t        objType;
    dsUInt32_t       reserved1;
    dsUInt32_t       reserved2;
    dsmBool_t        noRestoreOrder;
    dsmBool_t        noGroupInfo;
    dsChar_t        *hl;
} tsmQryBackupGroups;

#define tsmQryBackupGroupsVersion 4

/*-----+
| Type definition for proxynode queryBuffer on tsmBeginQuery() |
+-----*/
typedef struct tsmQryProxyNodeData
{
    dsUInt16_t      stVersion;                                /* structure version */
    dsChar_t        *targetNodeName;                          /* target node name */
} tsmQryProxyNodeData;

#define tsmQryProxyNodeDataVersion 1

/*-----+
| Type definition for qryRespProxyNodeData parameter used on tsmGetNextQObj() |
+-----*/
typedef struct tsmQryRespProxyNodeData
{
    dsUInt16_t      stVersion;                                /* structure version */
    dsChar_t        targetNodeName[DSM_MAX_ID_LENGTH+1];      /* target node name */
    dsChar_t        peerNodeName[DSM_MAX_ID_LENGTH+1];        /* peer node name */
    dsChar_t        hlAddress[DSM_MAX_ID_LENGTH+1];           /* peer hlAddress */
    dsChar_t        llAddress[DSM_MAX_ID_LENGTH+1];           /* peer llAddress */
} tsmQryRespProxyNodeData;

#define tsmQryRespProxyNodeDataVersion 1

/*-----+
| Type definition for WINNT and OS/2 Filespace attributes |
+-----*/
typedef struct tsmDosFSAttrib
{
    osChar_t        driveLetter;                               /* drive letter for filespace */
    dsUInt16_t      fsInfoLength;                               /* fsInfo length used */
    osChar_t        fsInfo[DSM_MAX_FSINFO_LENGTH];            /* caller-determined data */
} tsmDosFSAttrib;

/*-----+
| Type definition for UNIX Filespace attributes |
+-----*/

```

```

+-----*/
typedef struct tsmUnixFSAttrib
{
    dsUint16_t    fsInfoLength;          /* fsInfo length used          */
    osChar_t      fsInfo[DSM_MAX_FSINFO_LENGTH]; /* caller-determined data */
} tsmUnixFSAttrib ;

/*-----+
| Type definition for NetWare Filespace attributes |
+-----*/
typedef tsmUnixFSAttrib tsmNetwareFSAttrib;

/*-----+
| Type definition for Filespace attributes on all Filespace calls |
+-----*/
typedef union
{
    tsmNetwareFSAttrib  netwareFSAttr;
    tsmUnixFSAttrib     unixFSAttr ;
    tsmDosFSAttrib      dosFSAttr ;
} tsmFSAttr ;

/*-----+
| Type definition for fsUpd parameter on dsmUpdateFS() |
+-----*/
typedef struct    tsmFSUpd
{
    dsUint16_t      stVersion ;          /* structure version          */
    dsChar_t         *fsType ;           /* filespace type             */
    dsStruct64_t     occupancy ;         /* occupancy estimate         */
    dsStruct64_t     capacity ;          /* capacity estimate         */
    tsmFSAttr        fsAttr ;           /* platform specific attributes */
} tsmFSUpd ;

#define tsmFSUpdVersion 1

/*-----+
| Type definition for Filespace queryBuffer on dsmBeginQuery() |
+-----*/
typedef struct tsmQryFSData
{
    dsUint16_t  stVersion;          /* structure version */
    dsChar_t    *fsName;           /* File space name */
} tsmQryFSData;

#define tsmQryFSDataVersion 1

/*-----+
| Type definition for Query Filespace response on dsmGetNextQObj() |
+-----*/
typedef struct tsmQryRespFSData
{
    dsUint16_t      stVersion;          /* structure version          */
    dsChar_t        fsName[DSM_MAX_FSNAME_LENGTH + 1]; /* Filespace name          */
    dsChar_t        fsType[DSM_MAX_FSTYPE_LENGTH + 1]; /* Filespace type          */
    dsStruct64_t     occupancy;          /* Occupancy est. in bytes.  */
    dsStruct64_t     capacity;           /* Capacity est. in bytes.   */
    tsmFSAttr        fsAttr ;           /* platform specific attributes */
    dsmDate          backStartDate;      /* start backup date         */
    dsmDate          backCompleteDate;   /* end backup Date          */
    dsmDate          reserved1 ;         /* For future use           */
    dsmBool_t        bIsUnicode;
    dsUint32_t       fsID;
    dsmDate          lastReplStartDate;   /* The last time replication was started */
    dsmDate          lastReplCmpltDate;   /* The last time replication completed   */
    /* (could have had a failure,      */
    /* but it still completes)        */
    dsmDate          lastBackOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server            */
    dsmDate          lastArchOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server            */
    dsmDate          lastSpMgOpDateFromServer; /* The last store time stamp the client */
    /* saved on the server            */
    dsmDate          lastBackOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local            */
    dsmDate          lastArchOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local            */
    dsmDate          lastSpMgOpDateFromLocal; /* The last store time stamp the client */
    /* saved on the Local            */
    dsInt32_t        failOverWriteDelay; /* Minutes for client to wait before allowed */
    /* to store to this Repl srvr, Specail codes: */
    /* NO_ACCESS(-1), ACCESS_RDONLY (-2)          */
}

```

```

} tsmQryRespFSData;

#define tsmQryRespFSDataVersion 5

/*-----+
| Type definition for regFilespace parameter on dsmRegisterFS()
+-----*/
typedef struct tsmRegFSData
{
    dsUInt16_t    stVersion;                /* structure version */
    dsChar_t      *fsName;                  /* Filespace name */
    dsChar_t      *fsType;                  /* Filespace type */
    dsStruct64_t  occupancy;                 /* Occupancy est. in bytes. */
    dsStruct64_t  capacity;                 /* Capacity est. in bytes. */
    tsmFSAttr     fsAttr;                   /* platform specific attributes */
} tsmRegFSData;

#define tsmRegFSDataVersion 1

/*-----+
| Type definition for session info response on dsmQuerySessionInfo()
+-----*/
typedef struct
{
    dsUInt16_t    stVersion;                /* Structure version */
    /*-----*/
    /* Server information */
    /*-----*/
    dsChar_t      serverHost[DSM_MAX_SERVERNAME_LENGTH+1];
    /* Network host name of DSM server */
    dsUInt16_t    serverPort;               /* Server comm port on host */
    dsmDate       serverDate;               /* Server's date/time */
    dsChar_t      serverType[DSM_MAX_SERVERTYPE_LENGTH+1];
    /* Server's execution platform */
    dsUInt16_t    serverVer;                /* Server's version number */
    dsUInt16_t    serverRel;                /* Server's release number */
    dsUInt16_t    serverLev;                /* Server's level number */
    dsUInt16_t    serverSubLev;             /* Server's sublevel number */
    /*-----*/
    /* Client Defaults */
    /*-----*/
    dsChar_t      nodeType[DSM_MAX_PLATFORM_LENGTH+1]; /*node/application type*/
    dsChar_t      fsdelim;                  /* File space delimiter */
    dsChar_t      hlDelim;                  /* Delimiter betw highlev & lowlev */
    dsUInt8_t     compression;              /* Compression flag */
    dsUInt8_t     archDel;                  /* Archive delete permission */
    dsUInt8_t     backDel;                  /* Backup delete permission */
    dsUInt32_t    maxBytesPerTxn;           /* for future use */
    dsUInt16_t    maxObjPerTxn;             /* The max objects allowed in a txn */
    /*-----*/
    /* Session Information */
    /*-----*/
    dsChar_t      id[DSM_MAX_ID_LENGTH+1];  /* Sign-in id node name */
    dsChar_t      owner[DSM_MAX_OWNER_LENGTH+1]; /* Sign-in owner */
    /* (for multi-user platforms) */
    dsChar_t      confFile[DSM_PATH_MAX + DSM_NAME_MAX +1];
    /* len is platform dep */
    /* dsInit name of appl config file */
    dsUInt8_t     opNoTrace;                /* dsInit option - NoTrace = 1 */
    /*-----*/
    /* Policy Data */
    /*-----*/
    dsChar_t      domainName[DSM_MAX_DOMAIN_LENGTH+1]; /* Domain name */
    dsChar_t      policySetName[DSM_MAX_PS_NAME_LENGTH+1];
    /* Active policy set name */
    dsmDate       polActDate;               /* Policy set activation date */
    dsChar_t      gpfltMCName[DSM_MAX_MC_NAME_LENGTH+1]; /* Default Mgmt Class */
    dsUInt16_t    gpBackRetn;              /* Grace-period backup retention */
    dsUInt16_t    gpArchRetn;              /* Grace-period archive retention */
    dsChar_t      adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* adsm server name */
    dsmBool_t     archiveRetentionProtection; /* is server Retention protection enabled */
    dsUInt64_t    maxBytesPerTxn_64;       /* for future use */
    dsmBool_t     lanFreeEnabled;           /* lan free option is set */
    dsmDedupType  dedupType;               /* server or clientOrServer */
    dsChar_t      accessNode[DSM_MAX_ID_LENGTH+1]; /* as node node name */
    /*-----*/
    /* Replication and fail over information */
    /*-----*/
    dsmFailOvrCfgType failOverCfgType; /* status of fail over */
    dsChar_t      replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
    dsChar_t      homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */

```



```

    dsChar_t      replServerHost[DSM_MAX_SERVERNAME_LENGTH+1]; /* Network host name of DSM server */
    dsInt32_t     replServerPort; /* Server comm port on host */
} tsmApiSessInfo;

#define tsmApiSessInfoVersion 6

/*-----+
| Type definition for Query options response on dsmQueryCliOptions()
| and dsmQuerySessOptions()
+-----*/

typedef struct
{
    dsUInt16_t    stVersion;
    dsChar_t      dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t      dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t      serverName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsInt16_t     commMethod;
    dsChar_t      serverAddress[DSM_MAX_SERVER_ADDRESS];
    dsChar_t      nodeName[DSM_MAX_NODE_LENGTH+1];
    dsmBool_t     compression;
    dsmBool_t     compressalways;
    dsmBool_t     passwordAccess;
} tsmOptStruct ;

#define tsmOptStructVersion 1

/*-----+
| Type definition for qryRespAccessData parameter used on dsmQueryAccess()
+-----*/

typedef struct
{
    dsUInt16_t     stVersion ; /* structure version */
    dsChar_t       node[DSM_MAX_ID_LENGTH+1]; /* node name */
    dsChar_t       owner[DSM_MAX_OWNER_LENGTH+1]; /* owner */
    tsmObjName     objName ; /* object name */
    dsmAccessType  accessType; /* archive or backup */
    dsUInt32_t     ruleNumber ; /* Access rule id */
} tsmQryRespAccessData;

#define tsmQryRespAccessDataVersion 1

/*-----+
| Type definition for envSetUp parameter on dsmSetUp()
+-----*/

typedef struct tsmEnvSetUp
{
    dsUInt16_t     stVersion; /* structure version */
    dsChar_t       dsmiDir[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t       dsmiConfig[DSM_PATH_MAX + DSM_NAME_MAX +1];
    dsChar_t       dsmiLog[DSM_PATH_MAX + DSM_NAME_MAX +1];
    char           **argv; /* for executables name argv[0] */
    dsChar_t       logName[DSM_NAME_MAX +1];
    dsmBool_t      reserved1; /* for future use */
    dsmBool_t      reserved2; /* for future use */
} tsmEnvSetUp;

#define tsmEnvSetUpVersion 4

/*-----+
| Type definition for dsmInitExIn_t
+-----*/

typedef struct tsmInitExIn_t
{
    dsUInt16_t     stVersion; /* structure version */
    tsmApiVersionEx *apiVersionExP;
    dsChar_t       *clientNodeNameP;
    dsChar_t       *clientOwnerNameP;
    dsChar_t       *clientPasswordP;
    dsChar_t       *userNameP;
    dsChar_t       *userPasswordP;
    dsChar_t       *applicationTypeP;
    dsChar_t       *configfile;
    dsChar_t       *options;
    dsChar_t       dirDelimiter;
    dsmBool_t      useUnicode;
    dsmBool_t      bCrossPlatform;
    dsmBool_t      bService;
    dsmBool_t      bEncryptKeyEnabled;

```

```

    dsChar_t          *encryptionPasswordP;
    dsmBool_t         useTsmBuffers;
    dsUInt8_t         numTsmBuffers;
    tsmAppVersion     appVersionP;
} tsmInitExIn_t;

#define tsmInitExInVersion 5

/*-----+
| Type definition for dsmInitExOut_t
+-----*/
typedef struct tsmInitExOut_t
{
    dsUInt16_t         stVersion; /* structure version */
    dsInt16_t          userNameAuthorities;
    dsInt16_t          infoRC; /* error return code if encountered */
    /* adsm server name */
    dsChar_t           adsmServerName[DSM_MAX_SERVERNAME_LENGTH+1];
    dsUInt16_t         serverVer; /* Server's version number */
    dsUInt16_t         serverRel; /* Server's release number */
    dsUInt16_t         serverLev; /* Server's level number */
    dsUInt16_t         serverSubLev; /* Server's sublevel number */
    dsmBool_t          bIsFailOverMode; /* true if failover has occurred */
    dsChar_t           replServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* repl server name */
    dsChar_t           homeServerName[DSM_MAX_SERVERNAME_LENGTH+1]; /* home server name */
} tsmInitExOut_t;

#define tsmInitExOutVersion 3

/*-----+
| Type definition for dsmLogExIn_t
+-----*/
typedef struct tsmLogExIn_t
{
    dsUInt16_t         stVersion; /* structure version */
    dsmLogSeverity     severity;
    dsChar_t           appMsgID[8];
    dsmLogType         logType; /* log type : local, server, both */
    dsChar_t           *message; /* text of message to be logged */
    dsChar_t           appName[DSM_MAX_PLATFORM_LENGTH];
    dsChar_t           osPlatform[DSM_MAX_PLATFORM_LENGTH];
    dsChar_t           appVersion[DSM_MAX_PLATFORM_LENGTH];
} tsmLogExIn_t;

#define tsmLogExInVersion 2

/*-----+
| Type definition for dsmLogExOut_t
+-----*/
typedef struct tsmLogExOut_t
{
    dsUInt16_t         stVersion; /* structure version */
} tsmLogExOut_t;

#define tsmLogExOutVersion 1

/*-----+
| Type definition for dsmRenameIn_t
+-----*/
typedef struct tsmRenameIn_t
{
    dsUInt16_t         stVersion; /* structure version */
    dsUInt32_t         tsmHandle; /* handle for session */
    dsUInt8_t          repository; /* Backup or Archive */
    tsmObjName         *objNameP; /* object name */
    dsChar_t           newHl[DSM_MAX_HL_LENGTH + 1]; /* new High level name */
    dsChar_t           newLl[DSM_MAX_LL_LENGTH + 1]; /* new Low level name */
    dsmBool_t          merge; /* merge into existing name */
    ObjID              objId; /* objId for Archive */
} tsmRenameIn_t;

#define tsmRenameInVersion 1

/*-----+
| Type definition for dsmRenameOut_t
+-----*/
typedef struct tsmRenameOut_t
{
    dsUInt16_t         stVersion; /* structure version */
} tsmRenameOut_t;

```

```

#define tsmRenameOutVersion 1

/*-----+
| Type definition for tsmEndSendObjExIn_t
+-----*/
typedef struct tsmEndSendObjExIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt32_t      tsmHandle;                /* handle for session */
} tsmEndSendObjExIn_t;

#define tsmEndSendObjExInVersion 1

/*-----+
| Type definition for dsmEndSendObjExOut_t
+-----*/
typedef struct tsmEndSendObjExOut_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsStruct64_t    totalBytesSent;           /* total bytes read from app */
    dsmBool_t       objCompressed;            /* was object compressed */
    dsStruct64_t    totalCompressSize;        /* total size after compress */
    dsStruct64_t    totalLFBytesSent;         /* total bytes sent Lan Free */
    dsUInt8_t       encryptionType;          /* type of encryption used */
    dsmBool_t       objDeduplicated;          /* was object processed for dist. data dedup */
    dsStruct64_t    totalDedupSize;          /* total size after de-dup */
} tsmEndSendObjExOut_t;

#define tsmEndSendObjExOutVersion 3

/*-----+
| Type definition for tsmGroupHandlerIn_t
+-----*/
typedef struct tsmGroupHandlerIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt32_t      tsmHandle;                /* handle for session */
    dsUInt8_t       groupType;                /* Type of group */
    dsUInt8_t       actionType;               /* Type of group operation */
    dsUInt8_t       memberType;              /* Type of member: Leader or member */
    dsStruct64_t    leaderObjId;              /* OBJID of the groupleader */
    dsChar_t        *uniqueGroupTagP;         /* Unique group identifier */
    tsmObjName      *objNameP;               /* group leader object name */
    dsmGetList      memberObjList;           /* list of objects to remove, assign */
} tsmGroupHandlerIn_t;

#define tsmGroupHandlerInVersion 1

/*-----+
| Type definition for tsmGroupHandlerExOut_t
+-----*/
typedef struct tsmGroupHandlerOut_t
{
    dsUInt16_t      stVersion;                /* structure version */
} tsmGroupHandlerOut_t;

#define tsmGroupHandlerOutVersion 1

/*-----+
| Type definition for tsmEndTxnExIn_t
+-----*/
typedef struct tsmEndTxnExIn_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt32_t      tsmHandle;                /* handle for session */
    dsUInt8_t       vote;
} tsmEndTxnExIn_t;

#define tsmEndTxnExInVersion 1

/*-----+
| Type definition for tsmEndTxnExOut_t
+-----*/
typedef struct tsmEndTxnExOut_t
{
    dsUInt16_t      stVersion;                /* structure version */
    dsUInt16_t      reason;                   /* reason code */
    dsStruct64_t    groupleaderObjId;         /* groupleader obj id returned on */
    /* DSM_ACTION_OPEN */
    dsUInt8_t       reserved1;                /* future use */
    dsUInt16_t      reserved2;                /* future use */
} tsmEndTxnExOut_t;

```

```

#define tsmEndTxnExOutVersion 1

/*-----+
| Type definition for tsmEndGetDataExIn_t
+-----*/
typedef struct tsmEndGetDataExIn_t
{
    dsUint16_t      stVersion;    /* structure version */
    dsUint32_t      tsmHandle;    /* handle for session */
}tsmEndGetDataExIn_t;

#define tsmEndGetDataExInVersion 1

/*-----+
| Type definition for tsmEndGetDataExOut_t
+-----*/
typedef struct tsmEndGetDataExOut_t
{
    dsUint16_t      stVersion;    /* structure version */
    dsUint16_t      reason;       /* reason code */
    dsStruct64_t    totalLFBytesRecv; /* total lan free bytes recieved */
}tsmEndGetDataExOut_t;

#define tsmEndGetDataExOutVersion 1

/*-----+
| Type definition for on tsmRetentionEvent()
+-----*/
typedef struct tsmRetentionEventIn_t
{
    dsUint16_t      stVersion;    /* structure version */
    dsUint32_t      tsmHandle;    /* session Handle */
    dsmEventType_t  eventType;    /* Event type */
    dsmObjList_t    objList;      /* object ID */
}tsmRetentionEventIn_t;

#define tsmRetentionEventInVersion 1

/*-----+
| Type definition for on tsmRetentionEvent()
+-----*/
typedef struct tsmRetentionEventOut_t
{
    dsUint16_t      stVersion;    /* structure version */
}tsmRetentionEventOut_t;

#define tsmRetentionEventOutVersion 1

/*-----+
| Type definition for tsmUpdateObjExIn_t
+-----*/
typedef struct tsmUpdateObjExIn_t
{
    dsUint16_t      stVersion;    /* structure version */
    dsUint32_t      tsmHandle;    /* session Handle */
    tsmSendType     sendType;     /* send type back/arch */
    dsChar_t        *descrP;      /* archive description */
    tsmObjName      *objNameP;    /* objName */
    tsmObjAttr      *objAttrPtr;  /* attribute */
    dsUint32_t      objUpdAct;    /* update action */
    ObjID           archObjId;    /* objId for archive */
}tsmUpdateObjExIn_t;

#define tsmUpdateObjExInVersion 1

/*-----+
| Type definition for tsmUpdateObjExOut_t
+-----*/
typedef struct tsmUpdateObjExOut_t
{
    dsUint16_t      stVersion;    /* structure version */
}tsmUpdateObjExOut_t;

#define tsmUpdateObjExOutVersion 1

#if _OPSYS_TYPE == DS_WINNT
#pragma pack()
#endif

#ifdef _MAC

```



```

/* added for the extended restore order */
typedef struct
{
    dsUInt32_t top;
    dsUInt32_t hi_hi;
    dsUInt32_t hi_lo;
    dsUInt32_t lo_hi;
    dsUInt32_t lo_lo;
} dsUInt160_t ;

#if defined(_LONG_LONG)
    typedef __int64          dsInt64_t;
    typedef unsigned __int64 dsUInt64_t;
    /*=== A "true" unsigned 64-bit integer ===*/
    typedef __int64          dsLongLong_t;
#else
typedef struct tagUINT64_t
{
    dsUInt32_t hi;          /* Most significant 32 bits. */
    dsUInt32_t lo;          /* Least significant 32 bits. */
} dsUInt64_t;
#endif

/*-----+
| Type definition for bool_t
+-----*/
/*
 * Had to create a Boolean type that didn't clash with any other predefined
 * version in any operating system or windowing system.
 */
typedef enum
{
    dsmFalse = 0x00,
    dsmTrue  = 0x01
} dsmBool_t ;

/*=== for backward compatability ===*/
#define uint8      dsUInt8_t
#define int8       dsInt8_t
#define uint16     dsUInt16_t
#define int16      dsInt16_t
#define uint32     dsUInt32_t
#define int32      dsInt32_t
#define uint64     dsStruct64_t
#define bool_t     dsBool_t
#define dsBool_t   dsmBool_t
#define bTrue      dsmTrue
#define bFalse     dsmFalse

typedef struct
{
    dsUInt32_t hi;          /* Most significant 32 bits. */
    dsUInt32_t lo;          /* Least significant 32 bits. */
} dsStruct64_t ;

#endif /* DSMAPILIB */

#ifndef _WIN64
#pragma pack()
#endif
#endif /* _H_DSMAPIPS */

```

## release.h

```

/*****
 * IBM Storage Protect
 * Common Source Component
 *
 * (C) Copyright IBM Corporation 1993,2018
 *****/

/*****
 * Header File Name: release.h
 *
 * Environment: *****/

```

```

*                ** This is a platform-independent source file **
*                ****
*
* Design Notes:   This file contains the common information about
*                the actual version.release.level.sublevel
*
* Descriptive-name: Definitions for IBM Storage Protect version
*
* Note: This file should contain no LOG or CMVC information. It is
*       shipped with the API code.
*
*-----*/

#ifndef _H_RELEASE
#define _H_RELEASE

#define COMMON_VERSION      8
#define COMMON_RELEASE      1
#define COMMON_LEVEL       13
#define COMMON_SUBLEVEL     0
#define COMMON_DRIVER       dsTEXT("")

#define COMMON_VERSIONTXT "8.1.13.0"

#define SHIPYEARTXT ""
#define SHIPYEARTXTW dsTEXT("2021")
#define TSMPRODTXT "IBM Storage Protect"

/*=====
The following string definitions are used for VERSION information
and should not be converted to dsTEXT or osTEXT. They are used
only at link time.

These are also used when the Jar file is built on Unix. See the
the perl script tools/unx/mzbuild/createReleaseJava
=====*/
#define COMMON_VERSION_STR "8"
#define COMMON_RELEASE_STR "1"
#define COMMON_LEVEL_STR "13"
#define COMMON_SUBLEVEL_STR "0"
#define COMMON_DRIVER_STR ""

/*=== product names definitions ===*/
#define COMMON_NAME_DFDSM 1
#define COMMON_NAME_ADSM 2
#define COMMON_NAME_TSM 3
#define COMMON_NAME_ITSM 4
#define COMMON_NAME COMMON_NAME_ITSM

/*=====
Internal version, release, and level (build) version. This
should be unique for every version+release+ptf of a product.
This information is recorded in the file attributes and data
stream for diagnostic purposes.
NOTE: DO NOT MODIFY THESE VALUES. YOU CAN ONLY ADD NEW ENTRIES!
=====*/
#define COMMON_BUILD_TSM_510 1
#define COMMON_BUILD_TSM_511 2
#define COMMON_BUILD_TSM_515 3
#define COMMON_BUILD_TSM_516 4
#define COMMON_BUILD_TSM_520 5
#define COMMON_BUILD_TSM_522 6
#define COMMON_BUILD_TSM_517 7
#define COMMON_BUILD_TSM_523 8
#define COMMON_BUILD_TSM_530 9
#define COMMON_BUILD_TSM_524 10
#define COMMON_BUILD_TSM_532 11
#define COMMON_BUILD_TSM_533 12
#define COMMON_BUILD_TSM_525 13
#define COMMON_BUILD_TSM_534 14
#define COMMON_BUILD_TSM_540 15
#define COMMON_BUILD_TSM_535 16
#define COMMON_BUILD_TSM_541 17
#define COMMON_BUILD_TSM_550 18
#define COMMON_BUILD_TSM_542 19
#define COMMON_BUILD_TSM_551 20
#define COMMON_BUILD_TSM_610 21
#define COMMON_BUILD_TSM_552 22
#define COMMON_BUILD_TSM_611 23
#define COMMON_BUILD_TSM_543 24
#define COMMON_BUILD_TSM_620 25
#define COMMON_BUILD_TSM_612 26

```

```

#define COMMON_BUILD_TSM_553 27
#define COMMON_BUILD_TSM_613 28
#define COMMON_BUILD_TSM_621 29
#define COMMON_BUILD_TSM_622 30
#define COMMON_BUILD_TSM_614 31
#define COMMON_BUILD_TSM_623 32
#define COMMON_BUILD_TSM_630 33
#define COMMON_BUILD_TSM_615 34
#define COMMON_BUILD_TSM_624 35
#define COMMON_BUILD_TSM_631 36
#define COMMON_BUILD_TSM_640 37
#define COMMON_BUILD_TSM_710 38
#define COMMON_BUILD_TSM_625 39
#define COMMON_BUILD_TSM_641 40
#define COMMON_BUILD_TSM_711 41
#define COMMON_BUILD_TSM_712 42
#define COMMON_BUILD_TSM_713 43
#define COMMON_BUILD_TSM_714 44
#define COMMON_BUILD_TSM_720 45
#define COMMON_BUILD_TSM_721 46
#define COMMON_BUILD_TSM_642 47
#define COMMON_BUILD_TSM_643 48
#define COMMON_BUILD_TSM_715 49
#define COMMON_BUILD_TSM_716 50
#define COMMON_BUILD_TSM_810 51
#define COMMON_BUILD_TSM_811 52
#define COMMON_BUILD_TSM_812 53
#define COMMON_BUILD_TSM_718 54
#define COMMON_BUILD_TSM_814 55
#define COMMON_BUILD_TSM_816 56
#define COMMON_BUILD_TSM_817 57
#define COMMON_BUILD_TSM_818 58
#define COMMON_BUILD_TSM_819 59
#define COMMON_BUILD_TSM_8110 60
#define COMMON_BUILD_TSM_8111 61
#define COMMON_BUILD_TSM_8112 62
#define COMMON_BUILD_TSM_8113 63
#define COMMON_BUILD COMMON_BUILD_TSM_8113

/*=== define VRL as an Int for bitmap version compares ===*/
static const int VRL_712 = 712;
static const int VRL_713 = 713;
static const int VRL_714 = 714;
static const int VRL_715 = 715;
static const int VRL_716 = 716;
static const int VRL_718 = 718;
static const int VRL_810 = 810;
static const int VRL_811 = 811;
static const int VRL_812 = 812;
static const int VRL_814 = 814;
static const int VRL_816 = 816;
static const int VRL_817 = 817;
static const int VRL_818 = 818;
static const int VRL_819 = 819;
static const int VRL_8110 = 8110;
static const int VRL_8111 = 8111;
static const int VRL_8112 = 8112;
static const int VRL_8113 = 8113;

#define TDP4VE_PLATFORM_STRING_MBCS "TDP VMware"
#define TDP4VE_PLATFORM_STRING dsTEXT("TDP VMware")

#define TDP4HYPERV_PLATFORM_STRING_MBCS "TDP HyperV"
#define TDP4HYPERV_PLATFORM_STRING dsTEXT("TDP HyperV")

#endif /* _H_RELEASE */

```



## Appendix C. API function definitions source file

This appendix contains the `dsmapifp.h` header file, so you can see the function definitions for the API.

**Note: DSMLINKAGE** is defined differently for each operating system. See the definitions in the `dsmapifp.h` file for your specific operating system.

The information that is provided here contains a point-in-time copy of the files that are distributed with the API. View the files in the API distribution package for the latest version.

### dsmapifp.h

```
/*
 * IBM Storage Protect
 * API Client Component
 *
 * IBM Confidential
 * (IBM Confidential-Restricted when combined with the Aggregated OCO
 * source modules for this program)
 *
 * OCO Source Materials
 *
 * 5648-020 (C) Copyright IBM Corporation 1993, 2016
 */

/*
 * Header File Name: dsmapifp.h
 *
 * Descriptive-name: IBM Storage Protect API function prototypes
 */
#ifndef _H_DSMAPIFP
#define _H_DSMAPIFP

#ifdef __cplusplus
extern "C" {
#endif

#ifdef DYNALOAD_DSMAPI

/* function will be dynamically loaded */
#include "dsmapidl.h"

#else

/* functions will be implicitly loaded from library */

/*=====*/
/*          P U B L I C   F U N C T I O N S          */
/*=====*/

extern dsInt16_t DSMLINKAGE dsmBeginGetData(
    dsUInt32_t      dsmHandle,
    dsBool_t        mountWait,
    dsmGetType      getType,
    dsmGetList      *dsmGetObjListP
);

extern dsInt16_t DSMLINKAGE dsmBeginQuery(
    dsUInt32_t      dsmHandle,
    dsmQueryType    queryType,
    dsmQueryBuff    *queryBuffer
);

extern dsInt16_t DSMLINKAGE dsmBeginTxn(
    dsUInt32_t      dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmBindMC(
    dsUInt32_t      dsmHandle,
    dsmObjName      *objNameP,
    dsmSendType     sendType,
    mcBindKey       *mcBindKeyP
);

#endif
}
```

```

extern dsInt16_t DSMLINKAGE dsmChangePW(
    dsUInt32_t
    char
    char
    dsmHandle,
    *oldPW,
    *newPW
);

extern dsInt16_t DSMLINKAGE dsmCleanUp(
    dsBool_t
    mtFlag
);

extern dsInt16_t DSMLINKAGE dsmDeleteAccess(
    dsUInt32_t
    dsUInt32_t
    dsmHandle,
    ruleNum
);

extern dsInt16_t DSMLINKAGE dsmDeleteObj(
    dsUInt32_t
    dsUInt32_t
    dsmDelType
    dsmDelInfo
    dsmHandle,
    delType,
    delInfo
);

extern dsInt16_t DSMLINKAGE dsmDeleteFS(
    dsUInt32_t
    char
    dsUInt8_t
    dsmHandle,
    *fsName,
    repository
);

extern dsInt16_t DSMLINKAGE dsmEndGetData(
    dsUInt32_t
    dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndGetDataEx(
    dsInt16_t DSMLINKAGE dsmEndGetDataExIn_t *dsmEndGetDataExInP,
    dsInt16_t DSMLINKAGE dsmEndGetDataExOut_t *dsmEndGetDataExOutP
);

extern dsInt16_t DSMLINKAGE dsmEndGetObj(
    dsUInt32_t
    dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndQuery(
    dsUInt32_t
    dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndSendObj(
    dsUInt32_t
    dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmEndSendObjEx(
    dsInt16_t DSMLINKAGE dsmEndSendObjExIn_t *dsmEndSendObjExInP,
    dsInt16_t DSMLINKAGE dsmEndSendObjExOut_t *dsmEndSendObjExOutP
);

extern dsInt16_t DSMLINKAGE dsmEndTxnEx(
    dsInt16_t DSMLINKAGE dsmEndTxnExIn_t *dsmEndTxnExInP,
    dsInt16_t DSMLINKAGE dsmEndTxnExOut_t *dsmEndTxnExOutP
);

extern dsInt16_t DSMLINKAGE dsmEndTxn(
    dsUInt32_t
    dsUInt8_t
    dsUInt16_t
    dsmHandle,
    vote,
    *reason
);

extern dsInt16_t DSMLINKAGE dsmGetData(
    dsUInt32_t
    DataBlk
    dsmHandle,
    *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmGetBufferData(
    dsInt16_t DSMLINKAGE getBufferDataIn_t *dsmGetBufferDataInP,
    dsInt16_t DSMLINKAGE getBufferDataOut_t *dsmGetBufferDataOutP
);

extern dsInt16_t DSMLINKAGE dsmGetNextQObj(
    dsUInt32_t
    DataBlk
    dsmHandle,
    *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmGetObj(

```

```

        dsUInt32_t
        ObjID
        DataBlk
    );

extern dsInt16_t DSMLINKAGE dsmGroupHandler(
    dsmGroupHandlerIn_t
    dsmGroupHandlerOut_t
);

extern dsInt16_t DSMLINKAGE dsmInit(
    dsUInt32_t
    dsmApiVersion
    char
    char
    char
    char
    char
    char
);

extern dsInt16_t DSMLINKAGE dsmInitEx(
    dsUInt32_t
    dsmInitExIn_t
    dsmInitExOut_t
);

extern dsInt16_t DSMLINKAGE dsmLogEvent(
    dsUInt32_t
    logInfo
);

extern dsInt16_t DSMLINKAGE dsmLogEventEx(
    dsUInt32_t
    dsmLogExIn_t
    dsmLogExOut_t
);

extern dsInt16_t DSMLINKAGE dsmQueryAccess(
    dsUInt32_t
    qryRespAccessData
    dsUInt16_t
);

extern void DSMLINKAGE dsmQueryApiVersion(
    dsmApiVersion
);

extern void DSMLINKAGE dsmQueryApiVersionEx(
    dsmApiVersionEx
);

extern dsInt16_t DSMLINKAGE dsmQueryCliOptions(
    optStruct
);

extern dsInt16_t DSMLINKAGE dsmQuerySessInfo(
    dsUInt32_t
    ApiSessInfo
);

extern dsInt16_t DSMLINKAGE dsmQuerySessOptions(
    dsUInt32_t
    optStruct
);

extern dsInt16_t DSMLINKAGE dsmRCMsg(
    dsUInt32_t
    dsInt16_t
    char
);

extern dsInt16_t DSMLINKAGE dsmRegisterFS(
    dsUInt32_t
    regFSData
);

extern dsInt16_t DSMLINKAGE dsmReleaseBuffer(
    dsInt16_t
    releaseBufferIn_t
    releaseBufferOut_t
);

```

```

extern dsInt16_t DSMLINKAGE dsmRenameObj(
    dsmRenameIn_t
    dsmRenameOut_t
    *dsmRenameInP,
    *dsmRenameOutP
);

extern dsInt16_t DSMLINKAGE dsmRequestBuffer(
    requestBufferIn_t
    requestBufferOut_t
    *dsmRequestBufferInP,
    *dsmRequestBufferOutP
);

extern dsInt16_t DSMLINKAGE dsmRetentionEvent(
    dsmRetentionEventIn_t
    dsmRetentionEventOut_t
    *dsmRetentionEventInP,
    *dsmRetentionEventOutP
);

extern dsInt16_t DSMLINKAGE dsmSendBufferData(
    sendBufferDataIn_t
    sendBufferDataOut_t
    *dsmSendBufferDataInP,
    *dsmSendBufferDataOutP
);

extern dsInt16_t DSMLINKAGE dsmSendData(
    dsUInt32_t
    DataBlk
    dsmHandle,
    *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmSendObj(
    dsUInt32_t
    dsmSendType
    void
    dsmObjName
    ObjAttr
    DataBlk
    dsmHandle,
    sendType,
    *sendBuff,
    *objNameP,
    *objAttrPtr,
    *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE dsmSetAccess(
    dsUInt32_t
    dsmAccessType
    dsmObjName
    char
    char
    dsmHandle,
    accessType,
    *objNameP,
    *node,
    *owner
);

extern dsInt16_t DSMLINKAGE dsmSetUp(
    dsBool_t
    envSetUp
    mtFlag,
    *envSetUpP
);

extern dsInt16_t DSMLINKAGE dsmTerminate(
    dsUInt32_t
    dsmHandle
);

extern dsInt16_t DSMLINKAGE dsmUpdateFS(
    dsUInt32_t
    char
    dsmFSUpd
    dsUInt32_t
    dsmHandle,
    *fs,
    *fsUpdP,
    fsUpdAct
);

extern dsInt16_t DSMLINKAGE dsmUpdateObj(
    dsUInt32_t
    dsmSendType
    void
    dsmObjName
    ObjAttr
    dsUInt32_t
    dsmHandle,
    sendType,
    *sendBuff,
    *objNameP,
    *objAttrPtr,
    objUpdAct
);

extern dsInt16_t DSMLINKAGE dsmUpdateObjEx(
    dsmUpdateObjExIn_t
    dsmUpdateObjExOut_t
    *dsmUpdateObjExInP,
    *dsmUpdateObjExOutP
);

#endif /* ifdef DYNALOAD */

#ifdef __cplusplus
}
#endif

```

```
#endif /* _H_DSMAPIFP */
```

## tsmapifp.h

This section contains the function definitions for the API. It is a copy of the tsmapifp.h header file.

**Note: DSMLINKAGE** is defined differently for each operating system. See the definitions in the tsmapips.h file for your specific operating system.

```

/*****
* IBM Storage Protect
* API Client Component
*
* IBM Confidential
* (IBM Confidential-Restricted when combined with the Aggregated OCO
* source modules for this program)
*
* OCO Source Materials
*
* 5648-020 (C) Copyright IBM Corporation 1993, 2016
*****/

/*****/
/* Header File Name: tsmapifp.h */
/*
/* Descriptive-name: IBM Storage Protect API function prototypes */
/*****/
#ifndef _H_TSMAPIFP
#define _H_TSMAPIFP

#if defined(__cplusplus)
extern "C" {
#endif

#ifdef DYNALOAD_DSMAPI

/* function will be dynamically loaded */
#include "dsmapidl.h"

#else

/* functions will be implicitly loaded from library */

/*****
* P U B L I C   F U N C T I O N S
*****/

typedef void tsmQueryBuff;

extern dsInt16_t DSMLINKAGE tsmBeginGetData(
    dsUInt32_t tsmHandle,
    dsBool_t mountWait,
    tsmGetType_t getType,
    dsmGetList_t *dsmGetObjListP
);

extern dsInt16_t DSMLINKAGE tsmBeginQuery(
    dsUInt32_t tsmHandle,
    tsmQueryType_t queryType,
    tsmQueryBuff_t *queryBuffer
);

extern dsInt16_t DSMLINKAGE tsmBeginTxn(
    dsUInt32_t tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmBindMC(
    dsUInt32_t tsmHandle,
    tsmObjName_t *objNameP,
    tsmSendType_t sendType,
    tsmMcBindKey_t *mcBindKeyP
);

extern dsInt16_t DSMLINKAGE tsmChangePW(
    dsUInt32_t tsmHandle,
    dsChar_t oldPW,

```

```

);      dsChar_t          *newPW

extern dsInt16_t DSMLINKAGE tsmCleanUp(
);      dsBool_t          mtFlag

extern dsInt16_t DSMLINKAGE tsmDeleteAccess(
);      dsUint32_t        tsmHandle,
        dsUint32_t        ruleNum

extern dsInt16_t DSMLINKAGE tsmDeleteObj(
);      dsUint32_t        tsmHandle,
        tsmDelType        delType,
        tsmDelInfo        delInfo

extern dsInt16_t DSMLINKAGE tsmDeleteFS(
);      dsUint32_t        tsmHandle,
        dsChar_t          *fsName,
        dsUint8_t         repository

extern dsInt16_t DSMLINKAGE tsmEndGetData(
);      dsUint32_t        tsmHandle

extern dsInt16_t DSMLINKAGE tsmEndGetDataEx(
);      tsmEndGetDataExIn_t *tsmEndGetDataExInP,
        tsmEndGetDataExOut_t *tsmEndGetDataExOutP

extern dsInt16_t DSMLINKAGE tsmEndGetObj(
);      dsUint32_t        tsmHandle

extern dsInt16_t DSMLINKAGE tsmEndQuery(
);      dsUint32_t        tsmHandle

extern dsInt16_t DSMLINKAGE tsmEndSendObj(
);      dsUint32_t        tsmHandle

extern dsInt16_t DSMLINKAGE tsmEndSendObjEx(
);      tsmEndSendObjExIn_t *tsmEndSendObjExInP,
        tsmEndSendObjExOut_t *tsmEndSendObjExOutP

extern dsInt16_t DSMLINKAGE tsmEndTxn(
);      dsUint32_t        tsmHandle,
        dsUint8_t         vote,
        dsUint16_t        *reason

extern dsInt16_t DSMLINKAGE tsmEndTxnEx(
);      tsmEndTxnExIn_t    *tsmEndTxnExInP,
        tsmEndTxnExOut_t    *tsmEndTxnExOutP

extern dsInt16_t DSMLINKAGE tsmGetData(
);      dsUint32_t        tsmHandle,
        DataBlk*dataBlkPtr

extern dsInt16_t DSMLINKAGE tsmGetBufferData(
);      getBufferDataIn_t  *tsmGetBufferDataInP,
        getBufferDataOut_t *tsmGetBufferDataOutP

extern dsInt16_t DSMLINKAGE tsmGetNextQObj(
);      dsUint32_t        tsmHandle,
        DataBlk*dataBlkPtr

extern dsInt16_t DSMLINKAGE tsmGetObj(
);      dsUint32_t        tsmHandle,
        ObjID             *objIDP,
        DataBlk           *dataBlkPtr
);

```

```

extern dsInt16_t DSMLINKAGE tsmGroupHandler(
    tsmGroupHandlerIn_t
    tsmGroupHandlerOut_t
);

extern dsInt16_t DSMLINKAGE tsmInitEx(
    dsUInt32_t
    tsmInitExIn_t
    tsmInitExOut_t
);

extern dsInt16_t DSMLINKAGE tsmLogEventEx(
    dsUInt32_t
    tsmLogExIn_t
    tsmLogExOut_t
);

extern dsInt16_t DSMLINKAGE tsmQueryAccess(
    dsUInt32_t
    tsmQryRespAccessData
    dsUInt16_t
);

extern void DSMLINKAGE tsmQueryApiVersionEx(
    tsmApiVersionEx
);

extern dsInt16_t DSMLINKAGE tsmQueryCliOptions(
    tsmOptStruct
);

extern dsInt16_t DSMLINKAGE tsmQuerySessInfo(
    dsUInt32_t
    tsmApiSessInfo
);

extern dsInt16_t DSMLINKAGE tsmQuerySessOptions(
    dsUInt32_t
    tsmOptStruct
);

extern dsInt16_t DSMLINKAGE tsmRCMsg(
    dsUInt32_t
    dsInt16_t
    dsChar_t
);

extern dsInt16_t DSMLINKAGE tsmRegisterFS(
    dsUInt32_t
    tsmRegFSData
);

extern dsInt16_t DSMLINKAGE tsmReleaseBuffer(
    releaseBufferIn_t
    releaseBufferOut_t
);

extern dsInt16_t DSMLINKAGE tsmRenameObj(
    tsmRenameIn_t
    tsmRenameOut_t
);

extern dsInt16_t DSMLINKAGE tsmRequestBuffer(
    requestBufferIn_t
    requestBufferOut_t
);

extern dsInt16_t DSMLINKAGE tsmRetentionEvent(
    tsmRetentionEventIn_t
    tsmRetentionEventOut_t
);

extern dsInt16_t DSMLINKAGE tsmSendBufferData(
    sendBufferDataIn_t
    sendBufferDataOut_t
);

extern dsInt16_t DSMLINKAGE tsmSendData(
    dsUInt32_t

```

```

);
DataBlk          *dataBlkPtr

extern dsInt16_t DSMLINKAGE tsmSendObj(
    dsUInt32_t      tsmHandle,
    tsmSendType     sendType,
    void            *sendBuff,
    tsmObjName      *objNameP,
    tsmObjAttr      *objAttrPtr,
    DataBlk         *dataBlkPtr
);

extern dsInt16_t DSMLINKAGE tsmSetAccess(
    dsUInt32_t      tsmHandle,
    tsmAccessType   accessType,
    tsmObjName      *objNameP,
    dsChar_t        *node,
    dsChar_t        *owner
);

extern dsInt16_t DSMLINKAGE tsmSetUp(
    dsBool_t        mtFlag,
    tsmEnvSetUp     *envSetUpP
);

extern dsInt16_t DSMLINKAGE tsmTerminate(
    dsUInt32_t      tsmHandle
);

extern dsInt16_t DSMLINKAGE tsmUpdateFS(
    dsUInt32_t      tsmHandle,
    dsChar_t        *fs,
    tsmFSUpd        *fsUpdP,
    dsUInt32_t      fsUpdAct
);

extern dsInt16_t DSMLINKAGE tsmUpdateObj(
    dsUInt32_t      tsmHandle,
    tsmSendType     sendType,
    void            *sendBuff,
    tsmObjName      *objNameP,
    tsmObjAttr      *objAttrPtr,
    dsUInt32_t      objUpdAct
);

extern dsInt16_t DSMLINKAGE tsmUpdateObjEx(
    tsmUpdateObjExIn_t *tsmUpdateObjExInP,
    tsmUpdateObjExOut_t *tsmUpdateObjExOutP
);

#endif /* ifdef DYNALOAD */
#ifdef __cplusplus
}
#endif
#endif /* _H_TSMAPIFP */

```



---

## Appendix D. Accessibility features for the IBM Storage Protect product family

Accessibility features assist users who have a disability, such as restricted mobility or limited vision, to use information technology content successfully.

### Overview

The IBM Storage Protect family of products includes the following major accessibility features:

- Keyboard-only operation
- Operations that use a screen reader

The IBM Storage Protect family of products uses the latest W3C Standard, WAI-ARIA 1.0 ([www.w3.org/TR/wai-aria/](http://www.w3.org/TR/wai-aria/)), to ensure compliance with US Section 508 and Web Content Accessibility Guidelines (WCAG) 2.0 ([www.w3.org/TR/WCAG20/](http://www.w3.org/TR/WCAG20/)). To take advantage of accessibility features, use the latest release of your screen reader and the latest web browser that is supported by the product.

The product documentation in IBM Documentation is enabled for accessibility.

### Keyboard navigation

This product uses standard navigation keys.

### Interface information

User interfaces do not have content that flashes 2 - 55 times per second.

Web user interfaces rely on cascading style sheets to render content properly and to provide a usable experience. The application provides an equivalent way for low-vision users to use system display settings, including high-contrast mode. You can control font size by using the device or web browser settings.

Web user interfaces include WAI-ARIA navigational landmarks that you can use to quickly navigate to functional areas in the application.

### Vendor software

The IBM Storage Protect product family includes certain vendor software that is not covered under the IBM license agreement. IBM makes no representation about the accessibility features of these products. Contact the vendor for accessibility information about its products.

### Related accessibility information

In addition to standard IBM help desk and support websites, IBM has a TTY telephone service for use by deaf or hard of hearing customers to access sales and support services:

TTY service  
800-IBM-3383 (800-426-3383)  
(within North America)

For more information about the commitment that IBM has to accessibility, see [IBM Accessibility](http://www.ibm.com/able) ([www.ibm.com/able](http://www.ibm.com/able)).



## Notices

---

This information was developed for products and services offered in the US. This material might be available from IBM in other languages. However, you may be required to own a copy of the product or product version in that language in order to access it.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

*IBM Director of Licensing  
IBM Corporation  
North Castle Drive, MD-NC119  
Armonk, NY 10504-1785  
US*

For license inquiries regarding double-byte character set (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

*Intellectual Property Licensing  
Legal and Intellectual Property Law  
IBM Japan Ltd.  
19-21, Nihonbashi-Hakozakicho, Chuo-ku  
Tokyo 103-8510, Japan*

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

*IBM Director of Licensing  
IBM Corporation  
North Castle Drive, MD-NC119  
Armonk, NY 10504-1785  
US*

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

The performance data discussed herein is presented as derived under specific operating conditions. Actual results may vary.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

#### **COPYRIGHT LICENSE:**

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Each copy or any portion of these sample programs or any derivative work must include a copyright notice as follows: © (your company name) (year). Portions of this code are derived from IBM Corp. Sample Programs. © Copyright IBM Corp. \_enter the year or years\_.

#### **Trademarks**

IBM, the IBM logo, and [ibm.com](http://ibm.com)® are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at [www.ibm.com/legal/copytrade.shtml](http://www.ibm.com/legal/copytrade.shtml).

Adobe is a registered trademark of Adobe Systems Incorporated in the United States, and/or other countries.

Linear Tape-Open, LTO, and Ultrium are trademarks of HP, IBM Corp. and Quantum in the U.S. and other countries.

Intel and Itanium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

The registered trademark Linux is used pursuant to a sublicense from the Linux Foundation, the exclusive licensee of Linus Torvalds, owner of the mark on a worldwide basis.

Microsoft, Windows, and Windows NT are trademarks of Microsoft Corporation in the United States, other countries, or both.

Java™ and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Red Hat®, OpenShift®, Ansible®, and Ceph® are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

UNIX is a registered trademark of The Open Group in the United States and other countries.

VMware, VMware vCenter Server, and VMware vSphere are registered trademarks or trademarks of VMware, Inc. or its subsidiaries in the United States and/or other jurisdictions.

## **Terms and conditions for product documentation**

Permissions for the use of these publications are granted subject to the following terms and conditions.

### **Applicability**

These terms and conditions are in addition to any terms of use for the IBM website.

### **Personal use**

You may reproduce these publications for your personal, noncommercial use provided that all proprietary notices are preserved. You may not distribute, display or make derivative work of these publications, or any portion thereof, without the express consent of IBM.

### **Commercial use**

You may reproduce, distribute and display these publications solely within your enterprise provided that all proprietary notices are preserved. You may not make derivative works of these publications, or reproduce, distribute or display these publications or any portion thereof outside your enterprise, without the express consent of IBM.

### **Rights**

Except as expressly granted in this permission, no other permissions, licenses or rights are granted, either express or implied, to the publications or any information, data, software or other intellectual property contained therein.

IBM reserves the right to withdraw the permissions granted herein whenever, in its discretion, the use of the publications is detrimental to its interest or, as determined by IBM, the above instructions are not being properly followed.

You may not download, export or re-export this information except in full compliance with all applicable laws and regulations, including all United States export laws and regulations.

IBM MAKES NO GUARANTEE ABOUT THE CONTENT OF THESE PUBLICATIONS. THE PUBLICATIONS ARE PROVIDED "AS-IS" AND WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, AND FITNESS FOR A PARTICULAR PURPOSE.

## **Privacy policy considerations**

IBM Software products, including software as a service solutions, ("Software Offerings") may use cookies or other technologies to collect product usage information, to help improve the end user experience, to tailor interactions with the end user, or for other purposes. In many cases no personally identifiable information is collected by the Software Offerings. Some of our Software Offerings can help enable you to collect personally identifiable information. If this Software Offering uses cookies to collect personally identifiable information, specific information about this offering's use of cookies is set forth below.

This Software Offering does not use cookies or other technologies to collect personally identifiable information.

If the configurations deployed for this Software Offering provide you as customer the ability to collect personally identifiable information from end users via cookies and other technologies, you should seek your own legal advice about any laws applicable to such data collection, including any requirements for notice and consent.

For more information about the use of various technologies, including cookies, for these purposes, see IBM's Privacy Policy at <http://www.ibm.com/privacy> and IBM's Online Privacy Statement at <http://www.ibm.com/privacy/details> in the section entitled "Cookies, Web Beacons and Other Technologies," and the "IBM Software Products and Software-as-a-Service Privacy Statement" at <http://www.ibm.com/software/info/product-privacy>.



## Glossary

---

A glossary is available with terms and definitions for the IBM Storage Protect family of products.

See the [IBM Storage Protect glossary](#).





---

# Index

## Numerics

- 128-bit AES encryption support [41](#)
- 256-bit AES encryption support [41](#)
- 64-bit
  - compiling [1](#)
  - requirements [1](#)

## A

- access to objects
  - by user [21](#), [22](#)
- accessibility features [239](#)
- accessing to objects
  - across nodes [22](#)
- active copies of objects [32](#), [37](#)
- active version
  - deleting [65](#)
- administrative user
  - creating [19](#)
- administrator options [2](#)
- API
  - dsmInitEx
    - configuration file used by [2](#)
  - environment setup [3](#)
  - option string used by dsmInitEx [2](#)
  - overview [1](#)
  - sample applications [5](#)
  - using Unicode [71](#)
- API configuration file
  - used by dsmInitEx [15](#)
- API options list
  - used by dsmInitEx [15](#)
- application type [15](#), [100](#), [103](#)
- application version [ix](#)
- archive copy group [24](#)
- archive files
  - how long retained [24](#)
- archive objects
  - expiration [26](#)
  - release [26](#)
  - suspend [26](#)
- archiveretentionprotection [27](#)
- archiving objects [32](#), [37](#)
- asnodename [70](#)
- authorization rule
  - dsmDeleteAccess function [84](#)
- authorized user [18](#), [21](#)
- automated client failover [48](#)

## B

- backing up objects [32](#), [37](#)
- backup
  - multiple nodes [70](#)
  - using client node proxy [70](#)
- backup copy group [24](#)

- backup-archive client
  - interoperability [67](#)
- buffer copy elimination
  - overview [39](#)
  - restore and retrieve [40](#)

## C

- callbuff
  - IBM Storage Protect data buffer sample API applications [5](#)
- callevnt
  - event-based retention [5](#)
- callhold
  - detention hold sample API applications [5](#)
- callmt\*
  - multithreaded sample API applications [5](#)
- callmt1.c
  - sample [14](#)
- callret
  - data retention protection sample API applications [5](#)
- capacity
  - file space [22](#)
- character sets [71](#)
- client node proxy support [70](#)
- client owner authority [19](#)
- client performance monitor [34](#)
- client performance monitor options
  - PERFCOMMTIMEOUT [36](#)
  - PERFMONTCPPORT [35](#)
  - PERFMONTCPSERVERADDRESS [35](#)
- client-side data deduplication [45](#)
- code pages [71](#)
- commands
  - makemtu [71](#)
- compatibility
  - between versions of API [12](#)
- compiling
  - Unicode [71](#)
- compressalways
  - option [7](#)
- compression [37](#), [57](#)
- compression type
  - LZ4 [38](#)
  - LZW [38](#)
- configuration file
  - API [2](#)
- configuration sources
  - priority sequence [2](#)
- copy group [24](#)
- CTRL+C [15](#)

## D

- dapi\*
  - single-threaded, interactive sample API applications [5](#)
- data deduplication [44](#)

- data deduplication files
  - exclude [47](#)
  - include [47](#)
- data protection [27](#)
- data retention [27](#)
- data structures
  - size limits [12](#), [31](#)
  - version control [13](#)
- data transfer
  - LAN-free [32](#)
- DB Chg operation [9](#)
- DBCS [71](#)
- delete archive [68](#)
- delete filespace [68](#)
- design recommendations [9](#)
- dir
  - object type [21](#)
- disability [239](#)
- double-byte character set [71](#)
- dscenu.txt [3](#)
- dsierror.log [3](#)
- DSM\_MAX\_PLATFORM\_LENGTH [15](#)
- dsm.opt
  - asnodename option [70](#)
  - enablearchiveretentionprotection [27](#)
  - encryptkey [41](#)
- dsm.sys
  - asnodename option [70](#)
  - enablearchiveretentionprotection [27](#)
  - encryptkey [41](#)
- dsmapifp.h
  - header file [73](#), [175](#), [231](#)
- dsmapi.h header file [140](#), [195](#)
- dsmapi.h header file [140](#), [195](#)
- dsmapi.h header file [140](#), [195](#)
- dsmApiVersion function
  - session [15](#)
- dsmBeginGetData function
  - buffer management [40](#)
  - code example [63](#)
  - dsmEndGetData function [87](#)
  - dsmTerminate function [87](#)
  - in flowchart [62](#)
  - overview [75](#)
  - return codes [76](#)
  - state diagram [61](#), [65](#)
  - syntax [75](#)
- dsmBeginQuery function
  - dsmEndQuery function [88](#)
  - dsmGetNextQObj function [94](#)
  - flowchart [28](#)
  - management class [25](#)
  - overview [76](#)
  - querying [28](#)
  - querying example [31](#)
  - receiving data [58](#)
  - return codes [80](#)
  - sending data example [32](#)
  - state diagram [28](#), [65](#)
  - syntax [77](#)
- dsmBeginTxn [22](#)
- dsmBeginTxn function
  - buffer copy elimination [39](#)
- dsmBeginTxn function (*continued*)
  - code example [53](#)
  - deleting objects [65](#)
  - deletion [26](#)
  - dsmEndTxn function [90](#)
  - dsmRenameObj function [115](#)
  - dsmRetentionEvent function [117](#)
  - expiration [26](#)
  - overview [81](#)
  - retention policy [28](#)
  - return codes [81](#)
  - state diagram [65](#)
  - syntax [81](#)
- dsmBindMC
  - example [25](#)
- dsmBindMC function
  - buffer copy elimination [39](#)
  - code example [53](#)
  - dsmSendObj function [120](#)
  - general description [53](#)
  - include-exclude list [24](#)
  - information returned by [25](#)
  - management classes [25](#)
  - object names [21](#)
  - overview [82](#)
  - return codes [82](#)
  - state diagram [65](#)
  - syntax [82](#)
- dsmChangePW
  - general description [65](#)
- dsmChangePW function
  - overview [83](#)
  - return codes [83](#)
  - session security [16](#)
  - state diagram [65](#)
  - syntax [83](#)
- dsmCleanUp function
  - dsmSetUp function [124](#)
  - multithreading [14](#)
  - overview [84](#)
  - signals [15](#)
  - syntax [84](#)
- dsmclientV3.cat [3](#)
- dsmDeleteAccess function
  - accessing objects [22](#)
  - overview [84](#)
  - syntax [84](#)
- dsmDeleteFS function
  - example code [22](#)
  - file spaces [22](#)
  - file system management [23](#)
  - overview [85](#)
  - return codes [85](#)
  - state diagram [65](#)
  - syntax [85](#)
- dsmDeleteObj function
  - deleting objects [65](#)
  - dsmEndTxn function [90](#)
  - dsmSendObj function
    - management class [9](#)
  - object naming [9](#)
  - objects [32](#), [37](#)
  - overview [86](#)
  - return codes [87](#)

- dsmDeleteObj function (*continued*)
  - state diagram [65](#)
  - syntax [86](#)
- dsmEndGetData**
  - stopping process [61](#)
- dsmEndGetData function
  - buffer management [40](#)
  - code example [63](#)
  - in flowchart [62](#)
  - LAN-free [32](#)
  - overview [87](#)
  - state diagram [61](#), [65](#)
  - syntax [87](#)
- dsmEndGetDataEx function
  - overview [87](#)
  - syntax [87](#)
- dsmEndGetObj function
  - buffer management [40](#)
  - code example [63](#)
  - dsmBeginGetData function [75](#)
  - in flowchart [62](#)
  - overview [88](#)
  - return codes [88](#)
  - state diagram [61](#), [65](#)
  - syntax [88](#)
- dsmEndQuery
  - general description [28](#)
- dsmEndQuery function
  - dsmGetNextQObj function [94](#)
  - flowchart [28](#)
  - overview [88](#)
  - querying the server [58](#)
  - state diagram [28](#), [65](#)
  - syntax [88](#)
- dsmEndSendObj function
  - code example [53](#)
  - dsmEndTxn function [90](#)
  - dsmSendData function [119](#)
  - dsmSendObj function [120](#)
  - flowchart [51](#)
  - overview [89](#)
  - return codes [89](#)
  - sending objects [36](#)
  - state diagram [51](#), [65](#)
  - syntax [89](#)
- dsmEndSendObjEx function
  - compression [37](#)
  - encryption [41](#)
  - LAN-free [32](#)
  - overview [89](#)
  - return codes [90](#)
  - syntax [89](#)
- dsmEndTxn function
  - buffer copy elimination [39](#)
  - code example [53](#)
  - deleting objects [65](#)
  - dsmEndTxnEx function [91](#)
  - dsmRenameObj function [115](#)
  - dsmRetentionEvent function [117](#)
  - dsmSendObj function [120](#)
  - file grouping [54](#)
  - flowchart [51](#)
  - overview [90](#)
  - return codes [91](#)

- dsmEndTxn function (*continued*)
  - simultaneous-write operations [32](#)
  - state diagram [51](#), [65](#)
  - syntax [90](#)
- dsmEndTxnEx function
  - file grouping [54](#)
  - overview [91](#)
  - return codes [92](#)
  - syntax [91](#)
- dsmEventType function
  - retention policy [28](#)
- dsmGetBufferData function
  - overview [93](#)
  - return codes [94](#)
  - syntax [94](#)
- dsmGetData [61](#)
- dsmGetData function
  - code example [63](#)
  - in flowchart [62](#)
  - in state diagram [61](#)
  - overview [93](#)
  - return codes [93](#)
  - state diagram [65](#)
  - syntax [93](#)
- dsmGetDataEx function
  - dsmReleaseBuffer function [114](#)
  - dsmRequestBuffer function [116](#)
- dsmGetList function
  - dsmGetObj function [97](#)
- dsmGetNextObj
  - dsmDeleteObj function [86](#)
- dsmGetNextQObj**
  - dsmEndQuery function [88](#)
- dsmGetNextQObj function
  - dsmRetentionEvent function [117](#)
  - flowchart [28](#)
  - overview [94](#)
  - querying example [31](#)
  - return codes [96](#)
  - state diagram [28](#), [65](#)
  - syntax [94](#)
- dsmGetObj
  - receiving objects [61](#)
- dsmGetObj function
  - code example [63](#)
  - dsmBeginGetData function [75](#)
  - dsmEndGetObj function [88](#)
  - dsmGetData function [93](#)
  - in flowchart [62](#)
  - overview [97](#)
  - return codes [98](#)
  - state diagram [61](#), [65](#)
  - syntax [97](#)
- dsmGroupHandler function
  - dsmEndTxnEx function [91](#)
  - file grouping [54](#)
  - overview [98](#)
  - return codes [99](#)
  - syntax [98](#)
- dsmgrp.c [57](#)
- dsmgrp\*
  - logical object grouping sample API applications [5](#)
- dsmHandle [110](#), [111](#)

- dsmHandle function
  - session [15](#)
- DSMI\_CONFIG environment variable [3](#)
- DSMI\_DIR
  - environment variable [6](#)
- DSMI\_DIR environment variable [3](#)
- DSMI\_LOG environment variable [3](#)
- dsmInit function
  - overview [99](#)
  - retention protection [27](#)
  - return codes [101](#)
  - syntax [99](#)
- dsmInitEx function
  - administrative user [19](#)
  - asnodename option [70](#)
  - dsmChangePW function [83](#)
  - dsmEndGetData function [87](#)
  - dsmGetBufferData function [93](#)
  - dsmGetNextQObj function [94](#)
  - dsmLogEvent function [106](#)
  - dsmQueryCliOptions function [110](#)
  - dsmQuerySessOptions [111](#)
  - dsmReleaseBuffer function [114](#)
  - dsmSetUp function [124](#)
  - encryption [41](#)
  - expired password [16](#)
  - interoperability [69](#)
  - multithreading [14](#)
  - option string [2](#)
  - overview [102](#)
  - retention protection [27](#)
  - return codes [105](#)
  - session [15](#)
  - session owner, set [21](#)
  - session security [16](#)
  - specifying options [2](#)
  - starting session [15](#)
  - state diagram [65](#)
  - syntax [102](#)
- dsmInitEx function
  - dsmQuerySessInfo function [110](#)
- dsmLogEvent function
  - overview [106](#)
  - return codes [106](#)
  - syntax [106](#)
- dsmLogEventEx function
  - overview [107](#)
  - return codes [108](#)
  - syntax [107](#)
- dsmQuery function
  - multiple nodes [70](#)
- dsmQueryAccess function
  - dsmDeleteAccess function [84](#)
  - overview [108](#)
- dsmQueryApiVersion function
  - overview [109](#)
  - state diagram [65](#)
  - syntax [109](#)
- dsmQueryApiVersionEx function
  - overview [109](#)
  - syntax [109](#)
  - version control [12](#)
- dsmQueryAPIVersionEx function
  - multithreading [14](#)
- dsmQueryCliOptions function
  - dsmQuerySessOptions [111](#)
  - overview [110](#)
  - session [15](#)
  - syntax [110](#)
- dsmQuerySessInfo
  - dsmDeleteFS function [85](#)
- dsmQuerySessInfo function
  - dsmRetentionEvent function [117](#)
  - general description [15](#)
  - overview [110](#)
  - return codes [111](#)
  - state diagram [65](#)
  - syntax [111](#)
- dsmQuerySessOptions function
  - overview [111](#)
  - syntax [111](#)
- dsmrc.h
  - header file [131](#), [185](#)
- dsmRCMsg function
  - overview [112](#)
  - return codes [113](#)
  - syntax [113](#)
- dsmRegisterFS function
  - example code [22](#)
  - file spaces [22](#)
  - overview [113](#)
  - return codes [114](#)
  - state diagram [65](#)
  - syntax [113](#)
- dsmReleaseBuffer function
  - dsmGetBufferData function [93](#)
  - dsmReleaseBuffer function [114](#)
  - dsmRequestBuffer function [116](#)
  - dsmSendBufferData function [118](#)
  - overview [114](#)
  - return codes [115](#)
  - syntax [114](#)
- dsmRenameObj function
  - overview [115](#)
  - return codes [116](#)
  - syntax [115](#)
- dsmRequestBuffer function
  - buffer copy elimination [39](#)
  - overview [116](#)
  - return codes [117](#)
  - syntax [116](#)
- dsmRetentionEvent function
  - deletion [26](#)
  - expiration [26](#)
  - overview [117](#)
  - retention policy [28](#)
  - return codes [118](#)
  - syntax [117](#)
- dsmSendBufferData function
  - buffer copy elimination [39](#)
  - overview [118](#)
  - return codes [119](#)
  - syntax [118](#)
- dsmSendData function
  - code example [53](#)
  - compression [37](#)
  - dsmEndSendObj function [89](#)
  - dsmEndTxn function [90](#)

dsmSendData function (*continued*)

- dsmSendObj function [120](#)
- flowchart [51](#)
- multithreading [14](#)
- overview [119](#)
- performance [32](#)
- return codes [120](#)
- sending objects [36](#)
- state diagram [51](#), [65](#)
- syntax [119](#)

dsmSendObj

- retention policy [28](#)

dsmSendObj function

- accessing objects [21](#)
- backup copy group [25](#)
- code example [53](#)
- compression [37](#)
- copy groups [25](#)
- dsmEndTxn function [90](#)
- flowchart [51](#)
- in state diagram [51](#)
- object naming [9](#)
- overview [120](#)
- retention policy [28](#)
- sending objects [36](#)
- state diagram [65](#)
- syntax [121](#)

dsmSendObjfunction

- deleting objects [65](#)

dsmSendType function

- updating objects [64](#)

dsmSetAccess function

- accessing objects [22](#)
- overview [123](#)
- return codes [123](#)
- syntax [123](#)

dsmSetUp function

- LAN-free [9](#), [32](#)
- multithread [14](#)
- multithreading [14](#), [32](#)
- overview [124](#)
- passwordaccess [18](#)
- syntax [125](#)

dsmTerminate [61](#)

dsmTerminate function

- buffer [39](#)
- buffer copy elimination [39](#)
- dsmInit function [99](#)
- dsmReleaseBuffer function [114](#)
- dsmRequestBuffer function [116](#)
- dsmSetUp function [124](#)
- general description [15](#)
- overview [126](#)
- session [15](#)
- signals [15](#)
- state diagram [65](#)
- syntax [126](#)

dsmUpdateFS function

- example code [22](#)
- file space management [23](#)
- file spaces [22](#)
- overview [126](#)
- return codes [127](#)
- state diagram [65](#)

dsmUpdateFS function (*continued*)

- syntax [126](#)

dsmUpdateObj function

- change management class [24](#)
- overview [127](#)
- return codes [128](#)
- syntax [128](#)

dsmUpdateObject(Ex) function

- updating objects [64](#)

dsmUpdateObjEx function

- change management class [24](#)
- overview [129](#)
- return codes [130](#)
- syntax [129](#)

## E

enablearchiveretentionprotection

- dsm.opt [27](#)
- dsm.sys [27](#)

encryption

- application managed [41](#)
- authentication setting [41](#)
- interoperability [69](#)
- transparent [43](#)

encryption and compression using buffer copy elimination [41](#)

encryptkey [41](#)

ending a session

- with dsmTerminate [15](#)

environment

- setting up API [3](#)

environment variables

- by operating system [3](#)
- DSMI\_CONFIG [3](#)
- DSMI\_DIR [3](#)
- DSMI\_LOG [3](#)

envSetUp [125](#)

errorlogretention

- when to use [65](#)

event

- eventRetentionActivate [28](#)

event logging [65](#)

event-based

- retention policy [28](#)

eventRetentionActivate event [28](#)

exclude data deduplication files [47](#)

exclude objects [20](#)

## F

failover

- overview [48](#)
- status information [49](#)

fast path [29](#)

fast path queries [77](#)

file aggregation [32](#)

file grouping [54](#)

file space

- capacity [22](#)
- deleting [22](#)
- managing [22](#)
- registering [22](#)

- file space management
  - dsmUpdateFS [23](#)
- file space name
  - file aggregation [32](#)
  - overview [20](#)
- file spaces
  - non-Unicode [71](#)
- file system management
  - dsmDeleteFS [23](#)
- files
  - configuration [1](#)
  - object type [21](#)
  - option [1](#)
- flowchart
  - backup and archive example [51](#)
  - restore and retrieve [61](#)
- fromowner option [22](#)
- function calls
  - short descriptions [73](#)
- function definitions, API [175](#), [179](#), [231](#), [235](#)

## G

- group leader [54](#)

## H

- header file dsmapi.h [140](#), [195](#)
- header file dsmapi.h [140](#), [195](#)
- header file release.h [140](#), [195](#)
- header file tsmapi.h [140](#), [195](#)
- header files
  - dsmapi.h [175](#), [231](#)
  - dsmrc.h [131](#), [185](#)
  - tsmapif.h [179](#), [235](#)
- high-level names
  - dsmRenameObj function [115](#)
- high-level qualifier [67](#)
- HP thread stack [14](#)

## I

- IBM Documentation vii
- inactive copies of objects [32](#), [37](#)
- include data deduplication files [47](#)
- include objects [20](#)
- include-exclude
  - file [126](#)
- include-exclude list [24](#), [72](#)
- InSession state [106](#), [107](#)
- interoperability
  - access to API objects [67](#)
  - backup-archive client [67](#)
  - commands [68](#)
  - conventions
    - UNIX or Linux [67](#)
    - Windows [67](#)
  - naming API objects [67](#)
  - operating system [69](#)

## K

- keyboard [239](#)

## L

- LAN-free
  - data transfer [32](#)
  - dsmEndGetDataEX function [87](#)
  - dsmSetUp function [9](#)
- logging events [65](#)
- low-level names
  - dsmRenameObj function [115](#)
- low-level qualifier [67](#)
- LZ4 compression [38](#)
- LZW compression [38](#)

## M

- makemtu [71](#)
- management class
  - associating objects [24](#)
  - binding and rebinding to files [25](#)
  - dsmBindMC, assigned by [24](#)
  - querying [25](#)
- mbcs [71](#)
- messages
  - dsmRCMsg function [112](#)
- metadata
  - object naming [20](#)
- multithreading
  - flag [9](#)
  - mtflag value [14](#)
  - multithread option [14](#)
  - overview [14](#)
  - restrictions [14](#)

## N

- node replication [48](#)
- nodes
  - accessing across owners [22](#)
  - authorization [65](#)
  - names [9](#)
  - querying management classes [25](#)
  - with client proxy support [70](#)
- NULL
  - backup or archive group [24](#)

## O

- object
  - version control [32](#), [37](#)
- object ids, overview [20](#)
- object naming
  - dsmBindMC [21](#)
  - examples by OS [21](#)
  - file space name [20](#)
  - high-level
    - object name [21](#)
  - interoperability [67](#)
  - low-level
    - object name [21](#)
  - object type [21](#)
  - overview [20](#)
- object types [21](#)
- objectID values [9](#)

## objects

- access rules [21](#)
- active copies [32](#), [37](#)
- deleting [64](#)
- deleting from server [65](#)
- expiration cycle [65](#)
- inactive copies [32](#), [37](#)
- turning off [65](#)
- updating [64](#)
- operating system interoperability [69](#)
- option list
  - format [101](#), [104](#)
- option string
  - API [2](#)
  - fromowner [22](#)
- options
  - compressalways [2](#)
  - enablearchiveretentionprotection [27](#)
  - errorlogretention [65](#)
  - fromnode [22](#)
  - fromowner [22](#)
  - not supported on API [1](#)
  - passwordaccess [14](#), [99](#)
  - servername [2](#)
  - set by administrator [2](#)
  - tcpbuffsize [32](#)
  - tcpnodelay [32](#)
  - tcpserveraddr [2](#)
- options files
  - user [3](#)
- owner authority [19](#)
- owner name
  - NULL [21](#)

## P

- partial object restore or retrieve [57](#)
- passwordaccess
  - generate [126](#)
  - option [9](#), [41](#)
- passwordaccess option
  - dsmInit function [99](#)
  - generate [16](#)
  - multithreading [14](#)
  - userNamePswd value [20](#)
- passwordaccess prompt [16](#)
- passworddir option
  - in dsm.sys [18](#)
- path examples
  - by OS [21](#)
- path information
  - interoperability [67](#)
- PERFMONCOMMTIMEOUT [36](#)
- PERFMONTCPPORT [35](#)
- PERFMONTCPSERVERADDRESS [35](#)
- performance considerations
  - dsmSendData function [32](#)
- performance monitor
  - client [34](#)
- policies to store data [24](#)
- policy
  - retention policy [28](#)
- proxynode [70](#)
- publications [vii](#)

## Q

- qMCDData structure [31](#)
- qryRespArchiveData [26](#)
- qryRespBackupData
  - dsmDeleteObj function [86](#)
- qryRespBackupData structure [28](#)
- queries, system [28](#)
- query
  - actlog [106](#)
  - command [68](#)
  - nodes with client proxy node authority [70](#)

## R

- rcApiOut
  - example, details [16](#)
- rcApiOut function
  - session [15](#)
- receiving data from a server
  - general description [57](#)
  - partial object restore or retrieve [57](#)
  - procedure [58](#)
- recommendations
  - dsmGetObject
    - large amounts of data [97](#)
    - setting HP thread stack [14](#)
- registering file spaces [22](#)
- release.h header file [140](#), [195](#)
- replication status [49](#)
- restore
  - objects from a server [57](#)
- restrictions
  - encryption and compression using buffer copy
  - elimination [40](#)
  - multithreading [14](#)
- retention protection [27](#)
- retrieve
  - objects from a server [57](#)
- return codes
  - obtaining through dsmRCMsg [112](#)
  - source header file [131](#), [185](#)

## S

- sample API applications
  - callbuff [5](#)
  - callbuff - data buffer [5](#)
  - callevnt [5](#)
  - callevnt - event-based retention [5](#)
  - callhold [5](#)
  - callhold - detention hold [5](#)
  - callmt\* [5](#)
  - callmt\* - multithreaded sample API applications [5](#)
  - callmtu1.c [71](#)
  - callmtu2.c [71](#)
  - callret [5](#)
  - callret - data retention protection sample API applications [5](#)
  - dapi\* [5](#)
  - dapi\* - interactive, single-threaded [5](#)
  - dsmgrp [5](#)
  - dsmgrp\* - object grouping sample [5](#)

- sample API applications (*continued*)
  - UNIX or Linux [5](#)
  - Windows 64-bit [7](#)
- sample application
  - callmt1.c [14](#)
- sample code
  - dsmgrp.c [57](#)
- security [16](#)
- selecting objects
  - to restore [59](#)
- sending data
  - to non-Unicode file spaces [71](#)
- sending data to a server [32](#)
- server
  - deleting objects from [65](#)
- server-side data deduplication [48](#)
- servername [2](#)
- session
  - password
    - session [16](#)
  - security [16](#)
  - starting with dsmInitEx [15](#)
- set access [68](#)
- sign-on process [16](#)
- signal handlers [15](#)
- signals, using [15](#)
- simultaneous-write
  - operations
  - storage pools [32](#)
- size estimates [36](#)
- size limits
  - API data structures [12](#), [31](#)
- sizing objects [36](#)
- sorting objects
  - by restore order [59](#)
- starting a session [15](#)
- state
  - InSession [107](#)
- state diagram
  - backup and archive example [51](#)
  - restore and retrieve [61](#)
- stopping a session [15](#)
- storage pools
  - simultaneous-write operations
  - [32](#)
- structure
  - qryRespBackupData [28](#)
  - qryRespFSDData function [22](#)
- structures
  - qMCDData [31](#)
  - size limits [12](#), [31](#)
- system queries [28](#)

## T

- target nodes and traditional nodes [70](#)
- TCPport [17](#)
- TCPserver address [17](#)
- tcpserveraddr [2](#)
- transaction model
  - dsmBeginTxn function [81](#)
- tsmapifp.h [71](#)
- tsmapifp.h header file [179](#), [235](#)
- tsmapitd.h [71](#)

- tsmapitd.h header file [140](#), [195](#)
- turning off objects [65](#)

## U

- Unicode
  - mbcs [71](#)
  - non-Unicode file spaces [71](#)
  - setting up [71](#)
  - Windows [71](#)
- UNIX or Linux
  - sample API application [5](#)
- user
  - intervention [15](#)

## V

- version control
  - API data structures [13](#)
  - dsmQueryApiVersionEx, using [12](#)
  - managing backed-up copies [32](#), [37](#)
- versions
  - files retained [24](#)

## W

- Windows 64-bit
  - sample application [7](#)







Product Number: 5725-W98  
5725-W99  
5725-X15